PANDUAN LAB

Panduan Lab

Training Arsitektur Microservices dan Cloud dengan Golang

DISIAPKAN UNTUK

Peserta In-house Training

Training	**26 – 29 Oktober 2026**
Studi kasus	**Sistem simulasi perbankan: account, transfer, notification**
Repositori	**github.com/artivisi/training-microservices-2026-01**

PT ArtiVisi Intermedia
endy@artivisi.com · artivisi.com

Oktober 2026

1 Lab 1: Account Service dengan REST dan PostgreSQL

1.1 Tujuan

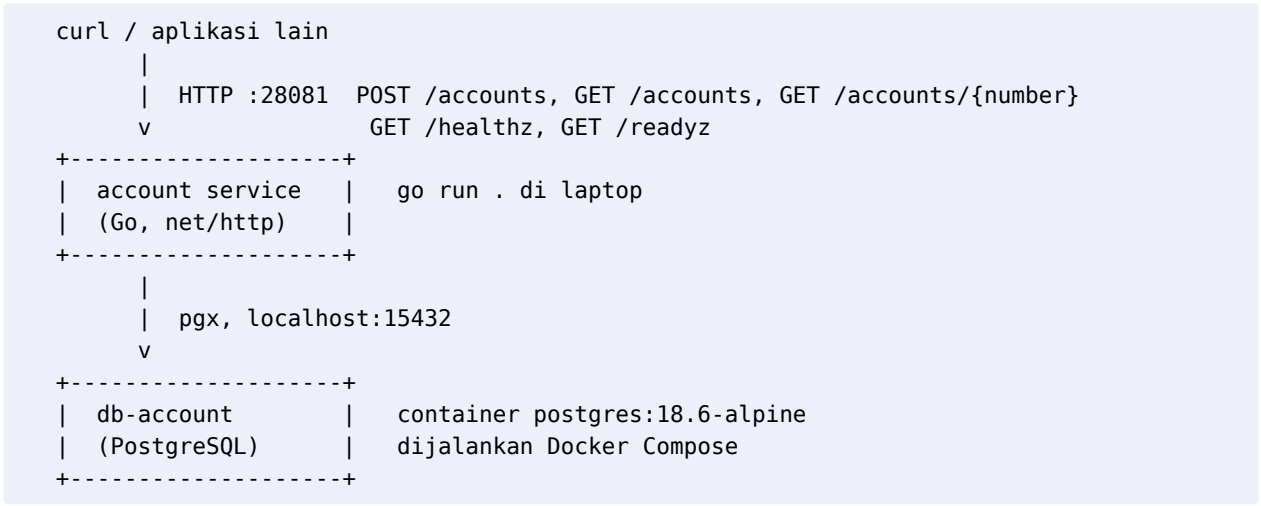
Setelah menyelesaikan lab ini, peserta mampu:

- menyusun Go *workspace* (`go.work`) yang menampung beberapa Go *module*, satu *module* untuk setiap *service*;
- menjalankan PostgreSQL sebagai infrastruktur lab dengan Docker Compose;
- membuat REST API dengan `net/http` bawaan Go, menggunakan *routing* berdasarkan *method* dan *path* (`ServeMux` sejak Go 1.22);
- menyimpan data ke PostgreSQL dengan pgx tanpa ORM, termasuk nilai uang bertipe `numeric`;
- membaca konfigurasi hanya dari *environment variable* dan menghentikan *service* saat *startup* bila ada variabel yang kosong;
- membedakan *liveness probe* (`/healthz`) dan *readiness probe* (`/readyz`);
- menguji *handler* HTTP tanpa database menggunakan `httptest` dan *fake store*.

1.2 Gambaran

Studi kasus pelatihan ini adalah sistem simulasi perbankan yang terdiri atas tiga *service*: `account`, `transfer`, dan `notification`. Lab 1 mengerjakan *service* pertama, yaitu `account`, yang menyimpan data rekening beserta saldonya. *Service* lain yang membutuhkan data rekening, misalnya `transfer` pada Lab 2, wajib mengaksesnya melalui API milik `account` dan tidak boleh membaca tabel `account` secara langsung.

Prinsip tersebut disebut *database-per-service*. Setiap *service* memiliki database sendiri, sehingga struktur tabel dapat diubah tanpa merusak *service* lain. Karena itu `compose.yaml` pada lab ini menjalankan satu *container* PostgreSQL khusus untuk `account`. Lab berikutnya menambahkan *container* PostgreSQL lain untuk `transfer` dan `notification`.



Selama Lab 1 hingga Lab 8, *service* Go dijalankan langsung di laptop dengan `go run`, sedangkan infrastruktur (database, Kafka, dan lain-lain) berjalan di *container*. Dengan pembagian ini, perubahan kode dapat langsung dicoba dengan menjalankan ulang `go run`, tanpa mem-*build image* baru. Port database sengaja dimulai dari 15432 agar tidak bentrok dengan PostgreSQL yang mungkin sudah terpasang di laptop pada port 5432.

Struktur kode di akhir lab ini:

```
studi-kasus/
├── go.work                    daftar module dalam workspace
```

```
├── compose.yaml                infrastruktur lab (PostgreSQL)
├── Makefile                    infra-up, infra-down, infra-reset, test
├── .gitignore
├── README.md                   disediakan oleh checkpoint
└── account/
    ├── go.mod
    ├── go.sum
    ├── Makefile                run, test, build
    ├── main.go                 run(): config, database, HTTP server, shutdown
    ├── config.go               LoadConfig() dari environment variable
    └── internal/account/
        ├── account.go          model, validasi, error, interface Store
        ├── schema.sql          tabel account
        ├── store.go            implementasi Store dengan pgx
        ├── http.go             handler dan routing
        └── http_test.go        unit test handler dengan fake store
```

Kode di dalam `internal/account` disusun dengan pola *package by feature* (pengelompokan menurut fitur): seluruh kode yang berkaitan dengan rekening berada di satu *package*. Direktori `internal` juga mencegah *module* lain mengimpor *package* tersebut, sehingga *service* lain tidak dapat bergantung pada detail implementasi `account`.

1.3 Titik Awal

Lab 1 dimulai dari *checkpoint* `lab-00`, yaitu repositori yang baru berisi materi dan `studi-kasus/README.md`. Bila repositori belum ada di laptop, lakukan *clone* terlebih dahulu. Peserta Windows menjalankan perintah berikut di terminal Ubuntu (WSL2), dengan repositori di *home* Ubuntu. Repositori yang di-*clone* dengan Git for Windows ke folder Windows dapat berisi akhir baris CRLF yang membuat skrip *shell* dan `Makefile` gagal:

```
git clone https://github.com/artivisi/training-microservices-2026-01.git
cd training-microservices-2026-01
git checkout -b lab-saya lab-00
cd studi-kasus
```

Perintah `git checkout -b lab-saya lab-00` membuat *branch* kerja baru bernama `lab-saya` dari tag `lab-00`. Seluruh pekerjaan lab disimpan di *branch* tersebut, sehingga tag *checkpoint* tetap utuh sebagai pembanding.

Pastikan Go dan Docker sudah siap. Versi Go minimal 1.26.

```
go version
docker compose version
```

Seluruh perintah pada lab ini dijalankan dari direktori `studi-kasus`, kecuali bila disebutkan lain.

1.4 Langkah

1.4.1 Membuat Go Module dan Go Workspace

Setiap *service* dibuat sebagai Go *module* tersendiri dengan `go.mod` masing-masing. Pemisahan ini membuat dependensi setiap *service* tercatat sendiri, dan setiap *service* dapat di-*build* menjadi *image* terpisah pada Lab 9. Agar ketiga *module* tetap dapat dikerjakan bersama dari satu direktori, Go menyediakan *workspace* melalui file `go.work`. File tersebut mendaftarkan *module* yang aktif, sehingga perintah `go` yang dijalankan dari `studi-kasus` mengenali seluruh *service*.

> **Mengapa *monorepo*?** Studi kasus menyimpan ketiga *service* dan kontrak protobuf dalam satu repositori (*monorepo*). Pilihan ini mempermudah training: satu `git clone` sudah berisi semua

service, satu *tag checkpoint* mencatat kondisi seluruh sistem di akhir setiap lab, dan perubahan kontrak gRPC beserta kode di `account` dan `transfer` masuk dalam satu *commit*. Setiap *service* tetap memiliki `go.mod`, `Dockerfile`, dan *pipeline* sendiri. *Pipeline* di Lab 10 hanya berjalan untuk *service* yang berubah, berdasarkan *path filter*.

Di organisasi dengan tim terpisah per *service*, satu repositori per *service* (*polyrepo*) sering lebih cocok, karena hak akses, riwayat, dan jadwal rilis setiap *service* terpisah dengan jelas. Konsekuensinya, kontrak bersama seperti modul `proto` harus dirilis sebagai modul berversi dan diimpor dengan versi tersebut, sedangkan di studi kasus modul ini cukup dirujuk dengan *directive* `replace` ke `../proto` (Lab 4).

```
mkdir -p account/internal/account
cd account
go mod init github.com/artivisi/training-microservices-2026-01/studi-kasus/account
go mod edit -go=1.26.0
cd ..
go work init ./account
go work edit -go=1.26.0
```

Perintah `go mod init` dan `go work init` menuliskan versi Go yang terpasang di laptop. Perintah `go mod edit -go=1.26.0` dan `go work edit -go=1.26.0` menurunkan versi minimal tersebut ke 1.26.0, sama dengan *checkpoint*, agar kode tetap dapat dikompilasi oleh Go 1.26 maupun versi yang lebih baru. Versi pada `go.work` tidak boleh lebih rendah dibandingkan versi pada `go.mod`. Bila aturan ini dilanggar, setiap perintah `go` berhenti dengan pesan error yang menyebutkan kedua versi tersebut.

Isi `go.work` sekarang:

```
go 1.26.0

use ./account
```

Tambahkan juga `.gitignore` agar hasil *build* tidak ikut masuk ke Git.

File `studi-kasus/.gitignore`

```
bin/
```

1.4.2 Menjalankan PostgreSQL dengan Docker Compose

Database `account` dijalankan sebagai *container* dari *image* `postgres:18.6-alpine`. *Service* Compose untuk database ini diberi nama `db-account`. Nama database dan nama *user* sama-sama `account`, dengan *password* `account123`. Kredensial ini hanya berlaku untuk lab lokal.

File `studi-kasus/compose.yaml`

```
# Lab infrastructure. Services run on the host (go run) and connect to these
# containers through the published ports (15432+, away from a locally installed
# PostgreSQL on 5432). One PostgreSQL container per service:
# database-per-service.
services:
  db-account:
    image: postgres:18.6-alpine
    environment:
      POSTGRES_DB: account
      POSTGRES_USER: account
      POSTGRES_PASSWORD: account123
```

```
    ports:
      - "15432:5432"
    volumes:
      - db-account:/var/lib/postgresql
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "account", "-d", "account"]
      interval: 2s
      retries: 30

volumes:
  db-account:
```

Beberapa bagian perlu diperhatikan:

- `ports: "15432:5432"` membuka port 5432 di dalam *container* sebagai port 15432 di laptop. *Service* Go yang berjalan di laptop terhubung ke `localhost:15432`.
- *Volume* `db-account` menyimpan data di luar *container*. Data tetap ada walaupun *container* dihentikan atau dibuat ulang. Mulai PostgreSQL 18, *image* resmi menyimpan data di bawah `/var/lib/postgresql`, sehingga *volume* di-*mount* ke direktori tersebut.
- `healthcheck` menjalankan `pg_isready` setiap 2 detik. Compose baru menganggap *container* siap setelah pemeriksaan ini berhasil.

Perintah Compose dibungkus dalam `Makefile` di direktori `studi-kasus`. Seluruh perintah lab, termasuk langkah *pipeline* CI/CD pada Lab 10, memanggil *target* `make`, sehingga perintah yang dijalankan di laptop sama dengan yang dijalankan di server.

File `studi-kasus/Makefile`

```
SERVICES := account

.PHONY: infra-up infra-down infra-reset test

infra-up:
  docker compose up -d --wait

infra-down:
  docker compose down

# Removes the database volumes too: all lab data is lost.
infra-reset:
  docker compose down -v

test:
  @for s in $(SERVICES); do $(MAKE) -C $$s test || exit 1; done
```

> Baris perintah di dalam *target* `Makefile` wajib diawali karakter *tab*, tidak boleh spasi. Bila `make` menampilkan `missing separator`, ganti spasi di awal baris tersebut dengan *tab*.

Opsi `--wait` membuat `docker compose up` menunggu hingga `healthcheck` berhasil sebelum perintah selesai. Jalankan infrastruktur lab:

```
make infra-up
docker compose ps
```

Kolom `STATUS` untuk `db-account` harus menampilkan `healthy`.

1.4.3 Konfigurasi dari Environment Variable

Service membutuhkan dua nilai konfigurasi: alamat *listen* HTTP *server* dan URL koneksi database. Nilai tersebut berbeda di setiap tempat *service* dijalankan (laptop, *container*, Kubernetes), sehingga dibaca dari *environment variable*.

Cara ini mengikuti *twelve-factor app* (12-factor), yaitu dua belas prinsip untuk aplikasi yang dijalankan sebagai *service* di *cloud*. Faktor III (*Config*) menetapkan bahwa nilai yang berbeda per *environment* disimpan di *environment variable*, terpisah dari kode. Dengan cara tersebut, *image* yang sama dapat dijalankan di laptop, di server *testing*, dan di *production* hanya dengan mengganti nilai variabelnya. Tabel berikut memuat faktor yang diterapkan di studi kasus beserta lab tempat penerapannya. Daftar lengkap kedua belas faktor ada di materi presentasi Hari 1.

Faktor	Penerapan di Studi Kasus	Lab
III. *Config*	Seluruh konfigurasi dibaca dari *environment variable*, tanpa nilai *default*. Di Kubernetes, nilainya berasal dari ConfigMap dan Secret	1, 9
IV. *Backing services*	Database, account service, Kafka, Hydra, dan *collector* OpenTelemetry dijangkau melalui alamat dari konfigurasi, sehingga dapat diganti tanpa mengubah kode	1, 2, 5, 7, 8
V. *Build, release, run*	Satu *image* hasil *build* digabungkan dengan konfigurasi per *environment* saat *deploy*	9, 10
VI. *Processes*	*Service* tidak menyimpan *state* di memori. Seluruh data disimpan di database	1, 9
VII. *Port binding*	*Service* membuka port HTTP sendiri (`HTTP_ADDR`), tanpa *web server* terpisah. Port gRPC (`GRPC_ADDR`) menyusul di Lab 4	1, 4
IX. *Disposability*	*Startup* cepat dan *graceful shutdown* saat menerima `SIGTERM`	1, 9, 10
XI. *Logs*	*Service* menulis *log* sebagai aliran *event* ke `stdout` (dan ke OTLP di Lab 8), tanpa mengelola *file log*. Penyimpanan dan pencarian *log* menjadi tugas platform	1, 8

Setiap variabel bersifat wajib. Bila salah satu kosong, *service* berhenti saat *startup* dengan pesan error yang menyebutkan nama variabelnya. Nilai *default* seperti `localhost:5432` sengaja tidak disediakan. Bila *service* menggunakan nilai *default* tanpa pesan apa pun dan terhubung ke database yang salah, penyebab masalahnya sulit ditemukan. *Service* yang langsung berhenti dengan pesan error jauh lebih mudah diperbaiki.

File `studi-kasus/account/config.go`

```
package main

import (
  "fmt"
  "os"
  "strings"
)

// Config comes from environment variables only (12-factor). Every variable is
// required: a missing value stops the service at startup instead of running
// with a guessed default.
```

```
type Config struct {
  HTTPAddr    string
  DatabaseURL string
}

func LoadConfig() (Config, error) {
  var missing []string
  get := func(name string) string {
    v := os.Getenv(name)
    if v == "" {
      missing = append(missing, name)
    }
    return v
  }
  cfg := Config{
    HTTPAddr:    get("HTTP_ADDR"),
    DatabaseURL: get("DATABASE_URL"),
  }
  if len(missing) > 0 {
    return Config{}, fmt.Errorf("missing required environment variables: %s",
strings.Join(missing, ", "))
  }
  return cfg, nil
}
```

Fungsi `get` merupakan *closure* yang mencatat setiap variabel kosong ke *slice* `missing` milik `LoadConfig`. Dengan cara ini, *startup* yang gagal langsung menampilkan sekaligus seluruh variabel yang belum diisi.

1.4.4 Model Domain dan Validasi

File `account.go` berisi struktur data rekening, data *request* pembuatan rekening, aturan validasi, dan *interface* `Store` untuk penyimpanan.

Saldo menggunakan `decimal.Decimal` dari *library* `shopspring/decimal`, tidak menggunakan `float64`. Tipe `float64` menyimpan bilangan dalam basis dua, sehingga nilai seperti 0,1 tidak dapat disimpan secara tepat dan selisih pembulatan menumpuk pada setiap operasi. Untuk nilai uang, selisih sekecil apa pun tidak dapat diterima. Di database, saldo disimpan sebagai `numeric(19,2)` yang juga bersifat desimal tepat.

Interface `Store` mendefinisikan operasi penyimpanan yang dibutuhkan *handler* HTTP. *Handler* bergantung pada *interface* tersebut dan tidak mengenal PostgreSQL secara langsung. Akibatnya, *handler* dapat diuji dengan *fake store* di memori (langkah pengujian di akhir lab).

File `studi-kasus/account/internal/account/account.go`

```
// Package account holds the account domain: model, validation, storage, and HTTP API.
package account

import (
  "context"
  "errors"
  "regexp"
  "strings"
  "time"

  "github.com/shopspring/decimal"
)

type Account struct {
```

```
  Number    string          `json:"number"`
  OwnerName string          `json:"owner_name"`
  Balance   decimal.Decimal `json:"balance"`
  CreatedAt time.Time       `json:"created_at"`
}

type CreateRequest struct {
  Number         string          `json:"number"`
  OwnerName      string          `json:"owner_name"`
  InitialBalance decimal.Decimal `json:"initial_balance"`
}

var (
  ErrNotFound      = errors.New("account not found")
  ErrAlreadyExists = errors.New("account already exists")
)

// ValidationError maps field name to the problem found in that field.
type ValidationError map[string]string

func (v ValidationError) Error() string { return "validation failed" }

var numberPattern = regexp.MustCompile(`^[0-9]{10}$`)

func (r CreateRequest) Validate() error {
  v := ValidationError{}
  if !numberPattern.MatchString(r.Number) {
    v["number"] = "must be exactly 10 digits"
  }
  if strings.TrimSpace(r.OwnerName) == "" {
    v["owner_name"] = "must not be empty"
  }
  if r.InitialBalance.IsNegative() {
    v["initial_balance"] = "must not be negative"
  }
  if len(v) > 0 {
    return v
  }
  return nil
}

// Store is the persistence contract; the HTTP handler depends on this, not on
PostgreSQL.
type Store interface {
  Create(ctx context.Context, req CreateRequest) (Account, error)
  Get(ctx context.Context, number string) (Account, error)
  List(ctx context.Context) ([]Account, error)
  Ping(ctx context.Context) error
}
```

Hal yang perlu dicermati pada file ini:

- *Struct tag* seperti `json:"owner_name"` menentukan nama *field* di JSON. Nama *field* di Go diawali huruf kapital agar dapat diakses *package* `encoding/json`.
- `ErrNotFound` dan `ErrAlreadyExists` adalah *sentinel error*: nilai error tetap yang nanti diperiksa dengan `errors.Is` untuk menentukan kode status HTTP.

- `ValidationError` adalah `map[string]string` yang memiliki *method* `Error()`, sehingga memenuhi *interface* `error`. Saat dikirim sebagai JSON, nilai ini menjadi objek berisi nama *field* dan masalahnya. Seluruh *field* diperiksa sekaligus, sehingga klien menerima semua kesalahan dalam satu respons.
- `Validate` mengembalikan `nil` secara eksplisit bila tidak ada kesalahan. Mengembalikan `v` yang kosong akan menghasilkan *interface* `error` yang tidak `nil`, karena *interface* yang berisi nilai bertipe `ValidationError` dianggap tidak `nil` walaupun *map*-nya kosong.

1.4.5 Skema Tabel dan Store PostgreSQL

Skema tabel ditulis dalam file SQL terpisah lalu di-*embed* ke dalam *binary* dengan `go:embed`. *Service* menjalankan skema tersebut pada setiap *startup*. Klausa `if not exists` membuat perintah aman dijalankan berulang kali. Cara ini cukup untuk lab, sedangkan sistem sungguhan umumnya menggunakan *tools* migrasi skema yang mencatat versi skema.

File `studi-kasus/account/internal/account/schema.sql`

```
create table if not exists account (
    number     varchar(10)    primary key,
    owner_name varchar(100)   not null,
    balance    numeric(19, 2) not null check (balance >= 0),
    created_at timestamptz    not null default now()
);
```

Constraint `check (balance >= 0)` menjaga saldo agar tidak pernah negatif, bahkan bila ada kode yang lupa memeriksanya. Aturan bisnis yang paling penting sebaiknya juga dijaga oleh database.

Implementasi `Store` menggunakan `pgxpool`, yaitu *connection pool* dari pgx. *Pool* menyimpan beberapa koneksi yang dapat digunakan bergantian oleh *request* yang datang bersamaan. Kolom `numeric` PostgreSQL dibaca ke `decimal.Decimal`. Tanpa *codec* tambahan, pgx sudah dapat membacanya melalui *interface* `sql.Scanner`, tetapi nilainya dikonversi lewat teks terlebih dahulu. *Package* `pgx-shopspring-decimal` mendaftarkan *codec* khusus pada setiap koneksi baru melalui `AfterConnect`, sehingga konversi `numeric` dilakukan langsung tanpa melalui teks.

File `studi-kasus/account/internal/account/store.go`

```
package account

import (
  "context"
  _ "embed"
  "errors"
  "fmt"

  pgxdecimal "github.com/jackc/pgx-shopspring-decimal"
  "github.com/jackc/pgx/v5"
  "github.com/jackc/pgx/v5/pgconn"
  "github.com/jackc/pgx/v5/pgxpool"
)

//go:embed schema.sql
var schema string

type PostgresStore struct {
  pool *pgxpool.Pool
}

// NewPostgresStore connects, verifies the connection, and applies the schema.
func NewPostgresStore(ctx context.Context, databaseURL string) (*PostgresStore, error) {
```

```
  cfg, err := pgxpool.ParseConfig(databaseURL)
  if err != nil {
    return nil, fmt.Errorf("parse DATABASE_URL: %w", err)
  }
  cfg.AfterConnect = func(_ context.Context, conn *pgx.Conn) error {
    pgxdecimal.Register(conn.TypeMap())
    return nil
  }
  pool, err := pgxpool.NewWithConfig(ctx, cfg)
  if err != nil {
    return nil, fmt.Errorf("create pool: %w", err)
  }
  if err := pool.Ping(ctx); err != nil {
    pool.Close()
    return nil, fmt.Errorf("connect to database: %w", err)
  }
  if _, err := pool.Exec(ctx, schema); err != nil {
    pool.Close()
    return nil, fmt.Errorf("apply schema: %w", err)
  }
  return &PostgresStore{pool: pool}, nil
}

func (s *PostgresStore) Close() { s.pool.Close() }

func (s *PostgresStore) Ping(ctx context.Context) error { return s.pool.Ping(ctx) }

func (s *PostgresStore) Create(ctx context.Context, req CreateRequest) (Account, error)
{
  rows, err := s.pool.Query(ctx, `
    insert into account (number, owner_name, balance)
    values (@number, @owner_name, @balance)
    returning number, owner_name, balance, created_at`,
    pgx.NamedArgs{"number": req.Number, "owner_name": req.OwnerName, "balance":
req.InitialBalance})
  if err != nil {
    return Account{}, err
  }
  a, err := pgx.CollectExactlyOneRow(rows, pgx.RowToStructByPos[Account])
  var pgErr *pgconn.PgError
  if errors.As(err, &pgErr) && pgErr.Code == "23505" { // unique_violation
    return Account{}, ErrAlreadyExists
  }
  return a, err
}

func (s *PostgresStore) Get(ctx context.Context, number string) (Account, error) {
  rows, err := s.pool.Query(ctx,
    `select number, owner_name, balance, created_at from account where number =
@number`,
    pgx.NamedArgs{"number": number})
  if err != nil {
    return Account{}, err
  }
  a, err := pgx.CollectExactlyOneRow(rows, pgx.RowToStructByPos[Account])
  if errors.Is(err, pgx.ErrNoRows) {
    return Account{}, ErrNotFound
  }
```

```
  return a, err
}

func (s *PostgresStore) List(ctx context.Context) ([]Account, error) {
  rows, err := s.pool.Query(ctx,
    `select number, owner_name, balance, created_at from account order by number limit
100`)
  if err != nil {
    return nil, err
  }
  return pgx.CollectRows(rows, pgx.RowToStructByPos[Account])
}
```

Penjelasan bagian penting:

- ***Named parameter.*** `pgx.NamedArgs` mengisi `@number`, `@owner_name`, dan `@balance`. Nilai dikirim terpisah dari teks SQL, sehingga kode aman dari *SQL injection*. Teks SQL tidak boleh disusun dengan menggabungkan *string* dari input pengguna.
- **Pemetaan baris ke *struct*.** `pgx.RowToStructByPos[Account]` mengisi *field* `Account` sesuai urutan kolom. Urutan kolom pada `select` dan `returning` harus sama dengan urutan *field* di *struct* `Account`.
- **Penanganan nomor rekening ganda.** Rekening dengan nomor yang sudah ada melanggar *primary key*, dan PostgreSQL mengembalikan kode error `23505` (`unique_violation`). Kode tersebut dipetakan menjadi `ErrAlreadyExists`, sehingga *handler* tidak perlu mengenal kode error PostgreSQL.
- **Rekening yang tidak ditemukan.** `CollectExactlyOneRow` mengembalikan `pgx.ErrNoRows` bila hasil *query* kosong, dan `Get` memetakannya menjadi `ErrNotFound`.
- **Batas jumlah data.** `List` dibatasi 100 baris agar satu *request* tidak memuat seluruh tabel. Pada sistem sungguhan, batas ini digantikan *pagination*.

1.4.6 Handler HTTP dan Routing

Sejak Go 1.22, `http.ServeMux` bawaan Go mendukung pola yang memuat *method* dan *path* sekaligus, misalnya `"GET /accounts/{number}"`. Nilai `{number}` dibaca dengan `r.PathValue("number")`. Fitur ini sudah mencukupi untuk API pada studi kasus, sehingga *router* pihak ketiga seperti chi atau gin tidak diperlukan. `ServeMux` juga otomatis mengembalikan status `405 Method Not Allowed` bila *path* cocok tetapi *method*-nya tidak terdaftar.

Bagian ini juga menyediakan dua *endpoint health check*, `/healthz` (*liveness*) dan `/readyz` (*readiness*). Perbedaan keduanya dibahas di bagian berikutnya, setelah kode *handler*.

File `studi-kasus/account/internal/account/http.go`

```
package account

import (
  "context"
  "encoding/json"
  "errors"
  "log/slog"
  "net/http"
  "time"
)

type Handler struct {
  store Store
  log   *slog.Logger
}

func NewHandler(store Store, log *slog.Logger) *Handler {
```

```
  return &Handler{store: store, log: log}
}

// Routes uses the Go 1.22+ ServeMux patterns: method + path, with {wildcards}.
func (h *Handler) Routes() *http.ServeMux {
  mux := http.NewServeMux()
  mux.HandleFunc("POST /accounts", h.create)
  mux.HandleFunc("GET /accounts", h.list)
  mux.HandleFunc("GET /accounts/{number}", h.get)
  mux.HandleFunc("GET /healthz", h.healthz)
  mux.HandleFunc("GET /readyz", h.readyz)
  return mux
}

func (h *Handler) create(w http.ResponseWriter, r *http.Request) {
  var req CreateRequest
  dec := json.NewDecoder(http.MaxBytesReader(w, r.Body, 1<<20))
  dec.DisallowUnknownFields()
  if err := dec.Decode(&req); err != nil {
    writeError(w, http.StatusBadRequest, "invalid JSON body: "+err.Error())
    return
  }
  if err := req.Validate(); err != nil {
    writeJSON(w, http.StatusBadRequest, map[string]any{"error": err.Error(), "fields":
err})
    return
  }
  a, err := h.store.Create(r.Context(), req)
  switch {
  case errors.Is(err, ErrAlreadyExists):
    writeError(w, http.StatusConflict, err.Error())
  case err != nil:
    h.internalError(w, r, err)
  default:
    w.Header().Set("Location", "/accounts/"+a.Number)
    writeJSON(w, http.StatusCreated, a)
  }
}

func (h *Handler) get(w http.ResponseWriter, r *http.Request) {
  a, err := h.store.Get(r.Context(), r.PathValue("number"))
  switch {
  case errors.Is(err, ErrNotFound):
    writeError(w, http.StatusNotFound, err.Error())
  case err != nil:
    h.internalError(w, r, err)
  default:
    writeJSON(w, http.StatusOK, a)
  }
}

func (h *Handler) list(w http.ResponseWriter, r *http.Request) {
  accounts, err := h.store.List(r.Context())
  if err != nil {
    h.internalError(w, r, err)
    return
  }
  writeJSON(w, http.StatusOK, accounts)
```

```
}

// healthz (liveness): the process is up and serving HTTP. No dependency checks here,
// otherwise a database outage makes Kubernetes restart every healthy pod.
func (h *Handler) healthz(w http.ResponseWriter, _ *http.Request) {
  writeJSON(w, http.StatusOK, map[string]string{"status": "UP"})
}

// readyz (readiness): the service can do useful work, i.e. the database answers.
func (h *Handler) readyz(w http.ResponseWriter, r *http.Request) {
  ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
  defer cancel()
  if err := h.store.Ping(ctx); err != nil {
    // Connection details stay in the log; callers only learn that the database is down.
    h.log.Warn("readiness check failed", "error", err)
    writeJSON(w, http.StatusServiceUnavailable, map[string]string{"status": "DOWN",
"database": "DOWN"})
    return
  }
  writeJSON(w, http.StatusOK, map[string]string{"status": "UP", "database": "UP"})
}

func (h *Handler) internalError(w http.ResponseWriter, r *http.Request, err error) {
  h.log.Error("request failed", "method", r.Method, "path", r.URL.Path, "error", err)
  writeError(w, http.StatusInternalServerError, "internal error")
}

func writeError(w http.ResponseWriter, status int, msg string) {
  writeJSON(w, status, map[string]string{"error": msg})
}

func writeJSON(w http.ResponseWriter, status int, body any) {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(status)
  _ = json.NewEncoder(w).Encode(body)
}
```

Alur `create` mengikuti urutan yang umum pada *handler* REST:

1. **Membaca *body*.** `http.MaxBytesReader` membatasi ukuran *body* hingga 1 MiB, sehingga *request* berukuran sangat besar tidak menghabiskan memori. `DisallowUnknownFields` menolak *field* yang tidak dikenal, sehingga salah ketik seperti `"nomor"` langsung menghasilkan error dan tidak diabaikan begitu saja.
2. **Validasi.** Kesalahan input dijawab dengan status 400 beserta daftar *field* yang bermasalah.
3. **Menyimpan.** Error dari `Store` dipetakan ke kode status HTTP: `ErrAlreadyExists` menjadi 409, error lain menjadi 500.
4. **Menjawab.** *Handler* mengirim rekening baru dengan status 201 dan *header* `Location` yang menunjuk ke alamat rekening tersebut.

Error yang tidak terduga ditangani `internalError`, yang menulis detail error ke *log* dan hanya mengirim `"internal error"` ke klien. Detail seperti nama tabel atau alamat database tidak boleh terkirim ke klien. Prinsip yang sama berlaku pada `readyz`.

1.4.7 *Liveness* dan *Readiness*

Kedua *endpoint* sama-sama mengembalikan 200 saat *service* sehat, tetapi menjawab pertanyaan yang berbeda, dan hasilnya memicu tindakan yang berbeda pula. Tindakan tersebut dijalankan oleh platform.

Di lab ini belum ada platform yang membaca hasilnya, sehingga keduanya diperiksa manual dengan `curl` (bagian Verifikasi). Kubernetes mulai menggunakannya di Lab 9.

Aspek	`/healthz` (*liveness*)	`/readyz` (*readiness*)
Pertanyaan	Apakah proses ini masih berjalan normal, atau macet sehingga perlu dijalankan ulang?	Apakah *instance* ini boleh menerima *request* saat ini?
Yang diperiksa di studi kasus	Tidak ada dependensi. Selama proses dapat menjawab HTTP, hasilnya 200	`Ping` ke database milik *service* itu sendiri dengan *timeout* 2 detik. Hasilnya 503 bila database tidak menjawab
Tindakan Kubernetes saat gagal	kubelet me-*restart container* setelah beberapa kegagalan berturut-turut	*Pod* dikeluarkan dari daftar *endpoint* Service sehingga tidak menerima *request* baru. *Container* tidak di-*restart*, dan *pod* kembali menerima *request* begitu `/readyz` kembali 200
Kesalahan desain yang umum	Ikut memeriksa database: saat database terganggu, semua *pod* di-*restart* bersamaan, padahal *restart* tidak memperbaiki database	Ikut memeriksa *service* lain: gangguan di satu *service* membuat semua pemanggilnya ikut keluar dari Service (kegagalan berantai)

Tabel berikut menunjukkan hasil kedua *endpoint* pada beberapa kondisi, beserta tindakan Kubernetes di Lab 9:

Kondisi	`/healthz`	`/readyz`	Tindakan Kubernetes	Alasan
Startup, *service* sedang membuka koneksi ke database	Belum menjawab, karena HTTP *server* baru dibuka setelah koneksi database berhasil	Belum menjawab	*Pod* belum *ready* dan belum menerima *request*. Bila database tidak dapat dihubungi, *service* berhenti dengan error, lalu kubelet menjalankan ulang *container*	*Request* tidak boleh sampai ke *instance* yang belum siap
Database mati sementara	200	503	*Pod* dikeluarkan dari Service, tidak di-*restart*	*Restart* tidak memperbaiki database. *Connection pool* membuka koneksi baru setelah database kembali berjalan
Proses macet dan tidak lagi menjawab HTTP	Tidak menjawab dalam batas waktu *probe*	Tidak menjawab	kubelet me-*restart container*	Proses yang macet hanya dapat dipulihkan dengan menjalankannya ulang
Overload, *request* menumpuk	200, tetapi bisa melewati batas waktu *probe*	Lambat, atau 503 bila *ping* ke database melewati 2 detik	*Readiness* yang gagal mengeluarkan *pod* sementara dari Service. *Liveness* baru memicu *restart* setelah 3 kegagalan berturut-turut	Ambang *liveness* dibuat longgar. *Restart* saat beban tinggi memindahkan beban ke *pod* lain dan memperparah keadaan. *Overload* diatasi dengan menambah *replica* (Hari 4)

Kondisi	/healthz	/readyz	Tindakan Kubernetes	Alasan
`account` lambat atau mati, dilihat dari `transfer`	200	200, karena `/readyz` milik `transfer` tidak memeriksa `account`	Tidak ada. *Pod* `transfer` tetap menerima *request*	Gangguan `account` ditangani *circuit breaker* (Lab 3). *Request* yang tidak membutuhkan `account`, misalnya membaca status transfer, tetap dilayani

Pertanyaan yang sering muncul: mengapa perlu dua *probe*, padahal *service* yang hidup tetapi tidak siap juga tidak berguna? Jawabannya, gangguan terbagi dua jenis yang membutuhkan tindakan berbeda. Gangguan jenis pertama hilang dengan sendirinya, misalnya database yang sementara mati atau beban sesaat. *Pod* cukup berhenti menerima *request* sambil menunggu, dan *restart* justru tidak membantu. Gangguan jenis kedua hanya hilang dengan *restart*, misalnya proses yang macet (baris ketiga tabel di atas). Tindakan untuk kedua jenis gangguan tersebut berlawanan, sehingga satu *probe* tidak cukup. Bila satu *probe* memicu *restart*, gangguan database membuat semua *pod* di-*restart*. Bila *probe* tersebut hanya menghentikan *request*, *pod* yang macet tetap macet selamanya. Karena itu *readiness* menangani jenis pertama dan *liveness* menangani jenis kedua.

Kubernetes juga menyediakan *startup probe* untuk *service* yang membutuhkan waktu lama saat *startup*, misalnya karena memuat data dalam jumlah besar. Selama *startup probe* belum berhasil, *liveness* dan *readiness probe* belum dijalankan, sehingga *service* tersebut tidak di-*restart* sebelum selesai *startup*. *Service* di studi kasus selesai *startup* dalam waktu kurang dari satu detik, sehingga tidak memerlukan *startup probe*.

1.4.8 Program Utama dan Graceful Shutdown

`main.go` merangkai seluruh bagian: membaca konfigurasi, membuka koneksi database, lalu menjalankan HTTP *server*. Logika tersebut ditempatkan di fungsi `run` yang mengembalikan `error`, sehingga `main` cukup mencatat error dan keluar dengan kode 1. Perintah `defer` di dalam `run`, misalnya `store.Close()`, tetap dijalankan sebelum program berakhir. Bila `os.Exit` dipanggil langsung di tengah logika, `defer` tidak sempat dijalankan.

Graceful shutdown berarti *service* yang menerima sinyal berhenti (Ctrl+C atau `SIGTERM` dari Docker dan Kubernetes) tidak langsung mati. *Server* berhenti menerima koneksi baru, menyelesaikan *request* yang sedang diproses (maksimal 10 detik), lalu menutup koneksi database.

File `studi-kasus/account/main.go`

```
package main

import (
  "context"
  "errors"
  "log/slog"
  "net/http"
  "os"
  "os/signal"
  "syscall"
  "time"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/account/internal/
account"
)

func main() {
```

```
  log := slog.New(slog.NewTextHandler(os.Stdout, nil))
  if err := run(log); err != nil {
    log.Error("account service stopped", "error", err)
    os.Exit(1)
  }
}

func run(log *slog.Logger) error {
  cfg, err := LoadConfig()
  if err != nil {
    return err
  }

  // ctx is cancelled on Ctrl+C (SIGINT) or on SIGTERM from Docker/Kubernetes.
  ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
  defer stop()

  store, err := account.NewPostgresStore(ctx, cfg.DatabaseURL)
  if err != nil {
    return err
  }
  defer store.Close()

  srv := &http.Server{
    Addr:              cfg.HTTPAddr,
    Handler:           account.NewHandler(store, log).Routes(),
    ReadHeaderTimeout: 5 * time.Second,
  }

  errCh := make(chan error, 1)
  go func() {
    log.Info("account service listening", "addr", cfg.HTTPAddr)
    errCh <- srv.ListenAndServe()
  }()

  select {
  case err := <-errCh:
    return err
  case <-ctx.Done():
    log.Info("shutting down")
    shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil && !errors.Is(err,
http.ErrServerClosed) {
      return err
    }
    return nil
  }
}
```

Beberapa konsep Go yang muncul di sini:

- `signal.NotifyContext` menghasilkan `context` yang dibatalkan ketika sinyal `SIGINT` atau `SIGTERM` diterima.
- `srv.ListenAndServe()` bersifat *blocking*, sehingga dijalankan di *goroutine* tersendiri. Hasilnya dikirim ke *channel* `errCh`.
- `select` menunggu salah satu dari dua kejadian: *server* gagal (misalnya port sudah digunakan) atau sinyal berhenti diterima.

- `ReadHeaderTimeout` membatasi waktu klien mengirim *header* HTTP. Tanpa batas ini, klien yang sangat lambat dapat menahan koneksi selamanya.
- *Logger* `slog` menulis *log* terstruktur berformat `key=value`. Pada Lab 8, format ini diganti menjadi JSON dan dilengkapi *correlation ID*.

1.4.9 Dependensi dan Makefile per Service

Tambahkan *library* yang digunakan dengan `go get`, pada versi yang sama seperti *checkpoint*:

```
cd account
go get github.com/jackc/pgx/v5@v5.11.0 \
  github.com/shopspring/decimal@v1.6.0 \
  github.com/jackc/pgx-shopspring-decimal@v0.0.0-20220624020537-1d36b5a1853e
go mod tidy
cd ..
```

`go mod tidy` menyesuaikan `go.mod` dengan *import* yang benar-benar ada di kode dan melengkapi `go.sum`. Kedua file tersebut di-*commit* bersama kode.

Setiap *service* memiliki `Makefile` sendiri dengan *target* `run`, `test`, dan `build`. *Target* `run` mengisi kedua *environment variable* dengan nilai untuk laptop. Nilai ini hanya ada di `Makefile`, tidak di dalam kode Go.

File `studi-kasus/account/Makefile`

```
# Every pipeline step calls a target here, so CI and laptops run the same commands.
SERVICE := account

.PHONY: run test build clean

run:
  HTTP_ADDR=:28081 \
  DATABASE_URL=postgres://account:account123@localhost:15432/account \
  go run .

test:
  go vet ./...
  go test ./...

build:
  CGO_ENABLED=0 go build -o bin/$(SERVICE) .

clean:
  rm -rf bin
```

Target `build` menggunakan `CGO_ENABLED=0` agar menghasilkan *static binary* yang tidak bergantung pada *library* C di sistem operasi. *Binary* seperti ini dapat dijalankan di *image* kosong (`FROM scratch`) pada Lab 9.

Semua *service* studi kasus menggunakan port di rentang 2xxxx, misalnya 28081 untuk `account`, sedangkan infrastruktur lab menggunakan rentang 1xxxx, sehingga tidak bentrok dengan aplikasi lain di laptop yang biasanya menggunakan port 8xxx.

Jalankan *service*:

```
make -C account run
```

Baris terakhir *output* yang diharapkan (waktu akan berbeda):

```
time=2026-10-26T09:15:02.123+07:00 level=INFO msg="account service listening"
addr=:28081
```

Biarkan terminal ini tetap berjalan, lalu buka terminal kedua untuk langkah berikutnya (unit test dan verifikasi).

1.4.10 Unit Test untuk Handler dengan Fake Store

Handler HTTP diuji tanpa PostgreSQL. `fakeStore` mengimplementasikan *interface* `Store` dengan *map* di memori, lalu `httptest.NewRecorder` menangkap respons *handler* tanpa membuka port jaringan. Test ini berjalan dalam hitungan milidetik dan dapat dijalankan di *pipeline* CI yang tidak memiliki database.

Test disusun sebagai *table-driven test*: setiap baris tabel berisi satu skenario (*method*, *path*, *body*, status yang diharapkan, dan potongan *body* respons yang diharapkan). Menambah skenario cukup dengan menambah satu baris.

File `studi-kasus/account/internal/account/http_test.go`

```
package account

import (
  "context"
  "io"
  "log/slog"
  "net/http"
  "net/http/httptest"
  "strings"
  "testing"
  "time"
)

// fakeStore keeps accounts in memory so the HTTP layer is tested without PostgreSQL.
type fakeStore struct {
  accounts map[string]Account
  pingErr  error
}

func newFakeStore() *fakeStore { return &fakeStore{accounts: map[string]Account{}} }

func (f *fakeStore) Create(_ context.Context, req CreateRequest) (Account, error) {
  if _, ok := f.accounts[req.Number]; ok {
    return Account{}, ErrAlreadyExists
  }
  a := Account{Number: req.Number, OwnerName: req.OwnerName, Balance:
req.InitialBalance, CreatedAt: time.Now()}
  f.accounts[a.Number] = a
  return a, nil
}

func (f *fakeStore) Get(_ context.Context, number string) (Account, error) {
  a, ok := f.accounts[number]
  if !ok {
    return Account{}, ErrNotFound
  }
  return a, nil
}

func (f *fakeStore) List(context.Context) ([]Account, error) {
  var out []Account
```

```
  for _, a := range f.accounts {
    out = append(out, a)
  }
  return out, nil
}

func (f *fakeStore) Ping(context.Context) error { return f.pingErr }

func do(t *testing.T, h http.Handler, method, path, body string)
*httptest.ResponseRecorder {
  t.Helper()
  rec := httptest.NewRecorder()
  h.ServeHTTP(rec, httptest.NewRequest(method, path, strings.NewReader(body)))
  return rec
}

func TestAccountAPI(t *testing.T) {
  h := NewHandler(newFakeStore(), slog.New(slog.NewTextHandler(io.Discard,
nil))).Routes()

  tests := []struct {
    name, method, path, body string
    wantStatus               int
    wantBody                 string
  }{
    {"create", "POST", "/accounts",
`{"number":"1234567890","owner_name":"Budi","initial_balance":"500000"}`, 201,
`"balance":"500000"`},
    {"duplicate", "POST", "/accounts",
`{"number":"1234567890","owner_name":"Budi","initial_balance":"0"}`, 409, "already
exists"},
    {"invalid fields", "POST", "/accounts", `{"number":"12","owner_name":"
","initial_balance":"-1"}`, 400, `"number":"must be exactly 10 digits"`},
    {"unknown field", "POST", "/accounts", `{"nomor":"1234567890"}`, 400, "unknown
field"},
    {"get", "GET", "/accounts/1234567890", "", 200, `"owner_name":"Budi"`},
    {"not found", "GET", "/accounts/0000000000", "", 404, "not found"},
    {"wrong method", "DELETE", "/accounts/1234567890", "", 405, ""},
    {"liveness", "GET", "/healthz", "", 200, "UP"},
    {"readiness", "GET", "/readyz", "", 200, `"database":"UP"`},
  }
  // Subtests run in order and share the handler: "duplicate" and "get" rely on
"create".
  for _, tc := range tests {
    t.Run(tc.name, func(t *testing.T) {
      rec := do(t, h, tc.method, tc.path, tc.body)
      if rec.Code != tc.wantStatus {
        t.Fatalf("status = %d, want %d; body: %s", rec.Code, tc.wantStatus, rec.Body)
      }
      if !strings.Contains(rec.Body.String(), tc.wantBody) {
        t.Fatalf("body %q does not contain %q", rec.Body, tc.wantBody)
      }
    })
  }
}

func TestReadinessDown(t *testing.T) {
  store := newFakeStore()
```

```
  store.pingErr = context.DeadlineExceeded
  h := NewHandler(store, slog.New(slog.NewTextHandler(io.Discard, nil))).Routes()
  if rec := do(t, h, "GET", "/readyz", ""); rec.Code != http.StatusServiceUnavailable {
    t.Fatalf("status = %d, want 503", rec.Code)
  }
}
```

Skenario `"wrong method"` memastikan pola `ServeMux` benar-benar membedakan *method*: `DELETE` ke *path* yang hanya terdaftar untuk `GET` menghasilkan 405. `TestReadinessDown` mensimulasikan database yang tidak menjawab dengan membuat `Ping` mengembalikan error.

Jalankan seluruh test dari direktori `studi-kasus`:

```
make test
```

Output yang diharapkan:

```
go vet ./...
go test ./...
?     github.com/artivisi/training-microservices-2026-01/studi-kasus/account  [no test
files]
ok    github.com/artivisi/training-microservices-2026-01/studi-kasus/account/internal/
account 0.012s
```

Pada Linux, `make` juga menampilkan baris `make[1]: Entering directory` dan `make[1]: Leaving directory`. Durasi test akan berbeda di setiap laptop. Untuk melihat setiap skenario, jalankan `go test -v ./...` dari direktori `account`.

1.5 Verifikasi

Jalankan perintah berikut di terminal kedua, sementara `make -C account run` masih berjalan di terminal pertama. Setiap respons JSON tampil dalam satu baris. Nama *field* pada objek JSON yang berasal dari *map* Go selalu diurutkan secara alfabetis.

1. *Liveness* dan *readiness*.

```
curl -i http://localhost:28081/healthz
curl -i http://localhost:28081/readyz
```

Keduanya mengembalikan `HTTP/1.1 200 OK` dengan *header* `Content-Type: application/json`. *Body* masing-masing:

```
{"status":"UP"}
{"database":"UP","status":"UP"}
```

2. Membuat rekening. Nilai uang dikirim sebagai *string* JSON agar tidak melewati konversi `float` di sisi klien.

```
curl -i -X POST http://localhost:28081/accounts \
  -H 'Content-Type: application/json' \
  -d '{"number":"1234567890","owner_name":"Budi Santoso","initial_balance":"500000"}'

curl -s -X POST http://localhost:28081/accounts \
  -H 'Content-Type: application/json' \
  -d '{"number":"9876543210","owner_name":"Siti Aminah","initial_balance":"250000.50"}'
```

Perintah pertama menghasilkan `HTTP/1.1 201 Created`, *header* `Location: /accounts/1234567890`, dan *body* seperti berikut (nilai `created_at` akan berbeda):

```
{"number":"1234567890","owner_name":"Budi
Santoso","balance":"500000","created_at":"2026-10-26T09:16:40.512345+07:00"}
```

Saldo `500000` disimpan di database sebagai `500000.00`. Angka nol di belakang koma tidak ditampilkan oleh `decimal.Decimal`, sehingga rekening kedua menampilkan saldo `"250000.5"`.

3. Membaca rekening.

```
curl -s http://localhost:28081/accounts/1234567890
curl -s http://localhost:28081/accounts
```

Perintah pertama mengembalikan satu objek rekening. Perintah kedua mengembalikan *array* berisi kedua rekening, diurutkan menurut nomor rekening. Sebelum ada rekening sama sekali, `GET /accounts` mengembalikan `[]`.

4. Kasus gagal.

```
# nomor rekening yang sama: 409
curl -s -w ' %{http_code}\n' -X POST http://localhost:28081/accounts \
  -H 'Content-Type: application/json' \
  -d '{"number":"1234567890","owner_name":"Budi","initial_balance":"0"}'

# validasi gagal: 400
curl -s -w ' %{http_code}\n' -X POST http://localhost:28081/accounts \
  -H 'Content-Type: application/json' \
  -d '{"number":"12","owner_name":" ","initial_balance":"-1"}'

# field tidak dikenal: 400
curl -s -w ' %{http_code}\n' -X POST http://localhost:28081/accounts \
  -H 'Content-Type: application/json' \
  -d '{"nomor":"1234567890"}'

# rekening tidak ada: 404
curl -s -w ' %{http_code}\n' http://localhost:28081/accounts/0000000000

# method tidak terdaftar: 405
curl -s -o /dev/null -w '%{http_code}\n' -X DELETE http://localhost:28081/accounts/
1234567890
```

Output yang diharapkan, berurutan:

```
{"error":"account already exists"}
 409
{"error":"validation failed","fields":{"initial_balance":"must not be
negative","number":"must be exactly 10 digits","owner_name":"must not be empty"}}
 400
{"error":"invalid JSON body: json: unknown field \"nomor\""}
 400
{"error":"account not found"}
 404
405
```

Kode status tampil di baris berikutnya karena `json.Encoder` menambahkan karakter *newline* di akhir setiap respons JSON.

5. *Readiness* saat database mati. Hentikan *container* database, periksa `/readyz` dan `/healthz`, lalu jalankan kembali database:

```
docker compose stop db-account
curl -s -w ' %{http_code}\n' http://localhost:28081/readyz
curl -s -w ' %{http_code}\n' http://localhost:28081/healthz
docker compose start db-account
```

Output yang diharapkan:

```
{"database":"DOWN","status":"DOWN"}
 503
{"status":"UP"}
 200
```

Terminal pertama menampilkan *log* `level=WARN msg="readiness check failed"` beserta detail error koneksi, sedangkan respons ke klien hanya berisi status. Setelah database kembali berjalan, `/readyz` kembali mengembalikan 200 tanpa perlu me-*restart service*, karena *connection pool* membuka koneksi baru.

6. Data tetap tersimpan. Hentikan *service* dengan Ctrl+C. *Log* menampilkan `msg="shutting down"`. Jalankan ulang `make -C account run`, lalu panggil `GET /accounts`. Kedua rekening masih ada karena data tersimpan di *volume* Docker.

7. Konfigurasi wajib. Jalankan *service* tanpa *environment variable* dari direktori `account`:

```
cd account
go run .
cd ..
```

Output yang diharapkan:

```
time=... level=ERROR msg="account service stopped" error="missing required environment
variables: HTTP_ADDR, DATABASE_URL"
exit status 1
```

1.6 Masalah Umum

Gejala	Penyebab	Solusi
`make infra-up` gagal dengan `port is already allocated`	Port 15432 sudah digunakan program lain, misalnya *container* dari latihan sebelumnya	Periksa dengan `docker ps` dan hentikan *container* yang menggunakan port tersebut. Pada Linux dan macOS, `lsof -i :15432` menampilkan proses yang menggunakan port
Service berhenti dengan `connect to database: ... connection refused`	*Container* `db-account` belum berjalan atau belum `healthy`	Jalankan `make infra-up` dan pastikan `docker compose ps` menampilkan status `healthy`
Service berhenti dengan `password authentication failed for user "account"`	*Volume* `db-account` dibuat sebelumnya dengan kredensial lain. PostgreSQL hanya membaca	Jalankan `make infra-reset` lalu `make infra-up`. Seluruh data lab di database akan terhapus

Gejala	Penyebab	Solusi
	`POSTGRES_PASSWORD` saat *volume* masih kosong	
`go: module ... listed in go.work file requires go >= 1.27...`	Versi Go di `go.mod` lebih tinggi dibandingkan versi di `go.work`	Jalankan `go mod edit -go=1.26.0` di direktori `account` dan `go work edit -go=1.26.0` di direktori `studi-kasus`
`make: *** missing separator`	Baris perintah di `Makefile` diawali spasi	Ganti spasi di awal baris perintah dengan karakter *tab*
`listen tcp :28081: bind: address already in use`	*Service* `account` dari terminal lain masih berjalan	Hentikan proses lama dengan Ctrl+C di terminal tersebut, lalu jalankan ulang
Field `balance` atau `created_at` kosong atau tertukar pada respons	Urutan kolom di `select`/`returning` tidak sama dengan urutan *field* di *struct* `Account`	Samakan urutan kolom dengan urutan *field*: `number`, `owner_name`, `balance`, `created_at`

1.7 Eksplorasi Mandiri

1. **Menambah *endpoint* penutupan rekening.** Tambahkan `DELETE /accounts/{number}` yang hanya berhasil bila saldo rekening nol. Tentukan kode status yang tepat untuk rekening yang tidak ditemukan dan untuk rekening yang saldonya belum nol, lalu tambahkan skenarionya ke tabel test. Skenario `"wrong method"` yang sudah ada perlu diperbarui.
2. ***Pagination.*** Ubah `GET /accounts` agar menerima parameter `limit` dan `after` (nomor rekening terakhir pada halaman sebelumnya), misalnya `GET /accounts?limit=10&after=1234567890`. Validasi `limit` antara 1 dan 100, dan jawab 400 bila nilainya di luar rentang.
3. **Test ke database sungguhan.** Tulis test untuk `PostgresStore` yang terhubung ke `db-account`. URL database dibaca dari *environment variable* `TEST_DATABASE_URL`, dan test dilewati dengan `t.Skip` bila variabel tersebut kosong. Test tersebut harus memastikan bahwa rekening ganda menghasilkan `ErrAlreadyExists` dan saldo negatif ditolak oleh *constraint* `check`.
4. **Melihat isi database.** Masuk ke database dengan `docker compose exec db-account psql -U account -d account`, lalu jalankan `\d account` dan `select * from account;`. Bandingkan nilai `balance` di database dengan nilai pada respons JSON.

1.8 Checkpoint

Kondisi kode di akhir lab ini tersimpan di tag `lab-01`. Untuk membandingkan hasil pekerjaan dengan *checkpoint*, jalankan dari direktori `studi-kasus`:

```
git diff lab-01 -- .
```

Perbedaan pada `README.md` dapat diabaikan, karena isi file tersebut disediakan oleh *checkpoint* dan tidak ditulis pada langkah lab.

Bila hasil pekerjaan belum selesai atau tidak dapat dijalankan, simpan pekerjaan lalu lanjutkan dari *checkpoint*:

```
git add -A && git commit -m "Lab 1 (pekerjaan sendiri)"
git checkout -b lab-01-checkpoint lab-01
```

Lab 2 dimulai dari kondisi ini: *service* `account` berjalan di port 28081 dengan database `db-account`. Lab 2 menambahkan *service* `transfer` yang memanggil API `account` melalui REST.

2 Lab 2: Transfer Service dan Panggilan REST Antar-Service

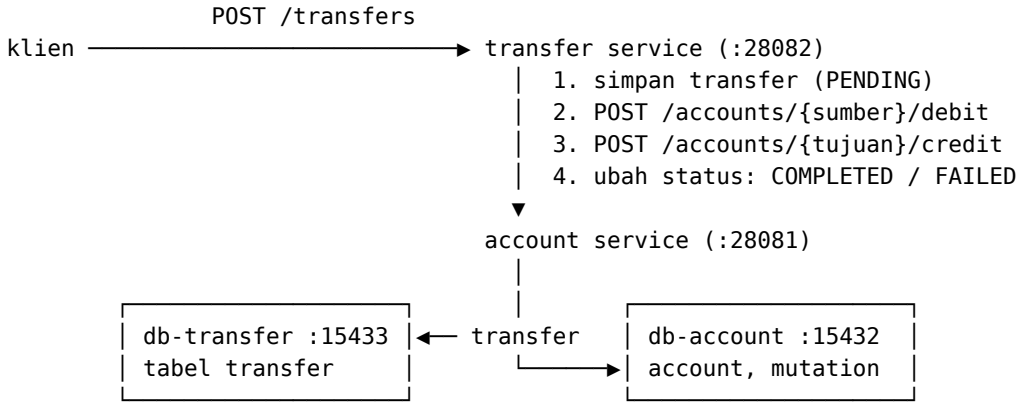
2.1 Tujuan

Setelah menyelesaikan lab ini, peserta mampu:

- menambahkan operasi debit dan kredit pada account service, lengkap dengan catatan mutasi dan pemeriksaan saldo yang aman terhadap *concurrent request*;
- membuat service kedua, transfer service, sebagai modul Go terpisah dengan database sendiri (*database-per-service*);
- memanggil account service dari transfer service melalui REST dengan `net/http`;
- memetakan respons HTTP dari service lain menjadi *error* bisnis yang dapat diperiksa dengan `errors.Is`;
- mengenali dua kelemahan yang sengaja dibiarkan pada lab ini: *HTTP client* tanpa *timeout* dan debit yang tidak dikembalikan ketika kredit gagal.

2.2 Gambaran

Hingga akhir Lab 1, sistem hanya berisi account service dengan database `account`. Lab 2 menambahkan transfer service. Transfer memindahkan sejumlah uang dari rekening sumber ke rekening tujuan. Data rekening dan saldo menjadi milik account service, sehingga transfer service tidak boleh membaca atau mengubah tabel `account` secara langsung. Transfer service hanya dapat mengakses data tersebut dengan memanggil API milik account service.



Setiap service memiliki database sendiri di *container* PostgreSQL yang berbeda. Konsekuensinya, debit dan kredit tidak dapat dibungkus dalam satu transaksi database. Debit terjadi di account service, sedangkan status transfer tersimpan di transfer service. Kedua service hanya terhubung melalui jaringan. Lab ini sengaja membuat versi paling sederhana dari alur tersebut supaya kelemahannya terlihat jelas:

- **Tanpa *timeout*.** Jika account service lambat, setiap transfer ikut menunggu tanpa batas. Lab 3 menambahkan *timeout*, *retry*, dan *circuit breaker*.
- **Tanpa kompensasi.** Jika debit berhasil lalu kredit gagal, uang sudah keluar dari rekening sumber tetapi tidak masuk ke rekening tujuan. Transfer service menandai transfer sebagai `FAILED` dan mencatatnya di *log* supaya saldo rekening sumber dapat dikoreksi manual. Lab 5 menyelesaikan masalah ini dengan pola *saga*.

Perubahan pada lab ini terbagi dua bagian. Bagian pertama melengkapi account service dengan *endpoint* debit dan kredit. Bagian kedua membuat transfer service dari awal.

2.3 Titik Awal

Lab ini dimulai dari *checkpoint* `lab-01`. Peserta yang sudah menyelesaikan Lab 1 dapat melanjutkan dari kode sendiri. Peserta yang tertinggal atau ingin memulai dari kondisi yang bersih dapat membuat *branch* kerja dari tag `lab-01`:

```
cd training-microservices-2026-01
git switch -c kerja-lab-02 lab-01
cd studi-kasus
make infra-up
```

Seluruh perintah pada lab ini dijalankan dari folder `studi-kasus/`, kecuali disebutkan lain. Pastikan account service dari Lab 1 masih dapat dijalankan dengan `make -C account run` sebelum melanjutkan.

2.4 Langkah

2.4.1 Tabel mutasi pada account service

Saldo rekening disimpan pada kolom `balance` di tabel `account`. Jika debit dan kredit hanya mengubah angka tersebut, tidak ada jejak tentang asal perubahan saldo. Sistem perbankan selalu menyimpan setiap mutasi sebagai baris tersendiri, sehingga saldo dapat ditelusuri kembali ke transaksi yang menyebabkannya. Kolom `reference` mengaitkan mutasi dengan transaksi bisnis pemicunya, dalam hal ini ID transfer.

Tambahkan definisi tabel berikut di akhir `account/internal/account/schema.sql`, setelah tabel `account`:

```
-- Every balance change is recorded; the balance column is the running total.
create table if not exists mutation (
    id             bigint generated always as identity primary key,
    account_number varchar(10)    not null references account (number),
    type           varchar(6)     not null check (type in ('DEBIT', 'CREDIT')),
    amount         numeric(19, 2) not null check (amount > 0),
    reference      varchar(64)    not null,
    created_at     timestamptz    not null default now()
);
```

Schema diterapkan otomatis ketika account service dijalankan (Lab 1 menggunakan `create table if not exists`), sehingga tabel baru langsung terbentuk pada *startup* berikutnya tanpa menghapus data lama.

2.4.2 Model mutasi dan kontrak `Store`

Debit dan kredit memiliki bentuk *request* yang sama: jumlah uang dan referensi. Perbedaannya hanya arah perubahan saldo, sehingga cukup satu tipe *request* dan satu tipe `MutationType`.

Pada `account/internal/account/account.go`, tambahkan kode berikut tepat setelah definisi `CreateRequest`:

```
// MutationRequest is the body of a debit or credit. Reference ties the mutation to
// the business transaction that caused it, e.g. a transfer ID.
type MutationRequest struct {
  Amount    decimal.Decimal `json:"amount"`
  Reference string          `json:"reference"`
}

type MutationType string

const (
  Debit  MutationType = "DEBIT"
  Credit MutationType = "CREDIT"
)
```

Saldo yang tidak mencukupi adalah penolakan bisnis, bukan kegagalan teknis. Penolakan ini perlu *error* tersendiri supaya *handler* dapat menjawab dengan status yang tepat. Ubah blok `var` yang berisi *error* menjadi:

```
var (
  ErrNotFound          = errors.New("account not found")
  ErrAlreadyExists     = errors.New("account already exists")
  ErrInsufficientFunds = errors.New("insufficient funds")
)
```

Validasi *request* mutasi diletakkan setelah *method* `CreateRequest.Validate`. Jumlah harus positif, dan referensi wajib diisi dengan panjang maksimal 64 karakter sesuai kolom `reference`:

```
func (r MutationRequest) Validate() error {
  v := ValidationError{}
  if !r.Amount.IsPositive() {
    v["amount"] = "must be greater than zero"
  }
  if strings.TrimSpace(r.Reference) == "" || len(r.Reference) > 64 {
    v["reference"] = "must be 1 to 64 characters"
  }
  if len(v) > 0 {
    return v
  }
  return nil
}
```

Terakhir, tambahkan *method* `Mutate` ke *interface* `Store`, di antara `List` dan `Ping`:

```
  // Mutate debits or credits the account and records the mutation, atomically.
  // It returns the account with its new balance.
  Mutate(ctx context.Context, number string, t MutationType, req MutationRequest) (Account,
error)
```

2.4.3 Implementasi `Mutate` di PostgreSQL

Perubahan saldo dan pencatatan mutasi harus terjadi bersama-sama. Jika salah satunya gagal, keduanya dibatalkan. Karena itu `Mutate` menggunakan satu transaksi database.

Bagian yang paling penting adalah pemeriksaan saldo. Cara yang sekilas tampak wajar adalah membaca saldo dengan `select`, membandingkannya di Go, lalu menjalankan `update`. Cara tersebut rawan *race condition*: dua debit yang datang bersamaan dapat sama-sama membaca saldo lama, sama-sama lolos pemeriksaan, lalu sama-sama mengurangi saldo. Solusinya adalah meletakkan pemeriksaan saldo di klausa `where` pada perintah `update`. PostgreSQL mengunci baris yang sedang diubah. Debit kedua menunggu *lock* tersebut dilepas, lalu mengevaluasi ulang kondisi `where` terhadap saldo terbaru.

Jika `update` tidak mengubah satu baris pun, ada dua kemungkinan: nomor rekening tidak ada, atau saldo tidak mencukupi. Kode membedakan keduanya dengan membaca rekening melalui `Get`.

Tambahkan *method* berikut ke `account/internal/account/store.go`, setelah *method* `Get`:

```
func (s *PostgresStore) Mutate(ctx context.Context, number string, t MutationType, req
MutationRequest) (Account, error) {
  tx, err := s.pool.Begin(ctx)
  if err != nil {
    return Account{}, err
  }
  defer tx.Rollback(ctx) // no-op after Commit

  delta := req.Amount
  if t == Debit {
    delta = delta.Neg()
  }
  // The balance check sits in the WHERE clause, so two concurrent debits cannot
  // both pass it: the second one waits for the row lock and re-evaluates.
  rows, err := tx.Query(ctx, `
```

```
    update account set balance = balance + @delta
    where number = @number and balance + @delta >= 0
    returning number, owner_name, balance, created_at`,
    pgx.NamedArgs{"number": number, "delta": delta})
  if err != nil {
    return Account{}, err
  }
  a, err := pgx.CollectExactlyOneRow(rows, pgx.RowToStructByPos[Account])
  if errors.Is(err, pgx.ErrNoRows) {
    // Nothing updated: either the account does not exist or the balance is too low.
    if _, getErr := s.Get(ctx, number); getErr != nil {
      return Account{}, getErr
    }
    return Account{}, ErrInsufficientFunds
  }
  if err != nil {
    return Account{}, err
  }

  if _, err := tx.Exec(ctx, `
    insert into mutation (account_number, type, amount, reference)
    values (@number, @type, @amount, @reference)`,
    pgx.NamedArgs{"number": number, "type": string(t), "amount": req.Amount, "reference":
req.Reference}); err != nil {
    return Account{}, err
  }
  return a, tx.Commit(ctx)
}
```

`defer tx.Rollback(ctx)` menjamin transaksi dibatalkan pada setiap `return` yang mengembalikan *error*. Setelah `Commit` berhasil, pemanggilan `Rollback` tidak berpengaruh.

2.4.4 Endpoint debit dan kredit

Account service kini membutuhkan dua *endpoint* baru: `POST /accounts/{number}/debit` dan `POST /accounts/{number}/credit`. Logika keduanya identik kecuali `MutationType`, sehingga satu fungsi pembuat *handler* cukup untuk keduanya.

Handler baru juga membutuhkan kode pembacaan JSON dari Lab 1. Pindahkan kode tersebut ke fungsi `decode` supaya tidak ditulis dua kali. Tambahkan fungsi berikut di `account/internal/account/http.go`, tepat sebelum `writeError`:

```
// decode reads a JSON body into dst and answers 400 itself when the body is invalid.
func decode(w http.ResponseWriter, r *http.Request, dst any) bool {
  dec := json.NewDecoder(http.MaxBytesReader(w, r.Body, 1<<20))
  dec.DisallowUnknownFields()
  if err := dec.Decode(dst); err != nil {
    writeError(w, http.StatusBadRequest, "invalid JSON body: "+err.Error())
    return false
  }
  return true
}
```

Lalu ganti awal *method* `create` sehingga menggunakan `decode`:

```
func (h *Handler) create(w http.ResponseWriter, r *http.Request) {
  var req CreateRequest
  if !decode(w, r, &req) {
    return
  }
  // ... validasi dan penyimpanan tetap seperti Lab 1
```

Tambahkan *handler* mutasi setelah *method* `get`. Pemetaan *error* ke status HTTP menjadi kontrak bagi pemanggil: `404` berarti rekening tidak ada, `422 Unprocessable Entity` berarti saldo tidak mencukupi, dan `500` berarti kegagalan teknis.

```
func (h *Handler) mutate(t MutationType) http.HandlerFunc {
  return func(w http.ResponseWriter, r *http.Request) {
    var req MutationRequest
    if !decode(w, r, &req) {
      return
    }
    if err := req.Validate(); err != nil {
      writeJSON(w, http.StatusBadRequest, map[string]any{"error": err.Error(), "fields":
err})
      return
    }
    a, err := h.store.Mutate(r.Context(), r.PathValue("number"), t, req)
    switch {
    case errors.Is(err, ErrNotFound):
      writeError(w, http.StatusNotFound, err.Error())
    case errors.Is(err, ErrInsufficientFunds):
      writeError(w, http.StatusUnprocessableEntity, err.Error())
    case err != nil:
      h.internalError(w, r, err)
    default:
      h.log.Info("account mutated", "number", a.Number, "type", t, "amount", req.Amount,
"reference", req.Reference)
      writeJSON(w, http.StatusOK, a)
    }
  }
}
```

Daftarkan kedua *route* di *method* `Routes`, setelah `GET /accounts/{number}`:

```
  mux.HandleFunc("POST /accounts/{number}/debit", h.mutate(Debit))
  mux.HandleFunc("POST /accounts/{number}/credit", h.mutate(Credit))
```

2.4.5 Pengujian account service

Test *handler* dari Lab 1 menggunakan `fakeStore`, implementasi `Store` di memori. Karena *interface* `Store` kini memiliki *method* `Mutate`, `fakeStore` tidak lagi memenuhi *interface* tersebut dan test gagal dikompilasi. Tambahkan *method* berikut di `account/internal/account/http_test.go`, sebelum *method* `Ping` milik `fakeStore`:

```
func (f *fakeStore) Mutate(_ context.Context, number string, t MutationType, req
MutationRequest) (Account, error) {
  a, ok := f.accounts[number]
  if !ok {
    return Account{}, ErrNotFound
  }
  if t == Debit {
    if a.Balance.LessThan(req.Amount) {
      return Account{}, ErrInsufficientFunds
    }
    a.Balance = a.Balance.Sub(req.Amount)
  } else {
    a.Balance = a.Balance.Add(req.Amount)
  }
  f.accounts[number] = a
  return a, nil
}
```

Kemudian tambahkan kasus uji berikut ke tabel test `TestAccountAPI`, setelah kasus `"wrong method"`. Kasus-kasus ini berjalan berurutan pada rekening yang sama: saldo 500000 didebit 200000, dikredit 50000, lalu debit sebesar 350000.01 harus ditolak.

```
    {"debit", "POST", "/accounts/1234567890/debit", `{"amount":"200000","reference":"T-1"}
`, 200, `"balance":"300000"`},
    {"credit", "POST", "/accounts/1234567890/credit", `{"amount":"50000","reference":"T-2"}
`, 200, `"balance":"350000"`},
    {"insufficient funds", "POST", "/accounts/1234567890/debit",
`{"amount":"350000.01","reference":"T-3"}`, 422, "insufficient funds"},
    {"debit unknown account", "POST", "/accounts/0000000000/debit",
`{"amount":"1","reference":"T-4"}`, 404, "not found"},
    {"invalid mutation", "POST", "/accounts/1234567890/credit",
`{"amount":"0","reference":""}`, 400, `"amount":"must be greater than zero"`},
```

```
make -C account test
```

Seluruh test harus lulus sebelum melanjutkan ke transfer service.

2.4.6 Database untuk transfer service

Transfer service mendapat *container* PostgreSQL sendiri. Port host `15433` dipilih supaya tidak bertabrakan dengan `db-account` (15432) maupun PostgreSQL lokal di port 5432. Tambahkan blok berikut di `compose.yaml`, di bawah *service* `db-account`:

```
  db-transfer:
    image: postgres:18.6-alpine
    environment:
      POSTGRES_DB: transfer
      POSTGRES_USER: transfer
      POSTGRES_PASSWORD: transfer123
    ports:
      - "15433:5432"
    volumes:
      - db-transfer:/var/lib/postgresql
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "transfer", "-d", "transfer"]
      interval: 2s
      retries: 30
```

Tambahkan juga *volume*-nya pada bagian `volumes` di akhir file:

```
volumes:
  db-account:
  db-transfer:
```

Jalankan ulang infrastruktur. `docker compose up` hanya membuat *container* yang belum ada, sehingga `db-account` beserta datanya tidak terganggu.

```
make infra-up
docker compose ps
```

Output `docker compose ps` harus menampilkan `db-account` dan `db-transfer` dengan status `healthy`.

2.4.7 Modul Go untuk transfer service

Setiap service adalah modul Go tersendiri dengan `go.mod` sendiri. Pemisahan ini membuat setiap service dapat di-*build*, diuji, dan di-*deploy* secara terpisah. File `go.work` menyatukan modul-modul tersebut selama pengembangan di laptop, sehingga perintah `go` dapat dijalankan dari folder mana pun di dalam `studi-kasus/`.

```
mkdir -p transfer/internal/transfer transfer/internal/accountclient
cd transfer
go mod init github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer
go mod edit -go=1.26.0
cd ..
go work use ./transfer
```

Seperti pada Lab 1, `go mod init` menuliskan versi Go yang terpasang di laptop, sehingga `go mod edit -go=1.26.0` menurunkan versi minimal modul `transfer` ke 1.26.0, sama dengan *checkpoint*. Jika baris `go` di `go.work` ikut naik setelah `go work use`, kembalikan dengan `go work edit -go=1.26.0`.

Setelah perintah di atas, `go.work` berisi dua modul:

```
go 1.26.0

use (
  ./account
  ./transfer
)
```

Daftarkan juga transfer service di `Makefile` di folder `studi-kasus/` supaya `make test` menguji kedua service. Ubah baris pertama menjadi:

```
SERVICES := account transfer
```

2.4.8 Domain transfer

Package `transfer` berisi model, validasi, dan dua *interface* yang dibutuhkan logika transfer, yaitu `Store` untuk mengakses database milik transfer service dan `AccountClient` untuk memanggil account service. Logika transfer hanya bergantung pada kedua *interface* ini. Implementasi konkretnya (PostgreSQL dan *HTTP client*) disambungkan di `main.go`, sehingga logika transfer dapat diuji dengan implementasi *fake* (tiruan untuk pengujian) tanpa database dan tanpa jaringan.

Transfer memiliki tiga status. `PENDING` berarti transfer sudah dicatat tetapi belum selesai diproses. `COMPLETED` berarti debit dan kredit berhasil. `FAILED` berarti salah satu langkah gagal, dengan alasan tersimpan di `failure_reason`.

Error bisnis `ErrAccountNotFound` dan `ErrInsufficientFunds` sengaja didefinisikan di *package* `transfer` dan tidak diletakkan di *package HTTP client*. Dengan begitu logika transfer dan *handler* HTTP dapat mengenali penolakan bisnis tanpa mengetahui bahwa account service diakses melalui REST. Pada Lab 4, REST diganti gRPC dan kedua *error* ini tetap sama.

Buat `transfer/internal/transfer/transfer.go`:

```
// Package transfer moves money between two accounts by calling the account service.
package transfer

import (
  "context"
  "errors"
  "regexp"
  "time"

  "github.com/shopspring/decimal"
)

type Status string

const (
  Pending   Status = "PENDING"
  Completed Status = "COMPLETED"
```

```
  Failed    Status = "FAILED"
)

type Transfer struct {
  ID            string          `json:"id"`
  FromAccount   string          `json:"from_account"`
  ToAccount     string          `json:"to_account"`
  Amount        decimal.Decimal `json:"amount"`
  Status        Status          `json:"status"`
  FailureReason string          `json:"failure_reason,omitempty"`
  CreatedAt     time.Time       `json:"created_at"`
  UpdatedAt     time.Time       `json:"updated_at"`
}

type CreateRequest struct {
  FromAccount string          `json:"from_account"`
  ToAccount   string          `json:"to_account"`
  Amount      decimal.Decimal `json:"amount"`
}

var (
  ErrNotFound = errors.New("transfer not found")
  // Business rejections from the account service: the request itself cannot succeed.
  ErrAccountNotFound   = errors.New("account not found")
  ErrInsufficientFunds = errors.New("insufficient funds")
)

type ValidationError map[string]string

func (v ValidationError) Error() string { return "validation failed" }

var numberPattern = regexp.MustCompile(`^[0-9]{10}$`)

func (r CreateRequest) Validate() error {
  v := ValidationError{}
  if !numberPattern.MatchString(r.FromAccount) {
    v["from_account"] = "must be exactly 10 digits"
  }
  if !numberPattern.MatchString(r.ToAccount) {
    v["to_account"] = "must be exactly 10 digits"
  }
  if r.FromAccount == r.ToAccount {
    v["to_account"] = "must differ from from_account"
  }
  if !r.Amount.IsPositive() {
    v["amount"] = "must be greater than zero"
  }
  if len(v) > 0 {
    return v
  }
  return nil
}

type Store interface {
  // Create records a new transfer in status PENDING.
  Create(ctx context.Context, req CreateRequest) (Transfer, error)
  Finish(ctx context.Context, id string, status Status, reason string) (Transfer, error)
  Get(ctx context.Context, id string) (Transfer, error)
  Ping(ctx context.Context) error
```

```
}

// AccountClient is what the transfer needs from the account service. The reference
// passed to the account service is the transfer ID.
type AccountClient interface {
  Debit(ctx context.Context, number string, amount decimal.Decimal, reference string) error
  Credit(ctx context.Context, number string, amount decimal.Decimal, reference string)
error
}
```

2.4.9 Penyimpanan transfer

Tabel `transfer` menggunakan UUID sebagai *primary key*. ID ini juga dikirim ke account service sebagai `reference` mutasi, sehingga setiap mutasi di database `account` dapat ditelusuri ke transfer pemicunya. Buat `transfer/internal/transfer/schema.sql`:

```
create table if not exists transfer (
    id             uuid           primary key default gen_random_uuid(),
    from_account   varchar(10)    not null,
    to_account     varchar(10)    not null,
    amount         numeric(19, 2) not null check (amount > 0),
    status         varchar(10)    not null check (status in ('PENDING', 'COMPLETED',
'FAILED')),
    failure_reason text,
    created_at     timestamptz    not null default now(),
    updated_at     timestamptz    not null default now()
);
```

Perhatikan bahwa kolom `from_account` dan `to_account` tidak memiliki *foreign key*. Tabel `account` berada di database lain, sehingga keberadaan rekening hanya dapat diperiksa melalui account service.

Implementasi `Store` mengikuti pola account service dari Lab 1: *connection pool* pgx, registrasi tipe `decimal`, dan *schema* yang di-*embed* ke dalam *binary*. Semua *query* mengembalikan kolom yang sama dalam urutan yang sama, sehingga cukup satu konstanta `columns` dan satu fungsi *helper* `one`. Ada satu kasus khusus: `GET /transfers/abc` mengirim teks yang bukan UUID, dan PostgreSQL menolaknya dengan kode `22P02`. Bagi pemanggil API, kasus ini sama saja dengan transfer yang tidak ditemukan.

Buat `transfer/internal/transfer/store.go`:

```
package transfer

import (
  "context"
  _ "embed"
  "errors"
  "fmt"

  pgxdecimal "github.com/jackc/pgx-shopspring-decimal"
  "github.com/jackc/pgx/v5"
  "github.com/jackc/pgx/v5/pgconn"
  "github.com/jackc/pgx/v5/pgxpool"
)

//go:embed schema.sql
var schema string

// columns is shared by every query so RowToStructByPos maps them onto Transfer in order.
const columns = `id::text, from_account, to_account, amount, status,
coalesce(failure_reason, ''), created_at, updated_at`

type PostgresStore struct {
```

```
  pool *pgxpool.Pool
}

// NewPostgresStore connects, verifies the connection, and applies the schema.
func NewPostgresStore(ctx context.Context, databaseURL string) (*PostgresStore, error) {
  cfg, err := pgxpool.ParseConfig(databaseURL)
  if err != nil {
    return nil, fmt.Errorf("parse DATABASE_URL: %w", err)
  }
  cfg.AfterConnect = func(_ context.Context, conn *pgx.Conn) error {
    pgxdecimal.Register(conn.TypeMap())
    return nil
  }
  pool, err := pgxpool.NewWithConfig(ctx, cfg)
  if err != nil {
    return nil, fmt.Errorf("create pool: %w", err)
  }
  if err := pool.Ping(ctx); err != nil {
    pool.Close()
    return nil, fmt.Errorf("connect to database: %w", err)
  }
  if _, err := pool.Exec(ctx, schema); err != nil {
    pool.Close()
    return nil, fmt.Errorf("apply schema: %w", err)
  }
  return &PostgresStore{pool: pool}, nil
}

func (s *PostgresStore) Close() { s.pool.Close() }

func (s *PostgresStore) Ping(ctx context.Context) error { return s.pool.Ping(ctx) }

func (s *PostgresStore) Create(ctx context.Context, req CreateRequest) (Transfer, error) {
  return s.one(ctx, `
    insert into transfer (from_account, to_account, amount, status)
    values (@from, @to, @amount, 'PENDING')
    returning `+columns,
    pgx.NamedArgs{"from": req.FromAccount, "to": req.ToAccount, "amount": req.Amount})
}

func (s *PostgresStore) Finish(ctx context.Context, id string, status Status, reason
string) (Transfer, error) {
  return s.one(ctx, `
    update transfer set status = @status, failure_reason = nullif(@reason, ''), updated_at
= now()
    where id = @id
    returning `+columns,
    pgx.NamedArgs{"id": id, "status": string(status), "reason": reason})
}

func (s *PostgresStore) Get(ctx context.Context, id string) (Transfer, error) {
  return s.one(ctx, `select `+columns+` from transfer where id = @id`, pgx.NamedArgs{"id":
id})
}

func (s *PostgresStore) one(ctx context.Context, sql string, args pgx.NamedArgs) (Transfer,
error) {
  rows, err := s.pool.Query(ctx, sql, args)
  if err != nil {
```

```
    return Transfer{}, err
  }
  t, err := pgx.CollectExactlyOneRow(rows, pgx.RowToStructByPos[Transfer])
  var pgErr *pgconn.PgError
  switch {
  case errors.Is(err, pgx.ErrNoRows):
    return Transfer{}, ErrNotFound
  case errors.As(err, &pgErr) && pgErr.Code == "22P02": // invalid_text_representation: not
a UUID
    return Transfer{}, ErrNotFound
  }
  return t, err
}
```

2.4.10 Logika transfer

`Service.Execute` menjalankan empat langkah secara berurutan: catat transfer dengan status `PENDING`, debit rekening sumber, kredit rekening tujuan, lalu ubah status menjadi `COMPLETED`. Jika debit atau kredit gagal, status menjadi `FAILED` dan alasannya disimpan.

Transfer service mencatat transfer lebih dahulu sebelum memanggil account service. Urutan ini memastikan setiap mutasi di account service selalu memiliki pasangan transfer yang dapat ditelusuri, termasuk ketika proses berhenti di tengah jalan.

Komentar *known gap* pada kode di bawah menjelaskan kelemahan utama lab ini. Debit dan kredit dijalankan sebagai dua panggilan terpisah ke service lain, tanpa transaksi database bersama. Ketika kredit gagal setelah debit berhasil, transfer ditandai `FAILED`, tetapi saldo rekening sumber tetap berkurang. Pada lab ini, kondisi tersebut hanya ditangani dengan *log* level `ERROR` sebagai penanda bahwa rekening sumber perlu dikoreksi manual.

Buat `transfer/internal/transfer/service.go`:

```
package transfer

import (
  "context"
  "errors"
  "fmt"
  "log/slog"
)

type Service struct {
  store    Store
  accounts AccountClient
  log      *slog.Logger
}

func NewService(store Store, accounts AccountClient, log *slog.Logger) *Service {
  return &Service{store: store, accounts: accounts, log: log}
}

// Execute records the transfer, debits the source, then credits the destination.
// The returned error is the reason the transfer FAILED; the transfer itself is
// returned whenever it was recorded.
//
// Known gap: debit and credit are two calls to another service, with no shared
// database transaction. When the credit fails after a successful debit, the money
// has left the source account and arrived nowhere. The transfer is marked FAILED
// and logged for manual reversal.
func (s *Service) Execute(ctx context.Context, req CreateRequest) (Transfer, error) {
  t, err := s.store.Create(ctx, req)
```

```
  if err != nil {
    return Transfer{}, fmt.Errorf("record transfer: %w", err)
  }

  if err := s.accounts.Debit(ctx, t.FromAccount, t.Amount, t.ID); err != nil {
    return s.fail(ctx, t, "debit failed", err)
  }
  if err := s.accounts.Credit(ctx, t.ToAccount, t.Amount, t.ID); err != nil {
    s.log.Error("credit failed after debit, source account needs manual reversal",
      "transfer_id", t.ID, "from_account", t.FromAccount, "amount", t.Amount, "error", err)
    return s.fail(ctx, t, "credit failed after debit", err)
  }

  done, err := s.store.Finish(ctx, t.ID, Completed, "")
  if err != nil {
    return t, fmt.Errorf("mark transfer completed: %w", err)
  }
  s.log.Info("transfer completed", "transfer_id", t.ID, "amount", t.Amount)
  return done, nil
}

func (s *Service) fail(ctx context.Context, t Transfer, step string, cause error)
(Transfer, error) {
  reason := fmt.Sprintf("%s: %v", step, cause)
  failed, err := s.store.Finish(ctx, t.ID, Failed, reason)
  if err != nil {
    return t, errors.Join(cause, fmt.Errorf("mark transfer failed: %w", err))
  }
  s.log.Warn("transfer failed", "transfer_id", t.ID, "reason", reason)
  return failed, cause
}

func (s *Service) Get(ctx context.Context, id string) (Transfer, error) {
  return s.store.Get(ctx, id)
}

func (s *Service) Ping(ctx context.Context) error { return s.store.Ping(ctx) }
```

Fungsi `fail` mengembalikan `cause` apa adanya, yaitu *error* dari `AccountClient`. Pemanggil `Execute` dapat memeriksa *error* ini dengan `errors.Is(err, ErrInsufficientFunds)` untuk menentukan status HTTP.

2.4.11 REST client ke account service

Package `accountclient` mengimplementasikan *interface* `AccountClient` dengan memanggil *endpoint* debit dan kredit milik account service. Tugas utamanya adalah menerjemahkan respons HTTP menjadi *error* yang dipahami *package* `transfer`:

Status dari Account Service	Error yang Dikembalikan
`200 OK`	Tidak ada *error*
`404 Not Found`	`transfer.ErrAccountNotFound`, dibungkus dengan `%w`
`422 Unprocessable Entity`	`transfer.ErrInsufficientFunds`, dibungkus dengan `%w`
Status lain	*Error* biasa berisi status dan isi respons
Gagal terhubung	*Error* dari `http.Client.Do`, misalnya *connection refused*

Wrapping dengan `%w` menambahkan konteks (operasi dan nomor rekening) ke pesan *error*, sementara `errors.Is` tetap dapat mengenali *error* aslinya.

Perhatikan konstruktor `New`. `http.Client{}` tanpa nilai `Timeout` akan menunggu respons tanpa batas waktu. Kelemahan ini sengaja dibiarkan dan dicatat di komentar, karena Lab 3 membahasnya secara khusus.

Buat `transfer/internal/accountclient/client.go`:

```
// Package accountclient calls the account service REST API.
package accountclient

import (
  "bytes"
  "context"
  "encoding/json"
  "fmt"
  "io"
  "net/http"

  "github.com/shopspring/decimal"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/
transfer"
)

type Client struct {
  baseURL string
  http    *http.Client
}

// New uses an http.Client without a timeout: a slow account service makes every
// transfer wait as long as it takes.
func New(baseURL string) *Client {
  return &Client{baseURL: baseURL, http: &http.Client{}}
}

func (c *Client) Debit(ctx context.Context, number string, amount decimal.Decimal,
reference string) error {
  return c.mutate(ctx, number, "debit", amount, reference)
}

func (c *Client) Credit(ctx context.Context, number string, amount decimal.Decimal,
reference string) error {
  return c.mutate(ctx, number, "credit", amount, reference)
}

func (c *Client) mutate(ctx context.Context, number, kind string, amount decimal.Decimal,
reference string) error {
  body, err := json.Marshal(map[string]any{"amount": amount, "reference": reference})
  if err != nil {
    return err
  }
  url := fmt.Sprintf("%s/accounts/%s/%s", c.baseURL, number, kind)
  req, err := http.NewRequestWithContext(ctx, http.MethodPost, url, bytes.NewReader(body))
  if err != nil {
    return err
  }
  req.Header.Set("Content-Type", "application/json")

  resp, err := c.http.Do(req)
  if err != nil {
```

```
    return fmt.Errorf("%s %s: %w", kind, number, err)
  }
  defer resp.Body.Close()
  msg, _ := io.ReadAll(io.LimitReader(resp.Body, 4096))

  switch resp.StatusCode {
  case http.StatusOK:
    return nil
  case http.StatusNotFound:
    return fmt.Errorf("%s %s: %w", kind, number, transfer.ErrAccountNotFound)
  case http.StatusUnprocessableEntity:
    return fmt.Errorf("%s %s: %w", kind, number, transfer.ErrInsufficientFunds)
  default:
    return fmt.Errorf("%s %s: account service answered %s: %s", kind, number, resp.Status,
bytes.TrimSpace(msg))
  }
}
```

`io.LimitReader` membatasi isi respons yang dibaca hingga 4096 byte. Isi respons hanya dibutuhkan untuk pesan *error*, dan pembatasan ini mencegah respons yang sangat besar menghabiskan memori.

2.4.12 HTTP API transfer service

Handler transfer mengikuti pola account service: `decode`, validasi, panggil logika, lalu petakan hasilnya ke status HTTP. Pemetaan untuk `POST /transfers`:

Status HTTP	Kondisi
`201 Created`	Debit dan kredit berhasil, transfer `COMPLETED`
`400 Bad Request`	JSON tidak valid atau validasi gagal, sehingga transfer tidak dicatat
`422 Unprocessable Entity`	Penolakan bisnis (rekening tidak ada, saldo kurang) sehingga transfer berstatus `FAILED`
`502 Bad Gateway`	Account service gagal atau tidak dapat dihubungi, sehingga transfer berstatus `FAILED`
`500 Internal Server Error`	Transfer bahkan tidak sempat dicatat, misalnya database transfer bermasalah

Pada respons `422` dan `502`, isi respons tetap berupa data transfer lengkap dengan `id`, `status`, dan `failure_reason`, sehingga klien dapat menelusuri transfer tersebut melalui `GET /transfers/{id}`.

Dua keputusan desain pada *handler* ini perlu diperhatikan.

`context.WithoutCancel`. Secara *default*, *context* dari `r.Context()` dibatalkan ketika klien memutus koneksi. Jika *context* tersebut diteruskan ke `Execute`, klien yang memutus koneksi di tengah proses dapat menghentikan transfer tepat di antara debit dan kredit. `context.WithoutCancel` mempertahankan nilai-nilai di dalam *context* tetapi melepas sinyal pembatalannya, sehingga transfer yang sudah dimulai tetap berjalan hingga selesai.

`readyz` hanya memeriksa database sendiri. Account service memiliki *probe* sendiri. Jika `readyz` milik transfer service ikut gagal ketika account service mati, transfer service akan dikeluarkan dari *load balancer* padahal sumber gangguannya ada di account service, dan `GET /transfers/{id}` yang tidak membutuhkan account service ikut tidak dapat diakses.

Buat `transfer/internal/transfer/http.go`:

```
package transfer

import (
```

```
  "context"
  "encoding/json"
  "errors"
  "log/slog"
  "net/http"
  "time"
)

type Handler struct {
  svc *Service
  log *slog.Logger
}

func NewHandler(svc *Service, log *slog.Logger) *Handler {
  return &Handler{svc: svc, log: log}
}

func (h *Handler) Routes() *http.ServeMux {
  mux := http.NewServeMux()
  mux.HandleFunc("POST /transfers", h.create)
  mux.HandleFunc("GET /transfers/{id}", h.get)
  mux.HandleFunc("GET /healthz", h.healthz)
  mux.HandleFunc("GET /readyz", h.readyz)
  return mux
}

func (h *Handler) create(w http.ResponseWriter, r *http.Request) {
  var req CreateRequest
  if !decode(w, r, &req) {
    return
  }
  if err := req.Validate(); err != nil {
    writeJSON(w, http.StatusBadRequest, map[string]any{"error": err.Error(), "fields":
err})
    return
  }
  // WithoutCancel: a client that disconnects must not abort a transfer halfway,
  // e.g. between the debit and the credit.
  t, err := h.svc.Execute(context.WithoutCancel(r.Context()), req)
  switch {
  case err == nil:
    w.Header().Set("Location", "/transfers/"+t.ID)
    writeJSON(w, http.StatusCreated, t)
  case t.ID == "":
    h.internalError(w, r, err)
  case errors.Is(err, ErrAccountNotFound), errors.Is(err, ErrInsufficientFunds):
    writeJSON(w, http.StatusUnprocessableEntity, t)
  default:
    // Recorded but not completed because a dependency failed.
    writeJSON(w, http.StatusBadGateway, t)
  }
}

func (h *Handler) get(w http.ResponseWriter, r *http.Request) {
  t, err := h.svc.Get(r.Context(), r.PathValue("id"))
  switch {
  case errors.Is(err, ErrNotFound):
    writeError(w, http.StatusNotFound, err.Error())
  case err != nil:
```

```
    h.internalError(w, r, err)
  default:
    writeJSON(w, http.StatusOK, t)
  }
}

func (h *Handler) healthz(w http.ResponseWriter, _ *http.Request) {
  writeJSON(w, http.StatusOK, map[string]string{"status": "UP"})
}

// readyz checks only this service's own database. The account service is a
// dependency with its own probes; failing here when it is down would take the
// transfer service out of rotation for an outage it cannot fix.
func (h *Handler) readyz(w http.ResponseWriter, r *http.Request) {
  ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
  defer cancel()
  if err := h.svc.Ping(ctx); err != nil {
    h.log.Warn("readiness check failed", "error", err)
    writeJSON(w, http.StatusServiceUnavailable, map[string]string{"status": "DOWN",
"database": "DOWN"})
    return
  }
  writeJSON(w, http.StatusOK, map[string]string{"status": "UP", "database": "UP"})
}

func (h *Handler) internalError(w http.ResponseWriter, r *http.Request, err error) {
  h.log.Error("request failed", "method", r.Method, "path", r.URL.Path, "error", err)
  writeError(w, http.StatusInternalServerError, "internal error")
}

// decode reads a JSON body into dst and answers 400 itself when the body is invalid.
func decode(w http.ResponseWriter, r *http.Request, dst any) bool {
  dec := json.NewDecoder(http.MaxBytesReader(w, r.Body, 1<<20))
  dec.DisallowUnknownFields()
  if err := dec.Decode(dst); err != nil {
    writeError(w, http.StatusBadRequest, "invalid JSON body: "+err.Error())
    return false
  }
  return true
}

func writeError(w http.ResponseWriter, status int, msg string) {
  writeJSON(w, status, map[string]string{"error": msg})
}

func writeJSON(w http.ResponseWriter, status int, body any) {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(status)
  _ = json.NewEncoder(w).Encode(body)
}
```

Fungsi `decode`, `writeError`, dan `writeJSON` memang sama persis dengan milik account service. Duplikasi kecil seperti ini disengaja. Jika kode tersebut dipindahkan ke modul bersama, kedua service akan saling terikat pada setiap perubahan modul itu.

2.4.13 Konfigurasi dan `main.go`

Transfer service membutuhkan tiga variabel *environment*: alamat HTTP, URL database, dan URL account service. Ketiganya wajib diisi. Jika salah satu kosong, service berhenti pada *startup* dengan

pesan yang menyebutkan nama variabelnya. Service tidak menebak nilainya sendiri, karena nilai tebakan dapat mengarah ke database atau service yang salah.

Buat `transfer/config.go`:

```
package main

import (
  "fmt"
  "os"
  "strings"
)

// Config comes from environment variables only (12-factor). Every variable is
// required: a missing value stops the service at startup instead of running
// with a guessed default.
type Config struct {
  HTTPAddr          string
  DatabaseURL       string
  AccountServiceURL string
}

func LoadConfig() (Config, error) {
  var missing []string
  get := func(name string) string {
    v := os.Getenv(name)
    if v == "" {
      missing = append(missing, name)
    }
    return v
  }
  cfg := Config{
    HTTPAddr:          get("HTTP_ADDR"),
    DatabaseURL:       get("DATABASE_URL"),
    AccountServiceURL: get("ACCOUNT_SERVICE_URL"),
  }
  if len(missing) > 0 {
    return Config{}, fmt.Errorf("missing required environment variables: %s",
strings.Join(missing, ", "))
  }
  return cfg, nil
}
```

`main.go` menyambungkan semua bagian: konfigurasi, `PostgresStore`, `accountclient`, `Service`, lalu `Handler`. Strukturnya sama dengan account service pada Lab 1, termasuk *graceful shutdown* ketika menerima `SIGINT` atau `SIGTERM`. Buat `transfer/main.go`:

```
package main

import (
  "context"
  "errors"
  "log/slog"
  "net/http"
  "os"
  "os/signal"
  "syscall"
  "time"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/
```

```
accountclient"
  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/
transfer"
)

func main() {
  log := slog.New(slog.NewTextHandler(os.Stdout, nil))
  if err := run(log); err != nil {
    log.Error("transfer service stopped", "error", err)
    os.Exit(1)
  }
}

func run(log *slog.Logger) error {
  cfg, err := LoadConfig()
  if err != nil {
    return err
  }

  ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
  defer stop()

  store, err := transfer.NewPostgresStore(ctx, cfg.DatabaseURL)
  if err != nil {
    return err
  }
  defer store.Close()

  svc := transfer.NewService(store, accountclient.New(cfg.AccountServiceURL), log)
  srv := &http.Server{
    Addr:              cfg.HTTPAddr,
    Handler:           transfer.NewHandler(svc, log).Routes(),
    ReadHeaderTimeout: 5 * time.Second,
  }

  errCh := make(chan error, 1)
  go func() {
    log.Info("transfer service listening", "addr", cfg.HTTPAddr, "account_service",
cfg.AccountServiceURL)
    errCh <- srv.ListenAndServe()
  }()

  select {
  case err := <-errCh:
    return err
  case <-ctx.Done():
    log.Info("shutting down")
    shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil && !errors.Is(err,
http.ErrServerClosed) {
      return err
    }
    return nil
  }
}
```

2.4.14 Makefile dan *dependency*

Setiap langkah *pipeline* CI pada Lab 10 memanggil *target* `make`, sehingga perintah di laptop dan di *runner* CI selalu sama. Buat `transfer/Makefile`. Ingat bahwa baris perintah di dalam *target* harus diawali karakter *tab*, bukan spasi.

```
# Every pipeline step calls a target here, so CI and laptops run the same commands.
SERVICE := transfer

.PHONY: run test build clean

run:
  HTTP_ADDR=:28082 \
  DATABASE_URL=postgres://transfer:transfer123@localhost:15433/transfer \
  ACCOUNT_SERVICE_URL=http://localhost:28081 \
  go run .

test:
  go vet ./...
  go test ./...

build:
  CGO_ENABLED=0 go build -o bin/$(SERVICE) .

clean:
  rm -rf bin
```

Setelah semua kode ditulis, unduh *dependency* dengan versi yang sama dengan account service, lalu rapikan `go.mod`:

```
cd transfer
go get github.com/jackc/pgx/v5@v5.11.0 \
  github.com/shopspring/decimal@v1.6.0 \
  github.com/jackc/pgx-shopspring-decimal@v0.0.0-20220624020537-1d36b5a1853e
go mod tidy
cd ..
```

2.4.15 Pengujian transfer service

Test logika transfer tidak membutuhkan database maupun account service. Keduanya diganti implementasi *fake*: `fakeStore` menyimpan transfer di *map*, dan `fakeAccounts` mengembalikan *error* yang sudah ditentukan sambil mencatat setiap panggilan. Tiga skenario diuji:

- transfer berhasil: status `COMPLETED`, dua panggilan (debit lalu kredit);
- saldo tidak cukup: status `FAILED`, hanya satu panggilan, karena kredit tidak boleh dijalankan setelah debit ditolak;
- kredit gagal setelah debit: status `FAILED`, dua panggilan. Skenario ini mendokumentasikan *known gap*: tidak ada panggilan ketiga untuk mengembalikan debit.

Buat `transfer/internal/transfer/service_test.go`:

```
package transfer

import (
  "context"
  "errors"
  "fmt"
  "io"
  "log/slog"
  "testing"
```

```
  "github.com/shopspring/decimal"
)

type fakeStore struct {
  transfers map[string]Transfer
  seq       int
}

func (f *fakeStore) Create(_ context.Context, req CreateRequest) (Transfer, error) {
  f.seq++
  t := Transfer{ID: fmt.Sprint(f.seq), FromAccount: req.FromAccount, ToAccount:
req.ToAccount, Amount: req.Amount, Status: Pending}
  f.transfers[t.ID] = t
  return t, nil
}

func (f *fakeStore) Finish(_ context.Context, id string, status Status, reason string)
(Transfer, error) {
  t := f.transfers[id]
  t.Status, t.FailureReason = status, reason
  f.transfers[id] = t
  return t, nil
}

func (f *fakeStore) Get(_ context.Context, id string) (Transfer, error) {
  t, ok := f.transfers[id]
  if !ok {
    return Transfer{}, ErrNotFound
  }
  return t, nil
}

func (f *fakeStore) Ping(context.Context) error { return nil }

// fakeAccounts returns the configured error for each operation and records the calls.
type fakeAccounts struct {
  debitErr, creditErr error
  calls               []string
}

func (f *fakeAccounts) Debit(_ context.Context, number string, amount decimal.Decimal, ref
string) error {
  f.calls = append(f.calls, "debit "+number)
  return f.debitErr
}

func (f *fakeAccounts) Credit(_ context.Context, number string, amount decimal.Decimal, ref
string) error {
  f.calls = append(f.calls, "credit "+number)
  return f.creditErr
}

func TestExecute(t *testing.T) {
  req := CreateRequest{FromAccount: "1111111111", ToAccount: "2222222222", Amount:
decimal.RequireFromString("100000")}
  unavailable := errors.New("connection refused")

  tests := []struct {
    name       string
```

```
    accounts   *fakeAccounts
    wantStatus Status
    wantErr    error
    wantCalls  int
  }{
    {"success", &fakeAccounts{}, Completed, nil, 2},
    {"insufficient funds stops before credit", &fakeAccounts{debitErr:
ErrInsufficientFunds}, Failed, ErrInsufficientFunds, 1},
    {"credit fails after debit", &fakeAccounts{creditErr: unavailable}, Failed,
unavailable, 2},
  }
  for _, tc := range tests {
    t.Run(tc.name, func(t *testing.T) {
      store := &fakeStore{transfers: map[string]Transfer{}}
      svc := NewService(store, tc.accounts, slog.New(slog.NewTextHandler(io.Discard, nil)))

      got, err := svc.Execute(context.Background(), req)
      if !errors.Is(err, tc.wantErr) {
        t.Fatalf("err = %v, want %v", err, tc.wantErr)
      }
      if got.Status != tc.wantStatus {
        t.Fatalf("status = %s, want %s", got.Status, tc.wantStatus)
      }
      if stored := store.transfers[got.ID]; stored.Status != tc.wantStatus {
        t.Fatalf("stored status = %s, want %s", stored.Status, tc.wantStatus)
      }
      if len(tc.accounts.calls) != tc.wantCalls {
        t.Fatalf("account calls = %v, want %d calls", tc.accounts.calls, tc.wantCalls)
      }
    })
  }
}

func TestValidate(t *testing.T) {
  err := CreateRequest{FromAccount: "1111111111", ToAccount: "1111111111", Amount:
decimal.Zero}.Validate()
  v, ok := err.(ValidationError)
  if !ok || v["to_account"] == "" || v["amount"] == "" {
    t.Fatalf("got %v, want errors on to_account and amount", err)
  }
}
```

Package `accountclient` diuji menggunakan server *fake* dari `net/http/httptest`. `httptest.NewServer` menjalankan server HTTP sungguhan di port acak pada `localhost`, sehingga pemetaan status HTTP ke *error* dapat diperiksa tanpa menjalankan account service. Buat `transfer/internal/accountclient/client_test.go`:

```
package accountclient

import (
  "context"
  "errors"
  "net/http"
  "net/http/httptest"
  "testing"

  "github.com/shopspring/decimal"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/
```

```
transfer"
)

func TestStatusMapping(t *testing.T) {
  tests := []struct {
    status  int
    wantErr error
  }{
    {http.StatusOK, nil},
    {http.StatusNotFound, transfer.ErrAccountNotFound},
    {http.StatusUnprocessableEntity, transfer.ErrInsufficientFunds},
  }
  for _, tc := range tests {
    srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request)
{
      if r.Method != http.MethodPost || r.URL.Path != "/accounts/1111111111/debit" {
        t.Errorf("unexpected request %s %s", r.Method, r.URL.Path)
      }
      w.WriteHeader(tc.status)
    }))
    err := New(srv.URL).Debit(context.Background(), "1111111111", decimal.NewFromInt(10),
"ref")
    srv.Close()
    if !errors.Is(err, tc.wantErr) {
      t.Errorf("status %d: err = %v, want %v", tc.status, err, tc.wantErr)
    }
  }
}

func TestServerError(t *testing.T) {
  srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, _ *http.Request) {
    http.Error(w, "boom", http.StatusInternalServerError)
  }))
  defer srv.Close()
  err := New(srv.URL).Credit(context.Background(), "1111111111", decimal.NewFromInt(10),
"ref")
  if err == nil || errors.Is(err, transfer.ErrAccountNotFound) || errors.Is(err,
transfer.ErrInsufficientFunds) {
    t.Fatalf("err = %v, want a non-business error", err)
  }
}
```

2.5 Verifikasi

2.5.1 Test otomatis

Jalankan test kedua service dari folder `studi-kasus/`:

```
make test
```

Output `make test` berisi baris `ok` untuk *package* `account/internal/account`, `transfer/internal/accountclient`, dan `transfer/internal/transfer`, tanpa satu pun `FAIL`.

2.5.2 Konfigurasi wajib

Jalankan transfer service tanpa variabel *environment*. Service harus langsung berhenti dan menyebutkan variabel yang belum diisi:

```
cd transfer && go run . ; cd ..
```

Service langsung berhenti dengan *exit code* 1 dan menulis *log* seperti berikut:

```
level=ERROR msg="transfer service stopped" error="missing required environment variables:
HTTP_ADDR, DATABASE_URL, ACCOUNT_SERVICE_URL"
```

2.5.3 Menjalankan kedua service

Buka dua terminal, keduanya di folder `studi-kasus/`. Account service berjalan di port 28081 dan transfer service di port 28082.

```
# terminal 1
make -C account run

# terminal 2
make -C transfer run
```

Terminal 2 menampilkan baris `msg="transfer service listening" addr=:28082 account_service=http://localhost:28081`. Buka terminal ketiga untuk perintah-perintah berikutnya, lalu buat dua rekening uji. Jika nomor rekening ini sudah dibuat pada Lab 1, account service menjawab `409` dan rekening yang lama dapat langsung digunakan. Dalam kasus tersebut, sesuaikan angka saldo pada langkah berikutnya.

```
curl -s -X POST localhost:28081/accounts -H 'Content-Type: application/json' \
  -d '{"number":"1111111111","owner_name":"Budi","initial_balance":"500000"}'
curl -s -X POST localhost:28081/accounts -H 'Content-Type: application/json' \
  -d '{"number":"2222222222","owner_name":"Sari","initial_balance":"100000"}'
```

2.5.4 Transfer berhasil

```
curl -s -i -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

Respons berstatus `201 Created` dengan *header* `Location: /transfers/<id>` dan isi seperti berikut (ID dan waktu akan berbeda):

```
{"id":"3f0c...","from_account":"1111111111","to_account":"2222222222","amount":"100000","status":"COMPLE
```

Terminal account service menampilkan dua baris `msg="account mutated"`, satu dengan `type=DEBIT` untuk `1111111111` dan satu dengan `type=CREDIT` untuk `2222222222`. Keduanya membawa `reference` yang sama dengan `id` transfer. Periksa saldo kedua rekening:

```
curl -s localhost:28081/accounts/1111111111
curl -s localhost:28081/accounts/2222222222
```

Saldo `1111111111` menjadi `"400000"` dan saldo `2222222222` menjadi `"200000"`. Catatan mutasi dapat dilihat langsung di database account:

```
docker compose exec db-account psql -U account -d account \
  -c "select account_number, type, amount, reference from mutation order by id"
```

2.5.5 Saldo tidak mencukupi

```
curl -s -i -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"2222222222","to_account":"1111111111","amount":"999999"}'
```

Respons berstatus `422 Unprocessable Entity`, dengan `"status":"FAILED"` dan `"failure_reason":"debit failed: debit 2222222222: insufficient funds"`. Saldo kedua rekening tidak berubah, dan tidak ada baris baru di tabel `mutation`.

2.5.6 Validasi dan transfer yang tidak ada

```
curl -s -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"1111111111","amount":"0"}'
curl -s -i localhost:28082/transfers/bukan-uuid
```

Perintah pertama menghasilkan `400` dengan `"fields":{"amount":"must be greater than zero","to_account":"must differ from from_account"}`. Perintah kedua menghasilkan `404` dengan `{"error":"transfer not found"}`.

2.5.7 Kredit gagal setelah debit

Skenario ini memperlihatkan *known gap* secara langsung. Rekening tujuan `9999999999` tidak ada, sehingga debit berhasil tetapi kredit ditolak.

```
curl -s -i -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"9999999999","amount":"50000"}'
curl -s localhost:28081/accounts/1111111111
```

Respons transfer berstatus `422` dengan `"status":"FAILED"` dan `"failure_reason":"credit failed after debit: credit 9999999999: account not found"`. Terminal transfer service menampilkan *log* `level=ERROR msg="credit failed after debit, source account needs manual reversal"`. Namun saldo `1111111111` turun dari `"400000"` menjadi `"350000"`: uang Rp50.000 sudah keluar dan tidak masuk ke mana pun. Tabel `mutation` mencatat satu `DEBIT` dengan `reference` transfer tersebut, tanpa pasangan `CREDIT`.

> Validasi nomor rekening tujuan sebelum debit memang dapat mencegah kasus ini secara khusus. Namun kredit tetap dapat gagal karena sebab lain, misalnya account service mati tepat setelah debit. Akar masalahnya, kedua panggilan tidak berada dalam satu transaksi database bersama. Lab 5 menyelesaikan masalah ini dengan kompensasi pada pola *saga*.

2.5.8 Account service mati

Hentikan account service di terminal 1 dengan `Ctrl+C`, lalu kirim transfer:

```
curl -s -i -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"10000"}'
curl -s localhost:28082/readyz
```

Transfer menghasilkan `502 Bad Gateway` dengan `"status":"FAILED"` dan `failure_reason` yang berisi `debit failed: debit 1111111111: Post "http://localhost:28081/accounts/1111111111/debit": dial tcp ...: connect: connection refused`. Sementara itu `readyz` tetap menjawab `{"database":"UP","status":"UP"}`, karena *readiness* transfer service hanya bergantung pada database miliknya sendiri. Jalankan kembali account service sebelum lanjut ke lab berikutnya.

2.6 Masalah Umum

`current directory is contained in a module that is not one of the workspace modules listed in go.work` atau `directory prefix . does not contain modules listed in go.work`. Modul `transfer` belum terdaftar di `go.work`. Pesan pertama muncul dari `go run .`, pesan kedua dari `go test ./...` di folder `transfer/`. Jalankan `go work use ./transfer` dari folder `studi-kasus/`, lalu periksa isi `go.work`.

`missing go.sum entry` atau `no required module provides package`. *Dependency* belum diunduh. Jalankan `go get` dan `go mod tidy` di folder `transfer/` seperti pada langkah Makefile dan *dependency*.

Test account gagal dikompilasi dengan pesan `*fakeStore does not implement Store (missing method Mutate)`. *Method* `Mutate` sudah ditambahkan ke *interface* `Store` tetapi belum ke `fakeStore` di `http_test.go`.

`make: *** missing separator. Stop.` Baris perintah di `transfer/Makefile` diawali spasi. Ganti dengan karakter *tab*. Sebagian editor mengubah *tab* menjadi spasi secara otomatis.

Transfer service gagal *startup* dengan `connect to database: ... connection refused` pada port 15433. *Container* `db-transfer` belum berjalan. Periksa `compose.yaml`, lalu jalankan `make infra-up` dan `docker compose ps`.

`relation "mutation" does not exist` di *log* account service. Account service yang sedang berjalan masih versi lama. Hentikan dengan `Ctrl+C` dan jalankan ulang `make -C account run` supaya *schema* baru diterapkan.

Semua transfer gagal dengan `422` dan `failure_reason debit failed: debit ...: account not found`, termasuk untuk rekening yang ada. Nilai `ACCOUNT_SERVICE_URL` diakhiri `/accounts`, sehingga *request* dikirim ke `/accounts/accounts/{number}/debit` dan account service menjawab `404`. *Package* `accountclient` menerjemahkan setiap `404` menjadi `ErrAccountNotFound`, sehingga kesalahan konfigurasi ini terlihat seperti penolakan bisnis. Hapus `/accounts` dari nilai tersebut.

`bind: address already in use` pada port 28081 atau 28082. Masih ada proses lama yang berjalan, misalnya dari terminal lain. Cari proses tersebut dengan `lsof -i :28082`, lalu hentikan.

2.7 Eksplorasi Mandiri

1. **Account service yang lambat.** Lab ini menyebut *HTTP client* tanpa *timeout* sebagai kelemahan. Buktikan dengan menghentikan sementara proses account service. Cari PID *binary* hasil `go run` dengan `pgrep -f 'exe/account'`, hentikan sementara dengan `kill -STOP <pid>`, lalu kirim transfer. Amati bahwa `curl` menunggu tanpa batas. Lanjutkan kembali proses tersebut dengan `kill -CONT <pid>` dan perhatikan bahwa transfer yang tertunda langsung selesai. Bayangkan dampaknya jika ratusan transfer menunggu bersamaan.
2. **Debit bersamaan.** Buat rekening dengan saldo 100000, lalu kirim sepuluh transfer masing-masing 20000 secara paralel, misalnya dengan `for i in $(seq 10); do curl ... & done; wait`. Hitung jumlah transfer `COMPLETED` dan periksa bahwa saldo tidak pernah negatif. Jelaskan peran klausa `where balance + @delta >= 0` dalam hasil tersebut.
3. **Rekonsiliasi.** Tulis *query* SQL di database `account` yang menampilkan referensi transfer dengan `DEBIT` tetapi tanpa `CREDIT`. *Query* ini menjadi laporan rekonsiliasi paling sederhana untuk koreksi manual pada *known gap*.
4. **Jumlah dengan tiga digit desimal.** Kirim transfer dengan `"amount":"100.005"`. Transfer tetap `COMPLETED`, tetapi jumlah yang tercatat adalah `"100.01"`. Kolom `numeric(19, 2)` di database transfer membulatkan nilai tersebut tanpa *error*, lalu debit dan kredit ke account service menggunakan nilai hasil pembulatan. Rumuskan aturan validasi `amount` yang menolak *request* ini dengan `400` sehingga jumlah tersebut tidak lagi dibulatkan tanpa *error*. Lab 4 menambahkan aturan tersebut pada kedua service.
5. ***Context* yang dibatalkan.** Ganti sementara `context.WithoutCancel(r.Context())` dengan `r.Context()`, gabungkan dengan percobaan nomor 1, lalu hentikan `curl` dengan `Ctrl+C` sebelum proses account service dilanjutkan dengan `kill -CONT`. Amati status akhir transfer tersebut, lalu kembalikan kodenya.

2.8 Checkpoint

Kondisi akhir lab ini tersedia sebagai tag `lab-02`. Bandingkan hasil kerja dengan *checkpoint* untuk menemukan bagian yang terlewat. Jalankan perintah berikut dari folder `studi-kasus/`:

```
git diff lab-02 -- .
```

Tag `lab-02` juga sudah berisi *fault injector* yang dibutuhkan Lab 3, yaitu *package* `account/internal/fault` dan perubahan `account/main.go` yang memasangnya. Perbedaan pada kedua bagian tersebut, serta pada `go.work.sum`, dapat diabaikan karena tidak ditulis pada langkah lab ini. Karena *package*

tersebut ada di *checkpoint*, `make test` pada tag `lab-02` juga menampilkan baris `ok` untuk `account/internal/fault`.

Peserta yang ingin langsung melanjutkan ke Lab 3 dari kondisi *checkpoint* dapat menjalankan:

```
git switch -c kerja-lab-03 lab-02
```

3 Lab 3: *Timeout*, *Retry*, dan *Circuit Breaker* dengan failsafe-go

3.1 Tujuan

Setelah menyelesaikan lab ini, peserta mampu:

- mensimulasikan `account` *service* yang lambat atau sering gagal dengan *fault injector*;
- memasang *timeout* per percobaan, *retry* dengan *backoff* (jeda antarpercobaan yang makin panjang), dan *circuit breaker* pada HTTP *client* menggunakan failsafe-go;
- menentukan jenis kegagalan yang aman untuk di-*retry* dan jenis kegagalan yang harus dihitung oleh *circuit breaker*;
- membaca perubahan status *circuit breaker* (*closed*, *open*, *half-open*) dari *log* dan dari *endpoint* `GET /debug/circuit-breaker`;
- menguji ketiga *policy* tersebut dengan `net/http/httptest` tanpa menjalankan `account` *service*.

3.2 Gambaran

Pada akhir Lab 2, `transfer` *service* memanggil `account` *service* dengan `http.Client` tanpa *timeout*. Kode tersebut mengandung dua masalah yang baru terlihat ketika `account` bermasalah:

- Bila `account` lambat, setiap transfer ikut menunggu tanpa batas hingga `account` menjawab. *Goroutine* dan koneksi di `transfer` menumpuk, sehingga `transfer` ikut lambat walaupun database milik `transfer` sehat.
- Bila `account` mati atau kewalahan, setiap transfer tetap mengirim *request* ke `account`. Setiap *request* tersebut gagal, dan beban tambahan tersebut membuat `account` makin lama kembali aktif.

Lab ini membungkus HTTP *client* di paket `transfer/internal/accountclient` dengan tiga *policy* dari *library* failsafe-go.

Policy	**Pengaturan**	**Alasan**
Timeout	2 detik untuk setiap percobaan	Membatasi lama transfer menunggu `account`. Batas ini berlaku per percobaan, sehingga setiap *retry* mendapat jatah waktu sendiri.
Retry	Maksimal 3 kali, *backoff* 100 ms hingga 1 detik dengan *jitter*	Hanya untuk kegagalan yang pasti belum diproses `account`: koneksi gagal dibuka dan HTTP 503. *Timeout* tidak pernah di-*retry*.
Circuit breaker	Beralih ke *open* setelah 5 gagal dari 10 panggilan terakhir, menunggu 10 detik, lalu memerlukan 3 panggilan uji yang berhasil	Menghentikan panggilan ke `account` yang sedang bermasalah. Selama *open*, transfer langsung ditolak (*fail fast*) tanpa *request* ke `account`.

Ketiga *policy* dipasang sebagai `http.RoundTripper`, yaitu komponen di dalam `http.Client` yang benar-benar mengirim *request*. Kode pemanggil (`Debit` dan `Credit`) tidak berubah: kode tersebut tetap memanggil `c.http.Do(req)`, dan *round tripper* menjalankan ketiga *policy* di setiap panggilan.

```
POST /transfers
      |
      v
transfer service
  accountclient.Client.Debit / Credit
    http.Client
      failsafehttp RoundTripper
        retry               (paling luar: mengulang seluruh percobaan)
          circuit breaker   (mencatat hasil setiap percobaan)
            timeout 2 dtk   (membatasi setiap percobaan)
              http.DefaultTransport ---------> account service :28081
                                                 fault injector (lab ini)
                                                   /accounts...
```

Urutan ketiga *policy* menentukan perilaku *round tripper*. *Retry* berada paling luar sehingga setiap percobaan ulang melewati *circuit breaker* dan *timeout* lagi. *Timeout* berada paling dalam sehingga batas 2 detik berlaku untuk setiap percobaan, dan *circuit breaker* ikut mencatat *timeout* sebagai kegagalan.

Setelah lab ini, `POST /transfers` menjawab dengan kode HTTP berikut.

Kode	Status Transfer	Penyebab
201	COMPLETED	*Debit* dan *credit* berhasil.
422	FAILED	*Debit* ditolak `account`: rekening sumber tidak ditemukan atau saldo tidak cukup. Belum ada saldo yang berubah.
503	FAILED	*Circuit breaker* berstatus *open* saat *debit* akan dikirim. Transfer ditolak tanpa panggilan ke `account`, belum ada saldo yang berubah, dan *request* aman diulang.
502	FAILED	Panggilan *debit* sudah dilakukan dan gagal: *timeout*, *retry* habis, atau HTTP 500.
502	FAILED	*Debit* berhasil, lalu *credit* gagal karena sebab apa pun: rekening tujuan tidak ditemukan, *circuit breaker* yang baru beralih ke *open*, atau kegagalan lain. Saldo sumber sudah berkurang dan perlu dikembalikan manual, sehingga *request* tidak boleh diulang.

Pembagian kode 503 dan 502 di atas menjadi pegangan bagi aplikasi pemanggil. Kode 503 hanya digunakan bila belum ada saldo yang berubah, sehingga pemanggil boleh mengirim ulang *request* yang sama nanti. Setelah *debit* berhasil, setiap kegagalan *credit* dijawab 502, walaupun penyebabnya *circuit breaker* atau rekening tujuan yang tidak ada. Bila kegagalan tersebut dijawab 503 atau 422, pemanggil dapat mengira transfer aman diulang, dan pengulangan tersebut memotong saldo sumber untuk kedua kalinya.

> **Pengaturan *policy* ditulis di kode.** Angka *timeout*, *retry*, dan *circuit breaker* tidak dibaca dari *environment variable*. Angka tersebut merupakan bagian dari desain, sehingga setiap perubahannya perlu melewati *review* dan *unit test* seperti perubahan kode lainnya. *Unit test* menggunakan angka yang lebih kecil agar selesai dalam hitungan milidetik.

3.3 Titik Awal

Lab ini berangkat dari kondisi akhir Lab 2. Peserta yang belum menyelesaikan Lab 2 dapat memulai dari *tag* `lab-02`, lalu membuat *branch* kerja sendiri.

```
git switch -c kerja-lab-03 lab-02
cd studi-kasus
make infra-up
make test
```

Seluruh perintah di lab ini dijalankan dari folder `studi-kasus`, kecuali disebutkan lain.

3.4 Langkah

3.4.1 Langkah 1: *Fault Injector* pada `account`

Perilaku *timeout*, *retry*, dan *circuit breaker* hanya terlihat ketika `account` lambat atau gagal. Untuk itu `account` diberi *fault injector*, yaitu *middleware* yang menambahkan jeda atau menjawab HTTP 503 sesuai konfigurasi. Selama `account` berjalan, konfigurasi tersebut dapat diubah melalui *endpoint* `/admin/fault` tanpa *restart*.

Fault injector hanya sarana latihan, sehingga kodenya tidak perlu diketik. Peserta yang mengerjakan Lab 2 sendiri mengambil paket `fault` (beserta *unit test*) dan `account/main.go` yang memasang paket tersebut dari *checkpoint* `lab-02`:

```
git checkout lab-02 -- account/internal/fault account/main.go
```

Penjelasan isi kode tersebut ada di bawah ini.

Dua keputusan pada *fault injector* berpengaruh pada hasil pengamatan di bagian Verifikasi:

- Jawaban 503 dikirim **sebelum** *request* diproses. Saldo tidak berubah, sehingga 503 aman di-*retry*.
- Bila pemanggil membatalkan *request* selama jeda berlangsung (misalnya karena *timeout* di sisi `transfer`), *fault injector* berhenti dan tidak meneruskan *request* ke *handler*. Akibatnya *debit* yang terkena *timeout* tidak mengubah saldo. Langkah 6 membahas mengapa perilaku ini tidak boleh dijadikan dasar untuk me-*retry timeout*.

Isi file `account/internal/fault/fault.go`:

```
// Package fault simulates a slow or failing service for the resiliency labs.
// It is a training tool: the admin endpoints let anyone degrade the service.
package fault

import (
  "encoding/json"
  "log/slog"
  "math/rand/v2"
  "net/http"
  "sync/atomic"
  "time"
)

type Config struct {
  // DelayMS is added before every request is processed.
  DelayMS int `json:"delay_ms"`
  // ErrorRate is the fraction of requests (0.0–1.0) answered with 503 without
  // being processed.
  ErrorRate float64 `json:"error_rate"`
}

type Injector struct {
```

```
  cfg atomic.Pointer[Config]
  log *slog.Logger
}

func NewInjector(log *slog.Logger) *Injector {
  i := &Injector{log: log}
  i.cfg.Store(&Config{})
  return i
}

// Wrap applies the current fault configuration to every request of next.
func (i *Injector) Wrap(next http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    cfg := i.cfg.Load()
    if cfg.DelayMS > 0 {
      select {
      case <-time.After(time.Duration(cfg.DelayMS) * time.Millisecond):
      case <-r.Context().Done(): // caller gave up; do not process the request
        return
      }
    }
    if cfg.ErrorRate > 0 && rand.Float64() < cfg.ErrorRate {
      w.Header().Set("Content-Type", "application/json")
      w.WriteHeader(http.StatusServiceUnavailable)
      _, _ = w.Write([]byte(`{"error":"injected fault"}` + "\n"))
      return
    }
    next.ServeHTTP(w, r)
  })
}

// Register adds GET/PUT/DELETE /admin/fault to mux.
func (i *Injector) Register(mux *http.ServeMux) {
  mux.HandleFunc("GET /admin/fault", func(w http.ResponseWriter, _ *http.Request) {
    writeJSON(w, http.StatusOK, i.cfg.Load())
  })
  mux.HandleFunc("PUT /admin/fault", func(w http.ResponseWriter, r *http.Request) {
    var cfg Config
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()
    if err := dec.Decode(&cfg); err != nil {
      writeJSON(w, http.StatusBadRequest, map[string]string{"error": "invalid JSON body: "
+ err.Error()})
      return
    }
    if cfg.DelayMS < 0 || cfg.ErrorRate < 0 || cfg.ErrorRate > 1 {
      writeJSON(w, http.StatusBadRequest, map[string]string{"error": "delay_ms must be >=
0, error_rate between 0 and 1"})
      return
    }
    i.cfg.Store(&cfg)
    i.log.Warn("fault injection changed", "delay_ms", cfg.DelayMS, "error_rate",
cfg.ErrorRate)
    writeJSON(w, http.StatusOK, cfg)
  })
  mux.HandleFunc("DELETE /admin/fault", func(w http.ResponseWriter, _ *http.Request) {
    i.cfg.Store(&Config{})
    i.log.Warn("fault injection cleared")
    writeJSON(w, http.StatusOK, Config{})
```

```
  })
}

func writeJSON(w http.ResponseWriter, status int, body any) {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(status)
  _ = json.NewEncoder(w).Encode(body)
}
```

Konfigurasi disimpan dalam `atomic.Pointer` karena *handler* `PUT /admin/fault` dan *request* ke `/accounts` berjalan di *goroutine* berbeda secara bersamaan. Setiap *request* membaca konfigurasi utuh sekali di awal, sehingga perubahan konfigurasi tidak pernah terbaca setengah jadi.

Fault injector hanya dipasang pada `/accounts` dan seluruh *path* di bawahnya. *Health check* (`/healthz`, `/readyz`) dan `/admin/fault` tetap cepat, sehingga *fault* dapat dimatikan lagi kapan pun. Pemasangannya ada di fungsi `run()` pada `account/main.go`, setelah `defer store.Close()`:

```
  // Faults apply to the account API only; probes and the admin endpoints stay fast.
  faults := fault.NewInjector(log)
  api := account.NewHandler(store, log).Routes()
  mux := http.NewServeMux()
  mux.Handle("/accounts", faults.Wrap(api))
  mux.Handle("/accounts/", faults.Wrap(api))
  mux.Handle("/", api)
  faults.Register(mux)

  srv := &http.Server{
    Addr:              cfg.HTTPAddr,
    Handler:           mux,
    ReadHeaderTimeout: 5 * time.Second,
  }
```

Pola `/accounts/` (diakhiri garis miring) pada `ServeMux` mencakup seluruh *path* di bawahnya, termasuk `/accounts/{number}/debit` dan `/accounts/{number}/credit`. Pola `/` menangkap *path* lain, yaitu *health check*, dan meneruskannya ke *handler* asli tanpa *fault*.

3.4.2 Langkah 2: Target `make` untuk Mengatur *Fault*

Perintah `curl` lengkap rawan salah ketik bila harus diketik setiap kali *fault* diubah. Karena itu `Makefile` di `studi-kasus` diberi beberapa target pembantu. Tambahkan dua variabel berikut di bawah baris `SERVICES := account transfer`, lalu perbarui baris `.PHONY`.

```
# Fault injection on the account service (Lab 3). Faults apply to /accounts*
# only; probes and /admin stay fast.
ACCOUNT_URL := http://localhost:28081
TRANSFER_URL := http://localhost:28082

.PHONY: infra-up infra-down infra-reset test fault-slow fault-error fault-clear fault-
status breaker
```

Tambahkan target berikut di akhir file. Baris perintah di dalam target `make` wajib diawali karakter *tab*.

```
# 3000 ms is above the transfer client's 2 s per-attempt timeout: every call
# times out and counts as a breaker failure.
fault-slow:
  curl -sS -X PUT $(ACCOUNT_URL)/admin/fault -d '{"delay_ms":3000,"error_rate":0}'

# Usage: make fault-error RATE=0.5 (fraction of requests answered 503).
fault-error:
  @test -n "$(RATE)" || { echo "RATE is required, e.g. make fault-error RATE=0.5" >&2; exit
1; }
  curl -sS -X PUT $(ACCOUNT_URL)/admin/fault -d '{"delay_ms":0,"error_rate":$(RATE)}'
```

```
fault-clear:
  curl -sS -X DELETE $(ACCOUNT_URL)/admin/fault

fault-status:
  curl -sS $(ACCOUNT_URL)/admin/fault

breaker:
  curl -sS $(TRANSFER_URL)/debug/circuit-breaker
```

Target `fault-error` sengaja tidak memiliki nilai *default* untuk `RATE`. Bila `RATE` tidak diisi, target tersebut berhenti dengan pesan error, sesuai prinsip konfigurasi di studi kasus ini: nilai yang wajib harus disebutkan secara eksplisit.

3.4.3 Langkah 3: Menambahkan Dependensi failsafe-go

failsafe-go menyediakan *policy timeout*, *retry*, *circuit breaker*, dan beberapa *policy* lain dengan API yang seragam. Paket `failsafehttp` di dalamnya membungkus *policy* tersebut menjadi `http.RoundTripper`.

```
cd transfer
go get github.com/failsafe-go/failsafe-go@v0.9.8
go mod tidy
cd ..
```

Setelah perintah tersebut, `transfer/go.mod` memuat baris `github.com/failsafe-go/failsafe-go v0.9.8` di blok `require`.

3.4.4 Langkah 4: Error untuk *Circuit Breaker* yang *Open* dan *Credit* yang Gagal

Paket `transfer` (domain) tidak boleh bergantung pada failsafe-go. Paket tersebut cukup mengenal dua kondisi baru:

- `account` sedang tidak tersedia, dan panggilan ditolak tanpa dikirim;
- *debit* sudah diproses, tetapi *credit* gagal, sehingga transfer tidak lagi aman diulang.

Tambahkan dua variabel berikut di blok `var` pada `transfer/internal/transfer/transfer.go`, tepat setelah `ErrInsufficientFunds`.

```
  // The circuit breaker to the account service is open: the call was rejected
  // without being sent.
  ErrAccountUnavailable = errors.New("account service unavailable")
  // The debit was applied but the credit was not: the source account needs a
  // manual reversal. Whatever made the credit fail, the transfer is no longer
  // safe to retry.
  ErrCreditFailed = errors.New("credit failed after debit")
```

Paket `accountclient` menerjemahkan error milik failsafe-go menjadi `ErrAccountUnavailable` (Langkah 8). Dengan cara ini, *handler* HTTP dapat memilih kode 503 tanpa mengetahui *library* apa yang digunakan di bawahnya. `Service.Execute` (Langkah 9) membungkus setiap kegagalan *credit* dengan `ErrCreditFailed`, apa pun penyebabnya.

3.4.5 Langkah 5: Pengaturan *Policy*

Seluruh angka *policy* dikumpulkan dalam satu *struct* agar *unit test* dapat memberikan angka yang lebih kecil. Di `transfer/internal/accountclient/client.go`, ganti komentar paket dan blok *import*, lalu tambahkan `Settings` dan `ProductionSettings` sebelum `type Client`.

```
// Package accountclient calls the account service REST API through failsafe-go
// policies: per-attempt timeout, retry with backoff, and a circuit breaker.
package accountclient

import (
  "bytes"
  "context"
```

```
  "encoding/json"
  "errors"
  "fmt"
  "io"
  "log/slog"
  "net"
  "net/http"
  "time"

  "github.com/failsafe-go/failsafe-go"
  "github.com/failsafe-go/failsafe-go/circuitbreaker"
  "github.com/failsafe-go/failsafe-go/failsafehttp"
  "github.com/failsafe-go/failsafe-go/retrypolicy"
  "github.com/failsafe-go/failsafe-go/timeout"
  "github.com/shopspring/decimal"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/
transfer"
)

// Settings are the resiliency parameters. They live in code, not in environment
// variables: changing them is a design decision that goes through review and
// tests like any other code change. Tests pass shorter durations.
type Settings struct {
  // AttemptTimeout bounds one HTTP attempt, not the whole call with its retries.
  AttemptTimeout time.Duration
  MaxRetries     int
  BackoffMin     time.Duration
  BackoffMax     time.Duration
  // The breaker opens when BreakerFailures of the last BreakerWindow calls failed,
  // stays open for BreakerDelay, then lets BreakerSuccesses trial calls through
  // (half-open). All of them must succeed to close it; one failure reopens it.
  BreakerFailures  uint
  BreakerWindow    uint
  BreakerDelay     time.Duration
  BreakerSuccesses uint
}

// ProductionSettings is what the transfer service runs with.
var ProductionSettings = Settings{
  AttemptTimeout:   2 * time.Second,
  MaxRetries:       3,
  BackoffMin:       100 * time.Millisecond,
  BackoffMax:       time.Second,
  BreakerFailures:  5,
  BreakerWindow:    10,
  BreakerDelay:     10 * time.Second,
  BreakerSuccesses: 3,
}
```

Arti setiap angka:

- `AttemptTimeout` 2 detik: *debit* atau *credit* yang sehat selesai dalam hitungan milidetik. Percobaan yang melewati 2 detik dianggap gagal.
- `MaxRetries` 3: total paling banyak 4 percobaan (1 percobaan awal ditambah 3 *retry*).
- `BackoffMin` dan `BackoffMax`: jeda sebelum *retry* dimulai dari 100 ms dan berlipat hingga paling lama 1 detik. Jeda yang makin panjang memberi waktu bagi `account` untuk kembali aktif.
- `BreakerFailures` dan `BreakerWindow`: *circuit breaker* beralih ke *open* bila 5 dari 10 panggilan terakhir gagal.

- `BreakerDelay` 10 detik: lama *circuit breaker* bertahan di *open* sebelum mengizinkan panggilan uji.
- `BreakerSuccesses` 3: jumlah panggilan uji yang harus berhasil di status *half-open* sebelum kembali ke *closed*. Satu kegagalan di *half-open* mengembalikan *circuit breaker* ke *open*.

3.4.6 Langkah 6: Menentukan Kegagalan yang Di-*retry* dan Dihitung

Langkah ini memuat inti keputusan desain di lab ini. Ada dua pertanyaan yang jawabannya berbeda.

Pertanyaan pertama: kegagalan mana yang aman untuk di-*retry*? *Debit* belum *idempotent*. Bila *debit* yang sama dikirim dua kali dan keduanya diproses, saldo terpotong dua kali. Karena itu *retry* hanya boleh dilakukan bila *request* dipastikan **belum diproses** oleh `account`:

- Koneksi gagal dibuka (*connection refused*, *host* tidak dikenal). *Request* belum terkirim sama sekali.
- HTTP 503. Di studi kasus ini, 503 berarti `account` menolak *request* sebelum memprosesnya.

Timeout tidak termasuk. Saat *timeout* terjadi, `transfer` tidak tahu apakah `account` sudah memproses *debit* dan hanya jawabannya yang terlambat. Di lab ini saldo memang tidak berubah ketika *timeout* terjadi, tetapi hal tersebut hanya karena *fault injector* berhenti saat pemanggil membatalkan *request* (Langkah 1). Database yang lambat di sistem sungguhan dapat saja sudah melakukan *commit* atas *debit* tersebut. HTTP 500 juga tidak di-*retry* dengan alasan yang sama: *request* bisa saja sudah setengah diproses.

Pertanyaan kedua: kegagalan mana yang dihitung oleh *circuit breaker*? *Circuit breaker* mengukur kesehatan `account` *service*. Error *transport*, *timeout*, dan HTTP 5xx menandakan `account` bermasalah. HTTP 4xx (rekening tidak ditemukan, saldo tidak cukup) justru menandakan `account` sehat dan menjawab secara normal, sehingga tidak boleh membuat *circuit breaker* beralih ke *open*. Tanpa pembedaan tersebut, sepuluh nasabah dengan saldo kurang dapat menghentikan transfer seluruh nasabah lain.

Tambahkan dua fungsi berikut di `client.go`. Letak fungsi di dalam file tidak berpengaruh. Di *checkpoint*, kedua fungsi tersebut berada setelah fungsi `New` dari Langkah 7.

```
// notProcessed reports whether the request provably never reached the account
// logic, the only case where a retry is safe today.
//
// A timeout is NOT retried: the account service may have applied the debit and
// only the answer was late. Debit is not idempotent yet, so a retry would debit
// twice. Lab 5 makes mutations idempotent by reference; only then are timeouts
// retried.
func notProcessed(resp *http.Response, err error) bool {
  if err != nil {
    // Dial failure (connection refused, unknown host): nothing was sent.
    var opErr *net.OpError
    return errors.As(err, &opErr) && opErr.Op == "dial"
  }
  // 503 means the service refused the request before processing it (the fault
  // injector answers 503 without touching the account).
  return resp.StatusCode == http.StatusServiceUnavailable
}

// countsAsFailure decides what the breaker counts against the account service:
// transport errors, timeouts and 5xx. A 4xx is a healthy service saying no
// (unknown account, insufficient funds) and must not open the breaker.
func countsAsFailure(resp *http.Response, err error) bool {
  return err != nil || resp.StatusCode >= 500
}
```

Kedua fungsi menerima pasangan `(resp, err)` yang sama dengan hasil `RoundTrip`. Bila `err` tidak `nil`, `resp` bernilai `nil`, sehingga `resp` hanya dibaca setelah `err` diperiksa. Pada `notProcessed`, error *timeout*

dari failsafe-go bukan `*net.OpError` dengan operasi `dial`, sehingga fungsi tersebut mengembalikan `false` dan *timeout* tidak di-*retry*.

3.4.7 Langkah 7: Menyusun *Policy* di Fungsi `New`

Ganti `type Client` dan fungsi `New` milik Lab 2 dengan versi berikut. Fungsi `New` kini menerima `Settings` dan `*slog.Logger`.

```
type Client struct {
  baseURL string
  http    *http.Client
  breaker circuitbreaker.CircuitBreaker[*http.Response]
}

func New(baseURL string, s Settings, log *slog.Logger) *Client {
  // One breaker for the whole account service: debit and credit share the same
  // failure domain, so a dead account service stops both.
  breaker := circuitbreaker.NewBuilder[*http.Response]().
    HandleIf(countsAsFailure).
    WithFailureThresholdRatio(s.BreakerFailures, s.BreakerWindow).
    WithDelay(s.BreakerDelay).
    WithSuccessThreshold(s.BreakerSuccesses).
    OnStateChanged(func(e circuitbreaker.StateChangedEvent) {
      log.Warn("circuit breaker state changed", "target", "account",
        "from", e.OldState.String(), "to", e.NewState.String())
    }).
    Build()

  retry := retrypolicy.NewBuilder[*http.Response]().
    HandleIf(notProcessed).
    // Explicit even though notProcessed already ignores ErrOpen: retrying against
    // an open breaker only burns backoff time and fails anyway.
    AbortOnErrors(circuitbreaker.ErrOpen).
    WithMaxRetries(s.MaxRetries).
    WithBackoff(s.BackoffMin, s.BackoffMax).
    // Jitter spreads the retries of many concurrent transfers so they do not all
    // hit a recovering account service in the same millisecond.
    WithJitterFactor(0.25).
    // Hand the last 503 back as a response instead of wrapping it in an
    // ExceededError, so the status mapping in mutate sees it.
    ReturnLastFailure().
    OnRetry(func(e failsafe.ExecutionEvent[*http.Response]) {
      // The response that triggered this retry is discarded; close it so its
      // connection goes back to the pool.
      cause := fmt.Sprint(e.LastError())
      if resp := e.LastResult(); resp != nil {
        cause = resp.Status
        _, _ = io.Copy(io.Discard, resp.Body)
        resp.Body.Close()
      }
      log.Warn("retrying account call", "next_attempt", e.Attempts(), "cause", cause)
    }).
    Build()

  // Order matters: retry wraps the breaker, which wraps the timeout. Every attempt
  // gets its own time budget, and every attempt, timeouts included, is counted by
  // the breaker.
  rt := failsafehttp.NewRoundTripper(nil, retry, breaker, timeout.New[*http.Response]
(s.AttemptTimeout))
  return &Client{baseURL: baseURL, http: &http.Client{Transport: rt}, breaker: breaker}
}
```

Beberapa hal yang perlu diperhatikan dari kode di atas:

- **Satu *circuit breaker* untuk `account`.** *Debit* dan *credit* menuju *service* yang sama. Bila `account` mati, keduanya sama-sama gagal, sehingga keduanya berbagi satu *circuit breaker*.
- **Urutan argumen `NewRoundTripper`.** *Policy* disusun dari luar ke dalam sesuai urutan argumen, yaitu `retry`, lalu `breaker`, lalu *timeout*. Argumen pertama `nil` berarti *round tripper* di bagian terdalam adalah `http.DefaultTransport`.
- **`AbortOnErrors(circuitbreaker.ErrOpen)`.** Bila *circuit breaker* beralih ke *open* di tengah rangkaian *retry*, *retry* berikutnya langsung dihentikan. Tidak ada gunanya menunggu *backoff* bila percobaan berikutnya pasti ditolak *circuit breaker*.
- ***Jitter*.** Faktor 0,25 mengacak jeda *backoff* hingga 25%. Bila seratus transfer gagal bersamaan, *retry* dari seratus transfer tersebut tidak tiba di `account` pada milidetik yang sama.
- **`ReturnLastFailure()`.** Bila *retry* habis, failsafe-go mengembalikan *response* 503 terakhir apa adanya. Dengan begitu fungsi `mutate` tetap membaca kode status seperti biasa.
- **`OnRetry` menutup *body*.** Kode lain tidak membaca lagi *response* 503 yang memicu *retry*. *Body* tersebut harus dibaca habis dan ditutup agar koneksinya kembali ke *connection pool*.
- ***Log* di setiap perubahan status.** `OnStateChanged` menulis *log* `circuit breaker state changed` dengan atribut `from` dan `to`. *Log* inilah yang diamati di bagian Verifikasi.

3.4.8 Langkah 8: Memetakan Error dan Menampilkan Status *Circuit Breaker*

Ketika *circuit breaker* berstatus *open*, `c.http.Do(req)` mengembalikan error yang membungkus `circuitbreaker.ErrOpen`. Di fungsi `mutate`, sisipkan pemeriksaan berikut tepat setelah baris `resp, err := c.http.Do(req)` dan sebelum pemeriksaan `if err != nil` yang sudah ada.

```
  resp, err := c.http.Do(req)
  if errors.Is(err, circuitbreaker.ErrOpen) {
    return fmt.Errorf("%s %s: %w", kind, number, transfer.ErrAccountUnavailable)
  }
  if err != nil {
    return fmt.Errorf("%s %s: %w", kind, number, err)
  }
```

Di fungsi yang sama, tambahkan komentar di atas `http.NewRequestWithContext`. *Body* dibuat dengan `bytes.NewReader` agar failsafe-go dapat mengirim ulang *body* yang sama di setiap *retry*.

```
  // bytes.Reader: failsafehttp replays the body on every retry attempt.
  req, err := http.NewRequestWithContext(ctx, http.MethodPost, url, bytes.NewReader(body))
```

Tambahkan dua *method* berikut setelah `countsAsFailure`. `BreakerState` digunakan oleh *unit test*, sedangkan `ServeCircuitBreaker` menjadi *handler* `GET /debug/circuit-breaker`.

```
// BreakerState is "closed", "open" or "half-open".
func (c *Client) BreakerState() string { return c.breaker.State().String() }

// ServeCircuitBreaker answers GET /debug/circuit-breaker on the transfer service.
func (c *Client) ServeCircuitBreaker(w http.ResponseWriter, _ *http.Request) {
  w.Header().Set("Content-Type", "application/json")
  _ = json.NewEncoder(w).Encode(map[string]string{"account": c.BreakerState()})
}
```

3.4.9 Langkah 9: Kode HTTP 503 dan 502, serta Pendaftaran *Endpoint*

Perubahan pertama ada di `Service.Execute` pada `transfer/internal/transfer/service.go`. Kegagalan *credit* kini dibungkus `ErrCreditFailed` agar *handler* dapat membedakannya dari kegagalan *debit*. Ubah fungsi `fail` sehingga menerima satu error yang sudah memuat nama langkahnya, lalu sesuaikan kedua pemanggilnya di `Execute`.

```
  if err := s.accounts.Debit(ctx, t.FromAccount, t.Amount, t.ID); err != nil {
    return s.fail(ctx, t, fmt.Errorf("debit failed: %w", err))
  }
```

```
  if err := s.accounts.Credit(ctx, t.ToAccount, t.Amount, t.ID); err != nil {
    s.log.Error("credit failed after debit, source account needs manual reversal",
      "transfer_id", t.ID, "from_account", t.FromAccount, "amount", t.Amount, "error", err)
    return s.fail(ctx, t, fmt.Errorf("%w: %w", ErrCreditFailed, err))
  }
func (s *Service) fail(ctx context.Context, t Transfer, cause error) (Transfer, error) {
  reason := cause.Error()
```

Format `"%w: %w"` membungkus dua error sekaligus. `errors.Is(err, ErrCreditFailed)` bernilai `true`, dan `errors.Is` terhadap penyebab aslinya, misalnya `ErrAccountNotFound`, juga tetap `true`. Teks `failure_reason` yang disimpan tidak berubah dari Lab 2, misalnya `debit failed: ...` atau `credit failed after debit: ...`.

Di `transfer/internal/transfer/http.go`, fungsi `create`, tambahkan dua *case* baru. *Case* `ErrCreditFailed` diletakkan **sebelum** *case* penolakan bisnis, sedangkan *case* `ErrAccountUnavailable` diletakkan di antara penolakan bisnis dan `default`.

```
  case errors.Is(err, ErrCreditFailed):
    // Checked first: the debit was applied, so this must not look retryable
    // (503) or like a plain business rejection (422). A retried POST would
    // debit again. The service already logged it for manual reversal.
    writeJSON(w, http.StatusBadGateway, t)
  case errors.Is(err, ErrAccountNotFound), errors.Is(err, ErrInsufficientFunds):
    writeJSON(w, http.StatusUnprocessableEntity, t)
  case errors.Is(err, ErrAccountUnavailable):
    // The open breaker refused the debit, so nothing was applied: 503 tells the
    // caller it is safe to come back later.
    writeJSON(w, http.StatusServiceUnavailable, t)
  default:
```

Urutan *case* menentukan hasilnya. Rekening tujuan yang tidak ditemukan menghasilkan error yang membungkus `ErrCreditFailed` sekaligus `ErrAccountNotFound`. Bila *case* 422 diperiksa terlebih dahulu, transfer tersebut dijawab 422 walaupun saldo sumber sudah terpotong. Hal yang sama berlaku untuk *circuit breaker* yang beralih ke *open* di antara *debit* dan *credit*: tanpa *case* `ErrCreditFailed` di urutan pertama, transfer tersebut dijawab 503 dan pemanggil mengira transfer aman diulang.

Di `transfer/main.go`, fungsi `run()`, ganti baris pembuatan `svc` dan `srv` menjadi seperti berikut.

```
  accounts := accountclient.New(cfg.AccountServiceURL, accountclient.ProductionSettings,
log)
  svc := transfer.NewService(store, accounts, log)
  mux := transfer.NewHandler(svc, log).Routes()
  mux.HandleFunc("GET /debug/circuit-breaker", accounts.ServeCircuitBreaker)
  srv := &http.Server{
    Addr:              cfg.HTTPAddr,
    Handler:           mux,
    ReadHeaderTimeout: 5 * time.Second,
  }
```

`Routes()` mengembalikan `*http.ServeMux`, sehingga *endpoint* tambahan dapat didaftarkan langsung di `main.go`. Paket `transfer` tetap tidak mengenal *circuit breaker*.

Terakhir, perbarui komentar di atas `Execute` pada `transfer/internal/transfer/service.go`. Celah dari Lab 2 masih ada, dan *circuit breaker* justru menambah peluang celah tersebut terjadi: *circuit breaker* dapat beralih ke *open* setelah *debit* berhasil dan sebelum *credit* dikirim.

```
// Execute records the transfer, debits the source, then credits the destination.
// The returned error is the reason the transfer FAILED; the transfer itself is
// returned whenever it was recorded. A credit failure is always wrapped in
// ErrCreditFailed, whatever its cause, so callers can tell it apart from a debit
```

```
// failure (nothing applied).
//
// Known gap: debit and credit are two calls to another service, with no shared
// database transaction. When the credit fails after a successful debit, the money
// has left the source account and arrived nowhere. The transfer is marked FAILED
// and logged for manual reversal. The circuit breaker (Lab 3) makes this more
// likely, not less: it can open between the debit and the credit. Lab 5 replaces
// this with a saga that compensates the debit.
```

3.4.10 Langkah 10: *Unit Test* untuk *Policy*

Ketiga *policy* diuji dengan `httptest.Server` yang perilakunya diatur oleh *test*. `account` *service* dan database tidak perlu berjalan. Angka *policy* diperkecil agar setiap *test* selesai dalam milidetik, sedangkan bentuk *policy*-nya tetap sama dengan `ProductionSettings`.

Di `transfer/internal/accountclient/client_test.go`, tambahkan *import* `bytes`, `io`, `log/slog`, `strings`, `sync/atomic`, dan `time`. *Import* `bytes` dan `strings` digunakan oleh *test* keempat. Lalu tambahkan fungsi pembantu berikut setelah blok *import*.

```
// testSettings keeps the production policy shape with short durations so the
// tests run in milliseconds.
var testSettings = Settings{
  AttemptTimeout:   100 * time.Millisecond,
  MaxRetries:       3,
  BackoffMin:       time.Millisecond,
  BackoffMax:       5 * time.Millisecond,
  BreakerFailures:  5,
  BreakerWindow:    10,
  BreakerDelay:     time.Hour, // stays open for the whole test
  BreakerSuccesses: 3,
}

func newTestClient(url string) *Client {
  return New(url, testSettings, slog.New(slog.NewTextHandler(io.Discard, nil)))
}

func debit(c *Client) error {
  return c.Debit(context.Background(), "1111111111", decimal.NewFromInt(10), "ref")
}

// countingServer answers each request with status(n), n = 1 for the first request.
func countingServer(t *testing.T, status func(n int64) int) (*httptest.Server,
*atomic.Int64) {
  var calls atomic.Int64
  srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, _ *http.Request) {
    w.WriteHeader(status(calls.Add(1)))
  }))
  t.Cleanup(srv.Close)
  return srv, &calls
}
```

`countingServer` menghitung jumlah *request* yang benar-benar diterima. Angka tersebut membuktikan berapa kali *retry* terjadi dan apakah *circuit breaker* yang *open* benar-benar tidak mengirim *request*.

Karena *signature* `New` berubah, *test* milik Lab 2 perlu disesuaikan. Di `TestStatusMapping`, ganti pemanggilan `New(srv.URL).Debit(...)` dengan `debit(newTestClient(srv.URL))`. Ganti juga isi `TestServerError` agar sekaligus memastikan HTTP 500 tidak di-*retry*.

```
func TestServerError(t *testing.T) {
  srv, calls := countingServer(t, func(int64) int { return
http.StatusInternalServerError })
```

```
  err := newTestClient(srv.URL).Credit(context.Background(), "1111111111",
decimal.NewFromInt(10), "ref")
  if err == nil || errors.Is(err, transfer.ErrAccountNotFound) || errors.Is(err,
transfer.ErrInsufficientFunds) {
    t.Fatalf("err = %v, want a non-business error", err)
  }
  // A 500 may come from a request that was half processed: no retry.
  if got := calls.Load(); got != 1 {
    t.Fatalf("calls = %d, want 1", got)
  }
}
```

Tambahkan empat *test* utama berikut. *Test* pertama memastikan 503 di-*retry* hingga berhasil.

```
func TestRetryOn503Succeeds(t *testing.T) {
  srv, calls := countingServer(t, func(n int64) int {
    if n <= 2 {
      return http.StatusServiceUnavailable
    }
    return http.StatusOK
  })
  if err := debit(newTestClient(srv.URL)); err != nil {
    t.Fatalf("err = %v, want success after retries", err)
  }
  if got := calls.Load(); got != 3 {
    t.Fatalf("calls = %d, want 3 (two 503, then 200)", got)
  }
}
```

Test kedua memastikan *timeout* tidak di-*retry*. *Server* menunggu 300 ms, lebih lama dari *timeout* 100 ms di `testSettings`. *Server* juga berhenti menunggu ketika *request* dibatalkan, sehingga `t.Cleanup(srv.Close)` tidak tertahan.

```
func TestNoRetryOnTimeout(t *testing.T) {
  var calls atomic.Int64
  srv := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    calls.Add(1)
    select {
    case <-time.After(300 * time.Millisecond):
    case <-r.Context().Done():
    }
  }))
  t.Cleanup(srv.Close)

  start := time.Now()
  err := debit(newTestClient(srv.URL))
  if err == nil {
    t.Fatal("err = nil, want timeout")
  }
  if got := calls.Load(); got != 1 {
    t.Fatalf("calls = %d, want 1: a timed-out debit may have been applied", got)
  }
  if elapsed := time.Since(start); elapsed > 500*time.Millisecond {
    t.Fatalf("took %s, want about the attempt timeout", elapsed)
  }
}
```

Test ketiga memastikan *circuit breaker* beralih ke *open* setelah 5 kegagalan, lalu menolak panggilan berikutnya dengan cepat tanpa mengirim *request* ke *server*.

```
func TestBreakerOpensAndRejectsFast(t *testing.T) {
  srv, calls := countingServer(t, func(int64) int { return
http.StatusInternalServerError })
  c := newTestClient(srv.URL)

  for range 5 {
    _ = debit(c)
  }
  if got := c.BreakerState(); got != "open" {
    t.Fatalf("breaker = %s, want open after 5 failures", got)
  }

  start := time.Now()
  err := debit(c)
  if !errors.Is(err, transfer.ErrAccountUnavailable) {
    t.Fatalf("err = %v, want ErrAccountUnavailable", err)
  }
  if got := calls.Load(); got != 5 {
    t.Fatalf("calls = %d, want 5: an open breaker must not reach the server", got)
  }
  if elapsed := time.Since(start); elapsed > 50*time.Millisecond {
    t.Fatalf("rejected in %s, want immediately", elapsed)
  }

  rec := httptest.NewRecorder()
  c.ServeCircuitBreaker(rec, httptest.NewRequest(http.MethodGet, "/debug/circuit-breaker",
nil))
  if body := rec.Body.String(); body != `{"account":"open"}`+"\n" {
    t.Fatalf("debug body = %q", body)
  }
}
```

Test keempat memastikan kegagalan koneksi di-*retry*. *Server* langsung ditutup, sehingga setiap percobaan menghasilkan *connection refused*. *Log* ditampung di `bytes.Buffer` agar jumlah baris `retrying account call` dapat dihitung. Empat kegagalan masih di bawah ambang 5, sehingga *circuit breaker* tetap *closed*.

```
func TestRetryOnConnectionRefused(t *testing.T) {
  srv := httptest.NewServer(http.NotFoundHandler())
  url := srv.URL
  srv.Close() // nothing listens on this port any more

  var logs bytes.Buffer
  c := New(url, testSettings, slog.New(slog.NewTextHandler(&logs, nil)))
  if err := debit(c); err == nil || errors.Is(err, transfer.ErrAccountUnavailable) {
    t.Fatalf("err = %v, want a connection error", err)
  }
  if got := strings.Count(logs.String(), "retrying account call"); got != 3 {
    t.Fatalf("retries = %d, want 3; logs:\n%s", got, logs.String())
  }
  // 4 refused attempts count as 4 breaker failures, still under 5.
  if got := c.BreakerState(); got != "closed" {
    t.Fatalf("breaker = %s, want closed", got)
  }
}
```

Checkpoint `lab-03` memuat tiga *test* tambahan dengan pola yang sama. Peserta disarankan menulisnya sendiri sebagai latihan sebelum membandingkan dengan *checkpoint*:

- `TestRetryGivesUpAfterMaxRetries`: *server* selalu menjawab 503, dan *server* harus menerima tepat 4 *request* (1 percobaan awal ditambah 3 *retry*).
- `TestBusinessRejectionsDoNotOpenBreaker`: sepuluh jawaban 422 berturut-turut tidak mengubah *circuit breaker* dari *closed*.
- `TestBreakerHalfOpenCloses`: dengan `BreakerDelay` 50 ms, *circuit breaker* beralih dari *open* ke *half-open* setelah panggilan uji pertama berhasil, lalu ke *closed* setelah panggilan uji ketiga.

Di `transfer/internal/transfer/service_test.go`, tabel `TestExecute` perlu tiga perubahan. Kasus `credit fails after debit` milik Lab 2 kini mengharapkan `ErrCreditFailed`. Dua kasus baru memastikan *credit* tidak dipanggil ketika *debit* ditolak *circuit breaker*, dan memastikan *circuit breaker* yang beralih ke *open* setelah *debit* tetap menghasilkan `ErrCreditFailed`.

```
    {"credit fails after debit", &fakeAccounts{creditErr: unavailable}, Failed,
ErrCreditFailed, 2},
    {"open circuit breaker fails the transfer", &fakeAccounts{debitErr:
ErrAccountUnavailable}, Failed, ErrAccountUnavailable, 1},
    {"breaker opens between debit and credit", &fakeAccounts{creditErr:
ErrAccountUnavailable}, Failed, ErrCreditFailed, 2},
```

Pemetaan error ke kode HTTP juga diuji. Buat file baru `transfer/internal/transfer/http_test.go`. *Test* ini menggunakan `fakeStore` dan `fakeAccounts` yang sudah ada di `service_test.go`, lalu mengirim `POST /transfers` ke `httptest.Server` untuk setiap kemungkinan hasil.

```
package transfer

import (
  "errors"
  "io"
  "log/slog"
  "net/http"
  "net/http/httptest"
  "strings"
  "testing"
)

// TestCreateStatus checks the HTTP status for each outcome. 503 is reserved for
// "nothing applied, safe to retry"; once the debit went through, every credit
// failure is a 502, even when the cause is the open breaker.
func TestCreateStatus(t *testing.T) {
  tests := []struct {
    name     string
    accounts *fakeAccounts
    want     int
  }{
    {"success", &fakeAccounts{}, http.StatusCreated},
    {"insufficient funds", &fakeAccounts{debitErr: ErrInsufficientFunds},
http.StatusUnprocessableEntity},
    {"breaker open before debit", &fakeAccounts{debitErr: ErrAccountUnavailable},
http.StatusServiceUnavailable},
    {"breaker opens between debit and credit", &fakeAccounts{creditErr:
ErrAccountUnavailable}, http.StatusBadGateway},
    {"destination not found after debit", &fakeAccounts{creditErr: ErrAccountNotFound},
http.StatusBadGateway},
    {"credit fails after debit", &fakeAccounts{creditErr: errors.New("connection
refused")}, http.StatusBadGateway},
    {"debit fails on transport error", &fakeAccounts{debitErr: errors.New("connection
refused")}, http.StatusBadGateway},
  }
  for _, tc := range tests {
```

```
    t.Run(tc.name, func(t *testing.T) {
      log := slog.New(slog.NewTextHandler(io.Discard, nil))
      svc := NewService(&fakeStore{transfers: map[string]Transfer{}}, tc.accounts, log)
      srv := httptest.NewServer(NewHandler(svc, log).Routes())
      defer srv.Close()

      body := `{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}`
      resp, err := http.Post(srv.URL+"/transfers", "application/json",
strings.NewReader(body))
      if err != nil {
        t.Fatal(err)
      }
      defer resp.Body.Close()
      if resp.StatusCode != tc.want {
        t.Fatalf("status = %d, want %d", resp.StatusCode, tc.want)
      }
    })
  }
}
```

Kasus `destination not found after debit` gagal bila *case* `ErrCreditFailed` diletakkan setelah *case* 422, dan kasus `breaker opens between debit and credit` gagal bila *case* tersebut diletakkan setelah *case* 503.

Jalankan seluruh *test*.

```
make test
```

Seluruh *test* harus lulus. `go vet` di target `test` juga memeriksa kode sebelum *test* dijalankan.

3.5 Verifikasi

Verifikasi berikut menjalankan `account` dan `transfer`, lalu mengamati perilaku *policy* dari sisi pemanggil dan dari *log*. Gunakan tiga terminal, semuanya di folder `studi-kasus`.

```
# terminal 1
make -C account run

# terminal 2
make -C transfer run

# terminal 3: perintah verifikasi di bawah
```

3.5.1 Kondisi Normal

Siapkan dua rekening. Bila rekening tersebut sudah dibuat di Lab 2, `account` menjawab `409` dan rekening lama tetap dapat digunakan.

```
curl -s -X POST localhost:28081/accounts -H 'Content-Type: application/json' \
  -d '{"number":"1111111111","owner_name":"Nasabah Satu","initial_balance":"1000000"}'
curl -s -X POST localhost:28081/accounts -H 'Content-Type: application/json' \
  -d '{"number":"2222222222","owner_name":"Nasabah Dua","initial_balance":"0"}'

curl -s -w '\nHTTP %{http_code}\n' -X POST localhost:28082/transfers \
  -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"10000"}'
make breaker
```

Transfer dijawab `HTTP 201` dengan `"status":"COMPLETED"`, dan `make breaker` menampilkan:

```
{"account":"closed"}
```

3.5.2 *Timeout* dan *Circuit Breaker* Saat `account` Lambat

Aktifkan jeda 3 detik, lebih lama dari *timeout* 2 detik. Catat saldo rekening sumber, lalu kirim enam transfer berturut-turut sambil mencetak kode HTTP dan lama setiap *request*.

```
make fault-slow
curl -s localhost:28081/accounts/1111111111

for i in 1 2 3 4 5 6; do
  curl -s -o /dev/null -w '%{http_code} %{time_total}s\n' -X POST localhost:28082/
transfers \
    -H 'Content-Type: application/json' \
    -d '{"from_account":"1111111111","to_account":"2222222222","amount":"10000"}'
done
make breaker
```

Hasil yang diharapkan (angka waktu di laptop masing-masing sedikit berbeda):

```
502 2.012841s
502 2.004377s
502 2.003915s
502 2.005102s
502 2.004260s
503 0.002118s
{"account":"open"}
```

Penjelasannya:

- Setiap transfer pertama hingga kelima menunggu sekitar 2 detik walaupun jeda di `account` 3 detik. *Timeout* menghentikan panggilan setelah 2 detik, dan karena *timeout* tidak di-*retry*, setiap transfer hanya melakukan satu percobaan *debit*.
- Setiap *timeout* dihitung satu kegagalan oleh *circuit breaker*. Setelah kegagalan kelima, *circuit breaker* beralih ke *open*.
- Transfer keenam dijawab `503` seketika. `transfer` tidak mengirim *request* apa pun ke `account`.

Di terminal 2, *log* `transfer` menampilkan baris seperti berikut (atribut `time` tidak ditampilkan agar ringkas).

```
level=WARN msg="transfer failed" transfer_id=... reason="debit failed: debit 1111111111:
Post \"http://localhost:28081/accounts/1111111111/debit\": timeout exceeded"
level=WARN msg="circuit breaker state changed" target=account from=closed to=open
level=WARN msg="transfer failed" transfer_id=... reason="debit failed: debit 1111111111:
account service unavailable"
```

Periksa lagi saldo rekening sumber dengan `curl -s localhost:28081/accounts/1111111111`. Saldo tidak berubah dibandingkan sebelum `make fault-slow`. Ingat penjelasan di Langkah 6: saldo tetap utuh hanya karena *fault injector* berhenti saat `transfer` membatalkan *request*. Database yang lambat di sistem sungguhan dapat saja sudah melakukan *commit* atas *debit* tersebut, dan itulah alasan *timeout* tidak di-*retry*.

3.5.3 Perpindahan *Half-Open* ke *Closed*

Matikan *fault*, tunggu lebih dari 10 detik, lalu periksa status *circuit breaker*.

```
make fault-clear
sleep 11
make breaker
```

Hasilnya masih `{"account":"open"}`. Status *circuit breaker* di failsafe-go diperbarui hanya ketika ada panggilan masuk. Selama tidak ada transfer, `/debug/circuit-breaker` tetap menampilkan `open` walaupun 10 detik sudah lewat.

Kirim dua transfer dan periksa status setelah masing-masing transfer.

```
curl -s -o /dev/null -w '%{http_code}\n' -X POST localhost:28082/transfers \
  -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"10000"}'
make breaker

curl -s -o /dev/null -w '%{http_code}\n' -X POST localhost:28082/transfers \
  -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"10000"}'
make breaker
```

Hasil yang diharapkan:

```
201
{"account":"half-open"}
201
{"account":"closed"}
```

Debit dan *credit* berbagi satu *circuit breaker*, sehingga tiga panggilan uji di status *half-open* terdiri dari *debit* dan *credit* transfer pertama, lalu *debit* transfer kedua. Setelah transfer pertama, baru dua panggilan uji yang berhasil, sehingga statusnya masih *half-open*. *Debit* transfer kedua melengkapi panggilan uji ketiga dan mengembalikan status ke *closed*. *Log* `transfer` mencatat kedua perpindahan tersebut:

```
level=WARN msg="circuit breaker state changed" target=account from=open to=half-open
level=WARN msg="circuit breaker state changed" target=account from=half-open to=closed
```

3.5.4 *Retry* Saat `account` Sering Gagal

Aktifkan jawaban 503 untuk 20% *request*, lalu kirim sepuluh transfer.

```
make fault-error RATE=0.2
for i in $(seq 1 10); do
  curl -s -o /dev/null -w '%{http_code}\n' -X POST localhost:28082/transfers \
    -H 'Content-Type: application/json' \
    -d '{"from_account":"1111111111","to_account":"2222222222","amount":"10000"}'
done
make breaker
make fault-clear
```

Sebagian besar transfer dijawab `201` walaupun sebagian *request* ke `account` gagal. *Log* `transfer` menampilkan baris `retrying account call` untuk setiap 503 yang di-*retry*:

```
level=WARN msg="retrying account call" next_attempt=... cause="503 Service Unavailable"
```

Karena hasilnya acak, jumlah *retry* dan status akhir *circuit breaker* berbeda di setiap percobaan. Ulangi skenario tersebut dengan `RATE=0.5`. *Circuit breaker* biasanya beralih ke *open* setelah dua atau tiga transfer saja. Hal ini terjadi karena *circuit breaker* berada di dalam *retry*, sehingga setiap percobaan ulang ikut dihitung. Satu transfer yang menghabiskan seluruh *retry* menyumbang empat kegagalan sekaligus. Untuk peragaan bertahap, gunakan `RATE=0.2` atau `make fault-slow`.

Dengan `RATE` yang tinggi, sesekali muncul transfer yang dijawab `502` dengan `failure_reason` berawalan `credit failed after debit`. Pada transfer tersebut, *debit* berhasil, lalu *credit* gagal karena keempat percobaannya dijawab 503, atau karena *circuit breaker* beralih ke *open* di tengah *retry* untuk *credit*. Karena hasilnya acak, bagian berikut memperagakan kegagalan *credit* dengan skenario yang dapat diulang dengan hasil yang selalu sama.

3.5.5 *Credit* Gagal Setelah *Debit*

Kirim transfer ke rekening tujuan yang tidak ada. *Debit* berhasil, lalu *credit* ditolak karena rekening tujuan tidak ditemukan.

Pastikan *fault* sudah dimatikan (`make fault-clear`) dan sudah lewat lebih dari 10 detik sejak *circuit breaker* terakhir beralih ke *open*. Sebelum jeda tersebut lewat, *debit* langsung ditolak dengan `HTTP 503`. Setelah jeda lewat, `make breaker` masih menampilkan `open` hingga transfer berikutnya masuk, sehingga tidak perlu menunggu *output* tersebut berubah.

```
curl -s localhost:28081/accounts/1111111111
curl -s -w '\nHTTP %{http_code}\n' -X POST localhost:28082/transfers \
  -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"9999999999","amount":"10000"}'
curl -s localhost:28081/accounts/1111111111
```

Transfer dijawab `HTTP 502` dengan `"status":"FAILED"` dan `"failure_reason":"credit failed after debit: credit 9999999999: account not found"`. Saldo rekening sumber berkurang 10000. Kode `422` tidak digunakan di sini walaupun penyebabnya rekening yang tidak ditemukan, karena saldo sudah berubah dan transfer tidak boleh diulang. *Log* `transfer` mencatat kasus tersebut untuk pengembalian manual:

```
level=ERROR msg="credit failed after debit, source account needs manual reversal"
transfer_id=... from_account=1111111111 amount=10000 error="credit 9999999999: account not
found"
level=WARN msg="transfer failed" transfer_id=... reason="credit failed after debit: credit
9999999999: account not found"
```

Bandingkan dengan rekening sumber yang tidak ada, misalnya `"from_account":"8888888888"`. Transfer tersebut dijawab `HTTP 422` dengan `failure_reason debit failed: debit 8888888888: account not found`, karena *debit* ditolak dan belum ada saldo yang berubah.

> **Ringkasan *log* yang perlu diamati.** `retrying account call` (atribut `next_attempt` dan `cause`) muncul di setiap *retry*. `circuit breaker state changed` (atribut `target`, `from`, `to`) muncul di setiap perubahan status. `transfer failed` (atribut `reason`) menjelaskan penyebab transfer berstatus `FAILED`. `credit failed after debit, source account needs manual reversal` (level `ERROR`) menandai transfer yang saldo sumbernya perlu dikembalikan manual. Di sisi `account`, `fault injection changed` dan `fault injection cleared` mencatat perubahan *fault*.

3.6 Masalah Umum

- **`make -C transfer run` gagal dengan `not enough arguments in call to accountclient.New`.** *Signature* `New` kini menerima `Settings` dan *logger*. Perbarui `transfer/main.go` sesuai Langkah 9 dan pemanggilan di `client_test.go` sesuai Langkah 10.
- **`missing go.sum entry` atau `no required module provides package github.com/failsafe-go/...`.** Dependensi belum ditambahkan ke modul `transfer`. Ulangi Langkah 3 dari folder `transfer`, bukan dari `studi-kasus`.
- **`make fault-error` berhenti dengan `RATE is required`.** Target tersebut wajib diberi nilai, misalnya `make fault-error RATE=0.2`.
- **`curl` ke `/admin/fault` menjawab `404`.** *Fault injector* berada di `account` (port 28081), bukan di `transfer` (port 28082). Pastikan juga paket `fault` sudah ada (Langkah 1) dan `account` sudah dijalankan ulang setelahnya.
- **Transfer tetap lambat 3 detik setelah `make fault-slow`.** *Timeout* belum terpasang. Periksa apakah `http.Client` di `New` menggunakan `Transport: rt`, dan apakah `main.go` memanggil `accountclient.New` dengan `ProductionSettings`.
- **`make breaker` menampilkan `open` lebih dari 10 detik setelah `make fault-clear`.** Perilaku ini normal. Status *circuit breaker* baru berpindah ke *half-open* ketika transfer berikutnya masuk.

- ***Circuit breaker* beralih ke *open* jauh lebih cepat dari perkiraan.** Setiap *retry* dihitung sebagai panggilan terpisah oleh *circuit breaker*. Satu transfer dengan *retry* habis menyumbang empat kegagalan.
- **`TestNoRetryOnTimeout` gagal dengan `calls = 2` atau lebih.** `notProcessed` mengembalikan `true` untuk *timeout*. Pastikan fungsi tersebut hanya menerima error `*net.OpError` dengan `Op == "dial"` dan HTTP 503.

3.7 Eksplorasi Mandiri

1. ***Connection refused.*** Hentikan `account` dengan Ctrl+C di terminal 1, lalu kirim beberapa transfer. Amati jumlah baris `retrying account call` untuk setiap transfer, nilai atribut `cause`, dan transfer ke berapa yang membuat *circuit breaker* beralih ke *open*. Jalankan lagi `account`, lalu amati perpindahan ke *half-open* dan *closed*.
2. **Mengubah angka *policy*.** Ubah `BreakerDelay` di `ProductionSettings` menjadi 30 detik dan `AttemptTimeout` menjadi 500 ms, lalu ulangi skenario `make fault-slow`. Catat perubahan lama setiap transfer dan lama *circuit breaker* bertahan di *open*. Pertimbangkan dampaknya bagi nasabah yang menunggu jawaban transfer.
3. ***Circuit breaker* terpisah untuk *debit* dan *credit*.** Ubah `Client` agar *debit* dan *credit* masing-masing memiliki *circuit breaker* sendiri. Bandingkan hasil skenario *half-open* dengan hasil di bagian Verifikasi, lalu diskusikan: dalam kondisi apa pemisahan tersebut berguna, dan dalam kondisi apa pemisahan tersebut justru menyesatkan?
4. ***Bulkhead.*** failsafe-go juga menyediakan *bulkhead*, yang membatasi jumlah panggilan bersamaan ke satu service. Di `transfer/internal/accountclient/client.go`, impor `github.com/failsafe-go/failsafe-go/bulkhead`, lalu pasang *bulkhead* sebagai *policy* paling luar:

   ```
   bh := bulkhead.New[*http.Response](5)
   rt := failsafehttp.NewRoundTripper(nil, bh, retry, breaker,
   timeout.New[*http.Response](s.AttemptTimeout))
   ```

   Jalankan ulang `transfer`, aktifkan `make fault-slow`, lalu kirim 20 transfer sekaligus dan catat *status code* serta lama setiap *request*:

   ```
   seq 20 | xargs -P 20 -I{} curl -sS -o /dev/null -w '%{http_code} %{time_total}\n' \
     -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
     -d '{"from_account":"1111111111","to_account":"2222222222","amount":"1000"}'
   ```

   Lima *request* menunggu hingga *timeout*, sekitar 2 detik. Sisanya langsung ditolak dalam hitungan milidetik, dengan `failure_reason` yang memuat `bulkhead full`. Diskusikan: bila `transfer` juga memanggil service lain selain `account`, apa yang terjadi pada panggilan tersebut tanpa *bulkhead* ketika `account` sedang lambat? Setelah selesai, jalankan `make fault-clear` dan kembalikan `client.go` dengan `git checkout -- transfer/internal/accountclient/client.go`.
5. **Celah *credit* setelah *debit*.** Baca kasus `breaker opens between debit and credit` di `service_test.go`. Kasus tersebut menunjukkan bahwa *circuit breaker* yang beralih ke *open* setelah *debit* berhasil membuat uang keluar dari rekening sumber tanpa masuk ke rekening tujuan. Rancang urutan langkah yang diperlukan untuk mengembalikan dana tersebut secara otomatis. Lab 5 menyelesaikan celah ini dengan pola *saga* dan kompensasi.

3.8 *Checkpoint*

Kondisi kode di akhir lab ini tersimpan di *tag* `lab-03`. Bandingkan hasil kerja dengan *checkpoint* tersebut:

```
git diff lab-03 -- .
```

Perbedaan pada `README.md` dan `go.work.sum` memang wajar. `README.md` di *checkpoint* memuat bagian baru tentang rekening contoh dan *resiliency*, sedangkan isi `go.work.sum` bergantung pada dependensi yang sudah di-*download* di laptop masing-masing. Keduanya dapat diambil dari *checkpoint*:

```
git checkout lab-03 -- README.md go.work.sum
```

Peserta yang tertinggal dapat langsung melanjutkan dari *checkpoint* tersebut ke Lab 4:

```
git checkout lab-03
```

4 Lab 4: gRPC untuk Komunikasi Antar-*Service*

4.1 Tujuan

Setelah menyelesaikan lab ini, peserta mampu:

- menulis kontrak *service* dalam *protocol buffers* (*protobuf*) dan menghasilkan kode Go darinya dengan `protoc`, `protoc-gen-go` serta `protoc-gen-go-grpc`;
- menjalankan server gRPC di *service* `account` berdampingan dengan REST API yang sudah ada, lengkap dengan *health service* dan *reflection*;
- memetakan *error* domain ke kode status gRPC (`NotFound`, `FailedPrecondition`, `InvalidArgument`) sehingga pemanggil dapat membedakan penolakan bisnis dari gangguan infrastruktur;
- memasang *unary server interceptor* untuk *logging* dan *fault injection*;
- memindahkan *policy* failsafe-go dari Lab 3 ke *unary client interceptor* di *service* `transfer`;
- menguji server dan *client* gRPC tanpa membuka port jaringan dengan `bufconn`.

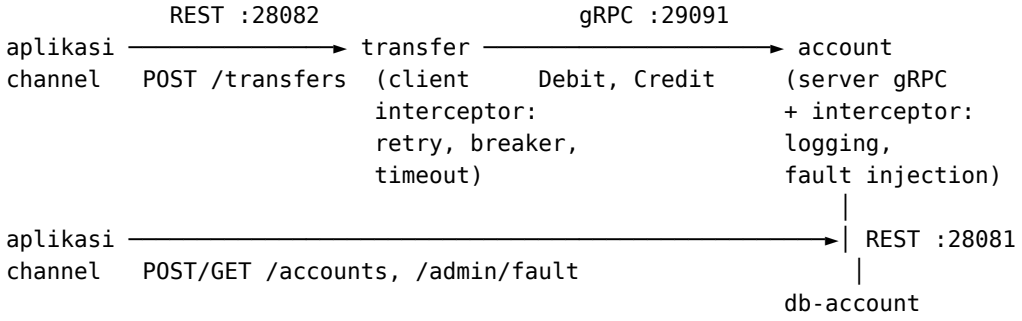
4.2 Gambaran

Hingga Lab 3, `transfer` memanggil `account` melalui REST: `POST /accounts/{number}/debit` dan `/credit`. REST API tersebut juga terbuka untuk aplikasi *channel* (aplikasi *mobile*, *web*, mitra). Akibatnya, aplikasi *channel* secara teknis dapat mengubah saldo tanpa melalui transfer.

Lab ini memisahkan kedua jenis pemanggil:

- **REST untuk *channel*.** Aplikasi *channel* tetap menggunakan REST API `account` (membuat dan membaca rekening). Data JSON yang dikirim melalui HTTP mudah diolah oleh *browser*, aplikasi *mobile* maupun sistem mitra.
- **gRPC untuk komunikasi internal.** *Service* lain memanggil `GetAccount`, `Debit` dan `Credit` melalui gRPC. Kontraknya ditulis dalam file `.proto`, kode *client* dan server dihasilkan dari kontrak tersebut, dan *payload*-nya dikodekan secara biner (*protobuf*) di atas HTTP/2. Perubahan kontrak terlihat sebagai *diff* pada file `.proto` dan pada kode hasil *generate*, sehingga ketidakcocokan antara *client* dan server terdeteksi saat kompilasi, sebelum *service* dijalankan.

Endpoint REST *debit* dan *credit* dihapus. Sejak lab ini, perubahan saldo hanya dapat dilakukan melalui gRPC.



Policy resiliency dari Lab 3 tidak berubah: *timeout* 2 detik per percobaan, *retry* maksimal 3 kali dengan *backoff* 100 ms hingga 1 detik ditambah *jitter*, dan *circuit breaker* yang terbuka setelah 5 kegagalan dari 10 panggilan terakhir. Yang berubah adalah tempat pemasangannya. Di Lab 3 *policy* tersebut dipasang sebagai *round tripper* HTTP. Di lab ini *policy* yang sama dipasang sebagai *interceptor* gRPC, dan aturan *retry* serta penghitungan kegagalan kini membaca kode status gRPC sebagai pengganti kode status HTTP.

Struktur yang bertambah atau berubah di lab ini:

```
studi-kasus/
├── go.work                     # + ./proto
├── Makefile                    # + target proto, test ikut memeriksa proto
├── proto/                      # modul baru
│   ├── go.mod
│   ├── Makefile                # generate, test
│   ├── account/v1/account.proto
│   └── gen/account/v1/         # hasil generate, ikut di-commit
├── account/
│   ├── config.go               # + GRPC_ADDR
│   ├── main.go                 # + server gRPC, health, reflection
│   └── internal/
│       ├── account/account.go  # + validasi format dan batas nominal
│       ├── account/grpc.go     # baru: implementasi AccountService
│       ├── account/http.go     # - endpoint debit/credit
│       ├── fault/fault.go      # + UnaryServerInterceptor
│       └── interceptor/logging.go   # baru
└── transfer/
    ├── config.go               # ACCOUNT_GRPC_ADDR menggantikan ACCOUNT_SERVICE_URL
    ├── main.go
    └── internal/
        ├── accountclient/client.go   # client gRPC + failsafegrpc
        └── transfer/transfer.go      # + batas 2 digit desimal dan 10^17 untuk amount
```

4.2.1 Alasan Memilih gRPC untuk Panggilan Internal

Pertimbangan	Keterangan
Kebutuhan	`transfer` memanggil `account` untuk mengubah saldo. Kontrak panggilan ini harus tegas dan diperiksa saat kompilasi, tidak boleh terbuka untuk aplikasi *channel*, dan batas waktu dari pemanggil perlu diketahui oleh `account`
Opsi yang dipertimbangkan	REST dengan JSON dan dokumentasi OpenAPI (seperti Lab 2 dan 3). gRPC dengan kontrak *protobuf*. *Messaging* tidak termasuk, karena `transfer` membutuhkan hasil *debit* saat itu juga
Pilihan di studi kasus	gRPC untuk panggilan antar-*service* internal. REST tetap digunakan untuk API yang dipanggil aplikasi *channel*
Yang diperoleh	Kode *client* dan server dihasilkan dari file `.proto`, sehingga ketidakcocokan kontrak terdeteksi saat kompilasi. HTTP/2 menjalankan banyak panggilan sekaligus di satu koneksi (*multiplexing*). *Deadline* pemanggil ikut terkirim, sehingga *context* di server dibatalkan saat batas waktu terlewati, dan *query* database yang menggunakan *context* tersebut ikut dihentikan. *Payload* biner lebih kecil dan lebih cepat diproses dibandingkan JSON. *Interceptor* menjadi satu tempat untuk *logging*, *resiliency*, dan autentikasi
Yang dikorbankan	Panggilan tidak dapat dicoba langsung dengan `curl` atau *browser*, sehingga diperlukan *client* gRPC seperti `grpcurl`, dibantu *reflection*. Koneksi HTTP/2 bersifat persisten, sehingga Service Kubernetes tidak menyebar

Pertimbangan	Keterangan
	panggilan ke semua *pod* tanpa *load balancing* di sisi klien atau *proxy* L7 (Lab 9). Perubahan kontrak wajib kompatibel dengan versi lama, dan nomor *field* tidak boleh digunakan ulang. `protoc` dan kode hasil *generate* harus dikelola. Tim memelihara dua gaya API: REST dan gRPC

4.3 Titik Awal

Lab ini melanjutkan kondisi kode di akhir Lab 3. Peserta yang menyelesaikan Lab 3 sendiri dapat langsung melanjutkan dari kode miliknya. Peserta yang tertinggal memulai dari *checkpoint* `lab-03`:

```
cd studi-kasus
git switch -c kerja-lab-04 lab-03
```

Pastikan `protoc` dan kedua *plugin* Go sudah terpasang (bagian dari panduan persiapan). Kedua *plugin* berada di `$(go env GOPATH)/bin`, sehingga folder tersebut perlu ada di `PATH`:

```
export PATH="$PATH:$(go env GOPATH)/bin"
protoc --version
protoc-gen-go --version        # protoc-gen-go v1.36.12
protoc-gen-go-grpc --version   # protoc-gen-go-grpc 1.6.2
```

Bila salah satu *plugin* belum ada, pasang dengan versi yang sama dengan *checkpoint*:

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@v1.36.12
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@v1.6.2
```

Jalankan infrastruktur lab (PostgreSQL) bila belum berjalan:

```
make infra-up
```

4.4 Langkah

4.4.1 Modul `proto` dan Kontrak `AccountService`

Kontrak gRPC ditempatkan di modul Go tersendiri, `studi-kasus/proto`. Dengan modul terpisah, `account` (server) dan `transfer` (*client*) mengimpor kode yang persis sama, dan tidak satu pun dari keduanya menjadi pemilik kontrak.

Buat folder dan file `proto/account/v1/account.proto`. Dengan segmen `v1` pada *path* dan nama *package* `account.v1`, perubahan yang tidak kompatibel kelak dapat ditulis sebagai `account.v2`, sementara `v1` tetap dilayani hingga semua *client* pindah.

```
// Internal contract between services. Channel-facing clients (mobile, web,
// partners) keep using the REST API; gRPC is for service-to-service calls.
syntax = "proto3";

package account.v1;

import "google/protobuf/timestamp.proto";

option go_package = "github.com/artivisi/training-microservices-2026-01/studi-kasus/proto/
gen/account/v1;accountv1";

// AccountService exposes balance reads and mutations to other services.
//
// Errors are gRPC status codes:
//   NOT_FOUND           the account does not exist
```

```
//   FAILED_PRECONDITION the balance is too low for a debit
//   INVALID_ARGUMENT    a field is malformed; the message names it
//   UNAVAILABLE         the request was refused before it was processed
//   DEADLINE_EXCEEDED   the caller's deadline passed while the request was
//                       running; the mutation may or may not have been applied
//   CANCELLED           the same, after the caller cancelled
service AccountService {
  rpc GetAccount(GetAccountRequest) returns (GetAccountResponse);
  rpc Debit(DebitRequest) returns (DebitResponse);
  rpc Credit(CreditRequest) returns (CreditResponse);
}

// Every RPC has its own request and response message, even when two of them
// look alike today: each can then gain fields without touching the others.

message GetAccountRequest {
  string number = 1;
}

message GetAccountResponse {
  Account account = 1;
}

// Amounts are decimal strings ("150000.50"), not double: binary floating point
// cannot represent most decimal fractions exactly (0.1 + 0.2 != 0.3), and money
// must not drift by rounding. Both sides parse them with shopspring/decimal.
// Format: digits with at most 2 decimal places ("150000", "150000.5",
// "150000.50"), no sign, no exponent; anything else is INVALID_ARGUMENT, never
// rounded.
message DebitRequest {
  string number = 1;
  string amount = 2;
  // reference ties the mutation to the business transaction, e.g. a transfer ID.
  string reference = 3;
}

message DebitResponse {
  Account account = 1;
}

message CreditRequest {
  string number = 1;
  string amount = 2;
  string reference = 3;
}

message CreditResponse {
  Account account = 1;
}

message Account {
  string number = 1;
  string owner_name = 2;
  // balance is a decimal string, like the amounts above.
  string balance = 3;
  google.protobuf.Timestamp created_at = 4;
}
```

Beberapa keputusan dalam kontrak tersebut perlu diperhatikan:

- **Nominal sebagai *string* desimal.** *Protobuf* memiliki tipe `double`, tetapi bilangan *floating point* biner tidak dapat menyimpan sebagian besar pecahan desimal secara tepat. Nilai `0.1 + 0.2` pada `double` menghasilkan `0.30000000000000004`. Untuk uang, selisih pembulatan sekecil apa pun tidak dapat diterima. Kedua sisi mengirim nominal sebagai teks (`"150000.50"`) dan mengolahnya dengan `shopspring/decimal`, sama seperti kolom `numeric(19,2)` di database. Formatnya dibatasi: hanya angka dengan paling banyak 2 digit desimal, tanpa tanda minus dan tanpa notasi eksponen (`"1e3"`). Format lain langsung ditolak dengan `INVALID_ARGUMENT` tanpa dibulatkan.
- **Nomor *field* adalah bagian dari kontrak.** Pada format biner, setiap *field* dikenali dari nomornya (`= 1`, `= 2`), bukan dari namanya. Nomor yang sudah digunakan tidak boleh diganti atau digunakan ulang untuk *field* lain. *Field* baru selalu mendapat nomor baru.
- **Satu *message request* dan satu *message response* untuk setiap RPC.** `DebitRequest` dan `CreditRequest` saat ini sama persis. Dengan *message* terpisah, salah satunya dapat ditambah *field* kelak tanpa mengubah RPC lainnya.
- **Kode *error* ditulis di komentar kontrak.** Pemanggil mengambil keputusan berdasarkan kode status (misalnya boleh *retry* atau tidak), sehingga daftar kode tersebut termasuk kontrak, sama pentingnya dengan *field*. `DEADLINE_EXCEEDED` dan `CANCELLED` juga dicantumkan karena pemanggil perlu tahu bahwa pada kedua kode tersebut mutasi saldo mungkin sudah terjadi, mungkin juga belum.

Opsi `go_package` menentukan *import path* kode Go hasil *generate* (`.../proto/gen/account/v1`) dan nama *package*-nya (`accountv1`, setelah titik koma).

Selanjutnya buat `proto/Makefile`:

```
# Generated code is committed: services build without protoc installed, and a
# contract change shows up in review as a diff of the generated API too.
MODULE := github.com/artivisi/training-microservices-2026-01/studi-kasus/proto
PROTO_FILES := $(shell find . -name '*.proto' -not -path './gen/*')
GOBIN := $(shell go env GOPATH)/bin

.PHONY: generate test

generate:
  PATH="$$PATH:$(GOBIN)" protoc -I . \
    --go_out=. --go_opt=module=$(MODULE) \
    --go-grpc_out=. --go-grpc_opt=module=$(MODULE) \
    $(PROTO_FILES)

# Nothing to test in generated code; vet catches a broken generation.
test:
  go vet ./...
```

Opsi `--go_opt=module=...` memotong awalan *module path* dari `go_package`, sehingga file hasil *generate* ditulis ke `gen/account/v1/` di dalam modul `proto` tanpa struktur folder `github.com/...` yang panjang.

Tambahkan dua target di `studi-kasus/Makefile`: `test` ikut memeriksa kode hasil *generate*, dan `proto` menjadi jalan pintas untuk *generate* ulang dari folder `studi-kasus`.

```
.PHONY: infra-up infra-down infra-reset test proto fault-slow fault-error fault-clear
fault-status breaker

test:
  $(MAKE) -C proto test
  @for s in $(SERVICES); do $(MAKE) -C $$s test || exit 1; done

# Regenerates proto/gen after a .proto change (Lab 4). Needs protoc,
# protoc-gen-go and protoc-gen-go-grpc.
```

```
proto:
  $(MAKE) -C proto generate
```

4.4.2 Kode Go Hasil *Generate*

Jalankan `protoc`, lalu jadikan folder `proto` modul Go dan daftarkan modul tersebut ke *workspace*:

```
make -C proto generate
cd proto
go mod init github.com/artivisi/training-microservices-2026-01/studi-kasus/proto
go get google.golang.org/grpc@v1.80.0 google.golang.org/protobuf@v1.36.12
go mod tidy
cd ..
go work use ./proto
```

Perintah `go work use` menambahkan baris `./proto` ke `go.work`, sehingga *workspace* kini berisi `account`, `proto` dan `transfer`.

`protoc` menghasilkan dua file di `proto/gen/account/v1/`:

File	Isi
`account.pb.go`	*Struct* Go untuk setiap *message* (`DebitRequest`, `Account`, dan seterusnya) beserta *getter* seperti `GetNumber()`. Dihasilkan oleh `protoc-gen-go`.
`account_grpc.pb.go`	*Interface* `AccountServiceClient` dan konstruktornya `NewAccountServiceClient`, *interface* `AccountServiceServer`, *struct* `UnimplementedAccountServiceServer`, fungsi `RegisterAccountServiceServer`, serta `AccountService_ServiceDesc` yang memuat nama *service*. Dihasilkan oleh `protoc-gen-go-grpc`.

Kedua file tersebut ikut di-*commit*. Dengan begitu, *service* dapat di-*build* (termasuk di *pipeline* CI dan di Docker pada Lab 9) tanpa `protoc`, dan setiap perubahan kontrak terlihat di *review* sebagai *diff* API Go yang dihasilkannya. File hasil *generate* tidak boleh diubah secara manual: perubahan selalu dimulai dari file `.proto`, lalu `make proto` dijalankan ulang.

> Bagian atas file hasil *generate* memuat komentar pembuka file `.proto` serta versi `protoc` yang digunakan (misalnya `protoc v7.36.2`). Bila versi `protoc` di laptop peserta berbeda dari versi yang digunakan untuk *checkpoint*, baris versi tersebut ikut berbeda saat dibandingkan dengan `git diff lab-04`. Perbedaan tersebut tidak memengaruhi kode.

4.4.3 Dependensi di *Service* `account`

Modul `account` kini mengimpor modul `proto`. Di dalam *workspace*, Go sudah menemukan modul tersebut lewat `go.work`. Namun `go mod tidy` dan *build* Docker pada Lab 9 bekerja per modul, tanpa `go.work`, sehingga letak modul `proto` perlu dicatat di `go.mod` milik `account` melalui direktif `replace`:

```
cd account
go mod edit \
  -require=github.com/artivisi/training-microservices-2026-01/studi-kasus/proto@v0.0.0 \
  -replace=github.com/artivisi/training-microservices-2026-01/studi-kasus/proto=../proto
cd ..
```

Versi `v0.0.0` hanya *placeholder*: dengan `replace`, Go selalu mengambil isi folder `../proto`. Jalankan `go mod tidy` di folder `account` setelah file `grpc.go` dan `main.go` pada langkah berikutnya selesai ditulis, agar `google.golang.org/grpc` ikut tercatat.

4.4.4 Implementasi `AccountService` di `account`

Buat file `account/internal/account/grpc.go`. File ini berada di *package* yang sama dengan *handler* REST (`http.go`) dan menggunakan `Store` yang sama. Perbedaannya hanya pada *transport* dan cara *error* dilaporkan (kode status gRPC sebagai pengganti kode HTTP).

```
package account

import (
  "context"
  "errors"
  "fmt"
  "log/slog"
  "maps"
  "slices"
  "strings"

  "google.golang.org/grpc/codes"
  "google.golang.org/grpc/status"
  "google.golang.org/protobuf/types/known/timestamppb"

  accountv1 "github.com/artivisi/training-microservices-2026-01/studi-kasus/proto/gen/
account/v1"
)

// GRPCServer is the internal API for other services. It uses the same Store as
// the REST handler; only the transport and the error vocabulary differ.
type GRPCServer struct {
  accountv1.UnimplementedAccountServiceServer
  store Store
  log   *slog.Logger
}

func NewGRPCServer(store Store, log *slog.Logger) *GRPCServer {
  return &GRPCServer{store: store, log: log}
}

func (s *GRPCServer) GetAccount(ctx context.Context, req *accountv1.GetAccountRequest)
(*accountv1.GetAccountResponse, error) {
  a, err := s.store.Get(ctx, req.GetNumber())
  if err != nil {
    return nil, s.toStatus(err)
  }
  return &accountv1.GetAccountResponse{Account: toProto(a)}, nil
}

func (s *GRPCServer) Debit(ctx context.Context, req *accountv1.DebitRequest)
(*accountv1.DebitResponse, error) {
  a, err := s.mutate(ctx, Debit, req.GetNumber(), req.GetAmount(), req.GetReference())
  if err != nil {
    return nil, err
  }
  return &accountv1.DebitResponse{Account: toProto(a)}, nil
}

func (s *GRPCServer) Credit(ctx context.Context, req *accountv1.CreditRequest)
(*accountv1.CreditResponse, error) {
  a, err := s.mutate(ctx, Credit, req.GetNumber(), req.GetAmount(), req.GetReference())
  if err != nil {
    return nil, err
```

```
  }
  return &accountv1.CreditResponse{Account: toProto(a)}, nil
}

func (s *GRPCServer) mutate(ctx context.Context, t MutationType, number, amount, reference
string) (Account, error) {
  parsed, err := parseAmount(amount)
  if err != nil {
    return Account{}, s.toStatus(err)
  }
  req := MutationRequest{Amount: parsed, Reference: reference}
  if err := req.Validate(); err != nil {
    return Account{}, s.toStatus(err)
  }
  a, err := s.store.Mutate(ctx, number, t, req)
  if err != nil {
    return Account{}, s.toStatus(err)
  }
  s.log.Info("account mutated", "number", a.Number, "type", t, "amount", req.Amount,
"reference", req.Reference)
  return a, nil
}

// toStatus maps domain errors to gRPC codes. Callers branch on the code, so the
// mapping is part of the contract (see account.proto):
//   - NotFound and FailedPrecondition are business answers from a healthy
//     service; clients must not retry them or count them against the breaker.
//   - Canceled / DeadlineExceeded: the caller gave up (its deadline passed or it
//     hung up) while the store was still working. Not our failure, so it is
//     not logged as one; the caller already has its own error.
//   - Anything unexpected becomes Internal, with the detail only in our log.
func (s *GRPCServer) toStatus(err error) error {
  var v ValidationError
  switch {
  case errors.Is(err, context.Canceled), errors.Is(err, context.DeadlineExceeded):
    st := status.FromContextError(err)
    s.log.Warn("caller gave up before the request finished", "code", st.Code().String(),
"error", err)
    return st.Err()
  case errors.As(err, &v):
    return status.Error(codes.InvalidArgument, v.describe())
  case errors.Is(err, ErrNotFound):
    return status.Error(codes.NotFound, err.Error())
  case errors.Is(err, ErrInsufficientFunds):
    return status.Error(codes.FailedPrecondition, err.Error())
  default:
    s.log.Error("grpc request failed", "error", err)
    return status.Error(codes.Internal, "internal error")
  }
}

// describe renders "amount: must be ...; reference: must be ..." in a stable order.
func (v ValidationError) describe() string {
  var parts []string
  for _, field := range slices.Sorted(maps.Keys(v)) {
    parts = append(parts, fmt.Sprintf("%s: %s", field, v[field]))
  }
  return strings.Join(parts, "; ")
}
```

```
func toProto(a Account) *accountv1.Account {
  return &accountv1.Account{
    Number:    a.Number,
    OwnerName: a.OwnerName,
    Balance:   a.Balance.StringFixed(2),
    CreatedAt: timestamppb.New(a.CreatedAt),
  }
}
```

Hal-hal yang perlu diperhatikan pada kode tersebut:

- **`UnimplementedAccountServiceServer` di-*embed* di dalam *struct*.** Bila kontrak kelak mendapat RPC baru, server lama tetap terkompilasi dan menjawab RPC baru tersebut dengan kode `Unimplemented`, sampai implementasinya ditulis.
- **Pemetaan *error* ke kode status di `toStatus`.** Tabel berikut menjadi acuan bagi pemanggil.

Kode gRPC	Penyebab	Padanan REST di Lab 2
`InvalidArgument`	Format nominal salah (lebih dari 2 digit desimal, tanda minus, notasi eksponen), nominal nol, nominal 10^17 ke atas, `reference` kosong	HTTP 400
`NotFound`	Nomor rekening tidak terdaftar	HTTP 404
`FailedPrecondition`	Saldo tidak cukup untuk *debit*	HTTP 422
`Canceled`, `DeadlineExceeded`	Pemanggil berhenti menunggu (*deadline* habis atau RPC dibatalkan) ketika database masih bekerja. Dicatat sebagai WARN, bukan ERROR	Tidak ada
`Internal`	*Error* lain, misalnya database bermasalah. Rinciannya hanya dicatat di *log* `account`	HTTP 500

`NotFound`, `FailedPrecondition` dan `InvalidArgument` merupakan jawaban bisnis dari *service* yang sehat. Di sisi `transfer`, kode-kode tersebut tidak di-*retry* dan tidak dihitung sebagai kegagalan oleh *circuit breaker*. Kode `Internal` hanya membawa teks `internal error` agar detail internal (nama tabel, alamat database) tidak bocor ke pemanggil.

Kasus `Canceled` dan `DeadlineExceeded` diperiksa paling awal di `toStatus`. Tanpa kasus tersebut, *error* dari pgx yang membungkus `context.Canceled` masuk ke cabang `default`: `account` mencatatnya sebagai ERROR dan menjawab `Internal`, padahal database sehat dan yang terjadi hanyalah pemanggil berhenti menunggu. `status.FromContextError` mengubah `context.Canceled` menjadi `Canceled` dan `context.DeadlineExceeded` menjadi `DeadlineExceeded`, sehingga arti *error* tersebut tidak hilang. Bagi pemanggil, kedua kode tersebut berarti mutasi mungkin sudah terjadi, mungkin juga belum.

4.4.5 Validasi Format Nominal

Kolom saldo dan nominal di database bertipe `numeric(19,2)`: paling banyak 17 digit sebelum titik desimal dan 2 digit sesudahnya. PostgreSQL membulatkan digit desimal ketiga tanpa pesan apa pun, sehingga nominal `100.005` tersimpan sebagai `100.01`. Nilai yang dikirim pemanggil dan nilai yang tersimpan menjadi berbeda. Karena itu nominal yang tidak muat di `numeric(19,2)` ditolak sebelum mencapai database.

Tambahkan kode berikut di `account/internal/account/account.go`, di bawah `numberPattern`:

```
// Balances and amounts are numeric(19,2) in the database: at most 17 digits
// before the decimal point and 2 after. PostgreSQL rounds extra decimal places
// without complaint (100.005 becomes 100.01), so they are rejected here instead.
var maxAmount = decimal.New(1, 17)

// amountProblem returns what is wrong with d as a numeric(19,2) value, or "".
// Sign is checked by the caller: a balance may be zero, an amount may not.
func amountProblem(d decimal.Decimal) string {
  switch {
  case d.Exponent() < -2:
    return "must have at most 2 decimal places"
  case d.Abs().GreaterThanOrEqual(maxAmount):
    return "must be less than 10^17"
  }
  return ""
}

// amountPattern is the wire format of an amount on the gRPC contract: plain
// digits with an optional fraction of 1 or 2 digits. No sign, no exponent ("1e3")
// and no grouping, so what the caller wrote is exactly what gets stored.
var amountPattern = regexp.MustCompile(`^[0-9]+(\.[0-9]{1,2})?$`)

// parseAmount parses a gRPC amount string. Range and sign are left to
// MutationRequest.Validate.
func parseAmount(s string) (decimal.Decimal, error) {
  if !amountPattern.MatchString(s) {
    return decimal.Decimal{}, ValidationError{"amount": "must be digits with at most 2
decimal places, e.g. 150000.50"}
  }
  return decimal.NewFromString(s)
}
```

Fungsi `mutate` di `grpc.go` memanggil `parseAmount` sebelum mengolah nominal. Pola *regex* tersebut menolak tanda minus, notasi eksponen dan spasi sekaligus, sehingga teks yang dikirim pemanggil persis sama dengan nilai yang disimpan. *Error*-nya berupa `ValidationError`, sehingga `toStatus` mengubahnya menjadi `InvalidArgument` dengan pesan `amount: must be digits with at most 2 decimal places, e.g. 150000.50`.

`amountProblem` juga dipasang di kedua fungsi `Validate`. Untuk `POST /accounts`, saldo awal dengan lebih dari 2 digit desimal atau bernilai 10^17 ke atas dijawab HTTP 400. *Request* JSON tidak melewati `parseAmount`, sehingga angka seperti `1e-3` tetap dapat dibaca, lalu ditolak oleh pemeriksaan jumlah digit desimal ini:

```
  // di dalam CreateRequest.Validate
  if r.InitialBalance.IsNegative() {
    v["initial_balance"] = "must not be negative"
  } else if p := amountProblem(r.InitialBalance); p != "" {
    v["initial_balance"] = p
  }

  // di dalam MutationRequest.Validate
  if !r.Amount.IsPositive() {
    v["amount"] = "must be greater than zero"
  } else if p := amountProblem(r.Amount); p != "" {
    v["amount"] = p
  }
```

Aturan yang sama perlu dipasang di `transfer`. Tabel `transfer` juga memiliki kolom `amount` bertipe `numeric(19,2)`. Transfer `100.005` disimpan sebagai `100.01`, lalu nilai hasil pembulatan tersebut yang

dikirim sebagai *debit*. Akibatnya, `account` hanya memeriksa nilai yang sudah dibulatkan, bukan nilai aslinya. Di `transfer/internal/transfer/transfer.go`, tambahkan `maxAmount` di bawah `numberPattern`, lalu ganti pemeriksaan `amount` di `CreateRequest.Validate`:

```
// The amount column is numeric(19,2), like the account service's. PostgreSQL
// would round a third decimal place without complaint (100.005 becomes 100.01)
// and the rounded value would then be debited, so it is rejected here instead.
var maxAmount = decimal.New(1, 17)

  // di dalam CreateRequest.Validate
  switch {
  case !r.Amount.IsPositive():
    v["amount"] = "must be greater than zero"
  case r.Amount.Exponent() < -2:
    v["amount"] = "must have at most 2 decimal places"
  case r.Amount.GreaterThanOrEqual(maxAmount):
    v["amount"] = "must be less than 10^17"
  }
```

Test `TestValidateAmount` di `transfer/internal/transfer/service_test.go` memeriksa aturan tersebut: `100.5` dan `100.50` diterima, `100.005` dan `1e-3` ditolak karena lebih dari 2 digit desimal, dan `100000000000000000` ditolak karena mencapai 10^17. Salin dari *checkpoint* bila tidak ingin menulisnya sendiri:

```
git show lab-04:studi-kasus/transfer/internal/transfer/service_test.go \
  > transfer/internal/transfer/service_test.go
```

Fungsi `Store.Mutate` dari Lab 2 digunakan kembali tanpa perubahan. *Struct* `MutationRequest` di `account.go` kini hanya diisi dari gRPC, sehingga *tag* JSON-nya dihapus:

```
// MutationRequest is a debit or credit, received over gRPC. Reference ties the
// mutation to the business transaction that caused it, e.g. a transfer ID.
type MutationRequest struct {
  Amount    decimal.Decimal
  Reference string
}
```

4.4.6 *Interceptor Logging*

Interceptor di gRPC berperan seperti *middleware* di `net/http`, yaitu fungsi yang membungkus setiap pemanggilan RPC sebelum dan sesudah *handler* berjalan. *Unary interceptor* berlaku untuk RPC dengan satu *request* dan satu *response*, yaitu jenis RPC yang digunakan di seluruh studi kasus ini.

Interceptor pertama mencatat setiap RPC beserta kode status dan durasinya, seperti *access log* pada server HTTP. Buat file `account/internal/interceptor/logging.go`:

```
// Package interceptor holds cross-cutting gRPC server middleware.
package interceptor

import (
  "context"
  "log/slog"
  "time"

  "google.golang.org/grpc"
  "google.golang.org/grpc/codes"
  "google.golang.org/grpc/status"
)

// Logging logs every unary call with its status code and duration: the gRPC
// equivalent of an HTTP access log. Business rejections (NotFound,
```

```
// FailedPrecondition, InvalidArgument) are logged at INFO because they are
// normal answers; everything else that failed is a WARN worth looking at.
func Logging(log *slog.Logger) grpc.UnaryServerInterceptor {
  return func(ctx context.Context, req any, info *grpc.UnaryServerInfo, handler
grpc.UnaryHandler) (any, error) {
    start := time.Now()
    resp, err := handler(ctx, req)
    code := status.Code(err)
    level := slog.LevelInfo
    switch code {
    case codes.OK, codes.NotFound, codes.FailedPrecondition, codes.InvalidArgument:
    default:
      level = slog.LevelWarn
    }
    log.Log(ctx, level, "grpc request", "method", info.FullMethod, "code", code.String(),
      "duration_ms", time.Since(start).Milliseconds())
    return resp, err
  }
}
```

Parameter `info.FullMethod` berisi nama lengkap RPC, misalnya `/account.v1.AccountService/Debit`. Parameter `handler` berisi langkah berikutnya dalam *chain*, yaitu *interceptor* lain atau implementasi `GRPCServer`.

4.4.7 *Fault Injection* untuk gRPC

Di Lab 3, `PUT /admin/fault` memperlambat atau menggagalkan REST API `account`. Agar latihan *resiliency* tetap berjalan setelah `transfer` beralih ke gRPC, *injector* yang sama perlu bekerja juga untuk gRPC. Satu konfigurasi berlaku untuk kedua *transport*, sehingga `make fault-slow` dan `make fault-error` tidak perlu diubah.

Di `account/internal/fault/fault.go`, logika jeda dan *error* dipindahkan dari `Wrap` ke fungsi bersama `inject`. Fungsi `Wrap` dan *interceptor* baru sama-sama memanggil fungsi tersebut. Tambahkan *import* berikut:

```
import (
  "context"
  // ... import yang sudah ada ...

  "google.golang.org/grpc"
  "google.golang.org/grpc/codes"
  "google.golang.org/grpc/status"
)
```

Ganti fungsi `Wrap`, lalu tambahkan `UnaryServerInterceptor` dan `inject` di bawahnya:

```
// Wrap applies the current fault configuration to every request of next.
func (i *Injector) Wrap(next http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    if i.inject(r.Context()) {
      w.Header().Set("Content-Type", "application/json")
      w.WriteHeader(http.StatusServiceUnavailable)
      _, _ = w.Write([]byte(`{"error":"injected fault"}` + "\n"))
      return
    }
    if r.Context().Err() != nil { // caller gave up during the delay; do not process
      return
    }
    next.ServeHTTP(w, r)
  })
}
```

```
// UnaryServerInterceptor is the gRPC counterpart of Wrap, driven by the same
// configuration: one PUT /admin/fault degrades REST and gRPC together. The
// injected error is Unavailable, the gRPC equivalent of HTTP 503 ("refused
// before processing"), which the transfer client is allowed to retry.
// skip names full methods that must stay fast, e.g. the health service.
func (i *Injector) UnaryServerInterceptor(skip func(fullMethod string) bool)
grpc.UnaryServerInterceptor {
  return func(ctx context.Context, req any, info *grpc.UnaryServerInfo, handler
grpc.UnaryHandler) (any, error) {
    if skip(info.FullMethod) {
      return handler(ctx, req)
    }
    if i.inject(ctx) {
      return nil, status.Error(codes.Unavailable, "injected fault")
    }
    if err := ctx.Err(); err != nil { // caller gave up during the delay; do not process
      return nil, status.FromContextError(err).Err()
    }
    return handler(ctx, req)
  }
}

// inject sleeps for the configured delay (or until ctx ends) and reports whether
// this request must be answered with an injected error.
func (i *Injector) inject(ctx context.Context) bool {
  cfg := i.cfg.Load()
  if cfg.DelayMS > 0 {
    select {
    case <-time.After(time.Duration(cfg.DelayMS) * time.Millisecond):
    case <-ctx.Done():
      return false
    }
  }
  return cfg.ErrorRate > 0 && rand.Float64() < cfg.ErrorRate
}
```

Ubah juga komentar *field* `ErrorRate` pada *struct* `Config` menjadi "answered with 503 (HTTP) or Unavailable (gRPC) without being processed".

Dua perilaku pada *interceptor* tersebut menentukan hasil latihan di bagian Verifikasi:

- Kegagalan hasil *fault injection* dijawab `codes.Unavailable` tanpa menjalankan *handler*. Kode tersebut setara dengan HTTP 503 di Lab 3: *request* ditolak sebelum diproses, sehingga `transfer` boleh mengulanginya.
- Bila pemanggil berhenti menunggu selama jeda berlangsung (misalnya karena *timeout* 2 detik di `transfer`), *handler* tidak dijalankan dan *interceptor* mengembalikan kode dari *context* yang berakhir, yaitu `Canceled` atau `DeadlineExceeded`.

4.4.8 Server gRPC di `main.go` `account`

Tambahkan `GRPC_ADDR` di `account/config.go`. Seperti variabel lain, `GRPC_ADDR` wajib diisi: bila kosong, *service* berhenti saat *startup* dengan pesan yang menyebut nama variabelnya.

```
type Config struct {
  HTTPAddr    string
  GRPCAddr    string
  DatabaseURL string
}

// di dalam LoadConfig
```

```
  cfg := Config{
    HTTPAddr:    get("HTTP_ADDR"),
    GRPCAddr:    get("GRPC_ADDR"),
    DatabaseURL: get("DATABASE_URL"),
  }
```

Tambahkan nilainya di target `run` pada `account/Makefile`:

```
run:
  HTTP_ADDR=:28081 \
  GRPC_ADDR=:29091 \
  DATABASE_URL=postgres://account:account123@localhost:15432/account \
  go run .
```

Di `account/main.go`, tambahkan *import* berikut:

```
  "fmt"
  "net"
  "strings"

  "google.golang.org/grpc"
  "google.golang.org/grpc/health"
  healthpb "google.golang.org/grpc/health/grpc_health_v1"
  "google.golang.org/grpc/reflection"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/account/internal/
interceptor"
  accountv1 "github.com/artivisi/training-microservices-2026-01/studi-kasus/proto/gen/
account/v1"
```

Sisipkan blok berikut setelah deklarasi `srv := &http.Server{...}` dan sebelum *goroutine* yang menjalankan server HTTP:

```
  // gRPC: the internal API (GetAccount, Debit, Credit) for other services.
  // Interceptors run in the order given: logging sees the code that the fault
  // injector produced, so injected failures show up in the log like real ones.
  // Health and reflection are excluded from faults, like the HTTP probes.
  grpcSrv := grpc.NewServer(grpc.ChainUnaryInterceptor(
    interceptor.Logging(log),
    faults.UnaryServerInterceptor(func(method string) bool {
      return !strings.HasPrefix(method,
"/"+accountv1.AccountService_ServiceDesc.ServiceName+"/")
    }),
  ))
  accountv1.RegisterAccountServiceServer(grpcSrv, account.NewGRPCServer(store, log))
  // Standard health protocol (grpc.health.v1): grpcurl and grpc-health-probe
  // understand it without custom code. It is NOT tied to the database: the
  // status is SERVING from startup until shutdown, so it only says "this process
  // accepts gRPC calls". Pod readiness is the HTTP /readyz, which pings the
  // database; do not replace it with a gRPC readinessProbe, which would report
  // ready while the database is down.
  healthSrv := health.NewServer()
  healthpb.RegisterHealthServer(grpcSrv, healthSrv)
  healthSrv.SetServingStatus(accountv1.AccountService_ServiceDesc.ServiceName,
healthpb.HealthCheckResponse_SERVING)
  // Reflection lets grpcurl / Postman list services and methods without the
  // .proto file. Handy in the lab; many teams switch it off in production.
  reflection.Register(grpcSrv)

  lis, err := net.Listen("tcp", cfg.GRPCAddr)
  if err != nil {
```

```
    return fmt.Errorf("listen on GRPC_ADDR %s: %w", cfg.GRPCAddr, err)
}
```

Penjelasan blok tersebut:

- **`grpc.ChainUnaryInterceptor`** menjalankan *interceptor* sesuai urutan argumennya. `Logging` menjadi *interceptor* terluar, sehingga *log* mencatat kode yang dihasilkan *fault injector*. Kegagalan buatan pun tercatat seperti kegagalan sungguhan.
- **Fungsi `skip`** mengecualikan semua RPC di luar `account.v1.AccountService`, yaitu *health service* dan *reflection*. Prinsipnya sama dengan Lab 3: *probe* tetap cepat walaupun API sedang diganggu.
- ***Health service*** (`grpc.health.v1`) mengikuti protokol standar, sehingga `grpc-health-probe` dan `grpcurl` dapat memeriksanya tanpa kode tambahan. Status *health* ini tidak bergantung pada kondisi database: nilainya `SERVING` sejak *startup* sampai *shutdown*, sehingga hanya menyatakan bahwa proses menerima panggilan gRPC. Kesiapan *pod* menerima *traffic* tetap ditentukan oleh `GET /readyz` di HTTP, yang memeriksa database. Karena itu *manifest* Kubernetes di Lab 9 hanya memasang *probe* HTTP. *Readiness probe* gRPC sebagai pengganti `/readyz` akan menyatakan *pod* siap walaupun database sedang mati.
- ***Reflection*** memungkinkan *tools* seperti `grpcurl` atau Postman menampilkan daftar *service* dan RPC tanpa file `.proto`. Fitur ini memudahkan eksplorasi di lab, dan banyak tim mematikannya di *production*.

Server HTTP dan server gRPC kini berjalan bersamaan, masing-masing di *goroutine* sendiri. Ganti bagian akhir fungsi `run`, mulai dari `errCh`, dengan kode berikut:

```
errCh := make(chan error, 2)
go func() {
  log.Info("account service listening", "protocol", "http", "addr", cfg.HTTPAddr)
  errCh <- srv.ListenAndServe()
}()
go func() {
  log.Info("account service listening", "protocol", "grpc", "addr", cfg.GRPCAddr)
  errCh <- grpcSrv.Serve(lis)
}()

select {
case err := <-errCh:
  srv.Close()
  grpcSrv.Stop()
  return err
case <-ctx.Done():
  log.Info("shutting down")
  // NOT_SERVING first, so health checkers stop routing new calls here
  // while in-flight ones finish.
  healthSrv.Shutdown()
  shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
  defer cancel()

  grpcDone := make(chan struct{})
  go func() {
    grpcSrv.GracefulStop()
    close(grpcDone)
  }()
  httpErr := srv.Shutdown(shutdownCtx)
  select {
  case <-grpcDone:
  case <-shutdownCtx.Done():
    // Same 10 s budget as HTTP: calls still running now are cut off.
    grpcSrv.Stop()
```

```
    }
    if httpErr != nil && !errors.Is(httpErr, http.ErrServerClosed) {
      return httpErr
    }
    return nil
  }
```

Kapasitas `errCh` menjadi 2 karena ada dua server yang dapat berhenti. Bila salah satu server gagal, server lainnya ikut dihentikan, sehingga tidak ada kondisi ketika hanya satu server yang masih menerima panggilan. Saat *shutdown*, `healthSrv.Shutdown()` mengubah status *health* menjadi `NOT_SERVING` lebih dahulu. Setelah itu `GracefulStop` menunggu RPC yang sedang berjalan selesai, dengan *timeout* 10 detik yang sama dengan server HTTP.

4.4.9 Penghapusan *Endpoint* REST *Debit* dan *Credit*

Di `account/internal/account/http.go`, hapus dua *route* berikut beserta fungsi `mutate` milik `Handler`:

```
  mux.HandleFunc("POST /accounts/{number}/debit", h.mutate(Debit))
  mux.HandleFunc("POST /accounts/{number}/credit", h.mutate(Credit))
```

Tambahkan komentar di atas fungsi `Routes`, agar pembaca kode berikutnya tahu ke mana kedua *endpoint* tersebut pindah:

```
// Routes uses the Go 1.22+ ServeMux patterns: method + path, with {wildcards}.
//
// This is the channel-facing API. Debit and credit are not here: since Lab 4
// other services mutate balances over gRPC only (see grpc.go), and no channel
// client should change a balance without going through a transfer.
```

Hapus juga *import* yang tidak lagi digunakan, karena Go menganggap *import* yang tidak terpakai sebagai *error* kompilasi. Di `http_test.go`, lima kasus *test debit* dan *credit* diganti dengan satu kasus yang memastikan *endpoint* tersebut sudah tidak ada:

```
    {"no debit over REST", "POST", "/accounts/1234567890/debit",
`{"amount":"1","reference":"T-1"}`, 404, ""},
```

> Sejak *checkpoint* `lab-04`, perintah `curl` dari Lab 2 dan Lab 3 yang memanggil `/accounts/{number}/debit` atau `/credit` secara langsung tidak berlaku lagi. Saldo hanya berubah melalui `POST /transfers`.

Setelah semua perubahan di `account` selesai, rapikan `go.mod` dan pastikan kode terkompilasi:

```
cd account
go mod tidy
go build ./...
cd ..
```

4.4.10 Pengujian Server gRPC dengan `bufconn`

Server gRPC dapat diuji melalui jalur yang sama dengan pemanggilan sungguhan, yaitu *encoding protobuf*, HTTP/2 dan kode status, tanpa membuka port TCP. *Package* `google.golang.org/grpc/test/bufconn` menyediakan *listener* di memori. Server menerima koneksi melalui *listener* tersebut, dan *client* terhubung dengannya melalui `WithContextDialer`.

Buat `account/internal/account/grpc_test.go`. Fungsi *helper*-nya sebagai berikut (`newFakeStore` sudah tersedia dari pengujian Lab 1):

```
// newGRPCClient serves GRPCServer over an in-memory listener (bufconn): the test
// goes through real protobuf encoding and status codes without opening a port.
func newGRPCClient(t *testing.T, store Store) accountv1.AccountServiceClient {
  t.Helper()
  lis := bufconn.Listen(1 << 20)
```

```
  srv := grpc.NewServer()
  accountv1.RegisterAccountServiceServer(srv, NewGRPCServer(store,
slog.New(slog.NewTextHandler(io.Discard, nil))))
  go func() { _ = srv.Serve(lis) }()
  t.Cleanup(srv.Stop)

  conn, err := grpc.NewClient("passthrough:///bufnet",
    grpc.WithContextDialer(func(ctx context.Context, _ string) (net.Conn, error) { return
lis.DialContext(ctx) }),
    grpc.WithTransportCredentials(insecure.NewCredentials()))
  if err != nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { conn.Close() })
  return accountv1.NewAccountServiceClient(conn)
}
```

Skema `passthrough:///` membuat gRPC meneruskan alamat `bufnet` apa adanya ke *dialer*, tanpa mencoba me-*resolve* nama tersebut lewat DNS.

Test `TestGRPCAccountService` di *checkpoint* memeriksa dua hal:

- Skenario sukses: `GetAccount` mengembalikan saldo `"500000.00"`, `Debit` sebesar `"200000.50"` menghasilkan saldo `"299999.50"`, dan `Credit` sebesar `"0.50"` menghasilkan `"300000.00"`. Saldo selalu dikirim dengan dua angka desimal karena `toProto` memanggil `StringFixed(2)`.
- Penolakan: rekening tidak ada menghasilkan `NotFound`, saldo kurang menghasilkan `FailedPrecondition`. Nominal `"1e"`, `"100.005"`, `"1e3"`, `"-5"`, `"100000000000000000"` (10^17) dan nominal nol tanpa `reference` menghasilkan `InvalidArgument`. Saldo tidak berubah setelah semua penolakan tersebut.

Test kedua di file yang sama, `TestGRPCCallerGaveUp`, membuat `Store` *fake* (tiruan untuk pengujian) mengembalikan `context.DeadlineExceeded` dan `context.Canceled` yang dibungkus `fmt.Errorf`, seperti *error* dari pgx. Kode yang diharapkan adalah `DeadlineExceeded` dan `Canceled`, bukan `Internal`. Untuk keperluan ini, `fakeStore` di `http_test.go` diberi *field* tambahan `mutateErr`, yang nilainya dikembalikan oleh `Mutate` bila diisi. `TestAccountAPI` di file yang sama juga mendapat tiga kasus HTTP 400 untuk `initial_balance`: `"100.005"`, angka JSON `1e-3`, dan `"100000000000000000"`.

Di `account/internal/fault/fault_test.go`, *test* `TestUnaryServerInterceptor` memanggil fungsi *interceptor* secara langsung dengan `grpc.UnaryServerInfo` buatan. *Test* tersebut memastikan `error_rate` 1 menghasilkan `Unavailable` tanpa menjalankan *handler*, RPC *health* tidak terkena gangguan, dan *deadline* pemanggil yang lebih pendek dari jeda menghasilkan `DeadlineExceeded` tanpa menjalankan *handler*. Salin kode lengkap ketiga file *test* tersebut dari *checkpoint* `lab-04`:

```
git show lab-04:studi-kasus/account/internal/account/grpc_test.go \
  > account/internal/account/grpc_test.go
git show lab-04:studi-kasus/account/internal/account/http_test.go \
  > account/internal/account/http_test.go
git show lab-04:studi-kasus/account/internal/fault/fault_test.go \
  > account/internal/fault/fault_test.go
```

4.4.11 Dependensi dan Konfigurasi di *Service* `transfer`

`transfer` juga mengimpor modul `proto`, sehingga membutuhkan direktif `replace` yang sama:

```
cd transfer
go mod edit \
  -require=github.com/artivisi/training-microservices-2026-01/studi-kasus/proto@v0.0.0 \
```

```
    -replace=github.com/artivisi/training-microservices-2026-01/studi-kasus/proto=../proto
  cd ..
```

Di `transfer/config.go`, `ACCOUNT_SERVICE_URL` diganti dengan `ACCOUNT_GRPC_ADDR`. Nilainya berupa `host:port`, tanpa skema `http://`.

```
type Config struct {
  HTTPAddr    string
  DatabaseURL string
  // AccountGRPCAddr is host:port of the account service's gRPC API.
  AccountGRPCAddr string
}

// di dalam LoadConfig
  cfg := Config{
    HTTPAddr:        get("HTTP_ADDR"),
    DatabaseURL:     get("DATABASE_URL"),
    AccountGRPCAddr: get("ACCOUNT_GRPC_ADDR"),
  }
```

Sesuaikan target `run` di `transfer/Makefile`:

```
run:
  HTTP_ADDR=:28082 \
  DATABASE_URL=postgres://transfer:transfer123@localhost:15433/transfer \
  ACCOUNT_GRPC_ADDR=localhost:29091 \
  go run .
```

4.4.12 *Client* gRPC dengan *Interceptor* failsafe-go

File `transfer/internal/accountclient/client.go` ditulis ulang. Struktur luarnya tetap: *struct* `Settings`, nilai `ProductionSettings`, *method* `Debit`, `Credit`, `BreakerState` dan `ServeCircuitBreaker`. Bagian dalamnya berganti dari HTTP ke gRPC. *Package* `transfer` (logika transfer) tidak berubah sama sekali, karena *package* tersebut hanya mengenal *interface* `AccountClient` dan *error* domain seperti `ErrInsufficientFunds`.

failsafe-go menyediakan integrasi gRPC di *package* `failsafegrpc`. Fungsi `failsafegrpc.NewUnaryClientInterceptor` menerima *policy* yang sama dengan `failsafehttp.NewRoundTripper` di Lab 3 dan menghasilkan *unary client interceptor*. Ganti blok *import* dengan:

```
import (
  "context"
  "encoding/json"
  "errors"
  "fmt"
  "log/slog"
  "net/http"
  "time"

  "github.com/failsafe-go/failsafe-go"
  "github.com/failsafe-go/failsafe-go/circuitbreaker"
  "github.com/failsafe-go/failsafe-go/failsafegrpc"
  "github.com/failsafe-go/failsafe-go/retrypolicy"
  "github.com/failsafe-go/failsafe-go/timeout"
  "github.com/shopspring/decimal"
  "google.golang.org/grpc"
  "google.golang.org/grpc/backoff"
  "google.golang.org/grpc/codes"
  "google.golang.org/grpc/credentials/insecure"
```

```
  "google.golang.org/grpc/status"

  accountv1 "github.com/artivisi/training-microservices-2026-01/studi-kasus/proto/gen/
account/v1"
  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/
transfer"
)
```

Struct `Settings` dan `ProductionSettings` tetap seperti Lab 3. Ganti *struct* `Client` dan fungsi `New`:

```
type Client struct {
  conn    *grpc.ClientConn
  api     accountv1.AccountServiceClient
  breaker circuitbreaker.CircuitBreaker[any]
}

// New connects lazily: no network traffic until the first call, so the transfer
// service starts even while the account service is down. opts are extra dial
// options; tests pass an in-memory dialer.
func New(target string, s Settings, log *slog.Logger, opts ...grpc.DialOption) (*Client,
error) {
  // R = any: the interceptor sees every RPC of the connection, whatever its
  // response type. The policies only look at the error.
  //
  // One breaker for the whole account service: debit and credit share the same
  // failure domain, so a dead account service stops both.
  breaker := circuitbreaker.NewBuilder[any]().
    HandleIf(countsAsFailure).
    WithFailureThresholdRatio(s.BreakerFailures, s.BreakerWindow).
    WithDelay(s.BreakerDelay).
    WithSuccessThreshold(s.BreakerSuccesses).
    OnStateChanged(func(e circuitbreaker.StateChangedEvent) {
      log.Warn("circuit breaker state changed", "target", "account",
        "from", e.OldState.String(), "to", e.NewState.String())
    }).
    Build()

  retry := retrypolicy.NewBuilder[any]().
    HandleIf(notProcessed).
    // Explicit even though notProcessed already ignores ErrOpen: retrying against
    // an open breaker only burns backoff time and fails anyway.
    AbortOnErrors(circuitbreaker.ErrOpen).
    WithMaxRetries(s.MaxRetries).
    WithBackoff(s.BackoffMin, s.BackoffMax).
    // Jitter spreads the retries of many concurrent transfers so they do not all
    // hit a recovering account service in the same millisecond.
    WithJitterFactor(0.25).
    // Hand the last status error back instead of wrapping it in an ExceededError,
    // so mapError sees the real gRPC code.
    ReturnLastFailure().
    OnRetry(func(e failsafe.ExecutionEvent[any]) {
      log.Warn("retrying account call", "next_attempt", e.Attempts(), "cause",
e.LastError())
    }).
    Build()

  // Order matters: retry wraps the breaker, which wraps the timeout. Every attempt
  // gets its own time budget, and every attempt, timeouts included, is counted by
  // the breaker. Same shape as the REST client of Lab 3, now as an interceptor.
  resilience := failsafegrpc.NewUnaryClientInterceptor[any](retry, breaker,
```

```
timeout.New[any](s.AttemptTimeout))

  opts = append([]grpc.DialOption{
    // Plaintext: fine on a laptop, not in production. Service-to-service calls
    // there go over TLS (or mTLS through a service mesh); Lab 7 adds tokens.
    grpc.WithTransportCredentials(insecure.NewCredentials()),
    grpc.WithUnaryInterceptor(resilience),
    // After a failed connect, gRPC waits before reconnecting, growing up to
    // 120 s by default. Capped at 5 s so a restarted account service is used
    // again within seconds, which is what the breaker's half-open trials need.
    grpc.WithConnectParams(grpc.ConnectParams{
      Backoff:           backoff.Config{BaseDelay: time.Second, Multiplier: 1.6, Jitter:
0.2, MaxDelay: 5 * time.Second},
      MinConnectTimeout: 5 * time.Second,
    }),
  }, opts...)
  conn, err := grpc.NewClient(target, opts...)
  if err != nil {
    return nil, fmt.Errorf("account gRPC client for %s: %w", target, err)
  }
  return &Client{conn: conn, api: accountv1.NewAccountServiceClient(conn), breaker:
breaker}, nil
}

func (c *Client) Close() error { return c.conn.Close() }
```

Perbedaan dengan versi REST di Lab 3:

- **Parameter tipe `[any]`.** Di Lab 3 *policy* bertipe `[*http.Response]`. Satu *interceptor* gRPC melayani semua RPC pada koneksi tersebut, dan setiap RPC memiliki tipe *response* sendiri. Karena keputusan *policy* hanya bergantung pada *error*, tipe `any` sudah cukup.
- **Urutan *policy* tetap sama:** *retry* membungkus *circuit breaker*, dan *circuit breaker* membungkus *timeout*. Setiap percobaan mendapat jatah waktu 2 detik sendiri, dan setiap percobaan dihitung oleh *circuit breaker*.
- **`grpc.NewClient` tidak langsung membuka koneksi.** Koneksi baru dibuat saat RPC pertama. Dengan perilaku ini, `transfer` tetap dapat dijalankan walaupun `account` sedang mati.
- **`insecure.NewCredentials()`** berarti koneksi tanpa TLS. Pilihan ini hanya untuk laptop. Di *production*, komunikasi antar-*service* menggunakan TLS atau mTLS (misalnya lewat *service mesh*).
- ***Reconnect backoff* dibatasi 5 detik.** Setelah gagal tersambung, gRPC menunggu sebelum mencoba lagi, dan secara *default* jeda tersebut dapat mencapai 120 detik. Batas 5 detik membuat `account` yang baru dijalankan ulang segera digunakan lagi, sehingga panggilan uji pada kondisi *half-open* tidak terbuang.

Ganti `notProcessed` dan `countsAsFailure`:

```
// notProcessed reports whether the call may be retried: only Unavailable, which
// means the server refused the call before running it (the fault injector does
// exactly that) or no connection could be made.
//
// DeadlineExceeded and our own attempt timeout are NOT retried: the account
// service may have applied the debit and only the answer was late. Debit is not
// idempotent yet, so a retry would debit twice. Lab 5 makes mutations idempotent
// by reference; only then are timeouts retried.
//
// Residual risk: gRPC also reports Unavailable when an established connection
// drops after the request was sent, so this is "almost provably" not processed.
// Lab 5's idempotent references close that gap as well.
func notProcessed(_ any, err error) bool {
```

```
  return status.Code(err) == codes.Unavailable
}

// countsAsFailure decides what the breaker counts against the account service:
// infrastructure codes and our own attempt timeout. Business codes (NotFound,
// FailedPrecondition, InvalidArgument) are a healthy service saying no and must
// not open the breaker.
func countsAsFailure(_ any, err error) bool {
  if err == nil {
    return false
  }
  if errors.Is(err, timeout.ErrExceeded) {
    return true
  }
  switch status.Code(err) {
  case codes.Unavailable, codes.DeadlineExceeded, codes.Internal, codes.Unknown:
    return true
  }
  return false
}
```

Aturan *retry* di Lab 3 menggunakan dua sinyal untuk "*request* pasti belum diproses": kegagalan *dial* dan HTTP 503. Di gRPC kedua sinyal tersebut sama-sama muncul sebagai `Unavailable`. Kode `DeadlineExceeded` dan *timeout* milik failsafe-go tetap tidak di-*retry* dengan alasan yang sama seperti di Lab 3: *debit* mungkin sudah dijalankan di `account` dan hanya jawabannya yang terlambat, sedangkan *debit* belum *idempotent* (aman dijalankan berulang tanpa memotong saldo dua kali). Komentar di atas `notProcessed` juga mencatat satu celah: gRPC mengembalikan `Unavailable` pula ketika koneksi yang sudah terbentuk putus setelah *request* terkirim. Celah tersebut ditutup di Lab 5, ketika mutasi saldo menjadi *idempotent* berdasarkan `reference`.

Ringkasan perlakuan setiap kode status di sisi `transfer`:

Kode	Arti	Di-*retry*	Dihitung *breaker*
`NotFound`	Rekening tidak ada	Tidak	Tidak
`FailedPrecondition`	Saldo tidak cukup	Tidak	Tidak
`InvalidArgument`	*Field* tidak valid	Tidak	Tidak
`Unavailable`	Ditolak sebelum diproses (termasuk *fault injection*) atau koneksi gagal	Ya	Ya
`DeadlineExceeded`, *timeout* 2 detik	Jawaban terlambat, *debit* mungkin sudah terjadi	Tidak	Ya
`Internal`, `Unknown`	Kesalahan di server	Tidak	Ya

Terakhir, ganti `Debit`, `Credit` dan fungsi `mutate` versi REST dengan pemanggilan gRPC dan fungsi `mapError`:

```
func (c *Client) Debit(ctx context.Context, number string, amount decimal.Decimal,
reference string) error {
  _, err := c.api.Debit(ctx, &accountv1.DebitRequest{Number: number, Amount:
amount.String(), Reference: reference})
  return mapError("debit", number, err)
}

func (c *Client) Credit(ctx context.Context, number string, amount decimal.Decimal,
```

```
reference string) error {
  _, err := c.api.Credit(ctx, &accountv1.CreditRequest{Number: number, Amount:
amount.String(), Reference: reference})
  return mapError("credit", number, err)
}

// mapError turns the account service's gRPC codes into the transfer domain's
// errors, so the transfer package never imports gRPC.
func mapError(kind, number string, err error) error {
  if err == nil {
    return nil
  }
  if errors.Is(err, circuitbreaker.ErrOpen) {
    return fmt.Errorf("%s %s: %w", kind, number, transfer.ErrAccountUnavailable)
  }
  switch status.Code(err) {
  case codes.NotFound:
    return fmt.Errorf("%s %s: %w", kind, number, transfer.ErrAccountNotFound)
  case codes.FailedPrecondition:
    return fmt.Errorf("%s %s: %w", kind, number, transfer.ErrInsufficientFunds)
  }
  return fmt.Errorf("%s %s: %w", kind, number, err)
}
```

Kode REST di Lab 3 harus menyusun URL, meng-*encode* JSON, membaca *body* dan menutupnya. Dengan gRPC, pemanggilan cukup satu baris `c.api.Debit(...)`. *Policy resiliency* tidak tampak di *method* `Debit`, karena *interceptor* sudah membungkus setiap RPC. Fungsi `mapError` menerjemahkan kode gRPC menjadi *error* domain, sehingga *package* `transfer` tidak perlu mengimpor gRPC dan *handler* HTTP `transfer` tetap memetakan *error* tersebut seperti di Lab 3. HTTP 503 hanya muncul bila *circuit breaker* yang terbuka menolak *debit*. Pada saat itu belum ada saldo yang berubah, sehingga *request* aman diulang. Setiap kegagalan *credit* sesudah *debit* berhasil, termasuk rekening tujuan yang tidak ada dan *circuit breaker* yang baru terbuka, dijawab HTTP 502. Saldo sumber sudah berkurang dan perlu dikembalikan secara manual, sehingga *request* tidak boleh diulang. HTTP 422 hanya digunakan untuk rekening sumber yang tidak ada atau saldo yang tidak cukup.

4.4.13 `main.go` di `transfer`

`accountclient.New` kini dapat mengembalikan *error* (misalnya alamat target tidak valid), dan koneksinya perlu ditutup saat *service* berhenti. Ubah `transfer/main.go`:

```
  accounts, err := accountclient.New(cfg.AccountGRPCAddr, accountclient.ProductionSettings,
log)
  if err != nil {
    return err
  }
  defer accounts.Close()
```

Sesuaikan juga *log* saat *startup*:

```
    log.Info("transfer service listening", "addr", cfg.HTTPAddr, "account_grpc",
cfg.AccountGRPCAddr)
```

Rapikan dependensi dan pastikan kode terkompilasi:

```
cd transfer
go mod tidy
go build ./...
cd ..
```

4.4.14 Pengujian *Client* dengan `bufconn`

Test client di Lab 3 menggunakan `httptest.Server`. Padanannya untuk gRPC adalah server gRPC *fake* yang berjalan di `bufconn`. Server *fake* ini mengimplementasikan `AccountServiceServer` dan menjawab setiap panggilan sesuai fungsi `answer` yang ditentukan oleh masing-masing *test*.

```
// fakeAccountService answers every Debit/Credit with answer(n), n = 1 for the
// first call; answer returns the gRPC error (nil = success).
type fakeAccountService struct {
  accountv1.UnimplementedAccountServiceServer
  calls  atomic.Int64
  answer func(ctx context.Context, n int64) error
  last   atomic.Pointer[accountv1.DebitRequest]
}

// serve runs the fake on an in-memory listener (bufconn) and returns a client
// wired to it with s and log. No TCP port is opened.
func serve(t *testing.T, answer func(ctx context.Context, n int64) error, s Settings, log
*slog.Logger) (*Client, *fakeAccountService, *bufconn.Listener) {
  t.Helper()
  fake := &fakeAccountService{answer: answer}
  lis := bufconn.Listen(1 << 20)
  srv := grpc.NewServer()
  accountv1.RegisterAccountServiceServer(srv, fake)
  go func() { _ = srv.Serve(lis) }()
  t.Cleanup(srv.Stop)

  c, err := New("passthrough:///bufnet", s, log,
    grpc.WithContextDialer(func(ctx context.Context, _ string) (net.Conn, error) { return
lis.DialContext(ctx) }))
  if err != nil {
    t.Fatal(err)
  }
  t.Cleanup(func() { c.Close() })
  return c, fake, lis
}
```

Parameter *variadic* `opts` pada `New` memungkinkan *test* memasukkan *dialer* `bufconn`, sementara `main.go` tidak mengirim opsi tambahan. Nilai `testSettings` mempertahankan bentuk *policy* di `ProductionSettings` (3 *retry*, ambang 5 dari 10, 3 keberhasilan untuk menutup) dengan durasi pendek: *timeout* 100 ms dan *backoff* beberapa milidetik.

Daftar *test* di *checkpoint* `lab-04`:

Test	Yang dipastikan
`TestCodeMapping`	`NotFound` dan `FailedPrecondition` menjadi `ErrAccountNotFound` dan `ErrInsufficientFunds`. Nominal `10.50` terkirim sebagai `"10.5"`.
`TestInternalIsNotRetried`	Server yang menjawab `Internal` hanya dipanggil sekali, tanpa *retry*.
`TestRetryOnUnavailableSucceeds`	Dua kali `Unavailable` lalu OK: transfer berhasil setelah 3 panggilan.
`TestRetryGivesUpAfterMaxRetries`	`Unavailable` terus-menerus: 4 panggilan (1 percobaan + 3 *retry*), *error* terakhir dikembalikan apa adanya.
`TestNoRetryOnTimeout`	Server lambat: hanya 1 panggilan, selesai dalam waktu sekitar *timeout* percobaan.

`TestNoRetryOnDeadlineExceeded`	`DeadlineExceeded` dari server tidak di-*retry*.
`TestRetryOnConnectionRefused`	*Listener* ditutup: 3 kali *retry* tercatat di *log*, server tidak menerima panggilan, dan 4 kegagalan belum membuka *breaker*.
`TestBreakerOpensAndRejectsFast`	Setelah 5 kali `Internal`, *breaker* terbuka. Panggilan berikutnya ditolak seketika dengan `ErrAccountUnavailable` tanpa mencapai server, dan `/debug/circuit-breaker` menampilkan `{"account":"open"}`.
`TestBusinessRejectionsDoNotOpenBreaker`	Sepuluh kali `FailedPrecondition` tidak membuka *breaker*.
`TestBreakerHalfOpenCloses`	Setelah jeda berlalu, *breaker* masih *half-open* sesudah satu panggilan uji sukses dan baru tertutup sesudah tiga panggilan uji sukses.

Test terpenting untuk mencegah saldo terdebit dua kali memastikan *timeout* tidak di-*retry*:

```
func TestNoRetryOnTimeout(t *testing.T) {
  c, fake := newTestClient(t, func(ctx context.Context, _ int64) error {
    select {
    case <-time.After(300 * time.Millisecond):
    case <-ctx.Done():
    }
    return nil
  })
  start := time.Now()
  if err := debit(c); err == nil {
    t.Fatal("err = nil, want timeout")
  }
  if got := fake.calls.Load(); got != 1 {
    t.Fatalf("calls = %d, want 1: a timed-out debit may have been applied", got)
  }
  if elapsed := time.Since(start); elapsed > 250*time.Millisecond {
    t.Fatalf("took %s, want about the attempt timeout", elapsed)
  }
}
```

Salin file *test* lengkap dari *checkpoint* bila tidak ingin menulisnya sendiri:

```
git show lab-04:studi-kasus/transfer/internal/accountclient/client_test.go \
  > transfer/internal/accountclient/client_test.go
```

Jalankan seluruh *test* dari folder `studi-kasus`:

```
make test
```

Target tersebut menjalankan `go vet` pada modul `proto`, lalu `go vet` dan `go test` pada `account` dan `transfer`. Semua *package* harus berstatus `ok`.

4.5 Verifikasi

Jalankan kedua *service* di dua terminal terpisah dari folder `studi-kasus`:

```
make -C account run
```

```
make -C transfer run
```

Log `account` saat *startup* menampilkan dua *listener*:

```
level=INFO msg="account service listening" protocol=http addr=:28081
level=INFO msg="account service listening" protocol=grpc addr=:29091
```

Log `transfer` menampilkan alamat gRPC tujuan:

```
level=INFO msg="transfer service listening" addr=:28082 account_grpc=localhost:29091
```

4.5.1 Transfer Berhasil melalui gRPC

Buat dua rekening uji melalui REST API `account`. Bila rekening tersebut sudah ada dari lab sebelumnya, `account` menjawab HTTP 409 dan langkah ini boleh dilewati.

```
curl -sS -X POST localhost:28081/accounts \
  -d '{"number":"1111111111","owner_name":"Budi","initial_balance":"1000000"}'
curl -sS -X POST localhost:28081/accounts \
  -d '{"number":"2222222222","owner_name":"Sari","initial_balance":"500000"}'
```

Kirim transfer:

```
curl -sS -i -X POST localhost:28082/transfers \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

Hasil yang diharapkan adalah `HTTP/1.1 201 Created` dengan `"status":"COMPLETED"`. *Log* `account` mencatat dua RPC, yaitu *access log* gRPC yang ditulis oleh *interceptor* `Logging`:

```
level=INFO msg="account mutated" number=1111111111 type=DEBIT amount=100000
reference=...
level=INFO msg="grpc request" method=/account.v1.AccountService/Debit code=OK
duration_ms=3
level=INFO msg="account mutated" number=2222222222 type=CREDIT amount=100000
reference=...
level=INFO msg="grpc request" method=/account.v1.AccountService/Credit code=OK
duration_ms=2
```

Nilai `duration_ms` dan `reference` (ID transfer) berbeda di setiap laptop.

4.5.2 Penolakan Bisnis

Transfer dengan nominal melebihi saldo:

```
curl -sS -i -X POST localhost:28082/transfers \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"999999999"}'
```

Hasilnya `HTTP/1.1 422 Unprocessable Entity` dengan `"status":"FAILED"` dan `failure_reason` berisi `insufficient funds`. *Log* `account` mencatat `code=FailedPrecondition` pada level `INFO`, dan `transfer` tidak mencatat `retrying account call`. Penolakan bisnis tidak di-*retry* dan tidak dihitung oleh *circuit breaker*:

```
make breaker
# {"account":"closed"}
```

4.5.3 Nominal dengan Tiga Digit Desimal

```
curl -sS -i -X POST localhost:28082/transfers \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100.005"}'
```

Hasilnya `HTTP/1.1 400 Bad Request` dengan *body* berikut, dan tidak ada transfer yang tercatat:

```
{"error":"validation failed","fields":{"amount":"must have at most 2 decimal places"}}
```

`POST /accounts` dengan `"initial_balance":"100.005"` juga dijawab HTTP 400, dengan `fields.initial_balance` berisi pesan yang sama.

4.5.4 Rekening Tujuan Tidak Ada

```
curl -sS -i -X POST localhost:28082/transfers \
  -d '{"from_account":"1111111111","to_account":"9999999999","amount":"1"}'
```

Hasilnya `HTTP/1.1 502 Bad Gateway` dengan `"status":"FAILED"` dan `failure_reason` berikut:

```
credit failed after debit: credit 9999999999: account not found
```

Debit sudah berhasil sebelum *credit* ditolak `NotFound`, sehingga saldo rekening `1111111111` berkurang 1 dan *log* `transfer` mencatat ERROR `credit failed after debit, source account needs manual reversal`. `transfer` menjawab dengan 502 alih-alih 422 agar aplikasi *channel* tahu bahwa *request* ini tidak boleh diulang. Lab 5 menambahkan kompensasi otomatis untuk kasus ini.

4.5.5 *Endpoint* REST *Debit* Sudah Tidak Ada

```
curl -sS -i -X POST localhost:28081/accounts/1111111111/debit \
  -d '{"amount":"1","reference":"X"}'
```

Hasilnya `HTTP/1.1 404 Not Found`.

4.5.6 *Retry* dan *Circuit Breaker* dengan `fault-error`

Buat semua panggilan ke `account` gagal:

```
make fault-error RATE=1
```

Kirim transfer pertama (perintah `curl` yang sama dengan transfer berhasil). *Log* `transfer` mencatat tiga kali `retrying account call` dengan `cause` berisi `code = Unavailable desc = injected fault`. Di *log* `account` tercatat empat baris `grpc request method=/account.v1.AccountService/Debit code=Unavailable` pada level `WARN`. Jawaban HTTP-nya `502 Bad Gateway` dengan `"status":"FAILED"` dan `failure_reason` seperti berikut:

```
debit failed: debit 1111111111: rpc error: code = Unavailable desc = injected fault
```

Keempat percobaan tersebut dihitung sebagai empat kegagalan oleh *circuit breaker*. Kirim transfer kedua. Percobaan pertamanya menjadi kegagalan kelima, dan *log* `transfer` mencatat:

```
level=WARN msg="circuit breaker state changed" target=account from=closed to=open
```

Sejak saat itu, transfer berikutnya dijawab seketika dengan `503 Service Unavailable`, `failure_reason` berisi `account service unavailable`, dan tidak ada baris `grpc request` baru di *log* `account`.

```
make breaker
# {"account":"open"}
```

4.5.7 *Circuit Breaker* Kembali *Closed* melalui *Half-Open*

Hapus gangguan, lalu tunggu lebih dari 10 detik (jeda *circuit breaker*):

```
make fault-clear
```

Kirim dua transfer. Setiap transfer terdiri dari dua RPC (*debit* dan *credit*), sehingga dua transfer menghasilkan empat panggilan sukses, lebih dari tiga panggilan uji yang dibutuhkan untuk menutup *circuit breaker*. *Log* `transfer` mencatat perpindahan `from=open to=half-open` lalu `from=half-open to=closed`, dan `make breaker` kembali menampilkan `{"account":"closed"}`.

4.5.8 *Timeout* Tidak Di-*retry* dengan `fault-slow`

```
make fault-slow
time curl -sS -X POST localhost:28082/transfers \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

Transfer gagal setelah sekitar 2,0 detik, yaitu batas *timeout* satu percobaan, dengan `failure_reason` berikut:

```
debit failed: debit 1111111111: timeout exceeded
```

Log `transfer` tidak memuat `retrying account call`. Di *log* `account` tercatat:

```
level=WARN msg="grpc request" method=/account.v1.AccountService/Debit code=Canceled
duration_ms=2000
```

Kode `Canceled` muncul karena `transfer` membatalkan RPC setelah 2 detik. *Fault injector* di `account` melihat *context* yang sudah berakhir, sehingga *handler* tidak dijalankan dan saldo tidak berubah. Ulangi transfer hingga lima kali: *circuit breaker* terbuka, dan transfer berikutnya langsung dijawab 503. Hapus gangguan, lalu tunggu lebih dari 10 detik:

```
make fault-clear
```

Kirim dua transfer agar *circuit breaker* melewati *half-open* dan kembali *closed*, sama seperti langkah sebelumnya. Pastikan statusnya sudah *closed* sebelum melanjutkan:

```
make breaker
# {"account":"closed"}
```

4.5.9 `account` Mati

Langkah ini membutuhkan *circuit breaker* dalam keadaan *closed*. Bila masih *open* atau *half-open*, panggilan langsung ditolak dan transfer dijawab 503 dengan `account service unavailable`. Hentikan `account` dengan Ctrl+C, lalu kirim satu transfer. `transfer` mencoba empat kali (1 percobaan + 3 *retry*), karena koneksi yang ditolak juga dilaporkan sebagai `Unavailable`. Jawabannya 502 dengan `failure_reason` yang memuat teks *error* gRPC lengkap, misalnya `... dial tcp 127.0.0.1:29091: connect: connection refused`. Empat percobaan yang ditolak dihitung sebagai empat kegagalan, masih di bawah ambang 5, sehingga *circuit breaker* tetap *closed* (sesuai `TestRetryOnConnectionRefused`).

> `failure_reason` saat ini memuat teks *error* gRPC apa adanya, termasuk alamat internal `account`. Informasi tersebut tidak layak diterima aplikasi *channel*. Pada sistem sungguhan, `failure_reason` untuk aplikasi *channel* sebaiknya berupa pesan umum, sedangkan rinciannya hanya dicatat di *log*.

Jalankan `account` kembali dengan `make -C account run`. Setelah *circuit breaker* (bila sempat terbuka) melewati jeda 10 detik dan koneksi gRPC tersambung ulang, transfer kembali berhasil.

4.6 Masalah Umum

`protoc-gen-go: program not found or is not executable.` *Plugin* belum terpasang atau folder `$(go env GOPATH)/bin` tidak ada di `PATH`. Target `generate` di `proto/Makefile` sudah menambahkan folder tersebut ke `PATH`. Bila `protoc` dijalankan langsung tanpa `make`, jalankan `export PATH="$PATH:$(go env GOPATH)/bin"` lebih dahulu. Bila *plugin* memang belum ada, pasang dengan `go install` seperti di bagian Titik Awal.

`missing required environment variables: GRPC_ADDR` atau `ACCOUNT_GRPC_ADDR`. *Service* dijalankan dengan perintah lama atau *environment variable* dari Lab 3. Jalankan melalui `make -C account run` dan `make -C transfer run`, yang sudah mengisi variabel baru. `ACCOUNT_SERVICE_URL` tidak dibaca lagi oleh `transfer`.

`listen on GRPC_ADDR :29091: bind: address already in use`. Proses `account` lain masih berjalan, biasanya di terminal lain. Hentikan proses tersebut terlebih dahulu.

`cannot find module providing package .../studi-kasus/proto/gen/account/v1` atau `go mod tidy` mencoba mengunduh modul `proto` dari GitHub. Direktif `replace` belum ada di `go.mod` milik `account` atau `transfer`, atau `./proto` belum terdaftar di `go.work`. Periksa kembali langkah dependensi di kedua *service*.

Transfer masih gagal beberapa detik setelah `account` dijalankan ulang. Ada dua penyebab yang dapat terjadi bersamaan. Pertama, *circuit breaker* yang sempat terbuka baru masuk *half-open* setelah 10 detik. Kedua, gRPC menunggu sebelum mencoba tersambung kembali, dengan jeda maksimal 5 detik sesuai `WithConnectParams`. Tunggu beberapa detik, lalu kirim transfer lagi.

`curl` ke `/accounts/{number}/debit` menghasilkan 404. Perilaku ini memang diharapkan. Sejak lab ini, perubahan saldo hanya melalui gRPC, dan dari sisi pengguna hanya melalui `POST /transfers`.

`git diff lab-04` menampilkan perbedaan di baris versi `protoc` pada file hasil *generate*. Versi `protoc` di laptop berbeda dari versi yang digunakan saat *checkpoint* dibuat. Perbedaan tersebut hanya pada komentar dan tidak memengaruhi kode.

Tidak ada `grpcurl` untuk mencoba RPC secara langsung. Panduan persiapan tidak menyertakan `grpcurl`. Seluruh verifikasi di atas berjalan melalui `transfer` dan *log* `account`. *Health service* dan *reflection* dapat dicoba dengan program Go kecil (lihat Eksplorasi Mandiri), atau dengan `grpcurl` maupun Postman bila tersedia.

4.7 Eksplorasi Mandiri

1. **Program pemeriksa *health*.** Tulis program Go kecil di dalam modul `account`, misalnya `account/cmd/grpchealth/main.go`, yang tersambung ke `localhost:29091` dengan `grpc.NewClient` dan `insecure.NewCredentials()`, lalu memanggil `grpc_health_v1.NewHealthClient(conn).Check` dengan `Service: "account.v1.AccountService"`. Jalankan dengan `go run ./cmd/grpchealth` dari folder `account`. Status yang diharapkan adalah `SERVING`. Hentikan `account` dengan Ctrl+C sambil menjalankan program tersebut berulang-ulang, lalu amati kapan statusnya berubah atau koneksinya ditolak. Pastikan juga bahwa `make fault-error RATE=1` tidak memengaruhi hasil pemeriksaan *health*.
2. **Evolusi kontrak.** Tambahkan *field* `string currency = 5;` pada *message* `Account`, jalankan `make proto`, lalu isi *field* tersebut di `toProto`. Jalankan `transfer` versi lama (belum di-*build* ulang) bersama `account` versi baru. Jelaskan mengapa `transfer` tetap berjalan, lalu coba hal sebaliknya: apa yang terjadi bila nomor *field* `balance` diubah dari `3` menjadi `6`?
3. **Urutan *interceptor*.** Tukar urutan `interceptor.Logging` dan `faults.UnaryServerInterceptor` di `grpc.ChainUnaryInterceptor`, lalu ulangi `make fault-error RATE=1` dan `make fault-slow`. Bandingkan baris `grpc request` di *log* `account` sebelum dan sesudah penukaran, terutama nilai `code` dan `duration_ms`, dan jelaskan perbedaannya.
4. **Pemanggil berhenti menunggu.** Pasang *lock* pada baris rekening `1111111111` selama 4 detik dari `psql`, lalu kirim transfer dari rekening tersebut di terminal lain dalam rentang waktu itu:

```
docker compose exec db-account psql -U account -d account -c \
  "begin; select number from account where number='1111111111' for update; select
pg_sleep(4); commit;"
```

Transfer dijawab 502 setelah sekitar 2 detik dengan `failure_reason debit failed: debit 1111111111: timeout exceeded`. *Log* `account` mencatat WARN `caller gave up before the request finished` dengan `code=Canceled`, bukan ERROR `grpc request failed`. Jelaskan mengapa kodenya `Canceled`, bukan `DeadlineExceeded`: siapa yang mengakhiri *context* RPC tersebut? Lalu pertimbangkan: bila *lock* pada baris tersebut dilepas tepat sebelum *context* berakhir, apakah *debit* mungkin sudah terjadi walaupun transfer tercatat `FAILED`?

5. ***Breaker* untuk kode `InvalidArgument`.** Tambahkan *test* di `client_test.go` yang memastikan sepuluh kali `InvalidArgument` tidak membuka *circuit breaker*, mengikuti pola `TestBusinessRejectionsDoNotOpenBreaker`. Kemudian pertimbangkan: apakah `ResourceExhausted` (misalnya *rate limit* di server) sebaiknya dihitung sebagai kegagalan, dan apakah boleh di-*retry*?

4.8 Checkpoint

Kondisi kode di akhir lab ini tersedia di *tag* `lab-04`. Untuk membandingkan hasil kerja sendiri dengan *checkpoint*:

```
git diff lab-04 -- studi-kasus
```

Untuk melanjutkan ke Lab 5 dari *checkpoint*:

```
git switch -c kerja-lab-05 lab-04
```

5 Lab 5: *Saga*, *Transactional Outbox*, dan Kafka

5.1 Tujuan

Setelah menyelesaikan lab ini, peserta mampu:

- membuat mutasi saldo di `account` menjadi *idempotent* dengan *unique index* atas `reference`, sehingga *retry* setelah *timeout* aman dilakukan;
- menjalankan transfer sebagai *saga* yang diatur oleh `transfer` (*orchestration*), lengkap dengan langkah kompensasi ketika *credit* gagal setelah *debit* berhasil;
- memberi langkah kompensasi *retry policy* tersendiri yang tidak melewati *circuit breaker*, sehingga gangguan singkat di `account` tidak membuat uang tertahan;
- menyimpan perubahan status transfer dan *event*-nya dalam satu transaksi database melalui pola *transactional outbox*;
- mengirim *event* dari tabel `outbox` ke Kafka dengan *relay* dan memahami jaminan *at-least-once* yang dihasilkannya;
- membuat service ketiga, `notification`, yang membaca *event* dari Kafka sebagai anggota *consumer group*.

5.2 Gambaran

Di akhir Lab 4, `transfer` memanggil `account` melalui gRPC dengan *timeout*, *retry*, dan *circuit breaker*. Dua celah dari Lab 2 masih tersisa. Pertama, *debit* yang terkena *timeout* tidak boleh di-*retry*, karena `account` mungkin sudah memproses *debit* tersebut dan *retry* akan memotong saldo dua kali. Kedua, bila *credit* ke rekening tujuan gagal setelah *debit* berhasil, uang sudah keluar dari rekening asal tetapi tidak masuk ke rekening tujuan. Transfer hanya ditandai `FAILED` (HTTP `502`, `ErrCreditFailed`) dan uang tersebut harus dikembalikan secara manual.

Lab ini menutup kedua celah tersebut dan sekaligus menambah service ketiga:

1. **Mutasi *idempotent*.** Setiap *debit* dan *credit* membawa `reference` (ID transfer). `account` menerapkan mutasi dengan kombinasi rekening, jenis, dan `reference` yang sama hanya sekali. Panggilan ulang dijawab sukses dengan saldo saat ini. Dengan jaminan tersebut, `transfer` boleh me-*retry* panggilan yang terkena *timeout*.
2. ***Saga*.** Transfer terdiri atas langkah-langkah lokal, dan setiap langkah yang sudah berhasil memiliki langkah kompensasi. Dua panggilan ke `account` tidak bisa disatukan dalam satu transaksi database, sehingga *rollback* diganti dengan langkah yang membatalkan efek langkah sebelumnya: *credit* balik ke rekening asal (*reversal*). *Reversal* dikirim lewat koneksi gRPC tersendiri yang tidak melewati *circuit breaker* dan terus di-*retry* hingga 30 detik.
3. ***Transactional outbox*.** Service lain perlu mengetahui hasil transfer. `transfer` tidak mengirim *event* langsung ke Kafka, tetapi menulisnya ke tabel `outbox` dalam transaksi yang sama dengan perubahan status. *Relay* di dalam `transfer` kemudian mengirim isi tabel tersebut ke Kafka.
4. **`notification`.** Service baru ini membaca *event* dari *topic* `transfer-events`, menyimpan satu baris notifikasi per *event*, lalu mencatat log `notification sent` sebagai pengganti pengiriman SMS atau *push notification*.

```
channel ── POST /transfers ──▶ transfer ── gRPC Debit / Credit ──▶ account
                                  │                                   │
                       db-transfer: transfer + outbox       db-account: account +
mutation
                       (satu transaksi per perubahan)       (unique: account, type,
reference)
                                  │
                            relay, setiap 500 ms
                                  ▼
                Kafka, topic transfer-events (3 partition, key = transfer_id)
                                  │
                       consumer group "notification"
                                  ▼
                           notification ──▶ db-notification
```

Status transfer mengikuti *state machine* (mesin status) berikut. *Event* hanya dihasilkan untuk status akhir yang perlu diketahui service lain.

Status	Kondisi	*Event*
`PENDING`	Transfer tercatat, *debit* belum berhasil	–
`DEBITED`	*Debit* rekening asal berhasil	–
`COMPLETED`	*Credit* rekening tujuan berhasil	`TransferCompleted`
`FAILED`	*Debit* gagal, saldo tidak berubah	`TransferFailed`
`COMPENSATED`	*Credit* gagal, lalu *debit* dikembalikan dengan *credit* ke rekening asal	`TransferCompensated`
`COMPENSATION_FAILED`	*Credit* gagal dan pengembalian juga gagal, sehingga perlu penanganan manual	–

Mengapa tidak langsung menulis ke Kafka setelah transaksi database selesai? Database dan Kafka tidak memiliki transaksi bersama. Bila proses berhenti di antara keduanya, ada dua kemungkinan buruk: status sudah tersimpan tetapi *event*-nya hilang, atau *event* terkirim untuk perubahan yang ternyata di-*rollback*. Dengan tabel `outbox`, perubahan status dan *event* ditulis ke database yang sama, sehingga keduanya dapat disimpan dalam satu transaksi.

5.2.1 Alasan Memilih *Messaging* dan Kafka

Pertimbangan	Keterangan
Kebutuhan	Hasil transfer perlu diketahui *service* lain: `notification` di lab ini, dan nantinya bisa juga audit atau analitik. Gangguan di `notification` tidak boleh menggagalkan transfer
Opsi yang dipertimbangkan	`transfer` memanggil `notification` secara sinkron (REST atau gRPC). `transfer` mengirim *event* ke *message broker*, yaitu Kafka atau RabbitMQ
Pilihan di studi kasus	*Messaging* dengan Kafka. `transfer` mengirim *event* melalui *outbox*, dan `notification` membacanya sebagai anggota *consumer group*
Yang diperoleh	*Temporal decoupling*: `notification` boleh mati sementara, dan *event* menunggu di Kafka hingga dibaca. *Load leveling*: lonjakan transfer tidak langsung membebani *consumer*, karena *consumer* membaca sesuai kapasitasnya. *Fan-out*: *consumer* baru cukup membaca *topic* yang sama dengan *consumer group* sendiri, tanpa perubahan di `transfer`
Yang dikorbankan	*Eventual consistency*: notifikasi tercatat beberapa saat setelah transfer selesai. Pengiriman *at-least-once* menghasilkan duplikat, sehingga *consumer* harus *idempotent* (Lab 6), dan urutan hanya terjamin di dalam satu *partition*. Kafka menjadi infrastruktur tambahan yang harus dioperasikan dan dipantau

Kafka menyimpan *event* di *log* dengan masa retensi tertentu. *Event* tetap tersimpan setelah dibaca, sehingga *consumer* baru atau *consumer* yang baru diperbaiki dapat membaca ulang (*replay*) dari *offset* sebelumnya. Urutan terjamin per *partition*, dan *consumer group* membagi *partition* ke beberapa *instance*. RabbitMQ menggunakan model *queue* dengan *routing* yang lebih fleksibel melalui *exchange*. Pada *queue* klasik, *message* dihapus setelah *consumer* mengirim *acknowledgement*, sehingga *replay* tidak tersedia. Studi kasus memilih Kafka karena *replay* digunakan pada Lab 5 dan Lab 6 (`make replay`), dan urutan *event* satu transfer terjaga dengan *key* berupa ID transfer.

5.3 Titik Awal

Lab ini melanjutkan kondisi akhir Lab 4. Peserta yang belum menyelesaikan Lab 4 dapat memulai dari *checkpoint*-nya:

```
git checkout lab-04
cd studi-kasus
```

Semua perintah di bab ini dijalankan dari folder `studi-kasus`, kecuali bila disebutkan lain.

Lab ini dimulai dengan database kosong. Hentikan service `account` dan `transfer` yang masih berjalan dari Lab 4 (`Ctrl+C`), lalu hapus seluruh data lab:

```
make infra-reset
```

Langkah ini wajib. *Unique index* baru di tabel `mutation` (bagian berikutnya) gagal dibuat bila database dari Lab 1–4 sudah berisi mutasi dengan `reference` ganda, dan `account` berhenti saat *startup* dengan *error* `apply schema: ... could not create unique index "mutation_idempotency" (SQLSTATE 23505)`. Infrastruktur dijalankan lagi setelah Kafka ditambahkan ke `compose.yaml`.

5.4 Langkah

5.4.1 Kafka dan Database Notification di Docker Compose

Kafka dijalankan sebagai satu *node* dalam mode KRaft: satu proses berperan sebagai *broker* sekaligus *controller*, tanpa ZooKeeper. Service berjalan di laptop, sedangkan *container* pembuat *topic* berjalan

di jaringan Docker Compose. Kedua kelompok *client* tersebut membutuhkan alamat yang berbeda, karena *client* Kafka hanya menggunakan alamat *bootstrap* untuk kontak pertama. Setelah itu *client* menghubungi *advertised listener*, yaitu alamat yang dikirimkan *broker* untuk *listener* tersebut. Karena itu ada dua *listener* untuk *client*: `HOST` (`localhost:19092`) untuk service di laptop dan `INTERNAL` (`kafka:29092`) untuk *container*. Port `HOST` di laptop adalah 19092, bukan port bawaan Kafka 9092, agar tidak bentrok dengan Kafka lain yang mungkin sudah berjalan di laptop.

Topic dibuat secara sengaja oleh *container* `kafka-init` yang hanya berjalan sekali, dan pembuatan *topic* otomatis dimatikan. Bila nama *topic* di konfigurasi service salah ketik, service langsung gagal dengan *error* dan *topic* baru tidak terbentuk. *Topic* `transfer-events` dibuat dengan 3 *partition* agar hingga tiga *instance* `notification` dapat bekerja paralel. Semua *event* dengan *key* yang sama (ID transfer) selalu masuk ke *partition* yang sama sehingga urutannya terjaga.

Tambahkan tiga service berikut ke `compose.yaml`, di bawah `db-transfer`:

```
  db-notification:
    image: postgres:18.6-alpine
    environment:
      POSTGRES_DB: notification
      POSTGRES_USER: notification
      POSTGRES_PASSWORD: notification123
    ports:
      - "15434:5432"
    volumes:
      - db-notification:/var/lib/postgresql
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "notification", "-d", "notification"]
      interval: 2s
      retries: 30

  # Single-node Kafka in KRaft mode: the same process is broker and controller,
  # no ZooKeeper. Two client listeners:
  #   HOST     localhost:19092 for the services running on the laptop
  #   INTERNAL kafka:29092 for containers on the compose network (kafka-init)
  # A client connects to the bootstrap address, then reconnects to the address
  # the broker advertises for its listener; that is why each audience needs its
  # own listener.
  kafka:
    image: apache/kafka:4.3.1
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: HOST://:19092,INTERNAL://:29092,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: HOST://localhost:19092,INTERNAL://kafka:29092
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
HOST:PLAINTEXT,INTERNAL:PLAINTEXT,CONTROLLER:PLAINTEXT
      KAFKA_INTER_BROKER_LISTENER_NAME: INTERNAL
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@localhost:9093
      # One broker: internal topics cannot have more than one replica.
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_REPLICATION_FACTOR: 1
      KAFKA_TRANSACTION_STATE_LOG_MIN_ISR: 1
      # A new consumer gets its partitions at once instead of after 3 s.
      KAFKA_GROUP_INITIAL_REBALANCE_DELAY_MS: 0
      # Topics are created on purpose (kafka-init), never by a typo in a client.
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"
      KAFKA_LOG_DIRS: /var/lib/kafka/data
    ports:
```

```
      - "19092:19092"
    volumes:
      - kafka:/var/lib/kafka/data
    healthcheck:
      test: ["CMD", "/opt/kafka/bin/kafka-broker-api-versions.sh", "--bootstrap-server",
"localhost:19092"]
      interval: 5s
      timeout: 10s
      retries: 30

  # One-shot: creates the topic and exits. 3 partitions = up to 3 consumers of
  # one group work in parallel; events with the same key (transfer ID) always
  # land in the same partition, so they stay in order.
  # Profile "init" keeps it out of "up --wait", which waits for running
  # containers and does not expect one that exits; make infra-up runs it with
  # "docker compose run" after Kafka is healthy.
  kafka-init:
    image: apache/kafka:4.3.1
    profiles: ["init"]
    depends_on:
      kafka:
        condition: service_healthy
    entrypoint: ["/opt/kafka/bin/kafka-topics.sh"]
    command: ["--bootstrap-server", "kafka:29092", "--create", "--if-not-exists",
              "--topic", "transfer-events", "--partitions", "3", "--replication-factor",
"1"]
```

Lengkapi daftar *volume* di akhir `compose.yaml`:

```
volumes:
  db-account:
  db-transfer:
  db-notification:
  kafka:
```

`kafka-init` ditempatkan di *profile* `init` karena `docker compose up --wait` menunggu semua *container* berstatus `healthy`, sedangkan `kafka-init` memang selesai lalu berhenti. Target `infra-up` di `Makefile` (folder `studi-kasus`) menjalankannya secara terpisah setelah Kafka berstatus `healthy`. Target `infra-down` dan `infra-reset` juga perlu `--profile init`, karena `docker compose down` melewati service dari *profile* yang tidak aktif, dan sisa *container* `kafka-init` akan menghalangi penghapusan jaringan Compose. Ubah ketiga target tersebut:

```
# kafka-init creates the topic (idempotent: --if-not-exists).
infra-up:
  docker compose up -d --wait
  docker compose run --rm kafka-init

# --profile init: down skips services of inactive profiles, and a leftover
# kafka-init container would keep the network from being removed.
infra-down:
  docker compose --profile init down

# Removes the database and Kafka volumes too: all lab data is lost.
infra-reset:
  docker compose --profile init down -v
```

> **Penting.** Baris *recipe* di `Makefile` harus diawali karakter *tab*, bukan spasi. Bila editor mengganti *tab* menjadi spasi, `make` berhenti dengan pesan `missing separator`.

Jalankan infrastruktur dan periksa bahwa *topic* sudah dibuat:

```
make infra-up
docker compose exec kafka /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server localhost:19092 --describe --topic transfer-events
```

Pada `make infra-up` pertama, `kafka-init` mencetak `Created topic transfer-events.`. Pada pemanggilan berikutnya `kafka-init` tidak mencetak apa pun karena opsi `--if-not-exists`. Perintah `--describe` menampilkan `PartitionCount: 3` beserta tiga baris `Partition: 0`, `1`, dan `2`.

5.4.2 Mutasi *Idempotent* di Account

Idempotency key untuk mutasi sudah ada sejak Lab 2: setiap *debit* dan *credit* membawa `reference` berisi ID transfer. Yang belum ada adalah aturan bahwa kombinasi rekening, jenis mutasi, dan `reference` hanya boleh diterapkan sekali. Aturan tersebut paling kuat bila dijaga oleh database sendiri melalui *unique index*, karena dua *retry* yang datang bersamaan pun tetap diperiksa oleh *index* yang sama.

Tambahkan di akhir `account/internal/account/schema.sql`:

```
-- Idempotency (Lab 5): one mutation per account, type and reference. A retried
-- debit with the same reference (the transfer ID) hits this index instead of
-- moving the money twice. Created separately so it also lands on a database
-- created in Labs 1-4, unless that database already holds duplicate references:
-- then this statement fails and the service does not start. Lab 5 starts with
-- make infra-reset for that reason.
create unique index if not exists mutation_idempotency
    on mutation (account_number, type, reference);
```

Jenis mutasi ikut menjadi bagian *idempotency key*. Dengan demikian *debit* dan *credit* dengan `reference` yang sama tetap dianggap dua mutasi yang berbeda.

Selanjutnya `Store.Mutate` perlu memberi tahu pemanggilnya apakah mutasi benar-benar diterapkan atau hanya pengulangan. Ubah deklarasi `Mutate` di *interface* `Store` (`account/internal/account/account.go`):

```
  // Mutate debits or credits the account and records the mutation, atomically.
  // It returns the account with its new balance. When a mutation with the same
  // account, type and reference exists already, nothing changes: it returns
  // the current account and applied = false.
  Mutate(ctx context.Context, number string, t MutationType, req MutationRequest) (a
Account, applied bool, err error)
```

Di Lab 2, `PostgresStore.Mutate` mengubah saldo terlebih dahulu, lalu mencatat baris `mutation`. Urutannya sekarang dibalik: baris `mutation` ditulis lebih dahulu dengan `on conflict do nothing` sebagai pemeriksaan *idempotency*. Bila tidak ada baris yang masuk, mutasi tersebut sudah pernah diterapkan, dan saldo tidak disentuh lagi. Urutan ini juga menangani *race condition*. Panggilan kedua dengan `reference` yang sama tertahan oleh *unique index* sampai transaksi pertama selesai, lalu mendapati konflik. *Foreign key* ke tabel `account` sekaligus memeriksa keberadaan rekening, sehingga pemanggilan `s.Get` untuk membedakan "rekening tidak ada" dari "saldo kurang" tidak diperlukan lagi.

Ganti seluruh fungsi `Mutate` di `account/internal/account/store.go`:

```
func (s *PostgresStore) Mutate(ctx context.Context, number string, t MutationType, req
MutationRequest) (Account, bool, error) {
  tx, err := s.pool.Begin(ctx)
  if err != nil {
    return Account{}, false, err
  }
  defer tx.Rollback(ctx) // no-op after Commit

  // The mutation row goes in first, as the idempotency check. A concurrent call
```

```
  // with the same reference waits on the unique index until this transaction
  // ends and then sees the conflict, so two racing retries still apply once.
  tag, err := tx.Exec(ctx, `
    insert into mutation (account_number, type, amount, reference)
    values (@number, @type, @amount, @reference)
    on conflict (account_number, type, reference) do nothing`,
    pgx.NamedArgs{"number": number, "type": string(t), "amount": req.Amount, "reference":
req.Reference})
  var pgErr *pgconn.PgError
  if errors.As(err, &pgErr) && pgErr.Code == "23503" { // foreign_key_violation: no such
account
    return Account{}, false, ErrNotFound
  }
  if err != nil {
    return Account{}, false, err
  }
  if tag.RowsAffected() == 0 {
    // Applied before: this is a retry of a call whose answer got lost (timeout,
    // dropped connection). Answer with the current account, as if it were the
    // first call. A stricter API would also compare the amount and reject a
    // different amount under the same reference.
    //
    // Read inside the same transaction, not with s.Get: that would take a second
    // pool connection while this one is still held, and under load (pool
    // exhausted by concurrent retries) wait for itself.
    rows, err := tx.Query(ctx,
      `select number, owner_name, balance, created_at from account where number = @number`,
      pgx.NamedArgs{"number": number})
    if err != nil {
      return Account{}, false, err
    }
    a, err := pgx.CollectExactlyOneRow(rows, pgx.RowToStructByPos[Account])
    if err != nil {
      return Account{}, false, err
    }
    return a, false, tx.Commit(ctx)
  }

  delta := req.Amount
  if t == Debit {
    delta = delta.Neg()
  }
  // The balance check sits in the WHERE clause, so two concurrent debits cannot
  // both pass it: the second one waits for the row lock and re-evaluates.
  rows, err := tx.Query(ctx, `
    update account set balance = balance + @delta
    where number = @number and balance + @delta >= 0
    returning number, owner_name, balance, created_at`,
    pgx.NamedArgs{"number": number, "delta": delta})
  if err != nil {
    return Account{}, false, err
  }
  a, err := pgx.CollectExactlyOneRow(rows, pgx.RowToStructByPos[Account])
  if errors.Is(err, pgx.ErrNoRows) {
    // The account exists (the foreign key accepted the mutation row), so the
    // balance is too low. Rollback removes the mutation row again: a later
    // retry with the same reference is evaluated afresh.
    return Account{}, false, ErrInsufficientFunds
  }
```

```
  if err != nil {
    return Account{}, false, err
  }
  return a, true, tx.Commit(ctx)
}
```

Query `update account` di tengah fungsi tidak berubah dari Lab 2. Yang hilang adalah `insert into mutation` di akhir fungsi (sudah pindah ke awal) serta pemanggilan `s.Get` di blok penanganan `pgx.ErrNoRows`.

Blok untuk panggilan ulang (`RowsAffected() == 0`) membaca saldo dengan `tx.Query` di transaksi yang sama, bukan dengan `s.Get`. `s.Get` meminjam koneksi kedua dari *pool* selagi koneksi pertama masih dipegang transaksi. Bila banyak *retry* datang bersamaan dan seluruh koneksi *pool* sudah dipegang, setiap pemanggilan memegang satu koneksi sambil menunggu koneksi kedua yang tidak pernah dilepas, sehingga tidak ada yang selesai.

Di sisi gRPC, mutasi yang tidak diterapkan ulang tetap dijawab sukses. Pemanggil yang melakukan *retry* karena jawaban pertama tidak sampai memang mengharapkan jawaban sukses, dan uangnya tetap hanya berpindah sekali. Ubah bagian akhir fungsi `mutate` di `account/internal/account/grpc.go`:

```
  a, applied, err := s.store.Mutate(ctx, number, t, req)
  if err != nil {
    return Account{}, s.toStatus(err)
  }
  if !applied {
    // Not an error: the caller retried after losing our first answer. It gets
    // a success with the current balance, and the money moved only once.
    s.log.Info("mutation already applied, not repeated", "number", a.Number, "type", t,
"reference", req.Reference)
    return a, nil
  }
  s.log.Info("account mutated", "number", a.Number, "type", t, "amount", req.Amount,
"reference", req.Reference)
  return a, nil
```

Aturan ini termasuk bagian kontrak API, sehingga perlu dituliskan di `proto/account/v1/account.proto`. Ganti komentar *field* `reference` di `DebitRequest` dan tambahkan komentar di `CreditRequest`:

```
message DebitRequest {
  string number = 1;
  string amount = 2;
  // reference ties the mutation to the business transaction, e.g. a transfer ID.
  // It is the idempotency key: a Debit or Credit repeated with the same number
  // and reference is not applied again and answers with the current account,
  // so callers may retry after a timeout.
  string reference = 3;
}

message CreditRequest {
  string number = 1;
  string amount = 2;
  // Same idempotency rule as DebitRequest.reference.
  string reference = 3;
}
```

Hanya komentar yang berubah, tetapi komentar tersebut ikut tersalin ke kode hasil *generate*. Jalankan ulang *generator* agar `proto/gen` tetap sesuai dengan `.proto`:

```
make proto
```

Test di `account` menggunakan `fakeStore` di `account/internal/account/http_test.go`. `fakeStore` harus mengikuti *signature* `Mutate` yang baru dan meniru *unique index* dengan *map*. *Field* `mutateErr` dari Lab 4 tetap ada. Ubah *struct*, konstruktor, dan metode `Mutate`-nya:

```
// fakeStore keeps accounts in memory so the HTTP layer is tested without PostgreSQL.
type fakeStore struct {
  accounts map[string]Account
  // applied holds "number type reference" of every mutation, like the unique
  // index on the mutation table.
  applied   map[string]bool
  pingErr   error
  mutateErr error // returned by Mutate when set, e.g. a context error
}

func newFakeStore() *fakeStore {
  return &fakeStore{accounts: map[string]Account{}, applied: map[string]bool{}}
}

func (f *fakeStore) Mutate(_ context.Context, number string, t MutationType, req
MutationRequest) (Account, bool, error) {
  if f.mutateErr != nil {
    return Account{}, false, f.mutateErr
  }
  a, ok := f.accounts[number]
  if !ok {
    return Account{}, false, ErrNotFound
  }
  key := number + " " + string(t) + " " + req.Reference
  if f.applied[key] {
    return a, false, nil
  }
  if t == Debit {
    if a.Balance.LessThan(req.Amount) {
      return Account{}, false, ErrInsufficientFunds
    }
    a.Balance = a.Balance.Sub(req.Amount)
  } else {
    a.Balance = a.Balance.Add(req.Amount)
  }
  f.accounts[number] = a
  f.applied[key] = true
  return a, true, nil
}
```

Tambahkan *test* berikut di akhir `account/internal/account/grpc_test.go`. *Test* ini mengirim *debit* yang sama dua kali, seperti *retry* setelah *timeout*, dan memeriksa bahwa saldo hanya berkurang sekali:

```
func TestGRPCMutationIsIdempotent(t *testing.T) {
  store := newFakeStore()
  store.accounts["1234567890"] = Account{Number: "1234567890", OwnerName: "Budi", Balance:
decimal.RequireFromString("500000")}
  c := newGRPCClient(t, store)
  ctx := context.Background()

  // The same debit twice, e.g. a retry after a timeout: applied once, and both
  // calls answer with the balance after that one debit.
  for i := range 2 {
    got, err := c.Debit(ctx, &accountv1.DebitRequest{Number: "1234567890", Amount:
"100000", Reference: "T-1"})
    if err != nil || got.GetAccount().GetBalance() != "400000.00" {
```

```
      t.Fatalf("call %d: Debit = %v, %v; want balance 400000.00", i+1, got, err)
    }
  }
  // Same reference, other type: a different mutation (a reversal uses its own
  // reference anyway, see the transfer saga).
  got, err := c.Credit(ctx, &accountv1.CreditRequest{Number: "1234567890", Amount:
"100000", Reference: "T-1"})
  if err != nil || got.GetAccount().GetBalance() != "500000.00" {
    t.Fatalf("Credit = %v, %v; want balance 500000.00", got, err)
  }
}
```

```
make -C account test
```

5.4.3 *Retry* untuk *Timeout* di Client Transfer

Di Lab 3 dan Lab 4, fungsi `notProcessed` di `transfer/internal/accountclient/client.go` hanya mengizinkan *retry* untuk `codes.Unavailable`. *Timeout* sengaja tidak di-*retry* karena *debit* yang terlambat dijawab mungkin sudah diterapkan. Setelah mutasi menjadi *idempotent*, alasan tersebut tidak berlaku lagi: *retry* dengan `reference` yang sama tidak menerapkan mutasi dua kali. Ganti fungsi `notProcessed` dengan `retryable`:

```
// retryable reports whether the call may be retried:
//   - Unavailable: the server refused the call before running it (the fault
//     injector does exactly that) or no connection could be made.
//   - DeadlineExceeded and our own attempt timeout: the answer was late, and the
//     account service may have applied the mutation anyway. Retrying is safe
//     since Lab 5 because Debit and Credit are idempotent by reference (the
//     transfer ID): a repeated call is not applied twice. Before Lab 5 a
//     timed-out debit was never retried, because a retry could debit twice.
//
// Internal and Unknown are not retried: they come from a bug or a broken
// dependency on the server side, and the same request will most likely fail the
// same way. Business codes are final answers.
func retryable(_ any, err error) bool {
  if errors.Is(err, timeout.ErrExceeded) {
    return true
  }
  switch status.Code(err) {
  case codes.Unavailable, codes.DeadlineExceeded:
    return true
  }
  return false
}
```

Sesuaikan juga pemanggilnya di fungsi `New`, pada *builder* `retrypolicy`:

```
  retry := retrypolicy.NewBuilder[any]().
    HandleIf(retryable).
    // Explicit even though retryable already ignores ErrOpen: retrying against
    // an open breaker only burns backoff time and fails anyway.
    AbortOnErrors(circuitbreaker.ErrOpen).
```

Dua *test* di `transfer/internal/accountclient/client_test.go` sekarang memeriksa perilaku yang sudah tidak berlaku. Ganti `TestNoRetryOnTimeout` dan `TestNoRetryOnDeadlineExceeded` dengan dua *test* berikut. *Test* pertama juga memeriksa bahwa *retry* membawa `reference` yang sama:

```
// A timed-out attempt is retried since Lab 5: the account service deduplicates
// by reference, so even a debit that was applied late is not applied twice.
func TestRetryOnTimeout(t *testing.T) {
  c, fake := newTestClient(t, func(ctx context.Context, n int64) error {
```

```
    if n == 1 {
      select {
      case <-time.After(300 * time.Millisecond):
      case <-ctx.Done():
      }
    }
    return nil
  })
  if err := debit(c); err != nil {
    t.Fatalf("err = %v, want success on the second attempt", err)
  }
  if got := fake.calls.Load(); got != 2 {
    t.Fatalf("calls = %d, want 2 (timeout, then OK)", got)
  }
  if req := fake.last.Load(); req.GetReference() != "ref" {
    t.Fatalf("retry reference = %q, want the same reference", req.GetReference())
  }
}

func TestRetryOnDeadlineExceeded(t *testing.T) {
  c, fake := newTestClient(t, always(codes.DeadlineExceeded))
  if err := debit(c); status.Code(err) != codes.DeadlineExceeded {
    t.Fatalf("err = %v, want DeadlineExceeded", err)
  }
  if got := fake.calls.Load(); got != 4 {
    t.Fatalf("calls = %d, want 4 (1 attempt + 3 retries)", got)
  }
}
```

5.4.4 *Policy* Tersendiri untuk *Reversal*

Langkah kompensasi *saga* (bagian berikutnya) adalah *credit* balik ke rekening asal, yang disebut *reversal*. *Reversal* dijalankan tepat setelah *credit* ke rekening tujuan gagal, dan penyebab kegagalan yang paling umum adalah `account` sedang mati atau *circuit breaker* baru saja terbuka. Bila *reversal* melewati *circuit breaker* yang sama, *reversal* langsung ditolak tanpa dikirim, dan transfer berakhir `COMPENSATION_FAILED` sehingga uang tertahan dan harus dikoreksi manual.

Circuit breaker mencegah pemanggil menunggu service yang mati dan mencegah service menerima panggilan dalam jumlah besar sekaligus. Pada *reversal*, kedua manfaat tersebut kalah penting dibanding risiko koreksi manual. *Reversal* jarang terjadi, sedangkan bila percobaannya dihentikan terlalu cepat, transfer harus dikoreksi manual. Karena itu *reversal* mendapat *policy* sendiri:

- tanpa *circuit breaker*;
- *retry* untuk semua kegagalan infrastruktur (`Unavailable`, `DeadlineExceeded`, `Internal`, `Unknown`, *timeout* per percobaan), dengan kriteria yang sama dengan `countsAsFailure`, sedangkan jawaban bisnis seperti `NotFound` tetap final;
- jumlah percobaan tidak dibatasi, tetapi durasinya dibatasi 30 detik, dengan *backoff* 200 ms hingga 5 detik ditambah *jitter*. Batas 30 detik cukup untuk menunggu *restart* `account` atau *pod* yang dipindahkan Kubernetes.

Pengulangan aman karena setiap percobaan membawa `reference` yang sama, `<transfer_id>:reversal`, dan `account` hanya memproses setiap `reference` satu kali.

Lengkapi juga komentar *package* di awal `transfer/internal/accountclient/client.go`:

```
// Package accountclient calls the account service gRPC API through failsafe-go
// policies: per-attempt timeout, retry with backoff, and a circuit breaker,
// installed as a gRPC unary client interceptor. The saga's reversal credit goes
// through a second connection with its own policy (see Reverse).
package accountclient
```

Interceptor gRPC melekat pada koneksi, sehingga *reversal* membutuhkan koneksi gRPC kedua ke `account`. Di `transfer/internal/accountclient/client.go`, tambahkan tiga *field* di akhir *struct* `Settings`, tiga fungsi bantu untuk menghitung durasi terburuk (digunakan *test* `transfer/main_test.go` di bagian berikutnya), lalu isi ketiga *field* tersebut di akhir `ProductionSettings`:

```
  // The reversal (saga compensation) keeps retrying for up to ReversalMaxDuration,
  // with backoff from ReversalBackoffMin to ReversalBackoffMax, and no breaker.
  ReversalMaxDuration time.Duration
  ReversalBackoffMin  time.Duration
  ReversalBackoffMax  time.Duration
}

// jitterFactor is the same for both policies: a delay varies by up to ±25%.
const jitterFactor = 0.25

// WorstCaseCall is the longest a Debit or Credit can take: every attempt runs
// into its timeout, with the longest backoff (plus jitter) between attempts.
func (s Settings) WorstCaseCall() time.Duration {
  attempts := time.Duration(s.MaxRetries + 1)
  backoff := time.Duration(float64(s.BackoffMax) * (1 + jitterFactor))
  return attempts*s.AttemptTimeout + time.Duration(s.MaxRetries)*backoff
}

// WorstCaseReversal is the longest a Reverse can take: the retry window, plus
// one last attempt that started just before the window closed.
func (s Settings) WorstCaseReversal() time.Duration {
  return s.ReversalMaxDuration + s.AttemptTimeout
}

// WorstCaseSaga is the longest the account calls of one transfer can take:
// debit, credit, and the reversal, each at its worst.
func (s Settings) WorstCaseSaga() time.Duration {
  return 2*s.WorstCaseCall() + s.WorstCaseReversal()
}
```

```
  BreakerSuccesses: 3,
  // 30 s covers a restart of the account service (or a pod rescheduled in
  // Kubernetes), which is the usual reason for the forward credit to fail.
  ReversalMaxDuration: 30 * time.Second,
  ReversalBackoffMin:  200 * time.Millisecond,
  ReversalBackoffMax:  5 * time.Second,
}
```

Ganti juga `WithJitterFactor(0.25)` di *builder* `retry` menjadi `WithJitterFactor(jitterFactor)`.

Struct `Client` mendapat koneksi kedua:

```
type Client struct {
  conn    *grpc.ClientConn
  api     accountv1.AccountServiceClient
  breaker circuitbreaker.CircuitBreaker[any]
  // reversalConn carries only Reverse, under the reversal policy.
  reversalConn *grpc.ClientConn
  reversalAPI  accountv1.AccountServiceClient
}
```

Di awal fungsi `New`, tolak durasi *reversal* yang tidak positif. *Policy reversal* tidak membatasi jumlah percobaan, sehingga durasi nol berarti *retry* selamanya:

```
  // A zero duration here would mean "retry forever": the reversal policy has
  // no retry count limit.
```

```
  if s.ReversalMaxDuration <= 0 {
    return nil, errors.New("accountclient: Settings.ReversalMaxDuration must be positive")
  }
```

Lalu ganti bagian akhir `New`, mulai dari `opts = append(...)` hingga `return &Client{...}`, dengan *policy reversal*, dua koneksi, dan fungsi bantu `dial`. Isi `dial` adalah opsi koneksi dari Lab 4 yang sebelumnya ditulis langsung di `New`:

```
  // The reversal policy: retry without a breaker, for a longer but bounded window.
  //
  // A reversal runs right after the forward credit failed, and the usual reason
  // for that failure is the account service being down or the breaker being
  // open. Behind the same breaker, the reversal would be rejected instantly and
  // the transfer would end in COMPENSATION_FAILED: money stuck, a person needed.
  // The breaker exists to protect callers from waiting on a dead service and to
  // spare the service a flood of calls; neither weighs as much here. A reversal
  // is rare, and giving up on it costs a manual correction. So it waits for the
  // account service to come back, up to ReversalMaxDuration.
  //
  // Repeating is safe: every attempt carries the same reference
  // (<transfer_id>:reversal), and the account service applies a reference once.
  reversalRetry := retrypolicy.NewBuilder[any]().
    // Every infrastructure failure, Internal included: giving up early is the
    // expensive outcome here. Business codes (NotFound) are still final.
    HandleIf(countsAsFailure).
    WithMaxRetries(-1). // bounded by the duration, not the count
    WithMaxDuration(s.ReversalMaxDuration).
    WithBackoff(s.ReversalBackoffMin, s.ReversalBackoffMax).
    WithJitterFactor(jitterFactor).
    ReturnLastFailure().
    OnRetry(func(e failsafe.ExecutionEvent[any]) {
      log.Warn("retrying reversal", "next_attempt", e.Attempts(), "elapsed",
e.ElapsedTime(), "cause", e.LastError())
    }).
    Build()
  reversalResilience := failsafegrpc.NewUnaryClientInterceptor[any](reversalRetry,
timeout.New[any](s.AttemptTimeout))

  // An interceptor belongs to a connection, so the reversal gets a connection of
  // its own. The cost is one more TCP connection to the account service.
  conn, err := dial(target, resilience, opts)
  if err != nil {
    return nil, err
  }
  reversalConn, err := dial(target, reversalResilience, opts)
  if err != nil {
    conn.Close()
    return nil, err
  }
  return &Client{
    conn: conn, api: accountv1.NewAccountServiceClient(conn), breaker: breaker,
    reversalConn: reversalConn, reversalAPI:
accountv1.NewAccountServiceClient(reversalConn),
  }, nil
}

func dial(target string, interceptor grpc.UnaryClientInterceptor, extra []grpc.DialOption)
(*grpc.ClientConn, error) {
  opts := append([]grpc.DialOption{
    // Plaintext: fine on a laptop, not in production. Service-to-service calls
```

```
    // there go over TLS (or mTLS through a service mesh); Lab 7 adds tokens.
    grpc.WithTransportCredentials(insecure.NewCredentials()),
    grpc.WithUnaryInterceptor(interceptor),
    // After a failed connect, gRPC waits before reconnecting, growing up to
    // 120 s by default. Capped at 5 s so a restarted account service is used
    // again within seconds, which is what the breaker's half-open trials and
    // the reversal retries need.
    grpc.WithConnectParams(grpc.ConnectParams{
      Backoff:           backoff.Config{BaseDelay: time.Second, Multiplier: 1.6, Jitter:
0.2, MaxDelay: 5 * time.Second},
      MinConnectTimeout: 5 * time.Second,
    }),
  }, extra...)
  conn, err := grpc.NewClient(target, opts...)
  if err != nil {
    return nil, fmt.Errorf("account gRPC client for %s: %w", target, err)
  }
  return conn, nil
}
```

`Close` menutup kedua koneksi, dan metode baru `Reverse` mengirim *credit* melalui koneksi *reversal*:

```
func (c *Client) Close() error { return errors.Join(c.conn.Close(),
c.reversalConn.Close()) }

// Reverse credits the amount back to the source account as the saga's
// compensating step, under the reversal policy (see New): no breaker, retries
// for up to ReversalMaxDuration.
func (c *Client) Reverse(ctx context.Context, number string, amount decimal.Decimal,
reference string) error {
  _, err := c.reversalAPI.Credit(ctx, &accountv1.CreditRequest{Number: number, Amount:
amount.String(), Reference: reference})
  return mapError("reverse", number, err)
}
```

`testSettings` di `client_test.go` juga membutuhkan ketiga *field* baru, karena `New` menolak `ReversalMaxDuration` nol. *File test* di *checkpoint* sudah memuat kedua *test retry* di atas, `testSettings` yang lengkap, dan empat *test reversal*:

- `TestReversalBypassesOpenBreaker`: *circuit breaker* terbuka dan *credit* biasa ditolak tanpa dikirim, sedangkan *reversal* tetap dikirim, di-*retry* selama `account` mati, berhasil setelah `account` kembali, dan selalu membawa `reference T-1:reversal`;
- `TestReversalGivesUpAfterMaxDuration`: *reversal* berhenti setelah batas durasinya;
- `TestReversalBusinessRejectionIsFinal`: `NotFound` tidak di-*retry*;
- `TestNewRejectsMissingReversalWindow`: `New` menolak `ReversalMaxDuration` nol.

Ambil *file* tersebut dari *checkpoint*:

```
git checkout lab-05 -- transfer/internal/accountclient/client_test.go
```

5.4.5 Status *Saga* dan *Event* Transfer

Model domain transfer mendapat tiga tambahan: status baru, jenis *event*, dan *struct* `Event` untuk *payload event*. `EventID` unik untuk setiap *event*, sehingga *consumer* nantinya dapat mengenali *event* yang dikirim ulang (Lab 6).

Di `transfer/internal/transfer/transfer.go`, ganti komentar *package* serta blok `Status` dan konstantanya:

```
// Package transfer moves money between two accounts by calling the account
// service. A transfer is a saga: a sequence of local steps, each with a
```

```
// compensating step that undoes it when a later step fails.
package transfer

// Status is the saga state:
//
//  PENDING -> DEBITED -> COMPLETED   happy path
//  PENDING -> FAILED                 debit failed, nothing to undo
//  DEBITED -> COMPENSATED            credit failed, debit reversed
//  DEBITED -> COMPENSATION_FAILED    credit failed, reversal failed too
type Status string

const (
  Pending   Status = "PENDING"
  Debited   Status = "DEBITED"
  Completed Status = "COMPLETED"
  Failed    Status = "FAILED"
  // Compensated: the credit failed and the debit was reversed. No money moved.
  Compensated Status = "COMPENSATED"
  // CompensationFailed: the source account is debited and the reversal did not
  // go through. Needs a person (or a reconciliation job) to fix.
  CompensationFailed Status = "COMPENSATION_FAILED"
)

// EventType names the integration events other services consume. Only the end
// states that someone outside cares about have an event.
type EventType string

const (
  EventCompleted   EventType = "TransferCompleted"
  EventFailed      EventType = "TransferFailed"
  EventCompensated EventType = "TransferCompensated"
)

// Event is the payload published to Kafka. EventID is unique per event, so a
// consumer can recognise a redelivered one (Lab 6).
type Event struct {
  EventID     string          `json:"event_id"`
  TransferID  string          `json:"transfer_id"`
  FromAccount string          `json:"from_account"`
  ToAccount   string          `json:"to_account"`
  Amount      decimal.Decimal `json:"amount"`
  Status      Status          `json:"status"`
  Reason      string          `json:"reason,omitempty"`
  OccurredAt  time.Time       `json:"occurred_at"`
}
```

Tambahkan `ErrStaleState` ke blok `var` berisi *error*, tepat di bawah `ErrNotFound`:

```
  // ErrStaleState: the transfer was not in the expected state, e.g. another
  // process moved it on already.
  ErrStaleState = errors.New("transfer is not in the expected state")
```

Di blok yang sama, ganti `ErrCreditFailed` dari Lab 3 dengan `ErrReversalFailed`. Sejak Lab 5, *credit* yang gagal setelah *debit* tidak lagi berarti uang tertahan, karena *debit* dikembalikan. Uang baru tertahan bila pengembaliannya juga gagal:

```
  // The credit failed and so did the reversal of the debit
  // (COMPENSATION_FAILED): the source account is short by the amount and needs
  // a manual correction. The transfer is not safe to retry.
  ErrReversalFailed = errors.New("debit reversal failed")
```

Interface `AccountClient` mendapat metode ketiga, `Reverse`, yang diimplementasikan `accountclient.Client` di bagian sebelumnya:

```
type AccountClient interface {
  Debit(ctx context.Context, number string, amount decimal.Decimal, reference string) error
  Credit(ctx context.Context, number string, amount decimal.Decimal, reference string)
error
  // Reverse is a credit that undoes a debit (the saga's compensating step). It
  // runs under its own resiliency policy, not behind the breaker that Debit
  // and Credit share.
  Reverse(ctx context.Context, number string, amount decimal.Decimal, reference string)
error
}
```

Metode `Finish` di *interface* `Store` diganti `Transition`. Perbedaannya ada di dua hal: `Transition` hanya mengubah status bila transfer masih berada di status asal yang diharapkan, dan `Transition` menulis *event* ke `outbox` dalam transaksi yang sama:

```
  // Transition moves the transfer from one status to the next and, when event
  // is not empty, writes that event to the outbox in the same database
  // transaction: either both are stored or neither. It returns ErrStaleState
  // when the transfer is not in status from.
  Transition(ctx context.Context, id string, from, to Status, reason string, event
EventType) (Transfer, error)
```

5.4.6 *Schema* Transfer dan Tabel Outbox

Kolom `status` di Lab 2 berukuran `varchar(10)` dengan *check constraint* untuk tiga status saja, sedangkan `COMPENSATION_FAILED` terdiri atas 19 karakter. Perubahan kolom dan *constraint* ditulis sebagai `alter table` yang dijalankan ulang setiap kali service dimulai. Dengan cara ini database yang dibuat di Lab 2–4 ikut diperbarui. Ganti isi `transfer/internal/transfer/schema.sql`:

```
create table if not exists transfer (
    id             uuid           primary key default gen_random_uuid(),
    from_account   varchar(10)    not null,
    to_account     varchar(10)    not null,
    amount         numeric(19, 2) not null check (amount > 0),
    status         varchar(20)    not null,
    failure_reason text,
    created_at     timestamptz    not null default now(),
    updated_at     timestamptz    not null default now()
);

-- Saga states (Lab 5). Re-applied on every start so a database created in
-- Labs 2-4, with only PENDING/COMPLETED/FAILED, gets the new states as well.
alter table transfer alter column status type varchar(20);
alter table transfer drop constraint if exists transfer_status_check;
alter table transfer add constraint transfer_status_check check (status in
    ('PENDING', 'DEBITED', 'COMPLETED', 'FAILED', 'COMPENSATED', 'COMPENSATION_FAILED'));

-- Transactional outbox (Lab 5). An event is inserted in the same transaction as
-- the state change it reports; the relay publishes it to Kafka afterwards and
-- sets published_at. Writing to the database and to Kafka directly would be two
-- systems without a shared transaction: a crash between them loses the event
-- or publishes one for a change that was rolled back.
create table if not exists outbox (
    id           uuid        primary key,
    aggregate_id uuid        not null,
    event_type   varchar(50) not null,
    payload      jsonb       not null,
    created_at   timestamptz not null default now(),
```

```
    published_at timestamptz
);

-- The relay only ever looks for unpublished rows; a partial index stays small
-- however many published rows pile up.
create index if not exists outbox_unpublished on outbox (created_at) where published_at is
null;
```

Kolom `aggregate_id` berisi ID transfer. Istilah *aggregate* menunjuk entitas yang perubahannya dilaporkan oleh *event* tersebut. Kolom `published_at` yang masih kosong menandakan *event* belum terkirim ke Kafka.

5.4.7 *Package* Outbox dan *Relay*

Package baru `outbox` memiliki dua tanggung jawab. Fungsi `Insert` menulis satu baris `outbox` di dalam transaksi milik pemanggil. `Relay` berjalan sebagai *goroutine*: setiap 500 ms *relay* mengambil hingga 100 baris yang belum terkirim, mengirimnya ke Kafka, lalu mengisi `published_at`.

Tiga keputusan desain di *relay* perlu dipahami sebelum menulis kodenya:

- **Penguncian baris.** Query `for update skip locked` mengunci baris yang sedang dikirim. Bila beberapa *instance* `transfer` berjalan, setiap *relay* melewati baris yang sudah dikunci *relay* lain, sehingga satu baris tidak dikirim dua kali oleh dua *relay* dan *relay* tidak saling menunggu.
- **Urutan kirim lalu tandai.** Kafka mengonfirmasi penerimaan terlebih dahulu, baru kemudian `published_at` diisi dan transaksi di-*commit*. Bila proses berhenti di antara kedua langkah tersebut, baris tetap berstatus belum terkirim dan dikirim lagi pada siklus berikutnya. Cara ini menghasilkan jaminan *at-least-once*: *event* tidak pernah hilang, tetapi sesekali terkirim dua kali. Urutan sebaliknya (tandai dulu, kirim kemudian) justru dapat menghilangkan *event*.
- **Konfigurasi *producer*.** *Key* Kafka berisi ID transfer dan `kafka.Hash` memilih *partition* berdasarkan *key* tersebut, sehingga semua *event* satu transfer tetap berurutan. `RequireAll` menunggu konfirmasi dari semua replika yang sinkron. Pembuatan *topic* otomatis dimatikan juga di sisi *client*.

Tambahkan *dependency* Kafka dan UUID ke modul `transfer`:

```
cd transfer
go get github.com/segmentio/kafka-go@v0.4.51 github.com/google/uuid@v1.6.0
cd ..
```

Buat *file* `transfer/internal/outbox/outbox.go`:

```
// Package outbox implements the transactional outbox: events are stored in the
// service's own database, in the same transaction as the state change they
// report, and a relay publishes them to Kafka afterwards.
package outbox

import (
  "context"
  "fmt"
  "log/slog"
  "time"

  "github.com/jackc/pgx/v5"
  "github.com/jackc/pgx/v5/pgxpool"
  "github.com/segmentio/kafka-go"
)

// Message is one outbox row. The table itself is created by the transfer schema.
type Message struct {
  ID          string // event ID, also sent as Kafka header event_id
  AggregateID string // transfer ID, used as the Kafka key
```

```
  EventType   string
  Payload     []byte // JSON
}

// Insert writes m inside tx. tx must be the transaction that also writes the
// state change, otherwise the outbox gives no guarantee at all.
func Insert(ctx context.Context, tx pgx.Tx, m Message) error {
  _, err := tx.Exec(ctx, `
    insert into outbox (id, aggregate_id, event_type, payload)
    values (@id, @aggregate_id, @event_type, @payload)`,
    pgx.NamedArgs{"id": m.ID, "aggregate_id": m.AggregateID, "event_type": m.EventType,
"payload": m.Payload})
  if err != nil {
    return fmt.Errorf("insert outbox message: %w", err)
  }
  return nil
}

// Publisher is the part of kafka.Writer the relay uses; tests can fake it.
type Publisher interface {
  WriteMessages(ctx context.Context, msgs ...kafka.Message) error
}

// NewKafkaWriter builds the producer for the relay.
func NewKafkaWriter(brokers []string, topic string) *kafka.Writer {
  return &kafka.Writer{
    Addr:  kafka.TCP(brokers...),
    Topic: topic,
    // Same key, same partition: all events of one transfer stay in order.
    Balancer: &kafka.Hash{},
    // Acknowledged by every in-sync replica before WriteMessages returns; only
    // then may the relay mark the rows published.
    RequiredAcks: kafka.RequireAll,
    // The topic is created on purpose, with its partition count (compose
    // kafka-init). A typo in KAFKA_TOPIC must fail, not create a new topic.
    AllowAutoTopicCreation: false,
    // The relay hands over a whole batch in one call; do not wait the default
    // 1 s for more messages that will not come.
    BatchTimeout: 10 * time.Millisecond,
    BatchSize:    BatchSize,
  }
}

const (
  // PollInterval is how long an event waits at most before the relay looks.
  PollInterval = 500 * time.Millisecond
  // BatchSize caps the rows claimed per poll.
  BatchSize = 100
)

type Relay struct {
  pool *pgxpool.Pool
  pub  Publisher
  log  *slog.Logger
}

func NewRelay(pool *pgxpool.Pool, pub Publisher, log *slog.Logger) *Relay {
  return &Relay{pool: pool, pub: pub, log: log}
}
```

```
// Run polls the outbox until ctx is cancelled. A failed batch (Kafka down,
// database down) is logged and tried again on the next tick: the rows stay
// unpublished, nothing is lost.
func (r *Relay) Run(ctx context.Context) {
  r.log.Info("outbox relay started", "interval", PollInterval.String())
  ticker := time.NewTicker(PollInterval)
  defer ticker.Stop()
  for {
    select {
    case <-ctx.Done():
      r.log.Info("outbox relay stopped")
      return
    case <-ticker.C:
    }
    n, err := r.publishBatch(ctx)
    if err != nil {
      if ctx.Err() == nil {
        r.log.Warn("outbox publish failed, will retry", "error", err)
      }
      continue
    }
    if n > 0 {
      r.log.Info("outbox events published", "count", n)
    }
  }
}

// publishBatch claims up to BatchSize unpublished rows, publishes them, and marks
// them published, all while holding the row locks of one transaction.
//
// "for update skip locked" lets several transfer instances run a relay each:
// rows locked by one relay are skipped by the others instead of being published
// twice or blocking them.
//
// Delivery is at-least-once. Kafka has acknowledged the batch before the update
// commits; when the commit fails or the process dies in between, the rows are
// still unpublished and go out again on the next poll. Consumers therefore see
// duplicates now and then and must be idempotent (Lab 6). The opposite order,
// commit first and publish after, would lose events instead: worse.
func (r *Relay) publishBatch(ctx context.Context) (int, error) {
  tx, err := r.pool.Begin(ctx)
  if err != nil {
    return 0, err
  }
  defer tx.Rollback(ctx) // no-op after Commit

  rows, err := tx.Query(ctx, `
    select id::text, aggregate_id::text, event_type, payload
    from outbox
    where published_at is null
    order by created_at
    limit @limit
    for update skip locked`,
    pgx.NamedArgs{"limit": BatchSize})
  if err != nil {
    return 0, err
  }
  pending, err := pgx.CollectRows(rows, pgx.RowToStructByPos[Message])
```

```
  if err != nil {
    return 0, err
  }
  if len(pending) == 0 {
    return 0, nil
  }

  msgs := make([]kafka.Message, len(pending))
  ids := make([]string, len(pending))
  for i, m := range pending {
    msgs[i] = kafka.Message{
      Key:   []byte(m.AggregateID),
      Value: m.Payload,
      Headers: []kafka.Header{
        {Key: "event_type", Value: []byte(m.EventType)},
        {Key: "event_id", Value: []byte(m.ID)},
      },
    }
    ids[i] = m.ID
  }
  // Bounded, so a hanging broker does not hold the row locks forever.
  pubCtx, cancel := context.WithTimeout(ctx, 10*time.Second)
  defer cancel()
  if err := r.pub.WriteMessages(pubCtx, msgs...); err != nil {
    return 0, fmt.Errorf("publish %d events: %w", len(msgs), err)
  }

  // The duplicate window described above starts here.
  if _, err := tx.Exec(ctx, `update outbox set published_at = now() where id =
any(@ids::uuid[])`,
    pgx.NamedArgs{"ids": ids}); err != nil {
    return 0, fmt.Errorf("mark %d events published: %w", len(ids), err)
  }
  return len(pending), tx.Commit(ctx)
}
```

5.4.8 Perubahan Status dan *Event* dalam Satu Transaksi

`PostgresStore.Transition` menjalankan dua perintah dalam satu transaksi: `update transfer` dan, bila ada *event*, `outbox.Insert`. Klausa `and status = @from` menjaga *state machine*. Transfer yang sudah dipindahkan ke status lain oleh proses lain tidak akan tertimpa, dan `Transition` mengembalikan `ErrStaleState`.

Fungsi bantu `one` yang semula metode `PostgresStore` diubah menjadi fungsi biasa yang menerima *interface* `querier`. *Interface* tersebut dipenuhi oleh *pool* maupun transaksi, sehingga query yang sama dapat dijalankan di keduanya.

Di `transfer/internal/transfer/store.go`, tambahkan *import* berikut:

```
  "encoding/json"

  "github.com/google/uuid"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/outbox"
```

Tambahkan metode `Pool` tepat di bawah `Close`. *Relay* membaca tabel `outbox` dari database yang sama, sehingga cukup menggunakan *pool* milik `PostgresStore`:

```
// Pool is shared with the outbox relay, which reads the outbox table of this
// same database.
func (s *PostgresStore) Pool() *pgxpool.Pool { return s.pool }
```

Ganti metode `Finish` dengan `Transition`:

```
func (s *PostgresStore) Transition(ctx context.Context, id string, from, to Status, reason
string, event EventType) (Transfer, error) {
  tx, err := s.pool.Begin(ctx)
  if err != nil {
    return Transfer{}, err
  }
  defer tx.Rollback(ctx) // no-op after Commit

  // "and status = @from" guards the state machine: a transfer that some other
  // process moved on already is not overwritten.
  t, err := one(ctx, tx, `
    update transfer set status = @to, failure_reason = nullif(@reason, ''), updated_at =
now()
    where id = @id and status = @from
    returning `+columns,
    pgx.NamedArgs{"id": id, "from": string(from), "to": string(to), "reason": reason})
  if errors.Is(err, ErrNotFound) {
    return Transfer{}, fmt.Errorf("%s -> %s: %w", from, to, ErrStaleState)
  }
  if err != nil {
    return Transfer{}, err
  }

  if event != "" {
    e := Event{
      EventID:     uuid.NewString(),
      TransferID:  t.ID,
      FromAccount: t.FromAccount,
      ToAccount:   t.ToAccount,
      Amount:      t.Amount,
      Status:      t.Status,
      Reason:      t.FailureReason,
      OccurredAt:  t.UpdatedAt,
    }
    payload, err := json.Marshal(e)
    if err != nil {
      return Transfer{}, err
    }
    if err := outbox.Insert(ctx, tx, outbox.Message{
      ID: e.EventID, AggregateID: t.ID, EventType: string(event), Payload: payload,
    }); err != nil {
      return Transfer{}, err
    }
  }
  return t, tx.Commit(ctx)
}
```

Terakhir, ubah `Create`, `Get`, dan fungsi bantu `one`. Isi `switch` di dalam `one` yang menerjemahkan `pgx.ErrNoRows` dan kode `22P02` menjadi `ErrNotFound` tidak berubah:

```
func (s *PostgresStore) Create(ctx context.Context, req CreateRequest) (Transfer, error) {
  return one(ctx, s.pool, `
    insert into transfer (from_account, to_account, amount, status)
    values (@from, @to, @amount, 'PENDING')
    returning `+columns,
    pgx.NamedArgs{"from": req.FromAccount, "to": req.ToAccount, "amount": req.Amount})
}
```

```
func (s *PostgresStore) Get(ctx context.Context, id string) (Transfer, error) {
  return one(ctx, s.pool, `select `+columns+` from transfer where id = @id`,
pgx.NamedArgs{"id": id})
}

// querier is what the pool and a transaction have in common.
type querier interface {
  Query(ctx context.Context, sql string, args ...any) (pgx.Rows, error)
}

func one(ctx context.Context, q querier, sql string, args pgx.NamedArgs) (Transfer, error)
{
  rows, err := q.Query(ctx, sql, args)
  // ... the rest of the former s.one is unchanged ...
}
```

5.4.9 *Saga Orchestrator* di Service Transfer

Seluruh alur *saga* berada di satu tempat, yaitu `Service.Execute`. Pendekatan ini disebut *orchestration*: satu service mengatur urutan langkah dan kompensasinya. Pada pendekatan lainnya, *choreography*, setiap service bereaksi terhadap *event* dari service lain tanpa pengatur pusat.

Kompensasi berupa *credit* ke rekening asal melalui `AccountClient.Reverse`, dengan `reference` tersendiri, yaitu ID transfer ditambah akhiran `:reversal`. Akhiran tersebut diperlukan karena aturan *idempotency* di `account`. Tanpa akhiran, *reversal* akan dianggap pengulangan *credit* lain dengan ID yang sama. Sebaliknya, *reversal* yang di-*retry* tetap menggunakan `reference` yang sama, sehingga *reversal* pun hanya diterapkan sekali.

Bila *reversal* tetap gagal setelah 30 detik *retry*, transfer berakhir di `COMPENSATION_FAILED` tanpa *event*, karena hasil akhirnya bagi nasabah belum pasti. Kondisi ini dicatat sebagai log `ERROR` untuk ditangani manual, dan *error* yang dikembalikan membungkus `ErrReversalFailed`.

Ganti seluruh isi `transfer/internal/transfer/service.go` mulai dari deklarasi `Service` hingga sebelum metode `Get`. Metode `Get` dan bagian sesudahnya tidak berubah:

```
// ReversalSuffix marks the compensating credit. The reversal needs its own
// reference: under the transfer ID alone, the account service would treat it
// as a repeat of a credit with that ID (see the idempotency rule in
// account.proto). A retried reversal reuses this reference and is therefore
// applied once as well.
const ReversalSuffix = ":reversal"

type Service struct {
  store    Store
  accounts AccountClient
  log      *slog.Logger
}

func NewService(store Store, accounts AccountClient, log *slog.Logger) *Service {
  return &Service{store: store, accounts: accounts, log: log}
}

// Execute runs the transfer saga, orchestrated from here:
//
//  1. record the transfer               -> PENDING
//  2. debit the source                  -> DEBITED   (failed: FAILED)
//  3. credit the destination            -> COMPLETED (failed: step 4)
//  4. compensate: credit the source back -> COMPENSATED
//
// There is no transaction across the two account calls; every step is a local
// transaction in one service, and a failed step is undone by a compensating
```

```
// step instead of a rollback. Each end state and its event are stored together
// (Store.Transition), and the outbox relay publishes the event afterwards.
//
// The returned error is the reason the transfer did not complete; the transfer
// itself is returned whenever it was recorded.
func (s *Service) Execute(ctx context.Context, req CreateRequest) (Transfer, error) {
  t, err := s.store.Create(ctx, req)
  if err != nil {
    return Transfer{}, fmt.Errorf("record transfer: %w", err)
  }

  if err := s.accounts.Debit(ctx, t.FromAccount, t.Amount, t.ID); err != nil {
    // Known gap: when every attempt timed out, the debit may still have been
    // applied, and FAILED is then wrong. Compensating blindly is no answer
    // either: a reversal of a debit that never happened creates money. The
    // production answer is a reconciliation job that repeats the debit with
    // the same reference (idempotent) until it gets a definite answer, then
    // continues or reverses. Here it stays FAILED and is logged.
    return s.finish(ctx, t, Pending, Failed, fmt.Sprintf("debit failed: %v", err),
EventFailed, err)
  }

  debited, err := s.store.Transition(ctx, t.ID, Pending, Debited, "", "")
  if err != nil {
    // The money left the source but our own database did not record it. The
    // transfer stays PENDING; the reconciliation above would find it.
    s.log.Error("debit applied but not recorded", "transfer_id", t.ID, "error", err)
    return t, fmt.Errorf("mark transfer debited: %w", err)
  }

  if err := s.accounts.Credit(ctx, debited.ToAccount, debited.Amount, debited.ID); err !=
nil {
    return s.compensate(ctx, debited, err)
  }

  return s.finish(ctx, debited, Debited, Completed, "", EventCompleted, nil)
}

// compensate undoes the debit by crediting the amount back to the source.
//
// The reversal goes through AccountClient.Reverse, not Credit: the forward credit
// has often just failed because the account service is down, which may well have
// opened the shared breaker. Reverse has no breaker and keeps retrying for a
// while (see accountclient), so a short outage ends in COMPENSATED instead of
// COMPENSATION_FAILED.
//
// Same ambiguity as the debit: a credit that timed out on every attempt may have
// arrived, and the reversal then gives the money back twice over (destination
// and source). Business rejections (destination not found) and an open breaker
// are certain not to have been applied; the reconciliation job resolves the rest.
func (s *Service) compensate(ctx context.Context, t Transfer, cause error) (Transfer,
error) {
  reason := fmt.Sprintf("credit failed: %v", cause)
  s.log.Warn("credit failed after debit, reversing the debit", "transfer_id", t.ID,
"error", cause)

  if err := s.accounts.Reverse(ctx, t.FromAccount, t.Amount, t.ID+ReversalSuffix); err !=
nil {
    // The source account is short by the amount. The reversal is idempotent, so
```

```
    // repeating it later is safe; production runs that from a background job.
    // Here a person repeats it. No event: the customer outcome is not final yet.
    s.log.Error("compensation failed, source account needs manual reversal",
      "transfer_id", t.ID, "from_account", t.FromAccount, "amount", t.Amount, "error", err)
    reason = fmt.Sprintf("%s; reversal failed: %v", reason, err)
    return s.finish(ctx, t, Debited, CompensationFailed, reason, "", fmt.Errorf("%w: %w",
ErrReversalFailed, errors.Join(cause, err)))
  }
  return s.finish(ctx, t, Debited, Compensated, reason, EventCompensated, cause)
}

// finish stores the end state with its event. cause is the saga's outcome error
// (nil on success) and is returned as is, so the HTTP layer can map it.
func (s *Service) finish(ctx context.Context, t Transfer, from, to Status, reason string,
event EventType, cause error) (Transfer, error) {
  done, err := s.store.Transition(ctx, t.ID, from, to, reason, event)
  if err != nil {
    s.log.Error("saga end state not recorded", "transfer_id", t.ID, "status", to, "error",
err)
    return t, errors.Join(cause, fmt.Errorf("mark transfer %s: %w", to, err))
  }
  switch to {
  case Completed:
    s.log.Info("transfer completed", "transfer_id", t.ID, "amount", t.Amount)
  case CompensationFailed:
    // logged in compensate
  default:
    s.log.Warn("transfer not completed", "transfer_id", t.ID, "status", to, "reason",
reason)
  }
  return done, cause
}
```

Komentar di `Execute` dan `compensate` menyebut celah yang sengaja dibiarkan. Bila semua percobaan *debit* atau *credit* berakhir *timeout*, hasilnya di `account` tidak pasti. Lab ini tetap menandai `FAILED` atau menjalankan kompensasi. Sistem *production* menjalankan *reconciliation job* yang mengulang mutasi dengan `reference` yang sama hingga jawabannya pasti, dan pengulangan tersebut aman karena mutasi sudah *idempotent*.

5.4.10 Kode Status HTTP untuk Hasil *Saga*

Di Lab 3 dan Lab 4, setiap kegagalan *credit* setelah *debit* dijawab `502` (`ErrCreditFailed`), karena uang sudah keluar dan *request* tidak boleh diulang. Dengan kompensasi, *credit* yang gagal berakhir `COMPENSATED`. *Debit* sudah dikembalikan sehingga tidak ada uang yang berpindah, dan kode HTTP-nya mengikuti penyebab kegagalan *credit*, sama seperti kegagalan *debit*. Uang hanya tertahan pada `COMPENSATION_FAILED`.

Kode	Kondisi
`201`	`COMPLETED`
`422`	Rekening tidak ada atau saldo kurang (`FAILED`), termasuk `COMPENSATED` karena rekening tujuan tidak ada
`503`	*Circuit breaker* terbuka saat *debit* (`FAILED`), atau saat *credit* lalu *debit* dikembalikan (`COMPENSATED`). Tidak ada uang yang berpindah, sehingga *request* aman diulang
`502`	*Dependency* gagal karena sebab lain (`FAILED` atau `COMPENSATED`)
`500`	`COMPENSATION_FAILED`: *debit* sudah terjadi dan pengembaliannya gagal

`COMPENSATION_FAILED` berarti uang tertahan karena kegagalan di sisi sistem sendiri, apa pun jawaban `account` sebelumnya. Kondisi tersebut dijawab `500` dan tidak boleh tampak sebagai `503` atau `422`, karena kedua kode tersebut membuat pemanggil mengulang *request*. Karena itu pemeriksaan `ErrReversalFailed` berada sebelum pemetaan *error* lain. Pemeriksaan ini menggunakan *error*, bukan `t.Status`, agar tetap berlaku ketika status `COMPENSATION_FAILED` sendiri gagal disimpan. Di `transfer/internal/transfer/http.go`, ubah `switch` di fungsi `create`: hapus kasus `ErrCreditFailed` dan sesuaikan komentarnya.

```
  switch {
  case err == nil:
    w.Header().Set("Location", "/transfers/"+t.ID)
    writeJSON(w, http.StatusCreated, t)
  case t.ID == "":
    h.internalError(w, r, err)
  case errors.Is(err, ErrReversalFailed):
    // Checked first: the source account is debited and the reversal did not go
    // through (COMPENSATION_FAILED, also when that state itself could not be
    // stored). Money is stuck on our side, whatever the account service
    // answered: this is our failure, not the caller's, and never a 503 or 422
    // that would invite a retry.
    writeJSON(w, http.StatusInternalServerError, t)
  case errors.Is(err, ErrAccountNotFound), errors.Is(err, ErrInsufficientFunds):
    // Includes COMPENSATED when the destination account does not exist.
    writeJSON(w, http.StatusUnprocessableEntity, t)
  case errors.Is(err, ErrAccountUnavailable):
    // The open breaker refused the debit (FAILED), or refused the credit and the
    // debit was reversed (COMPENSATED). Either way no money moved: 503 tells the
    // caller it is safe to come back later.
    writeJSON(w, http.StatusServiceUnavailable, t)
  default:
    // Recorded but not completed because a dependency failed (FAILED or
    // COMPENSATED).
    writeJSON(w, http.StatusBadGateway, t)
  }
```

Test `TestCreateStatus` di `transfer/internal/transfer/http_test.go` dari Lab 3 ikut berubah. Kasus "*breaker* terbuka di antara *debit* dan *credit*" sekarang mengharapkan `503`, kasus "rekening tujuan tidak ada" mengharapkan `422`, dan kasus baru "*reversal* juga gagal" mengharapkan `500`. *File*-nya diambil dari *checkpoint* di bagian *test saga*.

5.4.11 Konfigurasi dan *Relay* di `main.go` Transfer

`transfer` membutuhkan dua variabel baru, `KAFKA_BROKERS` dan `KAFKA_TOPIC`. Seperti variabel lain, keduanya wajib diisi, dan service berhenti saat *startup* bila salah satunya kosong. Di `transfer/config.go`, tambahkan dua *field* di *struct* `Config`:

```
  // KafkaBrokers is a comma-separated list of host:port.
  KafkaBrokers []string
  KafkaTopic   string
```

lalu isi keduanya di `LoadConfig`, di bawah `AccountGRPCAddr`:

```
    KafkaBrokers:    strings.Split(get("KAFKA_BROKERS"), ","),
    KafkaTopic:      get("KAFKA_TOPIC"),
```

Tambahkan kedua variabel ke target `run` di `transfer/Makefile`:

```
run:
  HTTP_ADDR=:28082 \
  DATABASE_URL=postgres://transfer:transfer123@localhost:15433/transfer \
  ACCOUNT_GRPC_ADDR=localhost:29091 \
```

```
KAFKA_BROKERS=localhost:19092 \
KAFKA_TOPIC=transfer-events \
go run .
```

Relay dijalankan sebagai *goroutine* di `transfer/main.go`. *Writer* Kafka tidak membuka koneksi sebelum pengiriman pertama, sehingga `transfer` tetap bisa dimulai dan menerima transfer selama Kafka mati. Urutan *shutdown* juga penting: *relay* harus berhenti sebelum *pool* database ditutup oleh `defer store.Close()`. `defer` dijalankan dalam urutan terbalik, sehingga fungsi untuk menghentikan *relay* yang didaftarkan belakangan akan berjalan lebih dahulu.

Tambahkan *import* `.../transfer/internal/outbox`, lalu sisipkan blok berikut di fungsi `run`, tepat setelah `srv := &http.Server{...}` dan sebelum `errCh := make(chan error, 1)`:

```
// The relay connects lazily too: the service starts and accepts transfers
// while Kafka is down; events wait in the outbox until it is back.
writer := outbox.NewKafkaWriter(cfg.KafkaBrokers, cfg.KafkaTopic)
defer writer.Close()
relayCtx, stopRelay := context.WithCancel(context.Background())
relayDone := make(chan struct{})
go func() {
  outbox.NewRelay(store.Pool(), writer, log).Run(relayCtx)
  close(relayDone)
}()
// Runs before the deferred store.Close: the relay must stop using the pool first.
defer func() {
  stopRelay()
  <-relayDone
}()
```

Lengkapi log *startup* dengan konfigurasi Kafka:

```
    log.Info("transfer service listening", "addr", cfg.HTTPAddr, "account_grpc",
cfg.AccountGRPCAddr,
      "kafka_brokers", cfg.KafkaBrokers, "kafka_topic", cfg.KafkaTopic)
```

Saat menerima SIGTERM, server HTTP menunggu transfer yang sedang berjalan. Di Lab 4 batasnya 10 detik, padahal durasi *saga* kini bisa jauh lebih panjang: *debit* dan *credit* masing-masing hingga 11,75 detik (4 percobaan × 2 detik ditambah 3 *backoff* × 1,25 detik), lalu *reversal* hingga 32 detik (durasi *retry* 30 detik ditambah satu percobaan terakhir 2 detik). Bila *saga* terhenti di tengah jalan, barisnya tertinggal di status `PENDING` atau `DEBITED` dan tidak ada proses yang melanjutkannya. Tambahkan konstanta berikut di `transfer/main.go`, di atas fungsi `main`:

```
// shutdownGrace is how long the HTTP server waits on SIGTERM for transfers in
// flight. A saga cut mid-way leaves a PENDING or DEBITED row that nothing
// resumes (see README), so the wait covers the worst case of one saga under
// accountclient.ProductionSettings:
//
//  debit    4 attempts x 2 s + 3 backoffs x 1.25 s  = 11.75 s
//  credit   same                                    = 11.75 s
//  reversal 30 s retry window + one last 2 s attempt = 32 s
//                                                      55.5 s
//
// plus a few seconds for the database writes in between. TestShutdownGrace
// fails when the policies grow past it. The platform must give the process at
// least this long before SIGKILL (Kubernetes: terminationGracePeriodSeconds).
const shutdownGrace = 60 * time.Second
```

Di cabang `case <-ctx.Done():`, ganti `10*time.Second` dengan `shutdownGrace`, lalu tambahkan komentar berikut tepat sebelum `srv.Shutdown`. Komentar ini menjelaskan mengapa urutan *shutdown* sudah benar tanpa kode tambahan:

```
    shutdownCtx, cancel := context.WithTimeout(context.Background(), shutdownGrace)
    defer cancel()
    // HTTP first: transfers in flight finish and write their outbox rows; the
    // deferred relay stop comes after, and anything left is published by the
    // next start.
```

Angka 60 detik harus tetap mengikuti *policy* di `accountclient`. Buat `transfer/main_test.go` yang gagal bila durasi terburuk *saga* melewati `shutdownGrace`:

```
package main

import (
  "testing"
  "time"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/
accountclient"
)

// TestShutdownGrace keeps shutdownGrace in step with the resiliency policies: a
// longer retry or reversal window must come with a longer shutdown wait.
func TestShutdownGrace(t *testing.T) {
  const dbMargin = 3 * time.Second
  if need := accountclient.ProductionSettings.WorstCaseSaga() + dbMargin; shutdownGrace <
need {
    t.Fatalf("shutdownGrace = %s, want at least %s (worst-case saga + %s)", shutdownGrace,
need, dbMargin)
  }
}
```

Masih ada satu celah. Bila proses dimatikan paksa (SIGKILL atau *crash*) di antara dua langkah, baris `PENDING` atau `DEBITED` tertinggal tanpa ada yang melanjutkannya. Sistem *production* menambahkan *recovery job* yang melanjutkan atau mengompensasi baris seperti itu. Di Lab 9, `terminationGracePeriodSeconds` untuk `transfer` disesuaikan dengan angka ini.

5.4.12 *Test* untuk *Saga*

`fakeStore` di `transfer/internal/transfer/service_test.go` harus mengganti `Finish` dengan `Transition` dan mencatat *event* yang ditulis, sehingga *test* dapat memeriksa urutan panggilan ke `account` sekaligus *event* yang masuk ke *outbox*. Bagian utamanya:

```
// fakeStore keeps transfers in memory and records the outbox events written
// with each transition.
type fakeStore struct {
  transfers map[string]Transfer
  events    []EventType
  seq       int
}

func (f *fakeStore) Transition(_ context.Context, id string, from, to Status, reason
string, event EventType) (Transfer, error) {
  t := f.transfers[id]
  if t.Status != from {
    return Transfer{}, ErrStaleState
  }
  t.Status, t.FailureReason = to, reason
  f.transfers[id] = t
  if event != "" {
    f.events = append(f.events, event)
  }
```

```
  return t, nil
}
```

`fakeAccounts` juga berubah. *Fake* ini kini mengenali setiap panggilan dari deskripsinya (`debit 1111111111`, `credit 2222222222`, `reverse 1111111111`), mengambil *error* dari *map*, dan mencatat setiap panggilan beserta `reference`-nya. Dengan begitu *test* dapat memeriksa bahwa pengembalian menggunakan `Reverse` dengan akhiran `:reversal`:

```
// fakeAccounts answers each call by its description ("debit 1111111111",
// "credit 2222222222", "reverse 1111111111") and records the calls with their
// references.
type fakeAccounts struct {
  errs  map[string]error
  calls []string
}

func (f *fakeAccounts) Reverse(_ context.Context, number string, _ decimal.Decimal, ref
string) error {
  return f.call("reverse "+number, ref)
}

func (f *fakeAccounts) call(desc, ref string) error {
  f.calls = append(f.calls, desc+" ref="+ref)
  return f.errs[desc]
}
```

Table-driven test `TestExecuteSaga` memeriksa enam skenario: sukses, saldo kurang, *circuit breaker* terbuka saat *debit*, *credit* gagal lalu dikembalikan, *circuit breaker* terbuka di antara *debit* dan *credit* (tetap dikembalikan karena `Reverse` tidak melewati *breaker*), dan pengembalian yang juga gagal. Sebagai contoh, skenario kompensasi mengharapkan tiga panggilan ke `account` dan satu *event*:

```
    {"credit fails after debit: debit reversed",
      map[string]error{"credit 2222222222": ErrAccountNotFound}, Compensated,
ErrAccountNotFound,
      []string{"debit 1111111111 ref=1", "credit 2222222222 ref=1", "reverse 1111111111
ref=1:reversal"},
      []EventType{EventCompensated}},
```

`http_test.go` menggunakan `fakeAccounts` yang sama, sehingga kedua *file* diambil bersama dari *checkpoint*. `service_test.go` di *checkpoint* juga tetap memuat `TestValidateAmount` dari Lab 4:

```
git checkout lab-05 -- transfer/internal/transfer/service_test.go transfer/internal/
transfer/http_test.go
cd transfer && go mod tidy && cd ..
make -C transfer test
```

5.4.13 Service `notification`

`notification` adalah modul Go baru di *workspace*. Service ini mengetahui hasil transfer hanya dari *event* di Kafka dan tidak pernah memanggil `transfer`. Struktur *payload event* ditulis ulang di modul `notification` dan tidak diimpor dari `transfer`. Kedua service hanya berbagi format data (kontrak), sedangkan kodenya tetap terpisah.

Buat modul dan daftarkan ke `go.work`:

```
mkdir -p notification/internal/notification
cd notification
go mod init github.com/artivisi/training-microservices-2026-01/studi-kasus/notification
go mod edit -go=1.26.0
```

```
cd ..
go work use ./notification
```

`go mod edit -go=1.26.0` menyamakan versi Go modul dengan `go.work`, karena `go mod init` menuliskan versi *toolchain* yang terpasang di laptop.

5.4.13.1 Konfigurasi

Selain variabel yang sudah dikenal dari service lain, `notification` membutuhkan `KAFKA_GROUP_ID`. Kafka menyimpan *committed offset* (posisi baca terakhir) per *consumer group*. Semua *instance* `notification` menggunakan *group* yang sama, sehingga setiap *event* ditangani oleh salah satu *instance* saja.

Buat `notification/config.go`:

```
package main

import (
  "fmt"
  "os"
  "strings"
)

// Config comes from environment variables only (12-factor). Every variable is
// required: a missing value stops the service at startup instead of running
// with a guessed default.
type Config struct {
  HTTPAddr    string
  DatabaseURL string
  // KafkaBrokers is a comma-separated list of host:port.
  KafkaBrokers []string
  KafkaTopic   string
  // KafkaGroupID is the consumer group. Kafka keeps the committed offsets per
  // group; every instance of this service uses the same group, so each event is
  // handled by one of them.
  KafkaGroupID string
}

func LoadConfig() (Config, error) {
  var missing []string
  get := func(name string) string {
    v := os.Getenv(name)
    if v == "" {
      missing = append(missing, name)
    }
    return v
  }
  cfg := Config{
    HTTPAddr:     get("HTTP_ADDR"),
    DatabaseURL:  get("DATABASE_URL"),
    KafkaBrokers: strings.Split(get("KAFKA_BROKERS"), ","),
    KafkaTopic:   get("KAFKA_TOPIC"),
    KafkaGroupID: get("KAFKA_GROUP_ID"),
  }
  if len(missing) > 0 {
    return Config{}, fmt.Errorf("missing required environment variables: %s",
strings.Join(missing, ", "))
  }
  return cfg, nil
}
```

Buat `notification/Makefile`. Seperti service lain, semua langkah *pipeline* nantinya memanggil target di *file* ini:

```
# Every pipeline step calls a target here, so CI and laptops run the same commands.
SERVICE := notification

.PHONY: run test build clean

run:
  HTTP_ADDR=:28083 \
  DATABASE_URL=postgres://notification:notification123@localhost:15434/notification \
  KAFKA_BROKERS=localhost:19092 \
  KAFKA_TOPIC=transfer-events \
  KAFKA_GROUP_ID=notification \
  go run .

test:
  go vet ./...
  go test ./...

build:
  CGO_ENABLED=0 go build -o bin/$(SERVICE) .

clean:
  rm -rf bin
```

5.4.13.2 Model dan *Schema*

Kolom `event_id` di tabel `notification` sengaja belum dibuat unik. *Event* yang dikirim ulang akan tersimpan dan dikirim lagi sebagai notifikasi ganda. Celah ini terlihat di bagian verifikasi dan ditutup di Lab 6.

Buat `notification/internal/notification/schema.sql`:

```
-- One row per notification sent to the customer (here: logged).
-- event_id is deliberately NOT unique yet: a redelivered event is stored and
-- sent again. Lab 6 makes the consumer idempotent.
create table if not exists notification (
    id          bigint generated always as identity primary key,
    event_id    uuid        not null,
    transfer_id uuid        not null,
    event_type  varchar(50) not null,
    message     text        not null,
    created_at  timestamptz not null default now()
);

create index if not exists notification_transfer on notification (transfer_id);
```

Buat `notification/internal/notification/notification.go`. Fungsi `Message` menyusun teks untuk nasabah berdasarkan jenis *event*. Jenis *event* yang tidak dikenal ditandai `ok = false` dan dilewati oleh *consumer*:

```
// Package notification tells customers about the outcome of their transfers. It
// learns about transfers only from events on Kafka; it never calls the transfer
// service.
package notification

import (
  "context"
  "fmt"
  "time"
```

```
  "github.com/shopspring/decimal"
)

type Notification struct {
  ID         int64     `json:"id"`
  EventID    string    `json:"event_id"`
  TransferID string    `json:"transfer_id"`
  EventType  string    `json:"event_type"`
  Message    string    `json:"message"`
  CreatedAt  time.Time `json:"created_at"`
}

// TransferEvent is the payload published by the transfer service. It is
// declared here again, not imported: services share the message format (the
// contract), not code. This side reads only the fields it needs.
type TransferEvent struct {
  EventID     string          `json:"event_id"`
  TransferID  string          `json:"transfer_id"`
  FromAccount string          `json:"from_account"`
  ToAccount   string          `json:"to_account"`
  Amount      decimal.Decimal `json:"amount"`
  Status      string          `json:"status"`
  OccurredAt  time.Time       `json:"occurred_at"`
}

// Message builds the customer-facing text for an event type. ok is false for an
// event type this service does not notify about.
func Message(eventType string, e TransferEvent) (msg string, ok bool) {
  amount := e.Amount.StringFixed(2)
  switch eventType {
  case "TransferCompleted":
    return fmt.Sprintf("Transfer %s dari %s ke %s berhasil.", amount, e.FromAccount,
e.ToAccount), true
  case "TransferFailed":
    return fmt.Sprintf("Transfer %s dari %s ke %s gagal. Saldo tidak berubah.", amount,
e.FromAccount, e.ToAccount), true
  case "TransferCompensated":
    return fmt.Sprintf("Transfer %s dari %s ke %s dibatalkan. Dana sudah dikembalikan ke
%s.",
      amount, e.FromAccount, e.ToAccount, e.FromAccount), true
  }
  return "", false
}

type Store interface {
  Insert(ctx context.Context, n Notification) (Notification, error)
  ListByTransfer(ctx context.Context, transferID string) ([]Notification, error)
  Ping(ctx context.Context) error
}
```

Buat `notification/internal/notification/store.go`:

```
package notification

import (
  "context"
  _ "embed"
  "fmt"
```

```
  "github.com/jackc/pgx/v5"
  "github.com/jackc/pgx/v5/pgxpool"
)

//go:embed schema.sql
var schema string

const columns = `id, event_id::text, transfer_id::text, event_type, message, created_at`

type PostgresStore struct {
  pool *pgxpool.Pool
}

// NewPostgresStore connects, verifies the connection, and applies the schema.
func NewPostgresStore(ctx context.Context, databaseURL string) (*PostgresStore, error) {
  pool, err := pgxpool.New(ctx, databaseURL)
  if err != nil {
    return nil, fmt.Errorf("parse DATABASE_URL: %w", err)
  }
  if err := pool.Ping(ctx); err != nil {
    pool.Close()
    return nil, fmt.Errorf("connect to database: %w", err)
  }
  if _, err := pool.Exec(ctx, schema); err != nil {
    pool.Close()
    return nil, fmt.Errorf("apply schema: %w", err)
  }
  return &PostgresStore{pool: pool}, nil
}

func (s *PostgresStore) Close() { s.pool.Close() }

func (s *PostgresStore) Ping(ctx context.Context) error { return s.pool.Ping(ctx) }

func (s *PostgresStore) Insert(ctx context.Context, n Notification) (Notification, error) {
  rows, err := s.pool.Query(ctx, `
    insert into notification (event_id, transfer_id, event_type, message)
    values (@event_id, @transfer_id, @event_type, @message)
    returning `+columns,
    pgx.NamedArgs{"event_id": n.EventID, "transfer_id": n.TransferID, "event_type":
n.EventType, "message": n.Message})
  if err != nil {
    return Notification{}, err
  }
  return pgx.CollectExactlyOneRow(rows, pgx.RowToStructByPos[Notification])
}

func (s *PostgresStore) ListByTransfer(ctx context.Context, transferID string)
([]Notification, error) {
  rows, err := s.pool.Query(ctx,
    `select `+columns+` from notification where transfer_id = @transfer_id order by id`,
    pgx.NamedArgs{"transfer_id": transferID})
  if err != nil {
    return nil, err
  }
  return pgx.CollectRows(rows, pgx.RowToStructByPos[Notification])
}
```

5.4.13.3 *Consumer* Kafka

Consumer menggunakan `kafka.Reader` dari kafka-go dengan `GroupID`. Kafka membagi *partition topic* ke anggota *group*. Karena *topic* memiliki 3 *partition*, pekerjaan dapat dibagi ke paling banyak tiga *instance*. *Group* yang belum memiliki *committed offset* mulai membaca dari *event* paling awal (`kafka.FirstOffset`), sehingga *event* yang dikirim ke Kafka sebelum `notification` pertama kali dijalankan tidak terlewat.

Urutan pemrosesan menentukan jaminan pengiriman. *Offset* baru di-*commit* setelah notifikasi tersimpan di database. Bila proses berhenti di antara kedua langkah tersebut, *event* dikirim ulang setelah *restart* (*at-least-once*) dan tidak pernah hilang. Konsekuensinya, *event* yang sama dapat diterima lebih dari sekali, baik karena urutan ini maupun karena duplikasi dari *relay outbox* yang sudah dibahas sebelumnya.

Dua jenis kegagalan ditangani dengan cara berbeda. Pada kegagalan sementara, seperti Kafka atau database yang mati, koneksi putus, atau *timeout*, service tidak berhenti dan menunggu hingga kondisinya kembali normal. Kegagalan permanen berasal dari *event* itu sendiri, sehingga setiap pengulangan pasti gagal lagi dan menahan semua *event* berikutnya di *partition* yang sama. *Event* seperti ini disebut *poison message*, dan *consumer* mencatatnya di log level `ERROR` lalu melewatinya:

- JSON yang tidak dapat dibaca;
- `event_id` atau `transfer_id` yang bukan UUID dalam bentuk standar 36 karakter (`xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx`), karena kolom tujuannya bertipe `uuid` dan *insert* akan gagal dengan `22P02` di setiap percobaan;
- *error* PostgreSQL kelas 22 (data tidak valid, misalnya `22P02` atau `22001`) dan kelas 23 (*constraint*, misalnya `23502` atau `23514`).

Error lain, termasuk kelas 42 (misalnya tabel tidak ada), tetap diulang. Kesalahan *schema* seperti itu menggagalkan setiap *event* tanpa kecuali, sehingga melewatinya berarti membuang seluruh notifikasi. Jenis *event* yang tidak dikenal juga dilewati, dengan log level `WARN`. Di sistem *production*, *event* yang dilewati dikirim ke *dead-letter topic* untuk diperiksa secara manual.

Tambahkan *dependency* modul:

```
cd notification
go get github.com/segmentio/kafka-go@v0.4.51 github.com/jackc/pgx/v5@v5.11.0 \
  github.com/google/uuid@v1.6.0 github.com/shopspring/decimal@v1.6.0
cd ..
```

Buat `notification/internal/notification/consumer.go`:

```
package notification

import (
  "context"
  "encoding/json"
  "errors"
  "log/slog"
  "strings"
  "time"

  "github.com/google/uuid"
  "github.com/jackc/pgx/v5/pgconn"
  "github.com/segmentio/kafka-go"
)

// Reader is the part of kafka.Reader the consumer uses; tests can fake it.
type Reader interface {
```

```
  FetchMessage(ctx context.Context) (kafka.Message, error)
  CommitMessages(ctx context.Context, msgs ...kafka.Message) error
}

// NewKafkaReader joins the consumer group. Kafka assigns the topic's partitions
// to the group's members; with 3 partitions, up to 3 instances share the work.
func NewKafkaReader(brokers []string, topic, groupID string) *kafka.Reader {
  return kafka.NewReader(kafka.ReaderConfig{
    Brokers: brokers,
    Topic:   topic,
    GroupID: groupID,
    // Where a group without a committed offset starts: from the oldest event,
    // so nothing published before the first start is missed.
    StartOffset: kafka.FirstOffset,
    // 0 = CommitMessages commits synchronously, at the moment we call it.
    CommitInterval: 0,
  })
}

type Consumer struct {
  reader Reader
  store  Store
  log    *slog.Logger
  // retryDelay is the pause before a failed fetch or insert is tried again.
  retryDelay time.Duration
}

func NewConsumer(reader Reader, store Store, log *slog.Logger) *Consumer {
  return &Consumer{reader: reader, store: store, log: log, retryDelay: time.Second}
}

// Run handles events one by one until ctx is cancelled. Kafka or database
// outages are waited out, never a reason to stop.
//
// The offset is committed only AFTER the notification is stored: a crash in
// between means the event is delivered again after the restart (at-least-once),
// never that it is lost. The price is duplicates; together with the outbox
// relay's own duplicate window this consumer does see the same event twice
// sometimes. It is deliberately NOT idempotent yet: a redelivered event becomes
// a second notification. Lab 6 fixes that.
func (c *Consumer) Run(ctx context.Context) {
  for {
    m, err := c.reader.FetchMessage(ctx)
    if err != nil {
      if ctx.Err() != nil {
        return
      }
      // Kafka unreachable or the group rebalancing. Not a reason to stop the
      // service: wait and fetch again; the committed offset says where.
      c.log.Warn("fetching event failed, retrying", "error", err)
      if !c.pause(ctx) {
        return
      }
      continue
    }
    if err := c.handle(ctx, m); err != nil {
      return // only ctx cancellation ends handle with an error
    }
    if err := c.reader.CommitMessages(ctx, m); err != nil {
```

```
      if ctx.Err() != nil {
        return
      }
      // The notification is stored but Kafka does not know. The event comes
      // again after a restart or rebalance: one more duplicate, nothing lost.
      c.log.Warn("committing offset failed", "partition", m.Partition, "offset", m.Offset,
"error", err)
    }
  }
}

// handle stores the notification for m. Errors fall in two groups:
//
//   - Transient (database down, connection lost, timeout): retried until it
//     succeeds. Skipping would lose the notification, and moving on to later
//     events of the same partition would break their order.
//   - Permanent (the event itself is bad): logged at ERROR and skipped, so the
//     offset is committed. Retrying would fail the same way forever and block
//     every later event of the partition behind it (a poison message). In
//     production the event would go to a dead-letter topic for a person to look
//     at.
//
// The event is validated before the insert, so most bad events never reach the
// database; permanentDBError catches what slips through.
func (c *Consumer) handle(ctx context.Context, m kafka.Message) error {
  eventType := header(m, "event_type")
  log := c.log.With("event_type", eventType, "partition", m.Partition, "offset", m.Offset)

  var e TransferEvent
  if err := json.Unmarshal(m.Value, &e); err != nil {
    log.Error("skipping unreadable event", "error", err, "payload", string(m.Value))
    return nil
  }
  // The columns are uuid: a malformed ID would fail the insert with 22P02 on
  // every try.
  if !isUUID(e.EventID) || !isUUID(e.TransferID) {
    log.Error("skipping event with invalid event_id or transfer_id", "event_id", e.EventID,
      "transfer_id", e.TransferID, "payload", string(m.Value))
    return nil
  }
  msg, ok := Message(eventType, e)
  if !ok {
    log.Warn("skipping event type without notification", "event_id", e.EventID)
    return nil
  }

  n := Notification{EventID: e.EventID, TransferID: e.TransferID, EventType: eventType,
Message: msg}
  for {
    stored, err := c.store.Insert(ctx, n)
    if err == nil {
      // Stand-in for the real channel (SMS, push, e-mail).
      log.Info("notification sent", "event_id", e.EventID, "transfer_id", e.TransferID,
        "notification_id", stored.ID, "message", msg)
      return nil
    }
    if permanentDBError(err) {
      log.Error("skipping event the database rejects", "event_id", e.EventID, "error", err,
        "payload", string(m.Value))
```

```
      return nil
    }
    log.Warn("storing notification failed, retrying", "event_id", e.EventID, "error", err)
    if !c.pause(ctx) {
      return errors.Join(ctx.Err(), err)
    }
  }
}

// isUUID accepts only the canonical 36-character form, the one the transfer
// service writes. uuid.Validate alone also accepts forms like "urn:uuid:...",
// which PostgreSQL rejects.
func isUUID(s string) bool {
  return len(s) == 36 && uuid.Validate(s) == nil
}

// permanentDBError reports whether PostgreSQL rejected the data itself, so the
// same insert fails the same way on every retry: SQLSTATE class 22 (data
// exception, e.g. 22P02 invalid uuid text, 22001 value too long) and class 23
// (integrity constraint, e.g. 23502 not null, 23514 check). Everything else is
// treated as transient and retried: connection errors, timeouts, and also
// schema errors (class 42, e.g. a missing table). Those fail every event, not
// one; retrying blocks consumption but loses nothing until a fix is deployed,
// while skipping would throw away every notification.
func permanentDBError(err error) bool {
  var pgErr *pgconn.PgError
  if !errors.As(err, &pgErr) {
    return false
  }
  return strings.HasPrefix(pgErr.Code, "22") || strings.HasPrefix(pgErr.Code, "23")
}

// pause waits retryDelay and reports false when ctx ended first.
func (c *Consumer) pause(ctx context.Context) bool {
  select {
  case <-ctx.Done():
    return false
  case <-time.After(c.retryDelay):
    return true
  }
}

func header(m kafka.Message, key string) string {
  for _, h := range m.Headers {
    if h.Key == key {
      return string(h.Value)
    }
  }
  return ""
}
```

Kegagalan sementara saat menyimpan ke database di-*retry* terus-menerus dan tidak dilewati. Melewati *event* berarti notifikasinya hilang, dan melanjutkan ke *event* berikutnya di *partition* yang sama akan merusak urutan. *Event* yang dilewati karena kegagalan permanen tetap di-*commit*, agar tidak menghambat *partition* selamanya.

5.4.13.4 API HTTP dan `main.go`

API `notification` hanya menyediakan `GET /notifications?transfer_id=...` beserta *probe*. Parameter `transfer_id` wajib diisi agar *channel* tidak dapat meminta daftar notifikasi seluruh nasabah sekaligus.

Readiness probe hanya memeriksa database. Selama Kafka mati, *consumer* cukup menunggu, dan API tetap dapat menjawab dari database.

Buat `notification/internal/notification/http.go`:

```
package notification

import (
  "context"
  "encoding/json"
  "log/slog"
  "net/http"
  "time"

  "github.com/google/uuid"
)

type Handler struct {
  store Store
  log   *slog.Logger
}

func NewHandler(store Store, log *slog.Logger) *Handler {
  return &Handler{store: store, log: log}
}

func (h *Handler) Routes() *http.ServeMux {
  mux := http.NewServeMux()
  mux.HandleFunc("GET /notifications", h.list)
  mux.HandleFunc("GET /healthz", h.healthz)
  mux.HandleFunc("GET /readyz", h.readyz)
  return mux
}

// list answers GET /notifications?transfer_id=... The filter is required: an
// unfiltered list of every customer's notifications is not something a channel
// should be able to ask for.
func (h *Handler) list(w http.ResponseWriter, r *http.Request) {
  transferID := r.URL.Query().Get("transfer_id")
  if _, err := uuid.Parse(transferID); err != nil {
    writeError(w, http.StatusBadRequest, "query parameter transfer_id must be a transfer ID
(UUID)")
    return
  }
  ns, err := h.store.ListByTransfer(r.Context(), transferID)
  if err != nil {
    h.log.Error("request failed", "method", r.Method, "path", r.URL.Path, "error", err)
    writeError(w, http.StatusInternalServerError, "internal error")
    return
  }
  if ns == nil {
    ns = []Notification{} // [] in JSON, not null
  }
  writeJSON(w, http.StatusOK, ns)
}

func (h *Handler) healthz(w http.ResponseWriter, _ *http.Request) {
  writeJSON(w, http.StatusOK, map[string]string{"status": "UP"})
}
```

```
// readyz checks the database only. Kafka is not checked: while it is down the
// consumer simply waits, and the HTTP API can still answer from the database.
func (h *Handler) readyz(w http.ResponseWriter, r *http.Request) {
  ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
  defer cancel()
  if err := h.store.Ping(ctx); err != nil {
    h.log.Warn("readiness check failed", "error", err)
    writeJSON(w, http.StatusServiceUnavailable, map[string]string{"status": "DOWN",
"database": "DOWN"})
    return
  }
  writeJSON(w, http.StatusOK, map[string]string{"status": "UP", "database": "UP"})
}

func writeError(w http.ResponseWriter, status int, msg string) {
  writeJSON(w, status, map[string]string{"error": msg})
}

func writeJSON(w http.ResponseWriter, status int, body any) {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(status)
  _ = json.NewEncoder(w).Encode(body)
}
```

`main.go` menjalankan server HTTP dan *consumer* sebagai dua *goroutine*. Saat berhenti, *consumer* dihentikan terlebih dahulu, lalu `reader.Close()` (lewat `defer`) mengirim permintaan untuk keluar dari *consumer group*, sehingga Kafka menyerahkan *partition* ke anggota lain tanpa menunggu *session timeout*. Meski begitu, Kafka masih dapat melaporkan *group* tersebut sebagai `Stable` selama beberapa detik setelah `notification` berhenti. Buat `notification/main.go`:

```
package main

import (
  "context"
  "errors"
  "log/slog"
  "net/http"
  "os"
  "os/signal"
  "syscall"
  "time"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/notification/internal/
notification"
)

func main() {
  log := slog.New(slog.NewTextHandler(os.Stdout, nil))
  if err := run(log); err != nil {
    log.Error("notification service stopped", "error", err)
    os.Exit(1)
  }
}

func run(log *slog.Logger) error {
  cfg, err := LoadConfig()
  if err != nil {
    return err
  }
```

```
  ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
  defer stop()

  store, err := notification.NewPostgresStore(ctx, cfg.DatabaseURL)
  if err != nil {
    return err
  }
  defer store.Close()

  srv := &http.Server{
    Addr:              cfg.HTTPAddr,
    Handler:           notification.NewHandler(store, log).Routes(),
    ReadHeaderTimeout: 5 * time.Second,
  }

  // Close leaves the consumer group, so Kafka hands this instance's partitions
  // to the remaining members right away instead of after a session timeout.
  reader := notification.NewKafkaReader(cfg.KafkaBrokers, cfg.KafkaTopic, cfg.KafkaGroupID)
  defer reader.Close()
  consumerCtx, stopConsumer := context.WithCancel(context.Background())
  defer stopConsumer()

  errCh := make(chan error, 1)
  go func() {
    log.Info("notification service listening", "addr", cfg.HTTPAddr)
    errCh <- srv.ListenAndServe()
  }()
  consumerDone := make(chan struct{})
  go func() {
    defer close(consumerDone)
    log.Info("consuming events", "kafka_brokers", cfg.KafkaBrokers, "topic",
cfg.KafkaTopic, "group", cfg.KafkaGroupID)
    notification.NewConsumer(reader, store, log).Run(consumerCtx)
  }()

  select {
  case err := <-errCh:
    stopConsumer()
    <-consumerDone
    return err
  case <-ctx.Done():
    log.Info("shutting down")
    // The event in progress is cut off before its offset commit, so it comes
    // again on the next start: never lost, a duplicate if its insert went through.
    stopConsumer()
    <-consumerDone
    shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil && !errors.Is(err,
http.ErrServerClosed) {
      return err
    }
    return nil
  }
}
```

Test untuk `notification` menggunakan *fake reader* dan *fake store*, dengan ID *event* dan ID transfer berupa UUID asli. Lima *test* di dalamnya memeriksa hal berikut:

- `TestConsumerStoresAndCommits`: *consumer* menunggu Kafka dan database yang mati, lalu meng-*commit* setiap *event*;
- `TestConsumerIsNotIdempotentYet`: *event* ganda masih menghasilkan dua notifikasi (celah yang ditutup Lab 6);
- `TestConsumerSkipsPoisonMessages`: *poison message* dilewati dan di-*commit*;
- `TestPermanentDBError`: kelas SQLSTATE dipilah dengan benar;
- `TestListNotifications`: parameter `transfer_id` divalidasi.

Ambil *file*-nya dari *checkpoint*, lalu rapikan `go.mod` dan jalankan:

```
git checkout lab-05 -- notification/internal/notification/consumer_test.go
cd notification && go mod tidy && cd ..
make -C notification test
```

5.4.14 *Make Target* untuk Lab 5

Tiga target baru di `Makefile` (folder `studi-kasus`) membantu mengamati alur *event* tanpa *tools* tambahan di laptop. Semua perintah Kafka dijalankan di dalam *container* `kafka` yang sudah membawa skrip-skrip Kafka:

- `events` menampilkan seluruh isi *topic* beserta *partition*, *header*, dan *key*-nya.
- `outbox` menampilkan 20 baris `outbox` terbaru. Baris dengan `published_at` kosong berisi *event* yang belum terkirim.
- `replay` mengembalikan *offset group* `notification` ke awal *topic*, sehingga semua *event* dikirim ulang. Kafka hanya mengizinkan *reset* pada *group* tanpa anggota aktif, sehingga `notification` harus dihentikan terlebih dahulu.

Tambahkan `notification` ke daftar service agar `make test` ikut menjalankan *test*-nya, lalu tambahkan variabel dan target berikut:

```
SERVICES := account transfer notification

KAFKA_TOPIC := transfer-events
KAFKA_BIN := /opt/kafka/bin

.PHONY: infra-up infra-down infra-reset test proto fault-slow fault-error fault-clear
fault-status breaker \
  events outbox replay

# Lab 5 helpers.

# Prints every event on the topic with its partition, headers and key, then
# exits after 5 s without new events (that exit is logged as a TimeoutException
# by the console consumer; it is not an error).
events:
  docker compose exec kafka $(KAFKA_BIN)/kafka-console-consumer.sh --bootstrap-server
localhost:19092 \
    --topic $(KAFKA_TOPIC) --from-beginning --timeout-ms 5000 \
    --property print.key=true --property print.headers=true --property print.partition=true

# The newest outbox rows in the transfer database; published_at empty = not on Kafka yet.
outbox:
  docker compose exec db-transfer psql -U transfer -d transfer -c \
    "select id, aggregate_id, event_type, created_at, published_at from outbox order by
created_at desc limit 20"

# Rewinds the notification consumer group to the first event, so every event is
# delivered again. Kafka only resets a group without active members: stop the
# notification service first.
replay:
```

```
  docker compose exec kafka $(KAFKA_BIN)/kafka-consumer-groups.sh --bootstrap-server
localhost:19092 \
    --group notification --topic $(KAFKA_TOPIC) --reset-offsets --to-earliest --execute
```

Baris `.PHONY` di atas menggantikan baris `.PHONY` yang lama. Target `events`, `outbox`, dan `replay` diletakkan di akhir *file*.

5.5 Verifikasi

Jalankan seluruh *test* terlebih dahulu:

```
make test
```

Semua *package* di `proto`, `account`, `transfer`, dan `notification` harus berakhir `ok`.

Jalankan infrastruktur, lalu ketiga service di tiga terminal terpisah (semuanya dari folder `studi-kasus`):

```
make infra-up
make -C account run        # terminal 1
make -C transfer run       # terminal 2
make -C notification run   # terminal 3
```

Log `transfer` menampilkan `msg="outbox relay started" interval=500ms`. Log `notification` menampilkan `msg="consuming events"` dengan `group=notification`.

Database sudah dikosongkan di awal lab, sehingga kedua rekening contoh perlu dibuat lagi:

```
curl -sS -X POST localhost:28081/accounts -H 'Content-Type: application/json' \
  -d '{"number":"1111111111","owner_name":"Budi","initial_balance":500000}'
curl -sS -X POST localhost:28081/accounts -H 'Content-Type: application/json' \
  -d '{"number":"2222222222","owner_name":"Siti","initial_balance":100000}'
```

5.5.1 Transfer Berhasil

```
curl -sS -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":100000}'
```

Respons HTTP `201` dengan `"status":"COMPLETED"`. Catat nilai `id` dari respons tersebut. Dalam waktu kurang dari satu detik, log `transfer` menampilkan `msg="outbox events published" count=1`, dan log `notification` menampilkan baris seperti berikut:

```
level=INFO msg="notification sent" event_type=TransferCompleted partition=1 offset=0
  event_id=... transfer_id=<id> notification_id=1
  message="Transfer 100000.00 dari 1111111111 ke 2222222222 berhasil."
```

Nomor *partition* dan *offset* dapat berbeda karena *partition* ditentukan oleh *hash* ID transfer. Periksa notifikasi lewat API `notification`, dengan `<id>` diganti ID transfer:

```
curl -sS "localhost:28083/notifications?transfer_id=<id>"
make outbox
make events
```

API `notification` mengembalikan satu notifikasi dengan `event_type TransferCompleted`. `make outbox` menampilkan baris `TransferCompleted` dengan `published_at` terisi. `make events` mencetak setiap *event* dalam satu baris: nomor *partition*, *header* `event_type` dan `event_id`, *key* berupa ID transfer, lalu *payload* JSON. Perintah tersebut berhenti setelah 5 detik tanpa *event* baru dan mencatat `TimeoutException`. Log tersebut menandai *console consumer* berhenti secara normal.

5.5.2 Saldo Kurang

```
curl -sS -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"2222222222","to_account":"1111111111","amount":99000000}'
```

Respons HTTP `422` dengan `"status":"FAILED"` dan `failure_reason` berisi `debit failed: debit 2222222222: insufficient funds`. `notification` mencatat `event_type=TransferFailed` dengan pesan `... gagal. Saldo tidak berubah.`

5.5.3 Kompensasi

Rekening tujuan yang tidak terdaftar membuat *credit* gagal setelah *debit* berhasil:

```
curl -sS -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"9999999999","amount":50000}'
```

Respons HTTP `422` dengan `"status":"COMPENSATED"` dan `failure_reason` berisi `credit failed: credit 9999999999: account not found`. Kode `422` digunakan karena penyebabnya rekening yang tidak ada, dan *debit* sudah dikembalikan. Log `transfer` menampilkan `msg="credit failed after debit, reversing the debit"`, dan log `account` menampilkan `msg="account mutated"` dengan `type=CREDIT` dan `reference` berakhiran `:reversal`. Saldo `1111111111` tidak berubah dibandingkan sebelum transfer ini. Mutasi di `db-account` menunjukkan dua baris untuk transfer tersebut, yaitu *debit* dengan `reference` berisi ID transfer dan *credit* dengan `reference` berakhiran `:reversal`:

```
docker compose exec db-account psql -U account -d account -c \
  "select type, amount, reference from mutation where reference like '<id>%' order by
id"
```

`notification` mencatat `event_type=TransferCompensated` dengan pesan `... dibatalkan. Dana sudah dikembalikan ke 1111111111.`

5.5.4 Kafka Mati

Skenario ini menunjukkan manfaat *outbox*: transfer tetap berjalan meskipun Kafka mati, dan *event* tersimpan di database sampai Kafka kembali aktif.

```
docker compose stop kafka
curl -sS -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":10000}'
make outbox
```

Transfer tetap `COMPLETED` dengan HTTP `201`. Log `transfer` berulang kali menampilkan `msg="outbox publish failed, will retry"`, dan `make outbox` menampilkan baris terbaru dengan `published_at` kosong. Log `notification` menampilkan `msg="fetching event failed, retrying"`. Jalankan kembali Kafka:

```
docker compose start kafka
make outbox
```

Setelah Kafka berstatus `healthy`, log `transfer` menampilkan `outbox events published` dan `published_at` terisi. `notification` membutuhkan waktu lebih lama, sekitar 15–20 detik, karena *consumer group* harus bergabung ulang terlebih dahulu. Setelah itu notifikasi untuk transfer tersebut muncul di log.

5.5.5 *Replay* dan Notifikasi Ganda

Skenario ini memperlihatkan celah yang ditutup di Lab 6. Hentikan `notification` dengan `Ctrl+C` di terminal 3, tunggu 10–30 detik hingga Kafka mencatat *group* tersebut tidak aktif, lalu kembalikan *offset group* ke awal:

```
make replay
```

Output `make replay` berupa tabel dengan kolom `GROUP`, `TOPIC`, `PARTITION`, dan `NEW-OFFSET`, dengan `NEW-OFFSET` bernilai `0` untuk ketiga *partition*. Jalankan kembali `notification`:

```
make -C notification run
```

Log menampilkan `notification sent` untuk setiap *event* yang pernah dikirim ke Kafka. Periksa isi `db-notification`:

```
docker compose exec db-notification psql -U notification -d notification -c \
  "select count(*) as baris, count(distinct event_id) as event_unik from notification"
```

Nilai `baris` sekarang dua kali `event_unik`: setiap *event* tersimpan dua kali. `GET /notifications` untuk transfer pertama juga mengembalikan dua notifikasi dengan `event_id` yang sama.

5.5.6 *Poison Message*

Event yang tidak mungkin diproses tidak boleh menahan *partition*. Kirim satu *event* rusak langsung ke *topic* dengan *console producer* Kafka. *Key* dan *payload* dipisahkan karakter `|`, dan `event_id` maupun `transfer_id`-nya bukan UUID:

```
echo 'k1|{"event_id":"abc","transfer_id":"def"}' | docker compose exec -T kafka \
  /opt/kafka/bin/kafka-console-producer.sh --bootstrap-server localhost:19092 \
  --topic transfer-events --reader-property parse.key=true --reader-property
key.separator='|'
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:19092 --describe --group notification
```

Log `notification` menampilkan `level=ERROR msg="skipping event with invalid event_id or transfer_id"` beserta *payload*-nya. *Output* `--describe` menampilkan kolom `LAG` bernilai `0` untuk ketiga *partition*: *offset event* rusak tersebut sudah di-*commit*, dan *event* berikutnya di *partition* yang sama tetap diproses.

5.5.7 *Reversal* Menunggu `account` Aktif Kembali

Dua skenario terakhir membutuhkan pengaturan waktu, karena *debit* harus berhasil dan *credit* harus gagal. *Delay* 1,5 detik masih di bawah *timeout* 2 detik, sehingga *debit* berhasil. Satu detik kemudian, selagi *debit* masih menunggu, `account` diubah agar menolak semua panggilan berikutnya. Lima detik setelahnya *fault injection* dihapus (`make fault-clear`) dan `account` kembali normal, selagi *reversal* masih di-*retry*:

```
curl -sS -X PUT localhost:28081/admin/fault -d '{"delay_ms":1500,"error_rate":0}'
curl -sS -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":10000}' &
sleep 1
curl -sS -X PUT localhost:28081/admin/fault -d '{"delay_ms":0,"error_rate":1}'
sleep 5
make fault-clear
wait
```

Sekitar delapan detik setelah dikirim, transfer dijawab dengan `"status":"COMPENSATED"`. Log `transfer` menampilkan `msg="credit failed after debit, reversing the debit"`, beberapa baris `msg="retrying reversal"` dengan `next_attempt` dan `elapsed` yang bertambah, lalu `msg="transfer not completed" status=COMPENSATED`. Kode HTTP-nya bergantung pada *circuit breaker* saat *credit* gagal:

- `502` dengan `failure_reason credit failed: credit 2222222222: rpc error: code = Unavailable desc = injected fault`, bila *breaker* tetap tertutup selama empat percobaan *credit*;

- `503` dengan `failure_reason credit failed: credit 2222222222: account service unavailable`, bila *breaker* terbuka di tengah percobaan *credit*, misalnya karena masih ada kegagalan lain di antara 10 panggilan terakhir.

Pada kedua kode tersebut tidak ada uang yang berpindah: saldo `1111111111` dan `2222222222` sama dengan sebelum transfer ini. Seandainya *reversal* melewati *breaker* yang sama, kasus `503` di atas akan langsung berakhir `COMPENSATION_FAILED`.

Sebelum skenario berikutnya, jalankan `make breaker`. Bila hasilnya `open`, tunggu 10 detik agar *debit* tidak langsung ditolak *circuit breaker*.

5.5.8 Kompensasi yang Gagal

Skenario terakhir sama dengan sebelumnya, tetapi *fault injection* baru dihapus setelah transfer selesai, sehingga *reversal* tidak pernah berhasil:

```
curl -sS -X PUT localhost:28081/admin/fault -d '{"delay_ms":1500,"error_rate":0}'
curl -sS -X POST localhost:28082/transfers -H 'Content-Type: application/json' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":10000}' &
sleep 1
curl -sS -X PUT localhost:28081/admin/fault -d '{"delay_ms":0,"error_rate":1}'
wait
make fault-clear
```

Jawaban datang setelah sekitar 32 detik: 2 detik untuk *debit* dan *credit*, lalu 30 detik *retry reversal*. Transfer dijawab HTTP `500` dengan `"status":"COMPENSATION_FAILED"`, dan `failure_reason` diakhiri `; reversal failed: reverse 1111111111: rpc error: code = Unavailable desc = injected fault`. Log `transfer` menampilkan sekitar sepuluh baris `retrying reversal`, lalu `level=ERROR msg="compensation failed, source account needs manual reversal"`. Tidak ada *event* yang dikirim ke Kafka untuk transfer ini, sehingga `notification` tidak mencatat apa pun. Saldo `1111111111` berkurang 10000 tanpa ada uang yang masuk ke `2222222222`.

Reversal tidak dihitung oleh *circuit breaker*, sehingga status *breaker* hanya ditentukan oleh percobaan *debit* dan *credit*. Periksa dengan `make breaker`. Bila hasilnya `open`, status tersebut tidak berubah tanpa panggilan. Tunggu 10 detik, lalu kirim tiga transfer kecil yang sah (misalnya 1000 dari `2222222222` ke `1111111111`). Ketiga panggilan tersebut menjadi panggilan uji pada status *half-open*. Setelah itu `make breaker` menampilkan `closed`.

5.6 Masalah Umum

`account` berhenti saat *startup* dengan `apply schema: ... could not create unique index "mutation_idempotency" (SQLSTATE 23505)`. Database masih berisi data Lab 1–4 dengan `reference` ganda di tabel `mutation`. Hentikan semua service, jalankan `make infra-reset` lalu `make infra-up`, dan buat ulang kedua rekening contoh.

`missing required environment variables: KAFKA_BROKERS, KAFKA_TOPIC.` `transfer` dijalankan dengan perintah lama. Jalankan dengan `make -C transfer run`, yang sudah memuat kedua variabel tersebut.

Log `transfer` terus menampilkan `outbox publish failed` dengan `Unknown Topic Or Partition`. *Topic* belum dibuat, misalnya karena Kafka dijalankan dengan `docker compose up` tanpa `make infra-up`. Buat *topic* dengan `docker compose run --rm kafka-init`. Pembuatan *topic* otomatis memang dimatikan, sehingga Kafka tidak membuatnya sendiri.

`make infra-up` gagal karena port 19092 sudah digunakan. Ada program lain di laptop yang menggunakan port tersebut. Pada Linux dan macOS, `lsof -i :19092` menampilkan proses yang menggunakan port. Hentikan program tersebut selama lab berlangsung.

`make replay` gagal dengan pesan bahwa *group* `notification` masih aktif (`Assignments can only be reset if the group 'notification' is inactive, but the current state is Stable`). `notification`

masih berjalan, atau baru saja dihentikan dan Kafka belum mencatat bahwa anggotanya sudah keluar dari *group*. Pastikan `notification` sudah berhenti (`Ctrl+C`), tunggu 10–30 detik, lalu ulangi `make replay`.

`notification` tidak menerima *event* baru setelah Kafka dijalankan ulang. *Consumer group* kafka-go membutuhkan sekitar 15–20 detik untuk bergabung ulang. Selama itu log menampilkan `fetching event failed, retrying`. Tunggu hingga log `notification sent` muncul.

Transfer pertama setelah `transfer` dijalankan terasa lambat, dan log menunjukkan *retry*. Di macOS, resolusi nama `localhost` untuk koneksi gRPC pertama dapat memakan waktu lebih dari 2 detik saat *cold start*, sehingga percobaan pertama terkena *timeout*. Di lab ini *timeout* di-*retry*, sehingga transfer tetap berhasil. Bila gejala ini mengganggu, ganti `ACCOUNT_GRPC_ADDR=localhost:29091` menjadi `ACCOUNT_GRPC_ADDR=127.0.0.1:29091` di `transfer/Makefile`.

`make fault-slow` sekarang membuat transfer gagal jauh lebih lama dibandingkan Lab 4. Perilaku ini memang diharapkan. *Timeout* kini di-*retry* hingga tiga kali, sehingga satu transfer menunggu sekitar empat kali 2 detik ditambah *backoff* sebelum berakhir `FAILED`. Setelah beberapa transfer, *circuit breaker* terbuka dan transfer berikutnya langsung ditolak dengan HTTP `503`.

Transfer berakhir di `COMPENSATION_FAILED` dan tidak bisa diperbaiki. Lab ini tidak menyediakan *tools* untuk mengulang pengembalian. Pengembalian dapat dilakukan dengan aman lewat *credit* ke rekening asal dengan `reference <transfer_id>:reversal`, karena `reference` yang sama menjamin *credit* tersebut hanya diterapkan sekali. Untuk keperluan lab, data dapat dibersihkan dengan `make infra-reset` (seluruh data lab terhapus).

`make infra-down` menampilkan peringatan bahwa jaringan masih digunakan. Ada sisa *container* `kafka-init` dari versi `Makefile` yang lama. Pastikan target `infra-down` sudah menggunakan `docker compose --profile init down`.

5.7 Eksplorasi Mandiri

1. Jalankan dua *instance* `notification` dengan `HTTP_ADDR` berbeda (misalnya `:28083` dan `:28084`), lalu kirim beberapa transfer. Amati nilai `partition` di log setiap *instance*: Kafka membagi tiga *partition* di antara kedua *instance*. Hentikan salah satunya dan perhatikan *partition* berpindah ke *instance* yang tersisa.
2. Jalankan dua *instance* `transfer` pada port berbeda, lalu kirim transfer bergantian ke keduanya selama Kafka mati. Setelah Kafka dijalankan kembali, periksa dengan `make events` bahwa setiap `event_id` hanya dikirim sekali ke Kafka. Hubungkan hasilnya dengan `for update skip locked` di *relay*.
3. Aturan *idempotency* di `account` hanya membandingkan `reference`. Ubah `PostgresStore.Mutate` agar panggilan ulang dengan `reference` yang sama tetapi nominal berbeda ditolak dengan `codes.InvalidArgument`. Tentukan bagaimana `transfer` sebaiknya memperlakukan jawaban tersebut.
4. Rancang *reconciliation job* untuk transfer yang tertahan di `PENDING`, `DEBITED`, atau `COMPENSATION_FAILED`: query apa yang menemukan transfer tersebut, panggilan apa yang diulang dengan `reference` yang sama, dan status apa yang dituju setelahnya.

5.8 Checkpoint

Kondisi kode di akhir lab ini tersedia di tag `lab-05`. Bandingkan hasil kerja dengan *checkpoint*, atau lanjutkan dari *checkpoint* bila tertinggal:

```
git diff lab-05 -- .
git checkout lab-05
```

6 Lab 6: *Idempotent Consumer* dan *Header* `Idempotency-Key`

6.1 Tujuan

Setelah menyelesaikan lab ini, peserta mampu:

1. menjelaskan mengapa *at-least-once delivery* selalu menghasilkan duplikat, dan mengapa duplikat tersebut harus ditangani di sisi penerima;
2. membuat `notification` service menjadi *idempotent consumer*: *event* yang dikirim ulang dilewati, sehingga notifikasi hanya tersimpan sekali;
3. menulis penanda "*event* sudah diproses" dan hasil pemrosesannya dalam satu transaksi database;
4. menerima *header* `Idempotency-Key` di `POST /transfers`, sehingga klien yang mengulang *request* mendapatkan transfer yang sama, dan rekening asal tidak terdebit dua kali;
5. membuktikan kedua perilaku tersebut dengan *replay* Kafka dan dengan *request* paralel.

6.2 Gambaran

Di Lab 5, sistem sudah memiliki tiga service. `transfer` menjalankan *saga*, menulis *event* ke tabel `outbox`, lalu *relay* mengirimnya ke Kafka. `notification` membaca *event* tersebut dan menyimpan satu notifikasi untuk setiap *event*.

Pengiriman *event* di Lab 5 bersifat *at-least-once*: setiap *event* pasti sampai, tetapi bisa sampai lebih dari sekali. *Event* yang sama dapat terkirim ulang dalam tiga situasi:

- *Relay* berhenti setelah Kafka menerima *event* tetapi sebelum kolom `published_at` terisi. Setelah *restart*, *relay* mengirim *event* tersebut lagi.
- `notification` berhenti setelah notifikasi tersimpan tetapi sebelum *offset* di-*commit*. Kafka mengirim *event* tersebut lagi ke anggota *consumer group* berikutnya.
- Operator sengaja melakukan *replay* (membaca ulang) *topic* dari awal, misalnya setelah memperbaiki *bug* di *consumer*.

Ketiga situasi tersebut wajar dan tidak dapat dihilangkan sepenuhnya. Jalan keluarnya ada di penerima: pemrosesan dibuat *idempotent*, artinya memproses *event* yang sama berkali-kali menghasilkan keadaan yang sama dengan memprosesnya sekali. Rumus yang menjadi pegangan lab ini:

$$\textit{at-least-once delivery} + \text{pemrosesan } \textit{idempotent} = \text{setiap efek terjadi tepat sekali}$$

Masalah yang sama muncul di sisi *channel*. Aplikasi *mobile banking* mengirim `POST /transfers`, lalu koneksinya putus sebelum *response* diterima. Aplikasi tidak tahu apakah transfer sudah dijalankan. Bila aplikasi mengirim ulang *request* yang sama, tanpa perlindungan apa pun rekening asal terdebit dua kali. *Header* `Idempotency-Key` menyelesaikan masalah tersebut: aplikasi membuat satu *key* (biasanya UUID) untuk setiap transfer yang ingin dilakukan nasabah, lalu mengirim *key* yang sama pada setiap pengulangan.



```
                                    topic transfer-events
                                         |
                                         v
                                    notification
                                    tabel processed_event (event_id primary key)
                                    + tabel notification, satu transaksi
```

Dengan lab ini, sistem memiliki tiga titik *idempotency* dengan pola yang sama: penerima menyimpan identitas unik, lalu tidak memproses ulang identitas yang sama.

Titik	Pengirim dan Penerima	Identitas Unik	Lab
Mutasi saldo	`transfer` ke `account` (gRPC)	`reference` = ID transfer, unik per rekening dan jenis mutasi	Lab 5
Consumer Kafka	Kafka ke `notification`	`event_id` di tabel `processed_event`	Lab 6
API transfer	Aplikasi *channel* ke `transfer` (REST)	Nilai *header* `Idempotency-Key`	Lab 6

Di Lab 5, *idempotency* sudah diterapkan antarservice pada mutasi saldo. *Header* `Idempotency-Key` menerapkan gagasan yang sama antara *channel* dan service.

6.3 Titik Awal

Lab ini dimulai dari kondisi akhir Lab 5. Peserta yang belum menyelesaikan Lab 5 dapat memulai dari tag `lab-05` dengan *branch* kerja baru:

```
cd studi-kasus
git switch -c lab-06-kerja lab-05
```

Jalankan infrastruktur dan ketiga service, masing-masing di terminal tersendiri:

```
make infra-up
make -C account run
make -C transfer run
make -C notification run
```

Database dari Lab 5 tidak perlu di-*reset*. Setiap service menjalankan `schema.sql` miliknya saat *startup*, dan semua perubahan *schema* di lab ini ditulis dengan `if not exists`, sehingga aman dijalankan pada database lama.

Contoh di lab ini menggunakan rekening `1111111111` dan `2222222222` dari lab sebelumnya. Bila rekening tersebut belum ada, buat terlebih dahulu:

```
curl -X POST localhost:28081/accounts \
  -d '{"number":"1111111111","owner_name":"Budi","initial_balance":"10000000"}'
curl -X POST localhost:28081/accounts \
  -d '{"number":"2222222222","owner_name":"Sari","initial_balance":"0"}'
```

6.4 Langkah

6.4.1 Mengamati notifikasi ganda di Lab 5

Sebelum mengubah kode, buktikan masalahnya terlebih dahulu. Buat satu transfer, lalu catat nilai `id` dari *response*-nya:

```
curl -X POST localhost:28082/transfers \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

Hentikan `notification` dengan `Ctrl+C`. Kafka hanya mengizinkan *offset* sebuah *consumer group* diubah ketika *group* tersebut tidak memiliki anggota aktif. Setelah itu, kembalikan *offset consumer group* `notification` ke awal *topic*, lalu jalankan `notification` lagi:

```
make replay
make -C notification run
```

Log `notification` menampilkan `notification sent` untuk setiap *event* yang pernah ada di *topic*, termasuk *event* yang sudah diproses sebelumnya. Periksa notifikasi untuk transfer tadi (ganti `<id>` dengan ID transfer):

```
curl "localhost:28083/notifications?transfer_id=<id>"
```

Hasilnya berisi dua notifikasi dengan `event_id` yang sama. Di sistem sungguhan, nasabah menerima SMS "Transfer berhasil" dua kali untuk satu transfer. *Replay* sengaja dipilih untuk demonstrasi karena mudah diulang dan melewati jalur kode yang sama dengan pengiriman ulang akibat *crash* atau *rebalance*.

6.4.2 Tabel `processed_event` di `notification`

Cara paling langsung untuk mengenali duplikat adalah mencatat setiap `event_id` yang sudah diproses. `event_id` dibuat oleh `transfer` saat menulis *outbox* (Lab 5) dan ikut dikirim di *payload* maupun *header* Kafka, sehingga nilainya sama pada setiap pengiriman ulang.

Pencatatan ini menggunakan tabel tersendiri, `processed_event`, dengan `event_id` sebagai *primary key*. Sebagai alternatif, dapat dipasang *unique index* pada `notification.event_id`, tetapi tabel tersendiri lebih tepat karena dua alasan:

- Tabel ini menjaga aturan "*event* ini sudah ditangani". Ada *event* yang tidak menghasilkan baris notifikasi, dan ada *event* yang mungkin menghasilkan beberapa baris. Tabel `processed_event` berlaku untuk semua kasus tersebut.
- Tabel `notification` dari Lab 5 sudah berisi baris ganda. Pembuatan *unique index* di atas data tersebut akan gagal, sedangkan tabel baru dapat dibuat tanpa menyentuh data lama.

Tambahkan bagian berikut di akhir `notification/internal/notification/schema.sql`. Hapus juga komentar lama di atas `create table notification` yang menyatakan `event_id` sengaja belum unik.

```
-- Idempotent consumer (Lab 6): one row per event already handled. It is
-- inserted in the same transaction as the notification, so "handled" and
-- "notification stored" are always both true or both false. A redelivered event
-- (relay duplicate, rebalance, replay) hits the primary key and is skipped.
--
-- A separate table, not a unique index on notification.event_id: the dedup rule
-- is "this event was handled", which also covers events that produce no row
-- or several rows. It also keeps the duplicate notifications of Lab 5 as they
-- are instead of failing the index creation on them.
create table if not exists processed_event (
    event_id     uuid        primary key,
    processed_at timestamptz not null default now()
);

-- Events handled in Lab 5, before this table existed, count as processed: a
-- replay after the upgrade must not notify about them once more. Re-running it
-- on every start changes nothing.
insert into processed_event (event_id)
```

```
select distinct event_id from notification
on conflict (event_id) do nothing;
```

Perintah `insert ... select` di bagian akhir menangani database yang sudah terisi di Lab 5. Tanpa perintah tersebut, `processed_event` kosong saat pertama kali dibuat, dan *replay* berikutnya akan mengirim ulang notifikasi untuk semua transfer lama. Karena menggunakan `on conflict do nothing`, perintah tersebut aman dijalankan pada setiap *startup*.

6.4.3 `Store.Insert` dalam satu transaksi

Penanda "sudah diproses" dan notifikasinya harus disimpan bersama. Bila keduanya disimpan terpisah, ada dua kemungkinan kegagalan:

- Penanda tersimpan, notifikasi gagal: *event* dianggap selesai padahal notifikasi tidak pernah ada.
- Notifikasi tersimpan, penanda gagal: pengiriman ulang menghasilkan notifikasi ganda lagi.

Dengan satu transaksi, keduanya selalu sama-sama ada atau sama-sama tidak ada.

Ubah kontrak `Store` di `notification/internal/notification/notification.go`. `Insert` sekarang mengembalikan nilai `inserted` yang bernilai `false` ketika *event* sudah pernah diproses:

```
type Store interface {
  // Insert stores the notification and marks its event as processed, in one
  // transaction. inserted is false, and nothing is stored, when the event was
  // processed before.
  Insert(ctx context.Context, n Notification) (stored Notification, inserted bool, err
error)
  ListByTransfer(ctx context.Context, transferID string) ([]Notification, error)
  Ping(ctx context.Context) error
}
```

Lalu ganti seluruh fungsi `Insert` di `notification/internal/notification/store.go`:

```
func (s *PostgresStore) Insert(ctx context.Context, n Notification) (Notification, bool,
error) {
  tx, err := s.pool.Begin(ctx)
  if err != nil {
    return Notification{}, false, err
  }
  defer tx.Rollback(ctx) // no-op after Commit

  // The processed_event row goes in first, as the duplicate check. A second
  // delivery of the same event being handled concurrently (another instance
  // after a rebalance) waits on the primary key until this transaction ends,
  // then sees the conflict: the notification is stored once.
  tag, err := tx.Exec(ctx,
    `insert into processed_event (event_id) values (@event_id) on conflict (event_id) do
nothing`,
    pgx.NamedArgs{"event_id": n.EventID})
  if err != nil {
    return Notification{}, false, err
  }
  if tag.RowsAffected() == 0 {
    return Notification{}, false, nil
  }

  rows, err := tx.Query(ctx, `
    insert into notification (event_id, transfer_id, event_type, message)
    values (@event_id, @transfer_id, @event_type, @message)
    returning `+columns,
    pgx.NamedArgs{"event_id": n.EventID, "transfer_id": n.TransferID, "event_type":
n.EventType, "message": n.Message})
```

```
  if err != nil {
    return Notification{}, false, err
  }
  stored, err := pgx.CollectExactlyOneRow(rows, pgx.RowToStructByPos[Notification])
  if err != nil {
    return Notification{}, false, err
  }
  return stored, true, tx.Commit(ctx)
}
```

Beberapa hal yang perlu diperhatikan dari kode tersebut:

- Baris `processed_event` ditulis lebih dahulu dan berfungsi sebagai pemeriksaan duplikat. `on conflict do nothing` tidak menghasilkan *error* bila `event_id` sudah ada, sehingga kondisi tersebut dikenali dari `RowsAffected() == 0`.
- Pemeriksaan dan penulisan terjadi dalam satu perintah SQL. Pola "`select` dulu, lalu `insert` bila belum ada" tidak aman, karena dua proses dapat sama-sama melihat tabel kosong lalu sama-sama menyimpan notifikasi.
- Bila dua *instance* `notification` memproses *event* yang sama secara bersamaan (misalnya saat *rebalance*), transaksi kedua tertahan oleh *lock* pada *primary key* hingga transaksi pertama selesai, lalu tidak menyimpan apa pun karena `event_id` tersebut sudah ada.
- `defer tx.Rollback(ctx)` membatalkan transaksi pada setiap jalur keluar sebelum `Commit`. Setelah `Commit` berhasil, `Rollback` tidak melakukan apa pun.

6.4.4 *Consumer* melewati duplikat

Di `notification/internal/notification/consumer.go`, fungsi `handle` memanggil `Insert`. Sesuaikan isi perulangan `for` di bagian akhir `handle` dengan tiga nilai kembalian yang baru. Blok `if permanentDBError(err)` dari Lab 5 tetap ada di antara jalur berhasil dan jalur *retry*:

```
  n := Notification{EventID: e.EventID, TransferID: e.TransferID, EventType: eventType,
Message: msg}
  for {
    stored, inserted, err := c.store.Insert(ctx, n)
    if err == nil && !inserted {
      // The offset is still committed by Run: the event is done, its
      // redelivery must not block the partition.
      log.Info("duplicate event skipped", "event_id", e.EventID, "transfer_id",
e.TransferID)
      return nil
    }
    if err == nil {
      // Stand-in for the real channel (SMS, push, e-mail). A real channel call
      // is outside the database transaction: send after commit and a crash in
      // between loses the message; send before commit and it may go twice. With
      // a provider that accepts an idempotency key (the event_id), sending
      // before the commit is safe.
      log.Info("notification sent", "event_id", e.EventID, "transfer_id", e.TransferID,
        "notification_id", stored.ID, "message", msg)
      return nil
    }
    if permanentDBError(err) {
      log.Error("skipping event the database rejects", "event_id", e.EventID, "error", err,
        "payload", string(m.Value))
      return nil
    }
    log.Warn("storing notification failed, retrying", "event_id", e.EventID, "error", err)
    if !c.pause(ctx) {
      return errors.Join(ctx.Err(), err)
```

```
    }
  }
```

Duplikat dikembalikan sebagai `nil`, sama seperti *event* yang berhasil diproses. Akibatnya, `Run` tetap melakukan *commit offset* untuk *event* duplikat. Langkah ini penting: bila *offset* tidak di-*commit*, Kafka terus mengirim *event* yang sama, dan semua *event* berikutnya di *partition* tersebut tertahan di belakangnya.

Komentar di jalur `notification sent` membahas keterbatasan solusi ini. Di lab ini, "mengirim notifikasi" hanya berupa baris log. Pengiriman sungguhan melalui SMS atau *push notification* memanggil sistem lain yang berada di luar transaksi database, sehingga transaksi tidak dapat menjaminnya:

- Bila dikirim setelah *commit* dan proses berhenti di antaranya, notifikasi tidak pernah terkirim.
- Bila dikirim sebelum *commit* dan transaksi gagal, pengiriman ulang *event* mengirim notifikasi kedua.

Solusinya ada di sisi *provider*: banyak *provider* SMS dan *push* menerima *idempotency key* di *request*-nya. Dengan `event_id` sebagai *key* tersebut, pengiriman sebelum *commit* menjadi aman.

Perbarui juga komentar di atas fungsi `Run` agar sesuai dengan perilaku baru. Ganti dua baris terakhir komentar tersebut (mulai `It is deliberately NOT idempotent yet`) dengan:

```
// sometimes. It is idempotent (Lab 6): the store remembers every processed
// event_id, and a redelivered event is skipped instead of notifying twice.
// at-least-once delivery + idempotent processing = each effect exactly once.
```

Dua komentar lain juga berubah: komentar di jalur gagal `CommitMessages` di dalam `Run`, dan komentar *shutdown* di `notification/main.go`. Keduanya sekarang menyebut bahwa *event* yang datang lagi dilewati sebagai duplikat. Perubahan komentar tersebut dan bagian *Idempotency* baru di `studi-kasus/README.md` tersedia di tag `lab-06` dan tidak memengaruhi perilaku program.

6.4.5 Memperbarui *test* `notification`

`fakeStore` di `notification/internal/notification/consumer_test.go` mengimplementasikan `Store`, sehingga harus mengikuti kontrak baru. Ganti `fakeStore` dan fungsi `Insert`-nya dengan versi yang mengingat `event_id`. *Field* `reject` dan `inserts` dari Lab 5 tetap dipertahankan karena digunakan oleh *test poison message*:

```
// fakeStore fails the first failures inserts, then stores each event once. An
// insert for an event ID in reject fails with that error on every try.
type fakeStore struct {
  rows      []Notification
  processed map[string]bool
  failures  int
  reject    map[string]error
  inserts   int
}

func (f *fakeStore) Insert(_ context.Context, n Notification) (Notification, bool, error) {
  f.inserts++
  if err := f.reject[n.EventID]; err != nil {
    return Notification{}, false, err
  }
  if f.failures > 0 {
    f.failures--
    return Notification{}, false, errors.New("database down")
  }
  if f.processed[n.EventID] {
    return Notification{}, false, nil
  }
  if f.processed == nil {
    f.processed = map[string]bool{}
```

```
  }
  f.processed[n.EventID] = true
  n.ID = int64(len(f.rows) + 1)
  f.rows = append(f.rows, n)
  return n, true, nil
}
```

Test `TestConsumerIsNotIdempotentYet` dari Lab 5 mendokumentasikan celah yang sekarang sudah ditutup. Ganti *test* tersebut dengan *test* yang membuktikan perilaku baru: empat *event* masuk, dua di antaranya duplikat, hasilnya dua notifikasi dan keempat *offset* tetap di-*commit*. *Test* ini menggunakan konstanta `eventID1` dan `eventID2` yang sudah ada sejak Lab 5. ID seperti `e-1` tidak dapat digunakan karena `handle` melewati *event* dengan `event_id` yang bukan UUID.

```
// The same event delivered twice (relay duplicate or replay) becomes one
// notification, and both deliveries are committed.
func TestConsumerSkipsDuplicates(t *testing.T) {
  store := &fakeStore{}
  reader := runConsumer(t, store,
    event(0, "TransferCompensated", eventID1),
    event(1, "TransferCompleted", eventID2),
    event(0, "TransferCompensated", eventID1), // replay from the start
    event(1, "TransferCompleted", eventID2),
  )
  if len(store.rows) != 2 {
    t.Fatalf("rows = %d, want 2 (duplicates skipped)", len(store.rows))
  }
  if fmt.Sprint(reader.committed) != "[0 1 0 1]" {
    t.Fatalf("committed = %v, want [0 1 0 1]", reader.committed)
  }
}
```

Jalankan *test* `notification`:

```
make -C notification test
```

6.4.6 *Make target* `duplicates`

Untuk memeriksa hasil *replay* tanpa membuka `psql` secara manual, tambahkan *target* berikut di akhir `Makefile` di direktori `studi-kasus`, lalu tambahkan `duplicates` ke daftar `.PHONY`:

```
# Lab 6: events with more than one notification. Empty after a replay = the
# consumer is idempotent. Rows from Lab 5 (before processed_event) stay listed.
duplicates:
  docker compose exec db-notification psql -U notification -d notification -c \
    "select event_id, event_type, count(*) as notifications from notification group by
event_id, event_type having count(*) > 1"
```

Baris perintah di dalam *target* `Makefile` wajib diawali karakter *tab*. Baris yang diawali spasi menghasilkan *error* `missing separator`.

6.4.7 Kolom `idempotency_key` di `transfer`

Sekarang beralih ke sisi *channel*. `transfer` menyimpan nilai *header* `Idempotency-Key` di tabel `transfer` dengan *unique index*. Tambahkan bagian berikut di akhir `transfer/internal/transfer/schema.sql`:

```
-- Idempotency key (Lab 6): the value of the Idempotency-Key header of POST
-- /transfers. A channel that retries a request whose answer it never got
-- (timeout, dropped connection) sends the same key again and gets the original
-- transfer back instead of a second debit. NULL for requests without the
-- header; PostgreSQL treats NULLs as distinct, so they never collide on the
-- unique index.
alter table transfer add column if not exists idempotency_key varchar(100);
create unique index if not exists transfer_idempotency_key on transfer (idempotency_key);
```

Kolom tersebut boleh `NULL`. *Request* tanpa *header* menyimpan `NULL`, dan PostgreSQL menganggap setiap `NULL` berbeda satu sama lain, sehingga transfer dari Lab 2–5 dan transfer baru tanpa *header* tidak melanggar *unique index*.

6.4.8 Validasi dan pembanding *request* di `transfer.go`

Tiga hal ditambahkan di `transfer/internal/transfer/transfer.go`: dua *error* baru, fungsi pembanding *request*, dan validasi format *key*.

Tambahkan `ErrIdempotencyKeyReused` dan `ErrInProgress` di dalam blok `var (...)`, tepat setelah `ErrStaleState`:

```
  // ErrIdempotencyKeyReused: the Idempotency-Key was used before for a
  // different transfer. Most likely a client bug (a key reused for a new
  // request); executing it would silently drop one of the two.
  ErrIdempotencyKeyReused = errors.New("idempotency key already used for a different
request")
  // ErrInProgress: an Idempotency-Key replay found the original transfer still
  // PENDING or DEBITED.
  ErrInProgress = errors.New("transfer still in progress")
```

`ErrIdempotencyKeyReused` dikembalikan ketika *key* yang sama datang dengan isi berbeda. `ErrInProgress` dikembalikan ketika pengulangan menemukan transfer asli yang belum selesai. *Handler* menerjemahkannya menjadi `202 Accepted`.

Setelah blok `var (...)` tersebut, tambahkan `SameRequest`. Ketika *key* yang sama datang lagi, `transfer` harus memastikan bahwa *request* tersebut memang pengulangan, bukan transfer baru yang secara keliru menggunakan *key* lama. Nominal dibandingkan sebagai desimal dengan `Equal`, sehingga `"100000"` dan `"100000.00"` dianggap sama.

```
// SameRequest reports whether t was created from req: the check behind an
// Idempotency-Key that comes again. Decimal comparison, so "100000" and
// "100000.00" are the same amount.
func (t Transfer) SameRequest(req CreateRequest) bool {
  return t.FromAccount == req.FromAccount && t.ToAccount == req.ToAccount &&
t.Amount.Equal(req.Amount)
}
```

Setelah deklarasi `maxAmount` (di bawah `numberPattern`), tambahkan validasi *key*. Batas 100 karakter mengikuti ukuran kolom, dan karakter yang diizinkan hanya ASCII yang dapat dicetak tanpa spasi (`\x21`–`\x7e`):

```
// keyPattern bounds the Idempotency-Key: printable ASCII, up to the column
// size. A UUID generated by the client is the usual choice.
var keyPattern = regexp.MustCompile(`^[\x21-\x7e]{1,100}$`)

// ValidateIdempotencyKey checks a non-empty Idempotency-Key header value.
func ValidateIdempotencyKey(key string) error {
  if !keyPattern.MatchString(key) {
    return ValidationError{"Idempotency-Key": "must be 1-100 printable ASCII characters
without spaces"}
  }
  return nil
}
```

Terakhir, ubah deklarasi `Create` di *interface* `Store`:

```
  // Create records a new transfer in status PENDING. With a non-empty
  // idempotencyKey that is already stored, nothing is inserted: it returns the
  // transfer stored under that key and created = false.
  Create(ctx context.Context, req CreateRequest, idempotencyKey string) (t Transfer,
created bool, err error)
```

6.4.9 `Create` di `store.go`

Pemeriksaan *key* dan penyimpanan transfer baru harus terjadi dalam satu perintah SQL, dengan alasan yang sama seperti di `notification`: pola "cari dulu, simpan bila belum ada" membuka celah bagi dua *request* paralel. Ganti fungsi `Create` di `transfer/internal/transfer/store.go`:

```
func (s *PostgresStore) Create(ctx context.Context, req CreateRequest, idempotencyKey
string) (Transfer, bool, error) {
  // "on conflict do nothing" + no row returned = the key exists. A concurrent
  // request with the same key waits on the unique index until the first insert
  // commits, then takes this path too: one transfer per key, even under a race.
  t, err := one(ctx, s.pool, `
    insert into transfer (from_account, to_account, amount, status, idempotency_key)
    values (@from, @to, @amount, 'PENDING', nullif(@key, ''))
    on conflict (idempotency_key) do nothing
    returning `+columns,
    pgx.NamedArgs{"from": req.FromAccount, "to": req.ToAccount, "amount": req.Amount,
"key": idempotencyKey})
  if errors.Is(err, ErrNotFound) && idempotencyKey != "" {
    existing, err := one(ctx, s.pool, `select `+columns+` from transfer where
idempotency_key = @key`,
      pgx.NamedArgs{"key": idempotencyKey})
    return existing, false, err
  }
  return t, err == nil, err
}
```

Cara kerjanya:

- `nullif(@key, '')` mengubah *key* kosong menjadi `NULL`, sehingga *request* tanpa *header* selalu menghasilkan transfer baru.
- Bila *key* sudah ada, `on conflict do nothing` membuat `returning` tidak menghasilkan baris. Fungsi `one` menerjemahkan kondisi tanpa baris tersebut menjadi `ErrNotFound`, lalu `Create` mengambil transfer yang tersimpan dengan *key* tersebut dan mengembalikan `created = false`.

6.4.10 `Execute` di `service.go`

Fungsi `Execute` di `transfer/internal/transfer/service.go` sekarang menerima *key* dan mengembalikan nilai tambahan `replayed`. Bagian *saga* (*debit*, *credit*, kompensasi) tidak berubah, hanya dipindahkan ke fungsi baru `run`. Tambahkan `"strings"` ke daftar *import*, lalu ganti *signature* dan bagian awal `Execute`, hingga dan termasuk baris `if err := s.accounts.Debit(...)`, menjadi:

```
//
// idempotencyKey (optional, from the Idempotency-Key header) makes a retried
// request safe: when the key was seen before, the saga does not run again and
// the original transfer is returned in whatever state it is now, with the
// second result (replayed) true. The error is then the one the original request
// ended with (see replayOutcome), so the HTTP layer answers the replay with the
// same status code. A request still in progress comes back as PENDING or
// DEBITED with ErrInProgress; the client polls GET /transfers/{id}.
//
// A row left PENDING or DEBITED by a crash (SIGKILL between two steps) is never
// resumed: nothing in this lab picks it up again (see the gap in the Lab 5
// README). A replay of its key keeps returning that state with ErrInProgress.
func (s *Service) Execute(ctx context.Context, req CreateRequest, idempotencyKey string)
(Transfer, bool, error) {
  t, created, err := s.store.Create(ctx, req, idempotencyKey)
  if err != nil {
    return Transfer{}, false, fmt.Errorf("record transfer: %w", err)
  }
  if !created {
    if !t.SameRequest(req) {
```

```
      s.log.Warn("idempotency key reused with a different request",
        "idempotency_key", idempotencyKey, "transfer_id", t.ID)
      return Transfer{}, false, ErrIdempotencyKeyReused
    }
    s.log.Info("idempotent replay, returning the original transfer",
      "idempotency_key", idempotencyKey, "transfer_id", t.ID, "status", t.Status)
    return t, true, replayOutcome(t)
  }
  t, err = s.run(ctx, t)
  return t, false, err
}

// replayOutcome rebuilds, from the stored transfer, the error the original
// request ended with: nil for COMPLETED, ErrReversalFailed for
// COMPENSATION_FAILED, ErrInProgress for PENDING and DEBITED. For FAILED and
// COMPENSATED the cause is only kept as text in failure_reason ("debit failed:
// debit 1111111111: insufficient funds"); the account client wraps the
// sentinels last, so the reason ends with the sentinel's message. Anything else
// was a dependency failure, the 502 of the original request.
func replayOutcome(t Transfer) error {
  switch t.Status {
  case Completed:
    return nil
  case Pending, Debited:
    return ErrInProgress
  case CompensationFailed:
    return fmt.Errorf("%w: %s", ErrReversalFailed, t.FailureReason)
  }
  for _, cause := range []error{ErrAccountNotFound, ErrInsufficientFunds,
ErrAccountUnavailable} {
    if strings.HasSuffix(t.FailureReason, ": "+cause.Error()) {
      return fmt.Errorf("%s: %w", t.Status, cause)
    }
  }
  return fmt.Errorf("%s: %s", t.Status, t.FailureReason)
}

// run executes the saga steps on a freshly recorded transfer.
func (s *Service) run(ctx context.Context, t Transfer) (Transfer, error) {

  if err := s.accounts.Debit(ctx, t.FromAccount, t.Amount, t.ID); err != nil {
```

Sisa isi fungsi lama, mulai dari penanganan *error* `Debit` hingga akhir, tetap seperti semula dan sekarang menjadi isi `run`. Komentar di atas fungsi yang menjelaskan langkah *saga* tetap dipertahankan, sedangkan paragraf `idempotencyKey` di atas ditambahkan di bawahnya.

Perhatikan bahwa *response* untuk pengulangan berisi transfer asli dengan status saat ini, ditambah *error* yang sama dengan hasil *request* pertama. *Error* tersebut disusun ulang oleh `replayOutcome` dari data yang tersimpan, sehingga *handler* dapat menjawab pengulangan dengan kode HTTP yang sama, kecuali bila status akhir *request* pertama gagal disimpan. Dengan begitu, klien yang kehilangan *response* pertama lalu mengulang *request* akan mengambil keputusan yang sama seperti bila *response* pertama diterima.

Untuk status `FAILED` dan `COMPENSATED`, penyebab kegagalan hanya tercatat sebagai teks di kolom `failure_reason`. `replayOutcome` memeriksa apakah teks tersebut diakhiri pesan salah satu *sentinel error*, misalnya `: insufficient funds`. Cara ini bergantung pada format teks dan cukup untuk lab. Di *production*, kode kegagalan sebaiknya disimpan di kolom tersendiri.

Bila *request* pertama masih berjalan, misalnya sedang menunggu `account`, *request* kedua menerima status `PENDING` atau `DEBITED` bersama `ErrInProgress`. *Saga* tidak dijalankan untuk kedua kalinya. Klien cukup memeriksa `GET /transfers/{id}` hingga statusnya final. Celah dari Lab 5 tetap berlaku: transfer yang tertinggal di `PENDING` atau `DEBITED` karena proses `transfer` mati di antara dua langkah tidak pernah dilanjutkan, sehingga pengulangan dengan *key* tersebut terus menerima status yang sama.

6.4.11 *Handler* `POST /transfers`

Di `transfer/internal/transfer/http.go`, fungsi `create` membaca *header*, memvalidasinya, lalu memilih kode HTTP. Tambahkan pembacaan *header* setelah validasi `req.Validate()` dan sebelum pemanggilan `Execute`:

```
  // Optional here so the requests of Labs 2-5 keep working. A channel-facing
  // payment API in production makes it required: without it, a client that
  // retries after a lost response cannot tell whether it pays twice.
  key := r.Header.Get("Idempotency-Key")
  if key != "" {
    if err := ValidateIdempotencyKey(key); err != nil {
      writeJSON(w, http.StatusBadRequest, map[string]any{"error": err.Error(), "fields":
err})
      return
    }
  }
```

Kode HTTP untuk *request* pertama maupun pengulangan sekarang dipilih dengan logika dari Lab 5. Karena itu, blok `switch` lama dipindahkan dari `create` ke fungsi baru `createStatus`. Ganti bagian `create` mulai dari komentar `WithoutCancel` di atas pemanggilan `Execute` hingga akhir fungsi, lalu tambahkan `createStatus` di bawahnya:

```
  // WithoutCancel: a client that disconnects must not abort a transfer halfway,
  // e.g. between the debit and the credit.
  t, replayed, err := h.svc.Execute(context.WithoutCancel(r.Context()), req, key)
  switch {
  case errors.Is(err, ErrIdempotencyKeyReused):
    writeError(w, http.StatusUnprocessableEntity, err.Error())
    return
  case t.ID == "":
    h.internalError(w, r, err)
    return
  }
  if replayed {
    // A replay answers with the status code the original request got for the
    // stored state, so a client whose first response was lost decides the same
    // way. The header tells it nothing was executed this time.
    w.Header().Set("Idempotent-Replayed", "true")
  }
  status := createStatus(err)
  if status == http.StatusCreated || status == http.StatusAccepted {
    w.Header().Set("Location", "/transfers/"+t.ID)
  }
  writeJSON(w, status, t)
}

// createStatus maps the outcome of a recorded transfer to the HTTP status of
// POST /transfers, for a first request and a replay alike.
func createStatus(err error) int {
  switch {
  case err == nil:
    return http.StatusCreated
  case errors.Is(err, ErrInProgress):
```

```
    // Replay only: the original request is still running (or was cut off by a
    // crash, see Service.Execute).
    return http.StatusAccepted
  case errors.Is(err, ErrReversalFailed):
    // Checked first: the source account is debited and the reversal did not go
    // through (COMPENSATION_FAILED, also when that state itself could not be
    // stored). Money is stuck on our side, whatever the account service
    // answered: this is our failure, not the caller's, and never a 503 or 422
    // that would invite a retry.
    return http.StatusInternalServerError
  case errors.Is(err, ErrAccountNotFound), errors.Is(err, ErrInsufficientFunds):
    // Includes COMPENSATED when the destination account does not exist.
    return http.StatusUnprocessableEntity
  case errors.Is(err, ErrAccountUnavailable):
    // The open breaker refused the debit (FAILED), or refused the credit and the
    // debit was reversed (COMPENSATED). Either way no money moved: 503 tells the
    // caller it is safe to come back later.
    return http.StatusServiceUnavailable
  default:
    // Recorded but not completed because a dependency failed (FAILED or
    // COMPENSATED).
    return http.StatusBadGateway
  }
}
```

Isi `createStatus` sama dengan `switch` dari Lab 5. Bedanya, setiap `case` mengembalikan kode HTTP alih-alih menulis *response*, dan ada satu `case` baru untuk `ErrInProgress`. Dua kondisi tetap ditangani langsung di `create` karena tidak memiliki transfer untuk dikembalikan: *key* yang digunakan ulang dengan *body* berbeda, dan kegagalan sebelum transfer tersimpan (`t.ID == ""`).

Kode HTTP untuk pengulangan dengan *key* yang sama:

Status Transfer yang Tersimpan	Kode HTTP Pengulangan
`COMPLETED`	201 Created, sama dengan *request* pertama
`FAILED` atau `COMPENSATED`	422, 503, atau 502, sesuai penyebab kegagalan di `failure_reason`
`COMPENSATION_FAILED`	500 Internal Server Error
`PENDING` atau `DEBITED`	202 Accepted: *request* pertama masih berjalan (atau terhenti oleh *crash* atau oleh kegagalan menyimpan status)

Setiap *response* pengulangan membawa *header* `Idempotent-Replayed: true`, yang memberi tahu klien bahwa kali ini tidak ada yang dijalankan. *Header* `Location` hanya diisi pada *response* 201 dan 202. Kondisi lain di luar tabel:

- **422 Unprocessable Entity** ketika *key* yang sama datang dengan *body* berbeda. Kondisi ini kemungkinan besar disebabkan oleh *bug* di klien yang menggunakan *key* lama untuk transfer baru. Menjalankannya berarti salah satu dari dua transfer tersebut hilang tanpa diketahui klien.
- **400 Bad Request** ketika format *key* tidak valid.

Pengulangan sengaja tidak dijawab dengan kode tersendiri seperti `200 OK`. Klien yang tidak menerima *response* pertama harus dapat mengambil keputusan dari *response* kedua dengan aturan yang sama: 201 berarti transfer selesai, 503 berarti tidak ada dana yang berpindah, 500 berarti dana tertahan dan perlu penanganan manual. Transfer yang tersimpan dengan status `FAILED` atau `COMPENSATED` sudah final. Pengulangan dengan *key* yang sama selalu mengembalikan hasil yang sama, sehingga klien yang ingin mencoba lagi setelah 503 mengirim transfer baru dengan *key* baru.

Header ini sengaja dibuat opsional agar perintah `curl` dari Lab 2–5 tetap berjalan. API pembayaran yang diakses *channel* di *production* mewajibkannya.

6.4.12 Memperbarui *test* `transfer`

Di `transfer/internal/transfer/service_test.go`, `fakeStore` harus mengikuti kontrak `Create` yang baru. Tambahkan *field* `keys` dan fungsi `newFakeStore`, lalu ganti `Create`:

```
type fakeStore struct {
  transfers map[string]Transfer
  keys      map[string]string // idempotency key -> transfer ID
  events    []EventType
  seq       int
}

func newFakeStore() *fakeStore {
  return &fakeStore{transfers: map[string]Transfer{}, keys: map[string]string{}}
}

func (f *fakeStore) Create(_ context.Context, req CreateRequest, key string) (Transfer,
bool, error) {
  if id, ok := f.keys[key]; ok && key != "" {
    return f.transfers[id], false, nil
  }
  f.seq++
  t := Transfer{ID: fmt.Sprint(f.seq), FromAccount: req.FromAccount, ToAccount:
req.ToAccount, Amount: req.Amount, Status: Pending}
  f.transfers[t.ID] = t
  if key != "" {
    f.keys[key] = t.ID
  }
  return t, true, nil
}
```

Di `TestExecuteSaga`, ganti pembuatan *store* dan pemanggilan `Execute`:

```
      store := newFakeStore()
      // ...
      got, _, err := svc.Execute(context.Background(), req, "")
```

Tambahkan *test* baru di *file* yang sama. *Test* ini memeriksa empat perilaku: transfer pertama berjalan, pengulangan dengan nominal yang ditulis berbeda tidak memanggil `account` lagi, *body* berbeda ditolak, dan *request* tanpa *key* selalu menjadi transfer baru.

```
func TestExecuteIdempotencyKey(t *testing.T) {
  req := CreateRequest{FromAccount: "1111111111", ToAccount: "2222222222", Amount:
decimal.RequireFromString("100000")}
  store := newFakeStore()
  accounts := &fakeAccounts{}
  svc := NewService(store, accounts, slog.New(slog.NewTextHandler(io.Discard, nil)))
  ctx := context.Background()

  first, replayed, err := svc.Execute(ctx, req, "key-1")
  if err != nil || replayed || first.Status != Completed {
    t.Fatalf("first: %+v replayed=%v err=%v, want COMPLETED", first, replayed, err)
  }

  // Same key, same body written differently: the original, no new account calls.
  same := req
  same.Amount = decimal.RequireFromString("100000.00")
  again, replayed, err := svc.Execute(ctx, same, "key-1")
  if err != nil || !replayed || again.ID != first.ID {
```

```
    t.Fatalf("replay: %+v replayed=%v err=%v, want transfer %s replayed", again, replayed,
err, first.ID)
  }
  if len(accounts.calls) != 2 {
    t.Fatalf("account calls = %q, want only the first transfer's debit and credit",
accounts.calls)
  }

  // Same key, different body.
  other := req
  other.Amount = decimal.RequireFromString("5")
  if _, _, err := svc.Execute(ctx, other, "key-1"); !errors.Is(err,
ErrIdempotencyKeyReused) {
    t.Fatalf("err = %v, want ErrIdempotencyKeyReused", err)
  }

  // Without a key every request is a new transfer.
  a, _, _ := svc.Execute(ctx, req, "")
  b, _, _ := svc.Execute(ctx, req, "")
  if a.ID == b.ID || a.ID == first.ID {
    t.Fatalf("ids %s, %s, %s: want three different transfers", first.ID, a.ID, b.ID)
  }
}
```

Tambahkan juga *test* untuk `replayOutcome`. *Test* ini memeriksa bahwa setiap kombinasi status dan `failure_reason` menghasilkan *sentinel* *error* yang sama dengan *request* pertama, dan bahwa kegagalan jaringan tidak cocok dengan `ErrAccountNotFound`, `ErrInsufficientFunds`, maupun `ErrAccountUnavailable`, sehingga tetap dijawab 502:

```
// TestReplayOutcome: a replay gets back the error class of the original
// request, rebuilt from the stored status and failure_reason.
func TestReplayOutcome(t *testing.T) {
  tests := []struct {
    status Status
    reason string
    want   error
  }{
    {Completed, "", nil},
    {Pending, "", ErrInProgress},
    {Debited, "", ErrInProgress},
    {Failed, "debit failed: debit 1111111111: insufficient funds", ErrInsufficientFunds},
    {Failed, "debit failed: debit 1111111111: account service unavailable",
ErrAccountUnavailable},
    {Compensated, "credit failed: credit 9999999999: account not found",
ErrAccountNotFound},
    {Compensated, "credit failed: credit 2222222222: account service unavailable",
ErrAccountUnavailable},
    {CompensationFailed, "credit failed: x; reversal failed: y", ErrReversalFailed},
  }
  for _, tc := range tests {
    err := replayOutcome(Transfer{Status: tc.status, FailureReason: tc.reason})
    if !errors.Is(err, tc.want) || (tc.want == nil && err != nil) {
      t.Errorf("%s %q: err = %v, want %v", tc.status, tc.reason, err, tc.want)
    }
  }
  // A transport failure maps to none of the sentinels: the 502 of the original.
  err := replayOutcome(Transfer{Status: Failed, FailureReason: "debit failed: debit
1111111111: rpc error: code = Unavailable"})
  for _, s := range []error{ErrAccountNotFound, ErrInsufficientFunds,
```

```
ErrAccountUnavailable, ErrReversalFailed, ErrInProgress} {
    if err == nil || errors.Is(err, s) {
      t.Fatalf("transport failure: err = %v, want a plain error", err)
    }
  }
}
```

Test kode HTTP sudah ada sejak Lab 3 di `transfer/internal/transfer/http_test.go` (`TestCreateStatus`). Di Lab 6, *file* tersebut mendapat tiga perubahan.

Pertama, tambahkan `"github.com/shopspring/decimal"` ke daftar *import*, lalu tambahkan fungsi *helper* `post` di bawah `fails`. Fungsi ini mengirim `POST /transfers` dengan *header* `Idempotency-Key`:

```
func post(t *testing.T, url, key string) *http.Response {
  t.Helper()
  body := `{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}`
  req, err := http.NewRequest(http.MethodPost, url+"/transfers", strings.NewReader(body))
  if err != nil {
    t.Fatal(err)
  }
  req.Header.Set("Idempotency-Key", key)
  resp, err := http.DefaultClient.Do(req)
  if err != nil {
    t.Fatal(err)
  }
  resp.Body.Close()
  return resp
}
```

Kedua, `TestCreateStatus` menjalankan setiap kasus dua kali dengan *key* yang sama. Pengulangan harus menghasilkan kode HTTP yang sama, membawa `Idempotent-Replayed: true`, dan tidak memanggil `account` lagi. Tambahkan paragraf berikut di akhir komentar di atas `TestCreateStatus`:

```
//
// The same cases run twice with an Idempotency-Key: the replay must answer with
// the status of the first request, plus Idempotent-Replayed: true, without
// calling the account service again.
```

Lalu ganti isi `t.Run` di dalam perulangan kasus:

```
    t.Run(tc.name, func(t *testing.T) {
      log := slog.New(slog.NewTextHandler(io.Discard, nil))
      svc := NewService(newFakeStore(), tc.accounts, log)
      srv := httptest.NewServer(NewHandler(svc, log).Routes())
      defer srv.Close()

      first := post(t, srv.URL, "k-1")
      if first.StatusCode != tc.want || first.Header.Get("Idempotent-Replayed") != "" {
        t.Fatalf("first: status = %d, replayed = %q, want %d", first.StatusCode,
first.Header.Get("Idempotent-Replayed"), tc.want)
      }
      calls := len(tc.accounts.calls)
      replay := post(t, srv.URL, "k-1")
      if replay.StatusCode != tc.want || replay.Header.Get("Idempotent-Replayed") != "true"
{
        t.Fatalf("replay: status = %d, replayed = %q, want %d, true", replay.StatusCode,
replay.Header.Get("Idempotent-Replayed"), tc.want)
      }
      if len(tc.accounts.calls) != calls {
        t.Fatalf("account calls = %q, want none for the replay", tc.accounts.calls[calls:])
```

```
      }
    })
```

Dengan perubahan ini, setiap baris tabel `TestCreateStatus` (201, 422, 503, 502, 500) sekaligus membuktikan bahwa `replayOutcome` menyusun ulang penyebab kegagalan dengan benar.

Ketiga, tambahkan *test* baru di akhir *file* untuk perilaku khusus *header*:

```
func TestCreateIdempotencyKeyHTTP(t *testing.T) {
  log := slog.New(slog.NewTextHandler(io.Discard, nil))
  store := newFakeStore()
  h := NewHandler(NewService(store, &fakeAccounts{}, log), log).Routes()
  body := `{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}`

  // A transfer whose first request is still running (or was cut off by a crash).
  store.transfers["99"] = Transfer{ID: "99", FromAccount: "1111111111", ToAccount:
"2222222222",
    Amount: decimal.RequireFromString("100000"), Status: Debited}
  store.keys["k-running"] = "99"

  tests := []struct {
    name         string
    key          string
    body         string
    wantStatus   int
    wantBody     string
    wantReplayed string
  }{
    {"new transfer", "k-1", body, http.StatusCreated, `"status":"COMPLETED"`, ""},
    {"same key again", "k-1", body, http.StatusCreated, `"id":"1"`, "true"},
    {"same key, still in progress", "k-running", body, http.StatusAccepted,
`"status":"DEBITED"`, "true"},
    {"same key, different body", "k-1", strings.Replace(body, "100000", "1", 1),
      http.StatusUnprocessableEntity, "idempotency key", ""},
    {"invalid key", "has space", body, http.StatusBadRequest, "Idempotency-Key", ""},
    {"no key", "", body, http.StatusCreated, `"id":"2"`, ""},
  }
  for _, tc := range tests {
    t.Run(tc.name, func(t *testing.T) {
      req := httptest.NewRequest(http.MethodPost, "/transfers", strings.NewReader(tc.body))
      if tc.key != "" {
        req.Header.Set("Idempotency-Key", tc.key)
      }
      rec := httptest.NewRecorder()
      h.ServeHTTP(rec, req)
      if rec.Code != tc.wantStatus || !strings.Contains(rec.Body.String(), tc.wantBody) {
        t.Fatalf("%d %s, want %d containing %q", rec.Code, rec.Body, tc.wantStatus,
tc.wantBody)
      }
      if got := rec.Header().Get("Idempotent-Replayed"); got != tc.wantReplayed {
        t.Fatalf("Idempotent-Replayed = %q, want %q", got, tc.wantReplayed)
      }
    })
  }
}
```

Transfer `99` berstatus `DEBITED` dimasukkan langsung ke *store* untuk meniru *request* pertama yang belum selesai, sehingga jalur 202 dapat diuji tanpa *goroutine* paralel. Kasus-kasus lain di tabel berjalan berurutan dan berbagi satu *store*, sehingga kasus kedua dapat memeriksa bahwa *key* `k-1` mengembalikan transfer `1` dari kasus pertama dengan kode 201. Jalankan seluruh *test* dari direktori `studi-kasus`:

```
make test
```

6.5 Verifikasi

Hentikan `transfer` dan `notification` dengan `Ctrl+C`, lalu jalankan lagi agar kode dan *schema* baru berlaku:

```
make -C transfer run
make -C notification run
```

6.5.1 *Replay* tanpa notifikasi ganda

Hentikan `notification`, kembalikan *offset* ke awal, lalu jalankan `notification` lagi:

```
make replay
make -C notification run
```

Log `notification` sekarang berisi `duplicate event skipped` untuk setiap *event* lama, tanpa `notification sent`. Contoh satu baris log (waktu, *offset*, dan ID akan berbeda):

```
time=2026-10-28T10:15:02.118+07:00 level=INFO msg="duplicate event skipped"
event_type=TransferCompleted partition=1 offset=0 event_id=5b0d... transfer_id=0e2c...
```

Periksa *event* yang memiliki lebih dari satu notifikasi:

```
make duplicates
```

Pada database yang dibuat dari awal di Lab 6 (misalnya setelah `make infra-reset`), hasilnya kosong:

```
 event_id | event_type | notifications
----------+------------+---------------
(0 rows)
```

Pada database dari Lab 5, daftar tersebut masih berisi duplikat hasil *replay* di langkah pertama lab ini. Baris-baris tersebut tersimpan sebelum `processed_event` ada, sehingga yang perlu dibuktikan hanyalah jumlah notifikasinya tidak bertambah. Jalankan `make replay` sekali lagi (dengan `notification` dihentikan lebih dahulu), lalu `make duplicates`: angka di kolom `notifications` tetap sama.

6.5.2 `Idempotency-Key` di `POST /transfers`

Gunakan `curl -i` agar kode status dan *header response* terlihat. Kirim transfer dengan sebuah *key*:

```
curl -i -X POST localhost:28082/transfers \
  -H 'Idempotency-Key: 7f9c2b1e-0d4a-4c1e-9b8f-1a2b3c4d5e6f' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

Response pertama berupa `201 Created` dengan transfer berstatus `COMPLETED` (potongan *response*; ID dan waktu akan berbeda):

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /transfers/3f1e...
...

{"id":"3f1e...","from_account":"1111111111","to_account":"2222222222","amount":"100000","status":"COMPL
```

Ulangi perintah yang sama persis. Kode HTTP-nya tetap `201 Created`, karena transfer asli berstatus `COMPLETED`, tetapi *response*-nya membawa *header* tambahan dan `id` yang sama:

```
HTTP/1.1 201 Created
Content-Type: application/json
Idempotent-Replayed: true
Location: /transfers/3f1e...
...
```

Log `transfer` mencatat `idempotent replay, returning the original transfer`, dan log `account` tidak mencatat *debit* baru. Periksa saldo rekening asal: saldo hanya berkurang sekali.

```
curl localhost:28081/accounts/1111111111
```

Kirim *key* yang sama dengan nominal berbeda:

```
curl -i -X POST localhost:28082/transfers \
  -H 'Idempotency-Key: 7f9c2b1e-0d4a-4c1e-9b8f-1a2b3c4d5e6f' \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"5000"}'
```

Response-nya `422 Unprocessable Entity`:

```
{"error":"idempotency key already used for a different request"}
```

Nominal `"100000.00"` dengan *key* yang sama tetap dianggap pengulangan (`201` dengan `Idempotent-Replayed: true`), karena nominal dibandingkan sebagai desimal. *Key* yang mengandung spasi, misalnya `-H 'Idempotency-Key: ada spasi'`, dijawab `400` dengan `fields` yang menyebut `Idempotency-Key`.

Pengulangan juga mempertahankan kode HTTP untuk transfer yang gagal. Kirim transfer ke rekening tujuan yang tidak ada dengan *key* baru, dua kali:

```
for i in 1 2; do
  curl -s -o /dev/null -w '%{http_code}\n' -X POST localhost:28082/transfers \
    -H 'Idempotency-Key: gagal-001' \
    -d '{"from_account":"1111111111","to_account":"9999999999","amount":"100000"}'
done
```

Kedua *response* berkode `422`. *Request* pertama menjalankan *debit*, gagal di *credit*, lalu membalikkan *debit* (status `COMPENSATED`, `failure_reason` diakhiri `account not found`). *Request* kedua tidak menjalankan apa pun, hanya membaca `failure_reason` tersebut melalui `replayOutcome` untuk menghasilkan kode yang sama.

6.5.3 *Request* paralel dengan *key* yang sama

Pengujian terakhir mensimulasikan aplikasi yang mengirim ulang *request* berkali-kali dalam waktu hampir bersamaan. Catat saldo rekening asal, lalu kirim enam *request* paralel dengan satu *key* baru:

```
curl localhost:28081/accounts/1111111111
KEY=$(uuidgen)
for i in 1 2 3 4 5 6; do
  curl -s -o /dev/null -w '%{http_code}\n' -X POST localhost:28082/transfers \
    -H "Idempotency-Key: $KEY" \
    -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}' &
done; wait
curl localhost:28081/accounts/1111111111
```

Bila `uuidgen` tidak tersedia, isi `KEY` dengan teks unik apa saja, misalnya `KEY=paralel-001`. Setiap *response* berupa `201` atau `202`, dengan urutan dan jumlah yang berbeda pada setiap percobaan, misalnya:

```
202
201
202
202
201
202
```

Saldo rekening asal berkurang tepat 100000. *Unique index* pada `idempotency_key` menjamin hanya satu `insert` yang berhasil, sedangkan kelima *request* lain mengambil transfer yang sudah tersimpan. Pengulangan yang tiba ketika *saga* dari *request* pertama belum selesai menerima `202 Accepted`, karena transfer masih berstatus `PENDING` atau `DEBITED`. Pengulangan yang tiba setelah *saga* selesai menerima `201`, sama dengan *request* pertama. Kode HTTP saja tidak dapat membedakan *request* pertama dari pengulangan. Pembedanya *header* `Idempotent-Replayed: true`, yang hanya muncul pada pengulangan.

6.6 Masalah Umum

`make replay` gagal dengan `Assignments can only be reset if the group 'notification' is inactive, but the current state is Stable.` `notification` masih berjalan, atau baru saja dihentikan dan Kafka belum mencatat bahwa anggota tersebut keluar dari *group*. Dengan `Ctrl+C`, `reader.Close()` langsung mengeluarkan `notification` dari *consumer group*. Bila proses mati tanpa sempat menutup *reader* (misalnya `kill -9` atau terminal ditutup), Kafka baru mengeluarkan anggota tersebut setelah *session timeout* habis, sekitar 30 detik. Pastikan `notification` sudah berhenti di terminalnya, tunggu hingga *session timeout* lewat bila perlu, lalu ulangi `make replay`.

Kompilasi gagal dengan `does not implement Store (wrong type for method Insert)` atau `wrong type for method Create`. Kontrak `Store` sudah diubah, tetapi `fakeStore` di *test* belum. Perbarui `fakeStore` di `consumer_test.go` (untuk `notification`) atau `service_test.go` (untuk `transfer`) sesuai bagian *test* di atas.

Kompilasi gagal dengan `assignment mismatch: 2 variables but h.svc.Execute returns 3 values`. Ada pemanggilan `Execute` yang belum diubah. Cari semua pemanggilan dengan `grep -rn "Execute(" transfer/` lalu tambahkan variabel `replayed` (atau `_`) dan argumen *key*.

Pengulangan dengan *key* yang sama menghasilkan transfer baru dengan `id` berbeda, tanpa *header* `Idempotent-Replayed`. `transfer` yang berjalan masih versi lama, karena service belum dijalankan ulang setelah kode diubah. Periksa juga penulisan nama *header* di perintah `curl`: `Idempotency-Key`, dengan tanda hubung.

`make duplicates` masih menampilkan baris setelah semua langkah selesai. Baris tersebut berasal dari *replay* di Lab 5 atau di langkah pertama lab ini, sebelum `processed_event` ada. Selama jumlahnya tidak bertambah setelah *replay* berikutnya, *consumer* sudah *idempotent*. Untuk memulai dari database bersih, jalankan `make infra-reset` lalu `make infra-up`. Perintah `make infra-reset` menghapus semua data lab, termasuk rekening, sehingga rekening contoh perlu dibuat ulang.

Pengulangan dijawab `202 Accepted` dengan status `PENDING` atau `DEBITED`. Perilaku ini memang diharapkan. *Request* pertama dengan *key* tersebut masih berjalan, dan *request* pengulangan tidak menunggunya. Periksa `GET /transfers/{id}` (alamatnya ada di *header* `Location`) beberapa saat kemudian. Bila statusnya tidak pernah berubah, proses `transfer` mati di tengah *saga*, atau status berikutnya gagal disimpan (log `debit applied but not recorded` atau `saga end state not recorded`). Di lab ini, tidak ada mekanisme yang melanjutkan transfer tersebut, sehingga pengulangan terus dijawab `202`, walaupun *request* pertama mungkin menerima kode lain.

Aplikasi klien mendapat `422` padahal hanya mengulang *request*. Klien membuat *body* yang berbeda pada pengulangan, atau menggunakan ulang *key* untuk transfer baru. Setiap transfer baru harus memiliki *key* sendiri, dan pengulangan harus mengirim *body* yang sama.

6.7 Eksplorasi Mandiri

1. ***Crash* di antara *commit* database dan *commit offset*.** Tambahkan `time.Sleep(10 * time.Second)` sementara di `Run` (`consumer.go`), tepat sebelum `c.reader.CommitMessages`. Buat satu transfer, tunggu hingga log `notification sent` muncul, lalu hentikan proses `notification` secara paksa sebelum 10 detik berlalu. `go run` menjalankan *binary* hasil kompilasi sebagai *child process*, sehingga proses itulah yang perlu dihentikan dengan`pkill -9 -f exe/notification`. Jalankan lagi `notification` dan amati bahwa *event* yang sama datang lagi lalu dilewati sebagai duplikat. Hapus kembali `time.Sleep` setelah selesai.
2. **Dua *instance* `notification`.** Jalankan *instance* kedua dari direktori `notification` dengan port HTTP lain: salin *environment variable* dari *target* `run` di `notification/Makefile`, ganti `HTTP_ADDR=:8093`, lalu jalankan `go run .`. Amati di log bagaimana tiga *partition* dibagi ke dua anggota *consumer group*. Hentikan keduanya, jalankan `make replay`, lalu jalankan keduanya lagi. Periksa dengan `make duplicates` bahwa tidak ada notifikasi baru yang ganda.
3. **Lingkup *key* per klien.** Di lab ini, *key* berlaku global untuk semua klien. Di *production*, dua klien berbeda bisa saja secara kebetulan membuat *key* yang sama, sehingga *unique index* biasanya dibuat pada pasangan (klien, *key*). Rancang perubahan *schema* dan kode untuk lingkup tersebut, misalnya dengan *header* `X-Client-ID` sementara. Di Lab 7, identitas klien akan tersedia dari *token* OAuth2.
4. **Masa berlaku *key*.** *Key* di lab ini tidak pernah kedaluwarsa. Sistem pembayaran umumnya menghapus *key* setelah periode tertentu, misalnya 24 jam. Tulis perintah SQL yang mengosongkan `idempotency_key` untuk transfer yang lebih tua dari 24 jam, lalu jelaskan apa yang terjadi bila klien mengulang *request* setelah *key*-nya dihapus.

6.8 Checkpoint

Kondisi kode di akhir lab ini tersedia di tag `lab-06`. Bandingkan hasil kerja dengan tag tersebut, atau lanjutkan langsung dari tag tersebut bila tertinggal:

```
git diff lab-06 -- .
git switch -c lab-07-kerja lab-06
```

7 Lab 7: Mengamankan Service dengan OAuth2 dan JWT

7.1 Tujuan

Setelah menyelesaikan lab ini, peserta mampu:

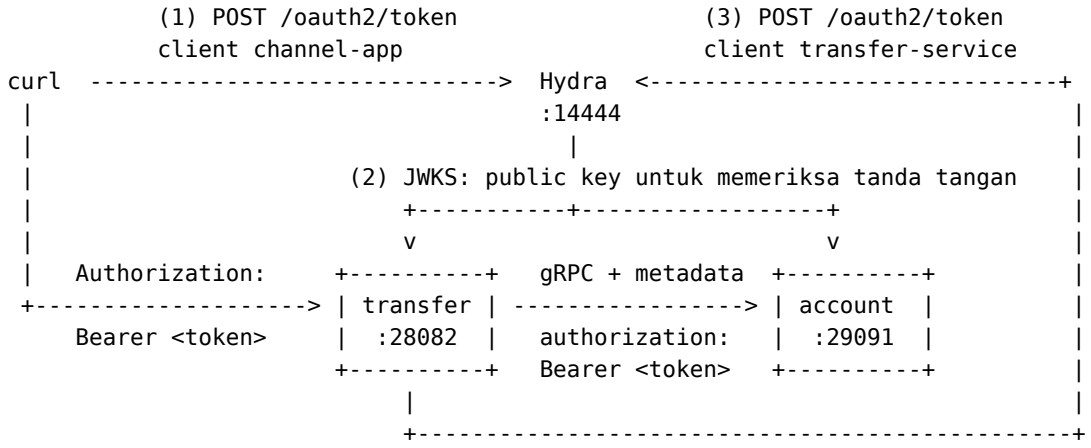
- menjelaskan peran *authorization server*, *resource server*, dan *client* pada OAuth2 *client credentials grant*;
- memverifikasi JWT *access token* di dalam service Go: *signature* (tanda tangan digital) dengan *public key* dari JWKS (*JSON Web Key Set*), *issuer* (penerbit *token*), masa berlaku, dan *scope*;
- menulis *middleware* REST yang menjawab 401 atau 403 sesuai jenis penolakan;
- menulis gRPC *unary server interceptor* yang menjawab `Unauthenticated` atau `PermissionDenied`;
- membuat service `transfer` mengambil *token* sendiri dari *authorization server* dan mengirimkannya di setiap panggilan gRPC ke `account`;
- menyembunyikan detail error internal dari respons API tanpa kehilangan detail tersebut di log.

7.2 Gambaran

Hingga Lab 6, siapa pun yang dapat menjangkau port 28082 bisa membuat transfer, dan siapa pun yang dapat menjangkau port 29091 bisa mendebit rekening. Lab ini menambahkan pemeriksaan identitas pemanggil di kedua pintu masuk tersebut.

Identitas pemanggil dibuktikan dengan *access token* berformat JWT (*JSON Web Token*). *Token* diterbitkan oleh Ory Hydra, sebuah *authorization server* OAuth2. Pemanggilnya berupa aplikasi, bukan

pengguna yang *login*, sehingga lab ini menggunakan alur *client credentials grant*: aplikasi mengirim *client ID* dan *secret* miliknya ke Hydra, lalu Hydra mengembalikan *token* yang berlaku selama satu jam.



Ada tiga peran dalam gambar di atas:

Peran OAuth2	Komponen	Tugas di Lab 7
Authorization server	Hydra	Menerbitkan dan menandatangani *token*, serta memublikasikan *public key* di JWKS. Konfigurasinya sudah disediakan.
Resource server	`account` (gRPC), `transfer` (REST)	Memeriksa *token* di setiap permintaan: *signature*, *issuer*, masa berlaku, dan *scope*.
Client	`curl` sebagai aplikasi *channel*, `transfer` saat memanggil `account`	Meminta *token* ke Hydra dan mengirimkannya di *header* `Authorization`.

Service `transfer` memegang dua peran sekaligus: *resource server* bagi aplikasi *channel* dan *client* bagi `account`.

Pemeriksaan *token* berlangsung di dalam service tanpa bertanya ke Hydra untuk setiap permintaan. Hydra menandatangani *token* dengan *private key* RSA, sedangkan service memeriksa *signature* tersebut dengan *public key* yang diunduh dari JWKS (*JSON Web Key Set*). Service cukup mengunduh JWKS sekali, lalu menyimpannya di memori. Akibatnya, setiap transfer tidak perlu menunggu jawaban Hydra.

Scope membatasi apa yang boleh dilakukan pemegang *token*. Lab ini menggunakan empat *scope*:

Service	Operasi	*Scope* yang Diperlukan
`account` gRPC	`GetAccount`	`account:read`
`account` gRPC	`Debit`, `Credit`	`account:write`
`transfer` REST	`POST /transfers`	`transfer:write`
`transfer` REST	`GET /transfers/{id}`	`transfer:read`

Beberapa *endpoint* tetap terbuka tanpa *token*: `/healthz`, `/readyz`, `/debug/circuit-breaker`, gRPC *health* dan *reflection*, `/admin/fault`, serta REST `account` (`/accounts`). *Probe* dipanggil oleh Kubernetes dan *load balancer* yang tidak membawa *token*. REST `account` dibiarkan terbuka agar langkah pembuatan rekening di lab tetap singkat. Di *production*, REST `account` dipasangi *middleware* yang sama dengan REST `transfer`.

Hasil pemeriksaan dibedakan menjadi dua jenis penolakan:

Kondisi	REST `transfer`	gRPC `account`
Tanpa *token*, atau *header* bukan `Bearer`	401	`Unauthenticated`
Token rusak, kedaluwarsa, *issuer* lain, *signature* tidak cocok	401	`Unauthenticated`
Token valid, tetapi *scope* tidak mencukupi	403	`PermissionDenied`

401 berarti "identitas pemanggil tidak dikenali", sedangkan 403 berarti "pemanggil dikenali, tetapi tidak berhak". Perbedaan ini penting bagi pemanggil: 401 diselesaikan dengan meminta *token* baru, sedangkan 403 tidak akan selesai hanya dengan mengulang permintaan.

> **Pembagian kerja di lab ini.** Konfigurasi Hydra beserta pendaftaran *client* sudah disediakan di `compose.yaml` dan `Makefile`. Peserta tidak mengonfigurasi Hydra. Yang ditulis peserta adalah sisi service: *interceptor* gRPC di `account`, *middleware* REST di `transfer`, dan pengiriman *token* dari `transfer` ke `account`.

7.3 Titik Awal

Lab ini dimulai dari kondisi akhir Lab 6. Peserta yang menyelesaikan Lab 6 dapat melanjutkan dari kode sendiri. Peserta yang tertinggal mengambil *checkpoint* Lab 6 dan membuat *branch* kerja:

```
git switch -c lab-07-kerja lab-06
cd studi-kasus
```

Semua perintah di bab ini dijalankan dari folder `studi-kasus`, kecuali disebutkan lain. Database dari Lab 6 tidak perlu di-*reset* karena lab ini tidak mengubah skema.

Pastikan service lama sudah berhenti. Hentikan `account`, `transfer`, dan `notification` dengan Ctrl+C di terminal masing-masing.

7.4 Langkah

7.4.1 Menjalankan Hydra

Konfigurasi Hydra tersedia di *checkpoint* `lab-07`. Ambil dua *file* infrastruktur dari *checkpoint* tersebut tanpa mengubah kode service:

```
git checkout lab-07 -- compose.yaml Makefile
```

Perubahan di `compose.yaml` hanya menambah dua service. Bagian yang perlu dipahami:

```
hydra:
  image: oryd/hydra:v26.2.0
  command: ["serve", "all", "--dev"]
  environment:
    DSN: memory
    URLS_SELF_ISSUER: http://localhost:14444/
    STRATEGIES_ACCESS_TOKEN: jwt
    SECRETS_SYSTEM: lab-only-system-secret-change-me
  ports:
    - "14444:4444"
    - "14445:4445"
```

- `--dev` mengizinkan HTTP tanpa TLS. Opsi ini hanya untuk laptop.
- `DSN: memory` menyimpan seluruh data Hydra di memori. Setelah Hydra di-*restart*, *client* dan *signing key* hilang. Sifat ini dimanfaatkan di bagian Eksplorasi Mandiri untuk mensimulasikan rotasi *key*.
- `URLS_SELF_ISSUER` menjadi isi *claim* `iss` di setiap *token*. Perhatikan garis miring di akhir alamat.

- `STRATEGIES_ACCESS_TOKEN: jwt` membuat Hydra menerbitkan JWT. Tanpa opsi ini, Hydra menerbitkan *opaque token* yang hanya bisa diperiksa dengan bertanya ke Hydra.
- Port 4444 adalah *public endpoint* (*token* dan JWKS), sedangkan port 4445 adalah *admin endpoint* untuk mengelola *client* dan tidak boleh dibuka ke luar di *production*. Di laptop, kedua port tersebut dipetakan ke 14444 dan 14445, di rentang 1xxxx seperti PostgreSQL dan Kafka, agar tidak bentrok dengan aplikasi lain.

Service kedua, `hydra-init`, berjalan sekali untuk mendaftarkan dua *client* dengan ID dan *secret* tetap:

Client ID	***Secret***	***Scope* yang Boleh Diminta**	**Digunakan oleh**
`transfer-service`	`transfer-service-secret`	`account:read`, `account:write`	`transfer` saat memanggil gRPC `account`
`channel-app`	`channel-app-secret`	`transfer:read`, `transfer:write`	Aplikasi *channel*; di lab ini diperankan oleh `curl`

`Makefile` mendapat dua tambahan: `infra-up` sekarang juga menjalankan `hydra-init`, dan target baru `token` meminta *token* `channel-app` ke Hydra. Jalankan infrastruktur:

```
make infra-up
```

Pastikan Hydra sudah menerbitkan *token*:

```
TOKEN=$(make -s token)
echo "$TOKEN"
```

Output-nya berupa satu baris panjang yang terdiri atas tiga bagian dipisah titik (`header.payload.signature`). Bagian tengah berisi *claim* dalam JSON yang di-*encode* dengan base64url. Isinya dapat dibaca dengan:

```
cut -d. -f2 <<<"$TOKEN" | base64 -d
```

Perintah `base64` dapat menampilkan error mengenai *padding* di akhir *output*, tetapi JSON-nya tetap terbaca. *Claim* yang diperiksa oleh service di lab ini:

Claim	**Isi**
`iss`	Penerbit *token*: `http://localhost:14444/`
`exp`	Waktu kedaluwarsa (detik sejak 1 Januari 1970), satu jam setelah diterbitkan
`client_id`	*Client* yang meminta *token*, misalnya `channel-app`
`scp`	*Scope* yang diberikan, dalam bentuk *array* JSON

> **Format *scope* berbeda di setiap *authorization server*.** Hydra menaruh *scope* di *claim* `scp` berupa *array*. Keycloak menaruhnya di *claim* `scope` berupa satu *string* yang dipisah spasi. Kode di lab ini khusus untuk format Hydra. Saat pindah ke *authorization server* lain, bagian yang berubah hanya *struct* `Claims`.

Hydra baru membuat *signing key* untuk *access token* saat menerima permintaan *token* pertama. Perintah `make -s token` di atas sekaligus memastikan *key* tersebut sudah ada.

7.4.2 Menambahkan *Dependency*

Lab ini menggunakan tiga *library* baru:

- `github.com/golang-jwt/jwt/v5`: mem-*parse* JWT, memeriksa *signature* dan *claim* standar;
- `github.com/MicahParks/keyfunc/v3`: mengunduh JWKS, menyimpannya di memori, dan mengunduh ulang saat menemukan *key ID* (`kid`) yang belum dikenal;
- `golang.org/x/time/rate`: membatasi frekuensi unduh ulang JWKS.

`transfer` juga membutuhkan `golang.org/x/oauth2` untuk meminta *token* ke Hydra.

```
(cd account && go get github.com/golang-jwt/jwt/v5@v5.3.1 \
  github.com/MicahParks/keyfunc/v3@v3.8.2 golang.org/x/time@v0.15.0)
(cd transfer && go get github.com/golang-jwt/jwt/v5@v5.3.1 \
  github.com/MicahParks/keyfunc/v3@v3.8.2 golang.org/x/time@v0.15.0 \
  golang.org/x/oauth2@v0.37.0)
```

Setelah semua kode di bab ini selesai ditulis, `go mod tidy` di setiap modul merapikan `go.mod`.

7.4.3 Konfigurasi OAuth2 di `account`

`account` membutuhkan dua nilai: *issuer* yang diharapkan dan alamat JWKS. Sesuai aturan studi kasus, keduanya wajib diisi dan tidak memiliki nilai *default*. Tambahkan dua *field* ke `Config` di `account/config.go`:

```
type Config struct {
  HTTPAddr    string
  GRPCAddr    string
  DatabaseURL string
  // OAuthIssuer must equal the "iss" claim of the tokens, character for
  // character (Hydra: http://localhost:14444/, trailing slash included).
  OAuthIssuer string
  // OAuthJWKSURL is where the authorization server publishes its public keys.
  OAuthJWKSURL string
}
```

Lalu isi kedua *field* di `LoadConfig`, di dalam *literal* `cfg := Config{...}`:

```
    OAuthIssuer:  get("OAUTH_ISSUER"),
    OAuthJWKSURL: get("OAUTH_JWKS_URL"),
```

Fungsi `get` yang sudah ada sejak Lab 1 mencatat variabel yang kosong, sehingga `account` berhenti saat *startup* bila salah satu variabel ini tidak diisi.

Tambahkan nilainya untuk laptop di target `run` pada `account/Makefile`, sebelum `go run .`:

```
  OAUTH_ISSUER=http://localhost:14444/ \
  OAUTH_JWKS_URL=http://localhost:14444/.well-known/jwks.json \
```

7.4.4 Memeriksa JWT dengan `Verifier`

Pemeriksaan *token* dipisahkan dari *transport*. Paket baru `account/internal/auth` berisi `Verifier` yang hanya menangani JWT, sedangkan *interceptor* gRPC di langkah berikutnya hanya menangani *metadata* gRPC. Pemisahan ini membuat `Verifier` yang sama dapat digunakan ulang di *middleware* REST `transfer`.

Hal-hal yang diperiksa `Verifier`, beserta alasannya:

- **Algoritma *signature* dibatasi pada RS256.** JWT membawa nama algoritmanya di *header*. Bila service menerima algoritma apa pun yang tertulis di *header*, penyerang dapat mengirim *token* dengan `alg: none` (tanpa *signature*) atau menandatangani dengan HS256 menggunakan *public key* sebagai *secret*.

- ***Issuer* harus sama persis**, termasuk garis miring di akhir. *Token* dari *authorization server* lain, atau dari *environment* lain, ditolak.
- **`exp` wajib ada.** *Token* tanpa `exp` berlaku selamanya.
- ***Leeway* 30 detik** untuk menoleransi selisih jam antarserver.
- ***Audience* (`aud`) tidak diperiksa** di lab ini, karena *token client credentials* dari Hydra tidak membawa `aud` kecuali diminta. Di *production*, setiap *resource server* didaftarkan sebagai *audience* tersendiri dan `aud` diperiksa, sehingga *token* untuk satu service tidak dapat digunakan di service lain.

JWKS diunduh oleh `keyfunc` saat *startup* dan diperbarui setiap jam. Bila *token* datang dengan `kid` yang belum dikenal (Hydra baru dibuat ulang, atau *key* dirotasi), `keyfunc` mengunduh ulang JWKS. Unduh ulang ini dibatasi paling sering sekali per 5 detik. Tanpa batas tersebut, penyerang yang mengirim *token* sampah dengan `kid` acak akan membuat service membanjiri Hydra dengan permintaan JWKS.

Kegagalan mengunduh JWKS saat *startup* hanya dicatat di log, tidak menghentikan service. `account` tetap berjalan ketika Hydra mati, dan hanya pemeriksaan *token* yang gagal sampai *key* berhasil diunduh.

Buat *file* `account/internal/auth/verifier.go`:

```
// Package auth checks the OAuth2 access tokens (JWT) that callers send. The
// account service is a resource server: it never issues tokens and never sees
// client secrets, it only verifies what the authorization server (Hydra) signed.
package auth

import (
  "context"
  "errors"
  "fmt"
  "log/slog"
  "slices"
  "time"

  "github.com/MicahParks/keyfunc/v3"
  "github.com/golang-jwt/jwt/v5"
  "golang.org/x/time/rate"
)

// Claims are the token fields this service reads. Hydra puts the granted
// scopes into "scp" as a JSON array; other servers (e.g. Keycloak) use a
// space-separated "scope" string, so this struct is specific to Hydra.
type Claims struct {
  jwt.RegisteredClaims
  ClientID string   `json:"client_id"`
  Scopes   []string `json:"scp"`
}

func (c Claims) HasScope(scope string) bool { return slices.Contains(c.Scopes, scope) }

var (
  // ErrNoToken: the request carries no bearer token at all.
  ErrNoToken = errors.New("missing bearer token")
  // ErrInvalidToken: bad signature, wrong issuer, expired, or not a JWT.
  ErrInvalidToken = errors.New("invalid token")
)

type Verifier struct {
  keys   keyfunc.Keyfunc
  parser *jwt.Parser
}
```

```
// NewVerifier fetches the signing keys from jwksURL once at startup and every
// hour after. A failed fetch is logged, not fatal. A token signed with an
// unknown key ID triggers a fresh download, at most once per 5 s.
// ctx ends the background refresh.
func NewVerifier(ctx context.Context, issuer, jwksURL string, log *slog.Logger) (*Verifier,
error) {
  keys, err := keyfunc.NewDefaultOverrideCtx(ctx, []string{jwksURL}, keyfunc.Override{
    RefreshUnknownKID: rate.NewLimiter(rate.Every(5*time.Second), 1),
    // Bounds both the wait for the rate limiter and the download itself: a
    // request with an unknown key ID waits at most 5 s, then gets 401.
    RateLimitWaitMax: 5 * time.Second,
    HTTPTimeout:      5 * time.Second,
    RefreshErrorHandlerFunc: func(u string) func(context.Context, error) {
      return func(ctx context.Context, err error) {
        log.ErrorContext(ctx, "fetching signing keys failed", "jwks_url", u, "error", err)
      }
    },
  })
  if err != nil {
    return nil, fmt.Errorf("JWKS from %s: %w", jwksURL, err)
  }
  return &Verifier{
    keys: keys,
    parser: jwt.NewParser(
      // Only the algorithm the authorization server uses: blocks "alg: none"
      // and RS/HS confusion.
      jwt.WithValidMethods([]string{"RS256"}),
      // Exact match, trailing slash included.
      jwt.WithIssuer(issuer),
      // A token without exp would be valid forever.
      jwt.WithExpirationRequired(),
      // Tolerates small clock differences between the servers.
      jwt.WithLeeway(30*time.Second),
    ),
  }, nil
}

// Verify checks signature, issuer and expiry, and returns the claims. Scope is
// the caller's business: each endpoint needs a different one.
//
// The audience is not checked: client credentials tokens from Hydra carry an
// empty "aud" unless the client asks for one. Production setups register an
// audience per resource server and check it here.
func (v *Verifier) Verify(token string) (Claims, error) {
  var c Claims
  if _, err := v.parser.ParseWithClaims(token, &c, v.keys.Keyfunc); err != nil {
    return Claims{}, fmt.Errorf("%w: %w", ErrInvalidToken, err)
  }
  return c, nil
}
```

`Verify` membungkus error asli dengan `ErrInvalidToken`. Pemanggil dapat menulis error lengkap ke log (misalnya `token is expired`) dan hanya mengirim `invalid token` ke *client*.

7.4.5 *Interceptor* gRPC di `account`

Interceptor membaca *token* dari *metadata* gRPC. *Metadata* gRPC dikirim sebagai *header* HTTP/2, sehingga nama *key*-nya selalu huruf kecil: `authorization`, bukan `Authorization`.

Urutan pemeriksaan di *interceptor*:

1. Metode publik (gRPC *health* dan *reflection*) langsung diteruskan tanpa *token*.
2. Metode yang tidak terdaftar di *map* `scopes` ditolak dengan `PermissionDenied`. Dengan begitu, RPC *unary* baru yang ditambahkan ke `AccountService` tanpa *scope* otomatis tertutup (*fail closed*).
3. *Token* tidak ada atau tidak valid: `Unauthenticated`.
4. *Token* valid tetapi *scope* kurang: `PermissionDenied`.
5. Lolos semua pemeriksaan: *claim* disimpan di `context` agar *handler* dapat membaca `client_id` pemanggil.

Sifat *fail closed* ini hanya berlaku untuk panggilan yang melewati *interceptor* dan tidak termasuk metode publik. Ada dua celah yang perlu diingat. Pertama, *interceptor* ini hanya untuk RPC *unary*. RPC *streaming* tidak pernah melewatinya, dan service ini tidak memasang *stream interceptor*, sehingga metode *streaming* akan terbuka. Kedua, `account/main.go` menganggap semua metode di luar `AccountService` sebagai metode publik. Service gRPC lain yang kelak didaftarkan di server yang sama otomatis terbuka. Sebelum menambah service seperti itu, persempit fungsi penentu metode publik agar hanya meloloskan *health* dan *reflection*.

Detail penolakan (kedaluwarsa, *issuer* salah, *signature* tidak cocok) hanya ditulis ke log `grpc call rejected`. Pemanggil hanya menerima `invalid token`. Informasi seperti "*issuer* yang diharapkan adalah X" tidak berguna bagi pemanggil yang sah, tetapi membantu penyerang.

Buat *file* `account/internal/auth/grpc.go`:

```
package auth

import (
  "context"
  "log/slog"
  "strings"

  "google.golang.org/grpc"
  "google.golang.org/grpc/codes"
  "google.golang.org/grpc/metadata"
  "google.golang.org/grpc/status"
)

type claimsKey struct{}

// ClaimsFrom returns the verified claims of the current call, if any.
func ClaimsFrom(ctx context.Context) (Claims, bool) {
  c, ok := ctx.Value(claimsKey{}).(Claims)
  return c, ok
}

// UnaryServerInterceptor requires a valid bearer token in the "authorization"
// metadata and the scope that scopes lists for the method:
//   - no token or an invalid one: Unauthenticated ("who are you?")
//   - valid token without the scope: PermissionDenied ("I know you, but no")
//
// public names full methods that need no token, e.g. the health service, which
// probes call without credentials. Every other method must be in scopes; one
// that is missing is denied, so a new unary RPC added to a protected service
// without a scope is closed, not open.
//
// Fail closed covers only what reaches this interceptor and is not public:
//   - It is a unary interceptor. A streaming RPC never passes through it, and
//     no stream interceptor is installed, so a streaming method would be open.
//   - The account service passes "not AccountService" as public, so any other
//     service registered on the same server later is open by default. Narrow
```

```
//     public to the health and reflection services before adding one.
func UnaryServerInterceptor(v *Verifier, scopes map[string]string, public func(fullMethod
string) bool, log *slog.Logger) grpc.UnaryServerInterceptor {
  return func(ctx context.Context, req any, info *grpc.UnaryServerInfo, handler
grpc.UnaryHandler) (any, error) {
    if public(info.FullMethod) {
      return handler(ctx, req)
    }
    scope, ok := scopes[info.FullMethod]
    if !ok {
      log.Error("no scope configured for gRPC method, denied", "method", info.FullMethod)
      return nil, status.Error(codes.PermissionDenied, "method not allowed")
    }

    token, err := bearerFromMetadata(ctx)
    if err != nil {
      log.Warn("grpc call rejected", "method", info.FullMethod, "error", err)
      return nil, status.Error(codes.Unauthenticated, err.Error())
    }
    claims, err := v.Verify(token)
    if err != nil {
      // The detail (expired, wrong issuer, bad signature) goes to our log only;
      // the caller learns that the token was not accepted.
      log.Warn("grpc call rejected", "method", info.FullMethod, "error", err)
      return nil, status.Error(codes.Unauthenticated, ErrInvalidToken.Error())
    }
    if !claims.HasScope(scope) {
      log.Warn("grpc call rejected", "method", info.FullMethod, "client_id",
claims.ClientID,
        "required_scope", scope, "token_scopes", claims.Scopes)
      return nil, status.Errorf(codes.PermissionDenied, "scope %s required", scope)
    }
    return handler(context.WithValue(ctx, claimsKey{}, claims), req)
  }
}

// bearerFromMetadata reads "authorization: Bearer <token>". gRPC metadata keys
// are lower case: it travels as an HTTP/2 header.
func bearerFromMetadata(ctx context.Context) (string, error) {
  md, _ := metadata.FromIncomingContext(ctx)
  values := md.Get("authorization")
  if len(values) == 0 {
    return "", ErrNoToken
  }
  scheme, token, ok := strings.Cut(values[0], " ")
  if !ok || !strings.EqualFold(scheme, "Bearer") || token == "" {
    return "", ErrNoToken
  }
  return token, nil
}
```

Key `claimsKey` sengaja berupa tipe privat. Paket lain tidak dapat membuat *key* yang sama, sehingga tidak ada kode lain yang bisa menimpa *claim* di `context`.

7.4.6 *Scope* per RPC dan Log Pemanggil

Map `GRPCScopes` diletakkan di paket `account`, di samping `GRPCServer`, karena paket tersebut yang tahu arti setiap RPC. Tambahkan di `account/internal/account/grpc.go`, tepat sebelum deklarasi `type GRPCServer struct`:

```
// GRPCScopes is the OAuth2 scope each RPC requires (Lab 7). Reading and moving
// money are separate scopes, so a client that only displays balances cannot
// debit. The auth interceptor denies any AccountService method missing here.
var GRPCScopes = map[string]string{
  accountv1.AccountService_GetAccount_FullMethodName: "account:read",
  accountv1.AccountService_Debit_FullMethodName:      "account:write",
  accountv1.AccountService_Credit_FullMethodName:     "account:write",
}
```

Konstanta `..._FullMethodName` dihasilkan oleh `protoc-gen-go-grpc` di Lab 4. Menggunakan konstanta ini lebih aman dibandingkan menulis *string* `"/account.v1.AccountService/Debit"` secara manual, karena salah ketik langsung terdeteksi sebagai error kompilasi.

Tambahkan juga *import* paket `auth` di *file* yang sama:

```
  "github.com/artivisi/training-microservices-2026-01/studi-kasus/account/internal/auth"
```

Sekarang setiap mutasi saldo dapat dicatat beserta *client* yang memintanya. Di fungsi `mutate`, tepat setelah pemanggilan `s.store.Mutate` beserta blok `if err != nil` miliknya, ambil *claim* dan tambahkan `client_id` ke kedua baris log:

```
  // client_id from the verified token (Lab 7): who moved the money, for the audit trail.
  caller, _ := auth.ClaimsFrom(ctx)
  if !applied {
    s.log.Info("mutation already applied, not repeated", "number", a.Number, "type", t,
"reference", req.Reference,
      "client_id", caller.ClientID)
    return a, nil
  }
  s.log.Info("account mutated", "number", a.Number, "type", t, "amount", req.Amount,
"reference", req.Reference,
    "client_id", caller.ClientID)
  return a, nil
```

7.4.7 Memasang *Interceptor* di `account/main.go`

Tambahkan *import* `.../account/internal/auth`, lalu buat `Verifier` sebelum `grpc.NewServer`. Fungsi penentu metode publik yang sebelumnya ditulis *inline* untuk *fault injector* kini digunakan bersama oleh dua *interceptor*, jadi fungsi tersebut diberi nama. Ganti blok pembuatan `grpcSrv` menjadi:

```
  // Lab 7: every AccountService call needs a JWT from the authorization
  // server. The account REST API on HTTP_ADDR stays open in this lab to keep
  // the setup steps (creating accounts) short; in production it gets the same
  // kind of middleware as the transfer REST API.
  verifier, err := auth.NewVerifier(ctx, cfg.OAuthIssuer, cfg.OAuthJWKSURL, log)
  if err != nil {
    return err
  }
  // Health and reflection are not AccountService: no token, no faults, like
  // the HTTP probes. Note the predicate opens every non-AccountService method,
  // not only those two, and auth is a unary interceptor only: see
  // auth.UnaryServerInterceptor before registering another service or a
  // streaming RPC.
  notAccountService := func(method string) bool {
    return !strings.HasPrefix(method,
"/"+accountv1.AccountService_ServiceDesc.ServiceName+"/")
  }

  // gRPC: the internal API (GetAccount, Debit, Credit) for other services.
  // Interceptors run in the order given: logging sees the code that auth or the
  // fault injector produced, so rejections and injected failures show up in
  // the log like real ones. Auth comes before faults: a caller without a valid
```

```
  // token is turned away before any work, simulated or real, is done.
  grpcSrv := grpc.NewServer(grpc.ChainUnaryInterceptor(
    interceptor.Logging(log),
    auth.UnaryServerInterceptor(verifier, account.GRPCScopes, notAccountService, log),
    faults.UnaryServerInterceptor(notAccountService),
  ))
```

Urutan *interceptor* di `ChainUnaryInterceptor` menentukan urutan eksekusi: yang pertama membungkus yang berikutnya.

- ***Interceptor* `Logging` di posisi pertama** agar log `grpc request` mencatat kode hasil akhir, termasuk `Unauthenticated` dari *auth* dan `Unavailable` dari *fault injector*.
- ***Auth* sebelum *fault injector*** agar pemanggil tanpa *token* valid ditolak sebelum menerima *delay* simulasi. Di sistem sungguhan, prinsipnya sama: permintaan yang tidak sah tidak boleh menghabiskan sumber daya.

7.4.8 *Resource Server* di `transfer`

`transfer` membutuhkan `Verifier` yang sama. Salin *file*-nya:

```
mkdir -p transfer/internal/auth
cp account/internal/auth/verifier.go transfer/internal/auth/verifier.go
```

Ubah komentar paket di baris pertama agar menyebut `transfer`. Setiap service sengaja memiliki salinan sendiri dan tidak berbagi modul *library*. Modul bersama mengikat jadwal rilis semua service, karena perubahan kecil di *library* memaksa semua service ikut di-*build* ulang dan di-*deploy*. Tim yang memilih *library* bersama perlu memberinya versi seperti *dependency* eksternal lainnya.

7.4.8.1 *Middleware* REST

Middleware ini merupakan padanan REST untuk *interceptor* gRPC di `account`. Bedanya, penolakan dinyatakan melalui status HTTP dan *header*. RFC 6750 mengatur cara *resource server* memberi tahu alasan penolakan melalui *header* `WWW-Authenticate`:

- tanpa *token*: `Bearer`;
- *token* tidak valid: `Bearer error="invalid_token"`;
- *scope* kurang: `Bearer error="insufficient_scope", scope="..."`.

Buat *file* `transfer/internal/auth/http.go`:

```
package auth

import (
  "context"
  "encoding/json"
  "log/slog"
  "net/http"
  "strings"
)

type claimsKey struct{}

// ClaimsFrom returns the verified claims of the current request, if any.
func ClaimsFrom(ctx context.Context) (Claims, bool) {
  c, ok := ctx.Value(claimsKey{}).(Claims)
  return c, ok
}

// Middleware builds the per-route guard for the REST API.
type Middleware struct {
  v   *Verifier
```

```
  log *slog.Logger
}

func NewMiddleware(v *Verifier, log *slog.Logger) *Middleware {
  return &Middleware{v: v, log: log}
}

// Require lets a request through only with a valid bearer token that carries
// scope:
//   - no token or an invalid one: 401 with WWW-Authenticate (RFC 6750)
//   - valid token without the scope: 403 insufficient_scope
//
// The routes that are not wrapped (probes, /debug) stay open.
func (m *Middleware) Require(scope string, next http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    token, ok := bearer(r.Header.Get("Authorization"))
    if !ok {
      m.log.Warn("request rejected", "method", r.Method, "path", r.URL.Path, "error",
ErrNoToken)
      w.Header().Set("WWW-Authenticate", `Bearer`)
      writeError(w, http.StatusUnauthorized, ErrNoToken.Error())
      return
    }
    claims, err := m.v.Verify(token)
    if err != nil {
      // Detail (expired, wrong issuer, bad signature) only in our log.
      m.log.Warn("request rejected", "method", r.Method, "path", r.URL.Path, "error", err)
      w.Header().Set("WWW-Authenticate", `Bearer error="invalid_token"`)
      writeError(w, http.StatusUnauthorized, ErrInvalidToken.Error())
      return
    }
    if !claims.HasScope(scope) {
      m.log.Warn("request rejected", "method", r.Method, "path", r.URL.Path, "client_id",
claims.ClientID,
        "required_scope", scope, "token_scopes", claims.Scopes)
      w.Header().Set("WWW-Authenticate", `Bearer error="insufficient_scope",
scope="`+scope+`"`)
      writeError(w, http.StatusForbidden, "scope "+scope+" required")
      return
    }
    next.ServeHTTP(w, r.WithContext(context.WithValue(r.Context(), claimsKey{}, claims)))
  })
}

func bearer(header string) (string, bool) {
  scheme, token, ok := strings.Cut(header, " ")
  if !ok || !strings.EqualFold(scheme, "Bearer") || token == "" {
    return "", false
  }
  return token, true
}

func writeError(w http.ResponseWriter, status int, msg string) {
  w.Header().Set("Content-Type", "application/json")
  w.WriteHeader(status)
  _ = json.NewEncoder(w).Encode(map[string]string{"error": msg})
}
```

7.4.8.2 Memasang *Middleware* per *Route*

Scope ditentukan per *route*, jadi *middleware* dipasang per *route*, bukan membungkus seluruh `ServeMux`. Agar paket `transfer` tidak bergantung pada paket `auth`, `Routes` menerima fungsi bertipe `RequireScope` yang didefinisikan paket `transfer` sendiri. `main.go` mengisinya dengan *middleware* JWT, sedangkan *unit test* mengisinya dengan fungsi yang meloloskan semua permintaan.

Di `transfer/internal/transfer/http.go`, ganti fungsi `Routes` menjadi:

```
// OAuth2 scopes of the transfer REST API (Lab 7). The channel app that only
// shows transfer status gets read; moving money needs write.
const (
  ScopeRead  = "transfer:read"
  ScopeWrite = "transfer:write"
)

// RequireScope wraps next so it runs only for a caller whose token carries
// scope. The transfer package defines the shape; main plugs in the JWT
// middleware, tests plug in a pass-through.
type RequireScope func(scope string, next http.Handler) http.Handler

// Routes registers the API behind requireScope. Probes stay open: the kubelet
// and load balancers call them without a token.
func (h *Handler) Routes(requireScope RequireScope) *http.ServeMux {
  mux := http.NewServeMux()
  mux.Handle("POST /transfers", requireScope(ScopeWrite, http.HandlerFunc(h.create)))
  mux.Handle("GET /transfers/{id}", requireScope(ScopeRead, http.HandlerFunc(h.get)))
  mux.HandleFunc("GET /healthz", h.healthz)
  mux.HandleFunc("GET /readyz", h.readyz)
  return mux
}
```

Method `(*Middleware).Require` memiliki *signature* yang sama dengan `RequireScope`, sehingga `Require` dapat langsung diteruskan sebagai nilai fungsi.

7.4.9 `transfer` sebagai *Client* OAuth2

Bagian ini membahas sisi pemanggil. Setiap panggilan gRPC dari `transfer` ke `account` harus membawa *token* `transfer-service`.

7.4.9.1 Konfigurasi

Tambahkan lima *field* ke `Config` di `transfer/config.go`, setelah `KafkaTopic`:

```
  // Lab 7, as resource server (the REST API): OAuthIssuer must equal the "iss"
  // claim of the tokens character for character; OAuthJWKSURL serves the
  // authorization server's public keys.
  OAuthIssuer  string
  OAuthJWKSURL string
  // Lab 7, as OAuth2 client (calling account over gRPC): client credentials
  // grant against OAuthTokenURL. The secret belongs in a Kubernetes Secret or
  // a vault, never in the image or the repository (Lab 9).
  OAuthClientID     string
  OAuthClientSecret string
  OAuthTokenURL     string
```

Isi di `LoadConfig`, di dalam *literal* `cfg := Config{...}`:

```
    OAuthIssuer:       get("OAUTH_ISSUER"),
    OAuthJWKSURL:      get("OAUTH_JWKS_URL"),
    OAuthClientID:     get("OAUTH_CLIENT_ID"),
    OAuthClientSecret: get("OAUTH_CLIENT_SECRET"),
    OAuthTokenURL:     get("OAUTH_TOKEN_URL"),
```

Tambahkan nilainya di target `run` pada `transfer/Makefile`, sebelum `go run .`:

```
  OAUTH_ISSUER=http://localhost:14444/ \
  OAUTH_JWKS_URL=http://localhost:14444/.well-known/jwks.json \
  OAUTH_CLIENT_ID=transfer-service \
  OAUTH_CLIENT_SECRET=transfer-service-secret \
  OAUTH_TOKEN_URL=http://localhost:14444/oauth2/token \
```

7.4.9.2 `PerRPCCredentials`

gRPC menyediakan *interface* `credentials.PerRPCCredentials` untuk menempelkan *metadata* ke setiap panggilan. gRPC memanggil `GetRequestMetadata` sebelum setiap percobaan, termasuk percobaan ulang oleh *retry* failsafe-go, sehingga setiap percobaan membawa *token*.

grpc-go sudah menyediakan implementasi siap pakai di paket `credentials/oauth`. Implementasi tersebut tidak digunakan di sini karena hanya mengirim *token* lewat koneksi TLS, sedangkan koneksi gRPC di lab berjalan tanpa TLS. Pembatasan itu tepat untuk *production*: *bearer token* yang terkirim tanpa enkripsi dapat disalin dan digunakan ulang oleh siapa pun yang melihat lalu lintas jaringan. Implementasi di lab ini mengembalikan `false` dari `RequireTransportSecurity`, dan komentar di kode mencatat bahwa *production* wajib menggunakan TLS.

Tiga keputusan desain di dalam `tokenCredentials`:

- ***Token* disimpan dan digunakan ulang** hingga 10 detik sebelum kedaluwarsa (`oauth2.Token.Valid()`). Selama masa berlaku satu *token*, Hydra hanya menerima satu permintaan, berapa pun jumlah transfernya.
- **Gagal mengambil *token* menghasilkan `Unauthenticated`**, bukan `Unavailable`. Kebijakan *retry* dari Lab 4 hanya mengulang `Unavailable` (dan `DeadlineExceeded` sejak Lab 5), dan *circuit breaker* hanya menghitung error infrastruktur. Hydra yang mati tidak berarti `account` bermasalah, jadi *circuit breaker* ke `account` tidak boleh terbuka karenanya. Error ini dibuat sebagai tipe tersendiri, `tokenFetchError`, agar dapat dibedakan dari `Unauthenticated` yang dikirim oleh `account`. Panggilan gRPC yang gagal mendapat *token* tidak pernah keluar dari `transfer`. Bila ini terjadi saat `Debit`, transfer berakhir `FAILED` dengan kode HTTP 502.
- ***Token* yang ditolak `account` diganti sekali.** *Token* bisa ditolak sebelum kedaluwarsa, misalnya karena Hydra merotasi *signing key*. Hydra di lab mendapat *key* baru setiap kali di-*restart*. Tanpa mekanisme ini, setiap panggilan gagal hingga *token* lama kedaluwarsa, paling lama satu jam. Penggantian hanya dilakukan bila penolakan datang dari `account`. Kegagalan mengambil *token* (`tokenFetchError`) tidak diulang, karena percobaan kedua dalam panggilan yang sama hanya menghubungi Hydra sekali lagi. Mengulang `Debit` atau `Credit` aman karena keduanya sudah *idempotent* berdasarkan `reference` sejak Lab 5.

Buat *file* `transfer/internal/accountclient/credentials.go`:

```
package accountclient

import (
  "context"
  "errors"
  "sync"

  "golang.org/x/oauth2"
  "google.golang.org/grpc"
  "google.golang.org/grpc/codes"
  "google.golang.org/grpc/credentials"
  "google.golang.org/grpc/status"
)

// FetchToken gets a new access token from the authorization server, e.g.
```

```
// (*clientcredentials.Config).Token.
type FetchToken func(ctx context.Context) (*oauth2.Token, error)

// tokenCredentials attaches "authorization: Bearer <token>" to every call
// (Lab 7). It reuses a token until shortly before it expires, so the
// authorization server sees one request per token lifetime, not one per gRPC
// call.
//
// grpc-go ships credentials/oauth for this, but it refuses to send a token
// over a connection without TLS, and the lab runs plaintext. Production runs
// TLS (or mTLS through a service mesh): a bearer token in clear text can be
// replayed by anyone who sees the traffic.
type tokenCredentials struct {
  fetch FetchToken
  mu    sync.Mutex
  tok   *oauth2.Token
}

var _ credentials.PerRPCCredentials = (*tokenCredentials)(nil)

func (c *tokenCredentials) GetRequestMetadata(ctx context.Context, _ ...string)
(map[string]string, error) {
  c.mu.Lock()
  defer c.mu.Unlock()
  // Valid() is false for a token that expires within the next 10 s, so a
  // token never runs out while the call is on its way.
  if !c.tok.Valid() {
    tok, err := c.fetch(ctx)
    if err != nil {
      return nil, &tokenFetchError{err: err}
    }
    c.tok = tok
  }
  return map[string]string{"authorization": c.tok.Type() + " " + c.tok.AccessToken}, nil
}

// tokenFetchError: no token, so the call never left the transfer service.
//
// Its gRPC code is Unauthenticated, not Unavailable: the account service is
// fine, our side has no token. The resiliency policies neither retry this nor
// count it against the account breaker. grpc-go passes an error with a
// GRPCStatus method through unchanged, so refreshOnUnauthenticated can tell it
// apart from an Unauthenticated sent by the account service.
type tokenFetchError struct{ err error }

func (e *tokenFetchError) Error() string { return "get access token: " + e.err.Error() }
func (e *tokenFetchError) Unwrap() error { return e.err }
func (e *tokenFetchError) GRPCStatus() *status.Status {
  return status.New(codes.Unauthenticated, e.Error())
}

// RequireTransportSecurity is false only because the lab has no TLS; see above.
func (*tokenCredentials) RequireTransportSecurity() bool { return false }

// refreshOnUnauthenticated drops the cached token when the account service
// rejects it, and repeats the call once with a new one. A token can stop being
// accepted before it expires: the authorization server rotated its signing keys
// (the lab's Hydra gets new keys on every restart) or the token was revoked.
// Without this, every call fails until the old token expires, up to an hour.
```

```
// Repeating Debit/Credit is safe: they are idempotent by reference (Lab 5).
//
// Only a rejection by the account service counts. When the token fetch itself
// failed, a second try would only hit the authorization server again within
// the same call.
func (c *tokenCredentials) refreshOnUnauthenticated(ctx context.Context, method string,
req, reply any,
  cc *grpc.ClientConn, invoker grpc.UnaryInvoker, opts ...grpc.CallOption) error {
  err := invoker(ctx, method, req, reply, cc, opts...)
  var fetchErr *tokenFetchError
  if status.Code(err) != codes.Unauthenticated || errors.As(err, &fetchErr) {
    return err
  }
  c.mu.Lock()
  c.tok = nil
  c.mu.Unlock()
  return invoker(ctx, method, req, reply, cc, opts...)
}
```

`c.tok.Valid()` aman dipanggil walaupun `c.tok` masih `nil`: *method* `Valid` pada `*oauth2.Token` memeriksa `nil` terlebih dahulu dan mengembalikan `false`.

grpc-go meneruskan error yang memiliki *method* `GRPCStatus` apa adanya ke pemanggil. Karena itu, `refreshOnUnauthenticated` dapat memeriksa dua hal sekaligus: `status.Code(err)` tetap `Unauthenticated` bagi kebijakan *retry* dan *circuit breaker*, sedangkan `errors.As` mengenali bahwa error tersebut berasal dari `tokenFetchError`, bukan dari `account`.

7.4.9.3 Memasang *Credentials* di `accountclient.New`

Di `transfer/internal/accountclient/client.go`, tambahkan parameter `fetch` ke `New`:

```
func New(target string, s Settings, fetch FetchToken, log *slog.Logger,
opts ...grpc.DialOption) (*Client, error) {
```

Sejak Lab 5, `New` membuka dua koneksi melalui fungsi `dial`: koneksi utama dengan kebijakan `resilience` dan koneksi *reversal* dengan kebijakan `reversalResilience`. Kedua koneksi memanggil service yang sama dengan identitas *client* yang sama, sehingga cukup satu *cache token* untuk keduanya. Di `New`, ganti dua pemanggilan `dial` menjadi:

```
  // One token cache for both connections: they talk to the same service with
  // the same client identity.
  tokens := &tokenCredentials{fetch: fetch}

  // An interceptor belongs to a connection, so the reversal gets a connection of
  // its own. The cost is one more TCP connection to the account service.
  conn, err := dial(target, tokens, resilience, opts)
  if err != nil {
    return nil, err
  }
  reversalConn, err := dial(target, tokens, reversalResilience, opts)
```

Lalu ubah *signature* `dial`, dan ganti baris `grpc.WithUnaryInterceptor(interceptor),` di dalamnya dengan dua opsi baru. Opsi lain (koneksi tanpa TLS dan pengaturan *backoff* koneksi) tidak berubah:

```
func dial(target string, tokens *tokenCredentials, resilience grpc.UnaryClientInterceptor,
extra []grpc.DialOption) (*grpc.ClientConn, error) {
  opts := append([]grpc.DialOption{
    // Plaintext: fine on a laptop, not in production. Service-to-service calls
    // there go over TLS (or mTLS through a service mesh).
    grpc.WithTransportCredentials(insecure.NewCredentials()),
    // The token is added per attempt, after the interceptors, so a retried
```

```
    // attempt carries a valid token too.
    grpc.WithPerRPCCredentials(tokens),
    // First interceptor = outermost: a call repeated with a new token goes
    // through the resiliency policies again.
    grpc.WithChainUnaryInterceptor(tokens.refreshOnUnauthenticated, resilience),
```

Urutannya penting. `refreshOnUnauthenticated` berada di luar kebijakan *resiliency*, sehingga panggilan yang diulang dengan *token* baru kembali melewati *timeout*, *retry*, dan *circuit breaker* (atau *retry* khusus *reversal* di koneksi kedua).

> **Batasan yang belum ditangani.** Kebijakan *retry reversal* dari Lab 5 hanya mengulang error infrastruktur (`Unavailable`, `DeadlineExceeded`, `Internal`, `Unknown`). Kegagalan mengambil *token* menghasilkan kode `Unauthenticated`, jadi tidak diulang. Bila Hydra mati tepat saat *reversal* berjalan dan *token* di *cache* sudah kedaluwarsa, transfer langsung berakhir `COMPENSATION_FAILED` (HTTP 500) dan perlu koreksi manual.

7.4.9.4 Memasang *Verifier* dan *Client* OAuth2 di `transfer/main.go`

Tambahkan dua *import*:

```
  "golang.org/x/oauth2/clientcredentials"

  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/auth"
```

Ganti pembuatan `accounts` dan `mux` menjadi:

```
  // Lab 7: the transfer service is an OAuth2 client of its own. It asks only
  // for account:write (Debit, Credit), not everything its client registration
  // allows: least privilege per token. accountclient caches the token and
  // fetches a new one shortly before expiry.
  tokenCfg := clientcredentials.Config{
    ClientID:     cfg.OAuthClientID,
    ClientSecret: cfg.OAuthClientSecret,
    TokenURL:     cfg.OAuthTokenURL,
    Scopes:       []string{"account:write"},
  }
  // The fetch runs inside a gRPC attempt and is bounded by its 2 s timeout.
  accounts, err := accountclient.New(cfg.AccountGRPCAddr, accountclient.ProductionSettings,
tokenCfg.Token, log)
  if err != nil {
    return err
  }
  defer accounts.Close()

  verifier, err := auth.NewVerifier(ctx, cfg.OAuthIssuer, cfg.OAuthJWKSURL, log)
  if err != nil {
    return err
  }
  svc := transfer.NewService(store, accounts, log)
  mux := transfer.NewHandler(svc, log).Routes(auth.NewMiddleware(verifier, log).Require)
  // Open on purpose, like the probes: an operator reads it during an incident.
  // Production keeps /debug on an internal port or behind the gateway.
  mux.HandleFunc("GET /debug/circuit-breaker", accounts.ServeCircuitBreaker)
```

Client `transfer-service` terdaftar dengan *scope* `account:read` dan `account:write`, tetapi `transfer` hanya meminta `account:write` karena hanya memanggil `Debit` dan `Credit`. *Token* yang bocor hanya membawa hak yang memang dibutuhkan (*least privilege*).

`tokenCfg.Token` adalah *method value* bertipe `func(context.Context) (*oauth2.Token, error)`, sama dengan tipe `FetchToken`. *Token* baru diambil saat panggilan gRPC pertama, bukan saat *startup*. `transfer` tetap bisa menyala dan lolos *readiness probe* walaupun Hydra sedang mati.

Agar log *startup* menunjukkan *issuer* yang digunakan, tambahkan `"oauth_issuer", cfg.OAuthIssuer` ke baris log `transfer service listening`. Informasi ini membantu saat mencari penyebab semua permintaan ditolak (lihat Masalah Umum).

7.4.10 `failure_reason` yang Aman untuk Pemanggil

Hingga Lab 6, `failure_reason` berisi teks error apa adanya, misalnya:

```
debit failed: rpc error: code = Unavailable desc = connection error: desc = "transport:
Error while dialing: dial tcp 127.0.0.1:29091: connect: connection refused"
```

Teks ini sekarang sampai ke aplikasi *channel* yang memegang *token* `transfer:read`, dan juga ikut terkirim di *event* Kafka. Alamat IP, port, dan pesan error dari *library* internal tidak berguna bagi pemanggil, tetapi berguna bagi penyerang yang memetakan jaringan. API cukup menyebut hasil bisnis, sedangkan detail teknisnya dicatat di log.

Tambahkan fungsi berikut di `transfer/internal/transfer/service.go`, misalnya tepat sebelum `func (s *Service) Get`:

```
// publicReason is the failure text stored in failure_reason, returned by the
// API and carried in the events (Lab 7). Business outcomes are named; any other
// error becomes one generic phrase, because its text can hold internal
// addresses, ports and library messages (e.g. "dial tcp 10.0.3.7:9091:
// connection refused") that the caller cannot act on and an attacker can
// use. The full error is in the log next to the transfer ID.
func publicReason(err error) string {
  switch {
  case errors.Is(err, ErrInsufficientFunds):
    return ErrInsufficientFunds.Error()
  case errors.Is(err, ErrAccountNotFound):
    return ErrAccountNotFound.Error()
  case errors.Is(err, ErrAccountUnavailable):
    return ErrAccountUnavailable.Error()
  default:
    return "account service error"
  }
}
```

Lalu ganti tiga tempat yang menyusun alasan dengan `fmt.Sprintf`:

```
// di run(), cabang debit gagal
return s.finish(ctx, t, Pending, Failed, "debit failed: "+publicReason(err), EventFailed,
err)

// di compensate(), baris pertama
reason := "credit failed: " + publicReason(cause)

// di compensate(), cabang reversal gagal
reason = reason + "; reversal failed: " + publicReason(err)
```

Cabang untuk *reversal* yang gagal tetap mengembalikan error yang membungkus `ErrReversalFailed` dari Lab 5, sehingga transfer `COMPENSATION_FAILED` tetap dijawab 500. Yang berubah hanya teks `reason`.

`publicReason` menaruh pesan *sentinel* (misalnya `insufficient funds`) di akhir teks. Fungsi `replayOutcome` dari Lab 6 bergantung pada sifat ini: kode HTTP untuk *replay* ditentukan dari akhiran `failure_reason`. Teks lama sebelum Lab 7 (`debit failed: debit 1111111111: insufficient funds`) juga berakhir dengan pesan yang sama, jadi *replay* untuk transfer lama tetap menghasilkan kode yang sama. Perbarui komentar di atas `replayOutcome` agar menyebut kedua format tersebut:

```
// replayOutcome rebuilds, from the stored transfer, the error the original
// request ended with: nil for COMPLETED, ErrReversalFailed for
// COMPENSATION_FAILED, ErrInProgress for PENDING and DEBITED. For FAILED and
// COMPENSATED the cause is only kept as text in failure_reason ("debit failed:
// insufficient funds"); publicReason puts the sentinel's message last. Rows
// written before Lab 7 hold the raw error ("debit failed: debit 1111111111:
// insufficient funds"), which ends the same way. Anything else was a dependency
// failure, the 502 of the original request.
```

Agar detail error tidak hilang, tambahkan `"error", cause` ke log `transfer not completed` di fungsi `finish`:

```
    s.log.Warn("transfer not completed", "transfer_id", t.ID, "status", to, "reason",
reason, "error", cause)
```

7.4.11 Memperbarui *Unit Test*

Dua *signature* berubah: `accountclient.New` mendapat parameter `fetch`, dan `Handler.Routes` mendapat parameter `requireScope`. *Test* lama tidak lagi bisa dikompilasi. Ambil versi yang sudah diperbarui dari *checkpoint*, sekaligus *test* baru untuk kode Lab 7:

```
git checkout lab-07 -- \
  account/internal/auth/grpc_test.go \
  transfer/internal/auth/http_test.go \
  transfer/internal/accountclient/client_test.go \
  transfer/internal/transfer/http_test.go \
  transfer/internal/transfer/service_test.go
(cd account && go mod tidy)
(cd transfer && go mod tidy)
make test
```

Baca `account/internal/auth/grpc_test.go`. *File* ini menunjukkan cara menguji validasi JWT tanpa Hydra:

1. `newTestVerifier` membuat *key* RSA baru, lalu menyajikan *public key*-nya sebagai JWKS dari `httptest.Server`, dengan format yang sama seperti `/.well-known/jwks.json` milik Hydra.
2. `sign` membuat *token* serupa *token* Hydra, dan menerima fungsi untuk mengubah *claim* tertentu.
3. *Table-driven test* mencakup *token* valid, *scheme* huruf kecil, tanpa *token*, *token* Basic, *token* sampah, kedaluwarsa, tanpa `exp`, *issuer* lain, ditandatangani *key* lain, *scope* kurang, metode tanpa *scope*, dan metode publik.
4. `TestSigningAlgorithmIsPinned` membuktikan *token* HS256 tidak lolos sebagai RS256.

Test di `accountclient` mencakup perilaku baru berikut. Fungsi bantu `serve` kini menerima parameter `fetch`, sehingga setiap *test* dapat memasang sumber *token* sendiri.

- `TestSendsBearerToken`: *token* terkirim di *metadata* `authorization`.
- `TestTokenFailureIsNotAnAccountFailure`: gagal mengambil *token* tidak di-*retry*, tidak membuka *circuit breaker*, dan tidak memicu pengambilan *token* kedua. Sepuluh panggilan menghasilkan tepat sepuluh kali pengambilan *token*.
- `TestRejectedTokenIsReplaced`: *token* yang ditolak `account` diganti, lalu panggilan diulang dan berhasil.
- `TestRejectedTokenIsReplacedOnlyOnce`: bila *token* pengganti juga ditolak, panggilan gagal dengan `Unauthenticated` setelah dua kali sampai di server dan dua kali pengambilan *token*, tanpa berulang terus.

`TestFailureReasonIsGeneric` di `service_test.go` memastikan alamat internal tidak muncul di `failure_reason`.

7.5 Verifikasi

7.5.1 Menjalankan Service

Jalankan ketiga service di terminal terpisah:

```
make -C account run
make -C transfer run
make -C notification run
```

Pastikan `account` dan `transfer` tidak berhenti dengan pesan `missing required environment variables`. Log *startup* `transfer` memuat `oauth_issuer=http://localhost:14444/`.

7.5.2 Empat Skenario REST

Siapkan dua *token*: satu dengan kedua *scope*, satu lagi *read-only* (hanya `transfer:read`):

```
TOKEN=$(make -s token)
RO=$(make -s token SCOPE=transfer:read)
```

Tanpa *token*: 401.

```
curl -i -X POST localhost:28082/transfers \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

```
HTTP/1.1 401 Unauthorized
Content-Type: application/json
Www-Authenticate: Bearer
...
{"error":"missing bearer token"}
```

Go menulis nama *header* dengan huruf besar di awal setiap kata (`Www-Authenticate`). Nama *header* HTTP tidak membedakan huruf besar dan kecil.

***Token read-only* untuk operasi tulis: 403.**

```
curl -i -X POST localhost:28082/transfers -H "Authorization: Bearer $RO" \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

```
HTTP/1.1 403 Forbidden
Content-Type: application/json
Www-Authenticate: Bearer error="insufficient_scope", scope="transfer:write"
...
{"error":"scope transfer:write required"}
```

***Token* dengan `transfer:write`: 201.**

```
curl -i -X POST localhost:28082/transfers -H "Authorization: Bearer $TOKEN" \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

Respons berstatus `201 Created` dengan `"status":"COMPLETED"` dan *header* `Location: /transfers/...`.

***Token read-only* untuk operasi baca: 200.** Ganti `ID` dengan nilai `id` dari respons sebelumnya:

```
curl -i -H "Authorization: Bearer $RO" localhost:28082/transfers/ID
```

Respons berstatus `200 OK` dan berisi transfer yang sama.

Endpoint yang terbuka tetap dapat diakses tanpa *token*:

```
curl localhost:28082/readyz
curl localhost:28082/debug/circuit-breaker
```

7.5.3 Log di Kedua Service

Di terminal `transfer`, permintaan yang ditolak menghasilkan log `request rejected`:

```
time=... level=WARN msg="request rejected" method=POST path=/transfers error="missing
bearer token"
time=... level=WARN msg="request rejected" method=POST path=/transfers client_id=channel-
app required_scope=transfer:write token_scopes=[transfer:read]
```

Di terminal `account`, transfer yang berhasil mencatat *client* pemanggil:

```
time=... level=INFO msg="account mutated" number=1111111111 type=DEBIT amount=100000
reference=... client_id=transfer-service
time=... level=INFO msg="account mutated" number=2222222222 type=CREDIT amount=100000
reference=... client_id=transfer-service
```

`client_id=transfer-service` membuktikan bahwa `transfer` mengirim *token* dan `account` memverifikasinya.

7.5.4 `failure_reason` Generik

Matikan `account` (Ctrl+C di terminalnya), lalu buat transfer:

```
curl -i -X POST localhost:28082/transfers -H "Authorization: Bearer $TOKEN" \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

Respons berstatus `502 Bad Gateway` dan berisi `"status":"FAILED"` serta `"failure_reason":"debit failed: account service error"`, tanpa alamat `127.0.0.1:29091`. Alamat tersebut tetap ada di log `transfer not completed` pada *field* `error`. Bila percobaan diulang hingga *circuit breaker* terbuka, respons berubah menjadi `503 Service Unavailable` dengan `failure_reason debit failed: account service unavailable`. Artinya, tidak ada uang yang berpindah dan permintaan aman diulang. Jalankan lagi `make -C account run` setelah selesai.

Format baru hanya berlaku untuk transfer yang dibuat setelah kode Lab 7 berjalan. Transfer yang tersimpan sebelumnya tetap menampilkan `failure_reason` lama yang memuat pesan asli, misalnya `debit failed: debit 1111111111: rpc error: ...`. Untuk melihat format baru, selalu periksa transfer gagal yang baru dibuat. *Replay* dengan `Idempotency-Key` milik transfer lama tetap menghasilkan kode HTTP yang sama seperti permintaan aslinya.

7.6 Masalah Umum

Semua permintaan dengan *token* dijawab 401. Periksa log `request rejected` di `transfer`. Bila error-nya menyebut *issuer* (`token has invalid issuer`), nilai `OAUTH_ISSUER` tidak sama persis dengan *claim* `iss`. Hydra menerbitkan `http://localhost:14444/` dengan garis miring di akhir, sedangkan `http://localhost:14444` tanpa garis miring dianggap *issuer* lain. Log *startup* `transfer` menampilkan nilai `oauth_issuer` yang sedang digunakan.

`make -s token` gagal dengan `invalid_client`. Hydra menyimpan *client* di memori. Setelah *container* Hydra di-*restart* (termasuk setelah Docker atau laptop di-*restart*), *client* hilang. Jalankan `make infra-up` lagi. Target ini menjalankan `hydra-init` yang menghapus lalu membuat ulang kedua *client*.

***Token* di variabel `TOKEN` tiba-tiba ditolak dengan `invalid token`.** Ada dua kemungkinan: *token* sudah lebih dari satu jam (kedaluwarsa), atau Hydra sudah di-*restart* sehingga *signing key*-nya berganti dan `transfer` sudah mengunduh JWKS yang baru. Ambil *token* baru dengan `TOKEN=$(make -s token)`. Log `request rejected` menyebut penyebab persisnya.

Permintaan pertama setelah Hydra di-*restart* lambat, sekitar setengah detik. Hydra yang baru menyala membuat *signing key* baru saat permintaan *token* pertama. Service yang sudah berjalan

mengunduh ulang JWKS saat menerima *token* dengan `kid` yang belum dikenal. Unduh ulang dibatasi sekali per 5 detik, dan permintaan yang menunggu batas tersebut paling lama menunggu 5 detik sebelum dijawab 401.

Log `fetching signing keys failed` saat *startup*. Hydra belum berjalan ketika service menyala. Service tetap berjalan, tetapi semua pemeriksaan *token* gagal hingga Hydra tersedia. Jalankan `make infra-up`, lalu ulangi permintaan.

Transfer dijawab 502, `FAILED` dengan alasan `debit failed: account service error`, padahal `account` berjalan. Lihat log `transfer not completed` di `transfer`. Bila isinya `get access token`, `transfer` gagal mengambil *token* dari Hydra. Periksa `OAUTH_CLIENT_ID`, `OAUTH_CLIENT_SECRET`, `OAUTH_TOKEN_URL`, dan pastikan `hydra-init` sudah berjalan. Panggilan ini tidak pernah sampai ke `account`, tidak diulang, dan tidak membuka *circuit breaker* ke `account`, sehingga `make breaker` tetap menunjukkan `closed`.

Kompilasi gagal di *test* dengan pesan `cannot use log (variable of type *slog.Logger) as FetchToken value in argument to New` atau `not enough arguments in call to ....Routes`. *Test* lama belum diganti. Jalankan perintah `git checkout lab-07 -- ...` di langkah Memperbarui *Unit Test*.

Service lama masih menggunakan port 28081 atau 28082. Hentikan berdasarkan port, misalnya `kill $(lsof -t -i :28081)`. *Binary* hasil `go run` di macOS berada di folder *cache* Go, sehingga pencarian berdasarkan nama proses sering tidak menemukannya.

7.7 Eksplorasi Mandiri

1. **Rotasi *signing key*.** Pastikan sudah ada satu transfer berhasil sejak `transfer` dijalankan, agar `transfer` menyimpan *token* `transfer-service` di *cache*. Jalankan `docker compose restart hydra` lalu `docker compose run --rm hydra-init`. Setelah itu, Hydra memiliki *signing key* baru. Me-*restart* Hydra saja belum memicu penolakan: `account` masih menyimpan *key* lama di JWKS yang sudah diunduh (diperbarui setiap jam), sehingga *token* lama di `transfer` tetap diterima. Karena itu, jalankan ulang `account` (Ctrl+C, lalu `make -C account run`) agar `account` hanya mengenal *key* baru. Buat transfer dengan *token* `channel-app` baru dari `make -s token`. Amati log `account`: muncul `grpc call rejected` dengan error `key not found` dan `grpc request` dengan `code=Unauthenticated`, lalu `Debit` yang diulang berhasil. Jelaskan komponen mana yang mengganti *token* dan komponen mana yang mengunduh ulang JWKS. Mengapa mengulang `Debit` setelah `Unauthenticated` aman?
2. ***Issuer* yang salah.** Hapus garis miring di akhir `OAUTH_ISSUER` pada `transfer/Makefile`, jalankan ulang `transfer`, lalu kirim permintaan dengan *token* valid. Temukan baris log yang menjelaskan penolakan. Kembalikan dengan `git checkout -- transfer/Makefile` atau perbaiki secara manual.
3. ***Least privilege*.** Ubah `Scopes` di `tokenCfg` menjadi `account:read` saja, jalankan ulang `transfer`, lalu buat transfer. Kode gRPC apa yang diterima `transfer`, apakah dihitung *circuit breaker*, dan apa isi `failure_reason`? Kembalikan ke `account:write` setelah selesai.
4. **Hydra mati.** Hentikan Hydra dengan `docker compose stop hydra`, jalankan ulang `transfer`, lalu periksa `/readyz` dan buat transfer dengan *token* lama yang belum kedaluwarsa. Bandingkan perilaku `transfer` sebagai *resource server* (JWKS sudah tersimpan atau belum) dengan perilakunya sebagai *client* (perlu *token* baru). Nyalakan kembali dengan `make infra-up`.

7.8 Checkpoint

Kondisi kode di akhir lab ini tersimpan di tag `lab-07`. Bagian OAuth2 dan JWT di `studi-kasus/README.md` tersedia di tag tersebut: `git checkout lab-07 -- README.md`. Bandingkan hasil kerja dengan *checkpoint*:

```
git diff lab-07 -- .
```

Peserta yang ingin langsung melanjutkan ke Lab 8 dari kode referensi membuat *branch* kerja baru dari tag tersebut:

```
git switch -c lab-08-kerja lab-07
```

8 Lab 8: *Observability* Tiga Service

8.1 Tujuan

Hingga Lab 7, gangguan pada sistem transfer hanya terlihat dari log teks di tiga terminal yang terpisah. Untuk mengetahui mengapa sebuah transfer gagal, peserta harus mencocokkan waktu di log `transfer`, `account`, dan `notification` secara manual. Lab ini menambahkan tiga sinyal *observability* ke setiap service sehingga satu transfer dapat diikuti dari *request* pertama hingga notifikasi terkirim.

Setelah menyelesaikan lab ini, peserta mampu:

- menulis log JSON dengan `slog` yang selalu membawa `correlation_id`, `trace_id`, dan `span_id`, lalu mengirimkannya ke Loki;
- membuat *correlation ID* di *middleware* HTTP dan meneruskannya lewat *metadata* gRPC serta *header* Kafka;
- menyediakan *metric* Prometheus di `GET /metrics`: durasi *request* per *route*, jumlah transfer per status, status *circuit breaker*, jumlah baris *outbox* yang belum terkirim, dan notifikasi per hasil;
- mengirim *trace* OpenTelemetry dari ketiga service, termasuk melewati Kafka, sehingga satu transfer menjadi satu *trace*;
- menemukan service yang lambat dari *dashboard* Grafana, *trace* di Tempo, dan log di Loki.

8.2 Gambaran

Ketiga sinyal tersebut saling melengkapi. *Metric* menunjukkan adanya masalah beserta besar dampaknya. *Trace* menunjukkan service yang menghabiskan waktu paling lama, dan log menjelaskan kejadian di dalam service tersebut.

Sinyal	Cara Keluar dari Service	Penyimpanan di `otel-lgtm`
Log	JSON ke *stdout* sekaligus OTLP gRPC ke `localhost:14317`	Loki
Trace	OTLP gRPC ke `localhost:14317`, dikirim per *batch* di latar belakang	Tempo
Metric	`GET /metrics` di port HTTP masing-masing service, diambil Prometheus setiap 5 detik	Prometheus

Seluruh *backend* tersebut berjalan di satu *container* `grafana/otel-lgtm:0.37.0`. *Container* ini berisi OpenTelemetry Collector, Prometheus, Tempo, Loki, dan Grafana, dan khusus ditujukan untuk pengembangan serta lab. Di *production*, setiap komponen berjalan terpisah dengan *storage* masing-masing.

```
account      :28081 -+
transfer     :28082 -+- OTLP :14317 (trace, log) -> otel-lgtm
notification :28083 -+                              |- Collector -> Tempo (trace)
     ^                                              |            -> Loki (log)
     |                                              |- Prometheus (metric)
     +--- GET /metrics setiap 5 detik ---------------+
          (host.docker.internal:2808x)               |- Grafana :13000
```

Arah *metric* berlawanan dengan dua sinyal lainnya. Log dan *trace* dikirim oleh service ke *collector* (model *push*), sedangkan *metric* diambil oleh Prometheus dari service (model *pull* atau *scrape*). Karena

service berjalan di laptop dan Prometheus di dalam *container*, Prometheus menghubungi laptop lewat nama `host.docker.internal`.

Tantangan utama lab ini terletak pada upaya menjaga satu *trace* tetap utuh melintasi tiga service. Untuk HTTP dan gRPC, OpenTelemetry sudah menyediakan instrumentasi (`otelhttp`, `otelgrpc`) yang menulis dan membaca *header* `traceparent` (standar W3C *Trace Context*). Jalur Kafka lebih rumit: *relay outbox* mengirim *event* dari *goroutine* terpisah, setelah *request* asal selesai. Pada saat itu *context* milik *request* sudah tidak ada. Karena itu *correlation ID* dan *trace context* disimpan di baris tabel `outbox` dalam transaksi yang sama dengan *event*, lalu *relay* menyalinnya ke *header* Kafka. Hasilnya satu *trace* seperti berikut. Setiap baris mewakili satu *span* (satu unit kerja beserta waktu mulai dan selesainya), dan indentasi menandai *span* yang menjadi *child* dari baris di atasnya:

```
transfer      POST /transfers
transfer        HTTP POST                          (token Hydra, bila belum ada)
transfer        account.v1.AccountService/Debit    (client, 1 span/percobaan)
account           account.v1.AccountService/Debit  (server)
transfer        account.v1.AccountService/Credit
account           account.v1.AccountService/Credit
transfer        publish transfer-events            (relay outbox)
notification      process transfer-events
```

Correlation ID dan *trace ID* memiliki peran berbeda. *Correlation ID* merupakan ID di sisi bisnis yang diketahui pemanggil, sehingga pemanggil, misalnya aplikasi *channel*, dapat melaporkan "transfer dengan *correlation ID* X gagal". *Trace ID* adalah ID teknis buatan OpenTelemetry. Keduanya ditulis di setiap baris log.

Jalur	*Correlation ID*	*Trace Context* (W3C)
REST	*Header* `X-Correlation-ID`, berisi UUID baru bila tidak dikirim atau formatnya tidak valid, lalu dikembalikan di respons	*Header* `traceparent`, ditangani `otelhttp`
gRPC `transfer` ke `account`	*Metadata* `x-correlation-id`	*Metadata* `traceparent`, ditangani `otelgrpc`
Kafka `transfer` ke `notification`	*Header* `correlation_id`	*Header* `traceparent`, ditulis dan dibaca sendiri

8.3 Titik Awal

Lab ini melanjutkan kode dari akhir Lab 7. Peserta yang belum menyelesaikan Lab 7 dapat memulai dari *checkpoint*:

```
cd training-microservices-2026-01
git switch -c lab-08-kerja lab-07
cd studi-kasus
make infra-up
```

Semua perintah di bab ini dijalankan dari folder `studi-kasus`, kecuali disebutkan lain.

Pastikan service lama sudah berhenti. Hentikan `account`, `transfer`, dan `notification` dengan Ctrl+C di terminal masing-masing. Ketiganya dijalankan ulang dengan variabel `OTEL_*` yang baru di langkah berikutnya.

Database dari Lab 7 tidak perlu di-*reset*. Kolom baru di tabel `outbox` dibuat otomatis oleh *schema* `transfer` saat service dijalankan.

Rekening `1111111111` dan `2222222222` dari lab sebelumnya digunakan di seluruh contoh. Bila nomor rekening di database berbeda, sesuaikan nomor di perintah `curl`.

8.4 Langkah

8.4.1 Menambahkan `otel-lgtm` ke Docker Compose

Konfigurasi Prometheus dan *dashboard* Grafana sudah disediakan, sama seperti konfigurasi Hydra di Lab 7. Fokus lab ini ada di kode service, sehingga *file* konfigurasi tersebut cukup disalin dari *checkpoint*:

```
git checkout lab-08 -- observability Makefile
ls observability observability/dashboards
```

`Makefile` di folder `studi-kasus` mendapat *target* `metrics`, yang membaca `/metrics` dari ketiga service lalu menyaring *metric* domain. Folder `observability` berisi tiga *file*:

File	Isi
`prometheus.yaml`	Tiga *scrape job* (`account`, `transfer`, `notification`) ke `host.docker.internal:28081`–`28083` setiap 5 detik, ditambah konfigurasi bawaan *image* yang tetap dibutuhkan *receiver* OTLP
`grafana-dashboards.yaml`	*Provider* yang memuat semua JSON di folder `dashboards` ke folder Grafana *Studi Kasus*
`dashboards/studi-kasus.json`	*Dashboard Studi Kasus: Transfer* dengan sembilan panel

`otel-lgtm` versi 0.37.0 tidak menyediakan opsi untuk menambah target *scrape*. Prometheus di dalam *image* selalu membaca `/otel-lgtm/prometheus.yaml`, sehingga *file* tersebut ditimpa lewat *volume*. Tambahkan service berikut di `studi-kasus/compose.yaml`, sebelum bagian `volumes:` di akhir *file*:

```
  # Observability backend in one container (Lab 8), for development and labs
  # only. Host ports: 13000 Grafana (no login), 14317 OTLP gRPC, 14318 OTLP HTTP.
  otel-lgtm:
    image: grafana/otel-lgtm:0.37.0
    ports:
      - "13000:3000"
      - "14317:4317"
      - "14318:4318"
    # host.docker.internal = the laptop. Docker Desktop has it built in; on a
    # Linux Docker Engine this line creates it.
    extra_hosts:
      - "host.docker.internal:host-gateway"
    volumes:
      - ./observability/prometheus.yaml:/otel-lgtm/prometheus.yaml:ro
      # Long form only to fit the page; same as the short form in the repo.
      - type: bind
        source: ./observability/grafana-dashboards.yaml
        target: /otel-lgtm/grafana/conf/provisioning/dashboards/studi-kasus.yaml
        read_only: true
      - ./observability/dashboards:/otel-lgtm/dashboards/studi-kasus:ro
    # The image checks every 30 s; up --wait would sit idle that long.
    healthcheck:
      test: ["CMD", "/otel-lgtm/docker/healthcheck.sh"]
      interval: 5s
      timeout: 5s
      retries: 60
```

Container ini tidak memiliki *volume* data, sehingga semua log, *trace*, dan *metric* hilang ketika *container* dibuat ulang. Untuk lab, kondisi tersebut justru memudahkan, karena setiap sesi dimulai dari data kosong.

```
make infra-up
docker compose ps otel-lgtm
```

Buka http://localhost:13000. Port Grafana 3000 dan port OTLP 4317/4318 di dalam *container* dipetakan ke 13000 dan 14317/14318 di laptop agar tidak bentrok dengan aplikasi lain yang menggunakan port bawaan tersebut. Grafana di *container* ini berjalan dengan akses anonim sebagai admin, tanpa halaman *login*. Folder *Dashboards* → *Studi Kasus* sudah berisi *Studi Kasus: Transfer*, tetapi semua panel masih kosong karena service belum menyediakan `/metrics`.

8.4.2 Menambahkan *Dependency*

Setiap service mendapat *library* Prometheus dan OpenTelemetry. Versi ditulis eksplisit agar hasilnya sama dengan *checkpoint*. Jalankan dari folder `studi-kasus`:

```
OTEL="github.com/prometheus/client_golang@v1.25.0 \
  go.opentelemetry.io/otel@v1.47.0 \
  go.opentelemetry.io/otel/trace@v1.47.0 \
  go.opentelemetry.io/otel/sdk@v1.47.0 \
  go.opentelemetry.io/otel/sdk/log@v1.47.0 \
  go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracegrpc@v1.47.0 \
  go.opentelemetry.io/otel/exporters/otlp/otlplog/otlploggrpc@v0.23.0 \
  go.opentelemetry.io/contrib/bridges/otelslog@v0.21.0 \
  go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp@v0.72.0"
CONTRIB=go.opentelemetry.io/contrib/instrumentation
GRPC_OTEL=$CONTRIB/google.golang.org/grpc/otelgrpc@v0.72.0

(cd account      && go get $OTEL $GRPC_OTEL github.com/google/uuid@v1.6.0)
(cd transfer     && go get $OTEL $GRPC_OTEL)
(cd notification && go get $OTEL)
go work sync
```

`otelgrpc` menaikkan versi `google.golang.org/grpc` ke v1.83.2. Perintah `go work sync` menyalin versi tersebut ke `go.mod` modul lain di *workspace*, termasuk `proto`, sehingga semua modul menggunakan versi gRPC yang sama. Jalankan `go mod tidy` di setiap modul setelah kode di langkah berikutnya selesai ditulis.

8.4.3 Konfigurasi OpenTelemetry yang Wajib

Ketiga service membutuhkan dua variabel baru. Nama variabel mengikuti standar OpenTelemetry, sehingga nilai yang sama berlaku untuk SDK OpenTelemetry bahasa apa pun.

Variabel	Nilai di Laptop	Keterangan
`OTEL_EXPORTER_OTLP_ENDPOINT`	`http://localhost:14317`	Alamat OTLP gRPC *collector*, wajib berbentuk URL `http://host:port` atau `https://host:port`, lengkap dengan nomor port
`OTEL_SERVICE_NAME`	`account`, `transfer`, `notification`	Nama service di Tempo dan Loki

Pemeriksaan format URL diperlukan karena *exporter* OTLP mengabaikan nilai yang formatnya salah lalu mengirim ke alamat bawaan tanpa peringatan apa pun. Akibatnya service tampak normal, tetapi *trace* tidak pernah sampai. Port juga wajib ditulis. Untuk URL tanpa port, misalnya `http://localhost`,

gRPC menghubungi port 443, sehingga service tetap berjalan tetapi setiap *export* gagal setelah *startup*. Tambahkan di `Config` pada `transfer/config.go`:

```
  // Lab 8: the OpenTelemetry collector's OTLP gRPC endpoint as a URL
  // (http://localhost:14317) and the name the traces and logs carry.
  OTelEndpoint    string
  OTelServiceName string
```

Lalu di `LoadConfig`, isi kedua *field* dengan fungsi `get` yang sudah ada dan periksa formatnya setelah pemeriksaan variabel wajib yang belum diisi:

```
    OTelEndpoint:      get("OTEL_EXPORTER_OTLP_ENDPOINT"),
    OTelServiceName:   get("OTEL_SERVICE_NAME"),
  }
  if len(missing) > 0 {
    return Config{}, fmt.Errorf("missing required environment variables: %s",
strings.Join(missing, ", "))
  }
  if err := checkEndpointURL("OTEL_EXPORTER_OTLP_ENDPOINT", cfg.OTelEndpoint); err != nil {
    return Config{}, err
  }
  return cfg, nil
}

// checkEndpointURL rejects anything but scheme://host:port. The OTLP exporter
// would otherwise ignore a malformed value and silently send to its built-in
// default address. The port is required too: without it gRPC dials port 443,
// and http://localhost (no port) would fail on every export, not at startup.
func checkEndpointURL(name, v string) error {
  u, err := url.Parse(v)
  if err != nil || (u.Scheme != "http" && u.Scheme != "https") || u.Hostname() == "" {
    return fmt.Errorf("%s must be a URL like http://localhost:14317, got %q", name, v)
  }
  if u.Port() == "" {
    return fmt.Errorf("%s must name the port explicitly, like http://localhost:14317, got
%q", name, v)
  }
  return nil
}
```

Tambahkan `"net/url"` di `import`. Lakukan perubahan yang sama di `account/config.go` dan `notification/config.go`. Terakhir, tambahkan kedua variabel di target `run` pada `Makefile` setiap service, tepat sebelum baris `go run .`, dengan `OTEL_SERVICE_NAME` sesuai nama service:

```
  OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:14317 \
  OTEL_SERVICE_NAME=transfer \
  go run .
```

8.4.4 Paket `telemetry` di `transfer`

Seluruh kode *observability* dikumpulkan di paket `internal/telemetry`. Paket ini sengaja disalin ke setiap service dan tidak dijadikan *library* bersama. Ketiga service cukup menyepakati konvensi yang sama (nama *header*, nama *metric*), sedangkan kodenya dimiliki masing-masing service. Dengan *library* bersama, ketiga service harus selalu dirilis bersamaan. Paket ditulis lebih dahulu di `transfer`, karena `transfer` menggunakan ketiga jalur (HTTP, gRPC, dan Kafka).

8.4.4.1 `telemetry.go`: *exporter* dan *logger*

Fungsi `Setup` menyiapkan *tracer provider*, *propagator*, dan *logger*. `Setup` tidak membuka koneksi apa pun, sehingga service tetap berjalan walaupun *collector* tidak aktif. Kegagalan pengiriman dicatat di *stdout* sebagai `telemetry export failed`, dan *span* serta log yang gagal dikirim dibuang. Keputusan ini disengaja agar gangguan pada *observability* tidak pernah menggagalkan transfer.

Buat `transfer/internal/telemetry/telemetry.go`:

```
// Package telemetry wires the three observability signals of the service
// (Lab 8): logs (stdout + OTLP), traces (OTLP) and metrics (Prometheus),
// plus the correlation ID that ties the log lines of one request together.
//
// Every service has its own copy of this package on purpose: services share
// conventions (header names, metric names), not code.
package telemetry

import (
  "context"
  "errors"
  "fmt"
  "log/slog"
  "os"

  "go.opentelemetry.io/contrib/bridges/otelslog"
  "go.opentelemetry.io/otel"
  "go.opentelemetry.io/otel/attribute"
  "go.opentelemetry.io/otel/exporters/otlp/otlplog/otlploggrpc"
  "go.opentelemetry.io/otel/exporters/otlp/otlptrace/otlptracegrpc"
  "go.opentelemetry.io/otel/propagation"
  sdklog "go.opentelemetry.io/otel/sdk/log"
  "go.opentelemetry.io/otel/sdk/resource"
  sdktrace "go.opentelemetry.io/otel/sdk/trace"
)

type Telemetry struct {
  // Logger writes JSON to stdout and OTLP logs to the collector. Log with a
  // context (InfoContext etc.) inside request handling, so the record gets
  // the correlation ID and trace ID.
  Logger *slog.Logger
  tp     *sdktrace.TracerProvider
  lp     *sdklog.LoggerProvider
}

// Setup installs the global tracer provider and propagator and builds the
// logger. Nothing connects here: a collector that is down does not stop the
// service, export failures are logged on stdout instead.
func Setup(ctx context.Context, serviceName, otlpEndpoint string) (*Telemetry, error) {
  // service.name is how Tempo, Loki and Grafana tell the services apart.
  res, err := resource.New(ctx,
    resource.WithAttributes(attribute.String("service.name", serviceName)),
    resource.WithHost(),
    resource.WithTelemetrySDK())
  if err != nil {
    return nil, fmt.Errorf("telemetry resource: %w", err)
  }

  stdout := slog.NewJSONHandler(os.Stdout, nil)
  // Export failures go to stdout only: sending them through the OTLP log
  // exporter would feed the failure back into the exporter that is failing.
  exportLog := slog.New(stdout)
  otel.SetErrorHandler(otel.ErrorHandlerFunc(func(err error) {
    exportLog.Warn("telemetry export failed", "error", err)
  }))

  traceExp, err := otlptracegrpc.New(ctx, otlptracegrpc.WithEndpointURL(otlpEndpoint))
```

```
  if err != nil {
    return nil, fmt.Errorf("OTLP trace exporter: %w", err)
  }
  tp := sdktrace.NewTracerProvider(
    // Spans are sent in batches in the background, never inside the request.
    sdktrace.WithBatcher(traceExp),
    sdktrace.WithResource(res),
    // Every trace is kept: right for a lab. Production keeps a fraction
    // (TraceIDRatioBased) or decides at the collector (tail sampling).
    sdktrace.WithSampler(sdktrace.ParentBased(sdktrace.AlwaysSample())),
  )
  otel.SetTracerProvider(tp)
  // W3C Trace Context: the traceparent header (HTTP), metadata (gRPC) and
  // Kafka header that carry the trace from one service to the next.
  //
  // Baggage is left out on purpose: it would accept arbitrary key-value pairs
  // from any caller of the channel-facing REST API and forward them to every
  // service downstream. A public edge (API gateway, the first service behind
  // it) often also ignores an incoming traceparent and starts a new trace, so
  // an outside caller cannot pick trace IDs or force sampling.
  otel.SetTextMapPropagator(propagation.TraceContext{})

  logExp, err := otlploggrpc.New(ctx, otlploggrpc.WithEndpointURL(otlpEndpoint))
  if err != nil {
    return nil, fmt.Errorf("OTLP log exporter: %w", err)
  }
  lp := sdklog.NewLoggerProvider(
    sdklog.WithProcessor(sdklog.NewBatchProcessor(logExp)), sdklog.WithResource(res))

  // Every record goes to both: stdout for `docker logs` / `kubectl logs`,
  // OTLP for Loki, where a log line links to its trace.
  handler := newContextHandler(slog.NewMultiHandler(
    stdout, otelslog.NewHandler(serviceName, otelslog.WithLoggerProvider(lp))))
  return &Telemetry{Logger: slog.New(handler), tp: tp, lp: lp}, nil
}

// Shutdown flushes the spans and logs still buffered. Call it last, after
// everything that logs has stopped.
func (t *Telemetry) Shutdown(ctx context.Context) error {
  return errors.Join(t.tp.Shutdown(ctx), t.lp.Shutdown(ctx))
}
```

Empat hal perlu diperhatikan. *Propagator* hanya berisi `propagation.TraceContext{}`, sehingga yang berpindah antarservice hanya `traceparent` (dan `tracestate` bila ada): di *header* HTTP, *metadata* gRPC, *header* Kafka, dan kolom `trace_context` di tabel `outbox`. *Baggage* (W3C *Baggage*, pasangan *key-value* bebas yang ikut berpindah bersama *trace*) sengaja tidak dipasang. REST API `transfer` dipanggil aplikasi *channel* dari luar. Bila *propagator Baggage* dipasang, *baggage* dari pemanggil luar ikut diteruskan ke setiap service di belakangnya. Di *production*, *public edge* (API *gateway* atau service pertama di belakangnya) sering juga mengabaikan `traceparent` dari luar dan memulai *trace* baru, agar pemanggil luar tidak dapat menentukan *trace ID* atau memaksa *sampling*. Lab ini tidak memiliki API *gateway* dan tidak memasang penyaringan di *public edge*, sehingga `otelhttp` di `transfer` tetap melanjutkan `traceparent` bila pemanggil mengirimnya.

`slog.NewMultiHandler` (tersedia sejak Go 1.26) meneruskan setiap *record* ke dua *handler* sekaligus: *stdout* untuk `docker logs` atau `kubectl logs`, dan `otelslog` untuk Loki. *Sampler* `AlwaysSample` menyimpan semua *trace*. Pengaturan ini cocok untuk lab dengan beberapa *request* per detik, tetapi di *production* biaya penyimpanan semua *trace* bisa melebihi biaya menjalankan service itu sendiri. Terakhir, `Shutdown` harus dipanggil paling akhir agar *span* dan log dari proses *shutdown* ikut terkirim.

8.4.4.2 `log.go`: ID di setiap baris log

`slog.Handler` dapat dibungkus untuk menambah atribut dari *context*. Dengan *wrapper* ini, kode bisnis cukup memanggil `log.InfoContext(ctx, ...)`, dan `correlation_id`, `trace_id`, serta `span_id` ditambahkan otomatis. Buat `transfer/internal/telemetry/log.go`:

```
package telemetry

import (
  "context"
  "log/slog"

  "go.opentelemetry.io/otel/trace"
)

// contextHandler adds correlation_id, trace_id and span_id from the context
// to every record. Only records logged WITH a context get them: inside
// request handling, use log.InfoContext(ctx, ...), not log.Info(...).
type contextHandler struct {
  slog.Handler
}

func newContextHandler(h slog.Handler) slog.Handler { return contextHandler{h} }

func (h contextHandler) Handle(ctx context.Context, r slog.Record) error {
  if id := CorrelationID(ctx); id != "" {
    r.AddAttrs(slog.String("correlation_id", id))
  }
  if sc := trace.SpanContextFromContext(ctx); sc.IsValid() {
    r.AddAttrs(slog.String("trace_id", sc.TraceID().String()),
      slog.String("span_id", sc.SpanID().String()))
  }
  return h.Handler.Handle(ctx, r)
}

func (h contextHandler) WithAttrs(attrs []slog.Attr) slog.Handler {
  return contextHandler{h.Handler.WithAttrs(attrs)}
}

func (h contextHandler) WithGroup(name string) slog.Handler {
  return contextHandler{h.Handler.WithGroup(name)}
}
```

`WithAttrs` dan `WithGroup` wajib dibungkus ulang. Tanpa keduanya, *logger* hasil `log.With(...)` kembali ke *handler* asli dan kehilangan ID.

8.4.4.3 `correlation.go`: *correlation ID* di HTTP dan gRPC

Correlation ID dari luar berasal dari *input* pemanggil dan akan muncul di setiap baris log. Karena itu panjangnya dibatasi dan karakternya dibatasi ke `A-Z a-z 0-9 . _ : -`. Nilai yang tidak memenuhi pola diganti UUID baru. Buat `transfer/internal/telemetry/correlation.go`:

```
package telemetry

import (
  "context"
  "net/http"
  "regexp"

  "github.com/google/uuid"
  "go.opentelemetry.io/otel/attribute"
```

```
  "go.opentelemetry.io/otel/trace"
  "google.golang.org/grpc"
  "google.golang.org/grpc/metadata"
)

// The correlation ID is a business-level ID the caller can quote; the trace
// ID is the technical one. Both end up in every log line.
const (
  CorrelationHeader = "X-Correlation-ID"
  // gRPC metadata keys are lower case.
  CorrelationMetadataKey = "x-correlation-id"
  // Kafka header, next to event_type and event_id.
  CorrelationKafkaHeader = "correlation_id"
)

type correlationKey struct{}

func WithCorrelationID(ctx context.Context, id string) context.Context {
  return context.WithValue(ctx, correlationKey{}, id)
}

// CorrelationID is "" when ctx has none.
func CorrelationID(ctx context.Context) string {
  id, _ := ctx.Value(correlationKey{}).(string)
  return id
}

// An incoming ID is caller input that ends up in every log line: bounded in
// length and restricted to characters that cannot fake log fields.
var correlationPattern = regexp.MustCompile(`^[A-Za-z0-9._:-]{1,100}$`)

// AcceptCorrelationID returns id when it is acceptable, otherwise a new one.
func AcceptCorrelationID(id string) string {
  if correlationPattern.MatchString(id) {
    return id
  }
  return uuid.NewString()
}

// CorrelationMiddleware takes X-Correlation-ID from the request or generates
// one, stores it in the context, echoes it in the response, and tags the
// server span with it, so Tempo finds the trace by correlation ID.
func CorrelationMiddleware(next http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    id := AcceptCorrelationID(r.Header.Get(CorrelationHeader))
    w.Header().Set(CorrelationHeader, id)
    trace.SpanFromContext(r.Context()).SetAttributes(attribute.String("correlation.id",
id))
    next.ServeHTTP(w, r.WithContext(WithCorrelationID(r.Context(), id)))
  })
}

// UnaryClientCorrelation sends the context's correlation ID to the called
// service as gRPC metadata. The trace context needs no code here: the otelgrpc
// stats handler sends it (traceparent).
func UnaryClientCorrelation() grpc.UnaryClientInterceptor {
  return func(ctx context.Context, method string, req, reply any, cc *grpc.ClientConn,
    invoker grpc.UnaryInvoker, opts ...grpc.CallOption) error {
    if id := CorrelationID(ctx); id != "" {
```

```
      ctx = metadata.AppendToOutgoingContext(ctx, CorrelationMetadataKey, id)
    }
    return invoker(ctx, method, req, reply, cc, opts...)
  }
}
```

Dengan atribut *span* `correlation.id`, *trace* dapat dicari di Tempo berdasarkan *correlation ID* yang dilaporkan pemanggil.

8.4.4.4 `http.go`: *metric* dan *span* per *route*

Metric durasi *request* diberi *label* `method`, `route`, dan `status`. `route` harus diisi *pattern* ServeMux (`/transfers/{id}`), bukan *path* lengkap (`/transfers/8f3c...`). Setiap kombinasi nilai *label* menjadi satu *time series* di Prometheus. Bila *path* lengkap dijadikan *label*, setiap ID transfer menambah satu *series* baru, jumlahnya tumbuh tanpa batas, dan akhirnya Prometheus kehabisan memori. Masalah ini dikenal sebagai *high cardinality*.

ServeMux menulis *pattern* yang cocok ke `r.Pattern` pada `*http.Request` yang diterimanya. Nilai tersebut hanya terbaca bila *wrapper* `route` menerima objek `*http.Request` yang sama dengan yang diproses ServeMux, sehingga `route` harus membungkus *mux* secara langsung, tanpa *middleware* lain yang memanggil `r.WithContext` di antaranya. Buat `transfer/internal/telemetry/http.go`:

```
package telemetry

import (
  "net/http"
  "strconv"
  "strings"
  "time"

  "github.com/prometheus/client_golang/prometheus"
  "github.com/prometheus/client_golang/prometheus/promauto"
  "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
  "go.opentelemetry.io/otel/attribute"
  "go.opentelemetry.io/otel/trace"
)

// requestDuration is the RED metric of the HTTP API: rate (count), errors
// (status), duration. route is the ServeMux pattern, never the raw path.
var requestDuration = promauto.NewHistogramVec(prometheus.HistogramOpts{
  Name:    "http_server_request_duration_seconds",
  Help:    "Duration of HTTP requests handled by this service.",
  Buckets: prometheus.DefBuckets, // 5 ms .. 10 s
}, []string{"method", "route", "status"})

// HTTPHandler wraps the service's mux, outermost first:
//
//  1. otelhttp: server span, continues the caller's trace from traceparent
//  2. correlation ID
//  3. route: metrics and span name by the matched route pattern
//
// Probes and /metrics are not traced: they are called every few seconds,
// and those traces would bury the real ones.
func HTTPHandler(mux http.Handler) http.Handler {
  return otelhttp.NewHandler(CorrelationMiddleware(route(mux)), "http",
    otelhttp.WithFilter(func(r *http.Request) bool {
      switch r.URL.Path {
      case "/healthz", "/readyz", "/metrics":
        return false
      }
```

```
      return true
    }))
}

// route must wrap the mux directly: ServeMux writes the matched pattern into
// the *Request it routes (r.Pattern).
func route(mux http.Handler) http.Handler {
  return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
    start := time.Now()
    rec := &statusRecorder{ResponseWriter: w, status: http.StatusOK}
    mux.ServeHTTP(rec, r)

    // "GET /transfers/{id}" -> "/transfers/{id}"; the method is its own label.
    pattern := r.Pattern
    if _, path, ok := strings.Cut(pattern, " "); ok {
      pattern = path
    }
    if pattern == "" {
      pattern = "unmatched" // 404 from the mux: one series, whatever the path
    }
    requestDuration.WithLabelValues(method(r.Method), pattern, strconv.Itoa(rec.status)).
      Observe(time.Since(start).Seconds())

    span := trace.SpanFromContext(r.Context())
    span.SetName(r.Method + " " + pattern)
    span.SetAttributes(attribute.String("http.route", pattern))
  })
}

// method bounds the label: any token is a valid HTTP method on the wire.
func method(m string) string {
  switch m {
  case http.MethodGet, http.MethodPost, http.MethodPut, http.MethodDelete,
    http.MethodPatch, http.MethodHead, http.MethodOptions:
    return m
  }
  return "OTHER"
}

type statusRecorder struct {
  http.ResponseWriter
  status int
}

func (s *statusRecorder) WriteHeader(code int) {
  s.status = code
  s.ResponseWriter.WriteHeader(code)
}

// Unwrap lets http.ResponseController reach the real writer.
func (s *statusRecorder) Unwrap() http.ResponseWriter { return s.ResponseWriter }
```

Fungsi `method` juga membatasi nilai *label*: metode HTTP yang dikirim klien dapat berupa token apa pun, sehingga metode di luar daftar digabung menjadi `OTHER`. Nama *span* diubah dari `http` menjadi `POST /transfers` agar mudah dibaca di Tempo.

8.4.4.5 `kafka.go`: *trace context* di *header* Kafka

`kafka-go` tidak memiliki instrumentasi OpenTelemetry. *Propagator* OpenTelemetry menulis dan membaca *trace context* lewat *interface* `TextMapCarrier` (`Get`, `Set`, `Keys`), sehingga cukup dibuat adaptor dari `[]kafka.Header` ke *interface* tersebut. Buat `transfer/internal/telemetry/kafka.go`:

```
package telemetry

import (
  "github.com/segmentio/kafka-go"
  "go.opentelemetry.io/otel/propagation"
)

// KafkaHeaders lets the OpenTelemetry propagator write the trace context
// (traceparent, tracestate) into Kafka message headers, the way otelhttp and
// otelgrpc write it into HTTP headers and gRPC metadata.
type KafkaHeaders struct {
  Headers *[]kafka.Header
}

var _ propagation.TextMapCarrier = KafkaHeaders{}

func (c KafkaHeaders) Get(key string) string {
  for _, h := range *c.Headers {
    if h.Key == key {
      return string(h.Value)
    }
  }
  return ""
}

func (c KafkaHeaders) Set(key, value string) {
  for i, h := range *c.Headers {
    if h.Key == key {
      (*c.Headers)[i].Value = []byte(value)
      return
    }
  }
  *c.Headers = append(*c.Headers, kafka.Header{Key: key, Value: []byte(value)})
}

func (c KafkaHeaders) Keys() []string {
  keys := make([]string, len(*c.Headers))
  for i, h := range *c.Headers {
    keys[i] = h.Key
  }
  return keys
}
```

8.4.5 Memasang Telemetri di `transfer/main.go`

Sebelum konfigurasi terbaca, alamat *collector* belum diketahui. Karena itu `main` membuat *logger* JSON sederhana ke *stdout* untuk log *error* saat *startup*. Ganti baris pertama `main`:

```
  // Until the configuration is read there is no collector address: startup
  // errors go to stdout only, as JSON like every later log line.
  log := slog.New(slog.NewJSONHandler(os.Stdout, nil))
```

Di `run`, tepat setelah `signal.NotifyContext` dan sebelum `transfer.NewPostgresStore`, siapkan telemetri lalu ganti `log` dengan *logger* baru:

```
  // Lab 8: from here on, log lines also go to the collector (OTLP) and carry
  // correlation_id / trace_id when logged with a request context.
  tel, err := telemetry.Setup(ctx, cfg.OTelServiceName, cfg.OTelEndpoint)
  if err != nil {
    return err
  }
  // Deferred first, so it runs last: spans and logs of the shutdown itself
  // are flushed too.
  defer func(boot *slog.Logger) {
    flushCtx, cancel := context.WithTimeout(context.Background(), 5*time.Second)
    defer cancel()
    if err := tel.Shutdown(flushCtx); err != nil {
      boot.Warn("flushing telemetry failed", "error", err)
    }
  }(log)
  log = tel.Logger
```

Pengambilan *token* OAuth2 ke Hydra juga dijadikan *span*. Transfer pertama setelah service dijalankan ulang lebih lambat karena harus mengambil *token*, dan *span* ini menunjukkan penyebabnya. Ganti pemanggilan `accountclient.New`:

```
  // Its HTTP call to the authorization server becomes a span of the transfer's
  // trace: a slow first transfer after a restart shows why.
  tokenHTTP := &http.Client{Transport: otelhttp.NewTransport(http.DefaultTransport)}
  fetchToken := func(ctx context.Context) (*oauth2.Token, error) {
    return tokenCfg.Token(context.WithValue(ctx, oauth2.HTTPClient, tokenHTTP))
  }
  accounts, err := accountclient.New(cfg.AccountGRPCAddr, accountclient.ProductionSettings,
fetchToken, log)
```

Daftarkan `/metrics` setelah `/debug/circuit-breaker`, lalu bungkus *mux* dengan `telemetry.HTTPHandler`:

```
  // Lab 8: Prometheus pulls the metrics from here. Open like the probes;
  // production restricts it to the monitoring network.
  mux.Handle("GET /metrics", promhttp.Handler())
  srv := &http.Server{
    Addr:              cfg.HTTPAddr,
    Handler:           telemetry.HTTPHandler(mux),
    ReadHeaderTimeout: 5 * time.Second,
  }
```

Relay outbox kini menerima nama *topic* untuk menamai *span*-nya (perubahan `NewRelay` ada di langkah *outbox*):

```
    outbox.NewRelay(store.Pool(), writer, cfg.KafkaTopic, log).Run(relayCtx)
```

Log `transfer service listening` mendapat atribut `"otlp_endpoint", cfg.OTelEndpoint` agar alamat *collector* terlihat saat *startup*.

Tambahkan *import* `promhttp`, `otelhttp`, `golang.org/x/oauth2`, dan paket `telemetry`:

```
  "github.com/prometheus/client_golang/prometheus/promhttp"
  "go.opentelemetry.io/contrib/instrumentation/net/http/otelhttp"
  "golang.org/x/oauth2"
  ...
  "github.com/artivisi/training-microservices-2026-01/studi-kasus/transfer/internal/
telemetry"
```

8.4.6 Telemetri pada Client gRPC dan *Circuit Breaker*

Di `transfer/internal/accountclient/client.go`, client gRPC mendapat tiga tambahan. *Interceptor* `UnaryClientCorrelation` dipasang paling depan di *interceptor chain* agar *correlation ID* ditambahkan satu kali, sebelum *token* dan *policy resiliency*. `otelgrpc.NewClientHandler` dipasang sebagai *stats*

handler, sehingga setiap percobaan, termasuk setiap *retry*, menjadi satu *span* tersendiri. Kedua opsi ini ditambahkan di fungsi `dial`, sehingga berlaku baik untuk koneksi utama (`Debit`, `Credit`) maupun untuk koneksi *reversal* dari Lab 5. Ganti bagian opsi di `dial`:

```
    grpc.WithPerRPCCredentials(tokens),
    // First interceptor = outermost: a call repeated with a new token goes
    // through the resiliency policies again. The correlation ID is added once,
    // before all of them.
    grpc.WithChainUnaryInterceptor(telemetry.UnaryClientCorrelation(),
      tokens.refreshOnUnauthenticated, resilience),
    // Lab 8: one client span per attempt, so retries and timeouts are visible
    // in the trace, and the trace context sent along in the metadata.
    grpc.WithStatsHandler(otelgrpc.NewClientHandler()),
```

Status *circuit breaker* dicatat dalam dua *metric*. *Gauge* `circuit_breaker_state` menyimpan status sebagai angka agar dapat digambar sebagai grafik dan dijadikan dasar *alert*. *Counter* `circuit_breaker_transitions_total` menghitung perpindahan status. *Counter* ini dibutuhkan karena *circuit breaker* yang berpindah `open` lalu `closed` di antara dua *scrape* tidak pernah terlihat di *gauge*. Tambahkan di atas `type Settings`:

```
// Breaker metrics (Lab 8). The state as a number so Grafana can graph it and
// alert on it; transitions as a counter, because a breaker that flaps between
// open and closed within one scrape interval never shows up in the gauge.
var (
  breakerState = promauto.NewGaugeVec(prometheus.GaugeOpts{
    Name: "circuit_breaker_state",
    Help: "Circuit breaker state per target: 0 closed, 1 half-open, 2 open.",
  }, []string{"target"})
  breakerTransitions = promauto.NewCounterVec(prometheus.CounterOpts{
    Name: "circuit_breaker_transitions_total",
    Help: "Circuit breaker state changes per target.",
  }, []string{"target", "from", "to"})
)

// stateValue maps the breaker state to the circuit_breaker_state value.
func stateValue(s circuitbreaker.State) float64 {
  switch s {
  case circuitbreaker.HalfOpenState:
    return 1
  case circuitbreaker.OpenState:
    return 2
  }
  return 0
}
```

Perbarui kedua *metric* di `OnStateChanged` dan isi nilai awal *gauge* setelah `Build()`:

```
    OnStateChanged(func(e circuitbreaker.StateChangedEvent) {
      // No request context: the change belongs to the breaker, not to the
      // call that happened to trigger it.
      log.Warn("circuit breaker state changed", "target", "account",
        "from", e.OldState.String(), "to", e.NewState.String())
      breakerState.WithLabelValues("account").Set(stateValue(e.NewState))
      breakerTransitions.WithLabelValues("account", e.OldState.String(),
e.NewState.String()).Inc()
    }).
    Build()
  breakerState.WithLabelValues("account").Set(stateValue(breaker.State()))
```

Log *retry* di kedua *policy* diubah menjadi `WarnContext` dengan *context* percobaan, agar baris log tersebut muncul di bawah *trace* transfer yang sama. Di *policy* `retry`:

```
    OnRetry(func(e failsafe.ExecutionEvent[any]) {
      log.WarnContext(e.Context(), "retrying account call",
        "next_attempt", e.Attempts(), "cause", e.LastError())
    }).
```

Di *policy* `reversalRetry`:

```
    OnRetry(func(e failsafe.ExecutionEvent[any]) {
      log.WarnContext(e.Context(), "retrying reversal", "next_attempt",
        e.Attempts(), "elapsed", e.ElapsedTime(), "cause", e.LastError())
    }).
```

Import yang ditambahkan: `prometheus`, `promauto`, `otelgrpc`, dan paket `telemetry` milik `transfer`.

8.4.7 *Metric* Bisnis dan Atribut *Span* Transfer

Metric HTTP menunjukkan kesehatan pemrosesan *request*, sedangkan `transfers_total` menunjukkan hasil transfer dana. Kenaikan jumlah `COMPENSATED` atau `FAILED` menandakan insiden bisnis, walaupun setiap *request* dijawab dengan cepat. Buat `transfer/internal/transfer/metrics.go`:

```
package transfer

import (
  "github.com/prometheus/client_golang/prometheus"
  "github.com/prometheus/client_golang/prometheus/promauto"
)

// transfersTotal counts saga outcomes by end state (Lab 8).
var transfersTotal = promauto.NewCounterVec(prometheus.CounterOpts{
  Name: "transfers_total",
  Help: "Transfers that reached an end state, by status.",
}, []string{"status"})

// Every status starts at 0. A series that first appears with the value 5 shows
// no increase to rate(): the first transfers after a start would be missing
// from the dashboard.
func init() {
  for _, s := range []Status{Completed, Failed, Compensated, CompensationFailed} {
    transfersTotal.WithLabelValues(string(s))
  }
}
```

Fungsi `init` menyiapkan setiap status dengan nilai 0. Fungsi `rate()` di Prometheus menghitung kenaikan antara dua sampel. *Series* yang baru muncul dengan nilai 5 tidak memiliki sampel sebelumnya, sehingga lima transfer pertama tidak pernah terlihat di grafik.

Di `transfer/internal/transfer/service.go`, `finish` menaikkan *counter* dan menandai *span request* dengan status akhir, tepat setelah pemeriksaan `err` dari `s.store.Transition`:

```
  transfersTotal.WithLabelValues(string(to)).Inc()
  trace.SpanFromContext(ctx).SetAttributes(attribute.String("transfer.status", string(to)))
```

Di `Execute`, setelah transfer tercatat dan sebelum pemeriksaan `!created`, tandai *span* dengan ID transfer:

```
  // Tags the request's span: Tempo finds the trace of a transfer by its ID,
  // e.g. TraceQL {span.transfer.id="<id>"}.
  trace.SpanFromContext(ctx).SetAttributes(attribute.String("transfer.id", t.ID))
```

Tambahkan *import* `go.opentelemetry.io/otel/attribute` dan `go.opentelemetry.io/otel/trace`. Kedua atribut ini memungkinkan pencarian di Tempo dengan `{ span.transfer.status = "FAILED" }` atau `{ span.transfer.id = "<id>" }`.

8.4.8 Log dengan *Context*

`contextHandler` hanya dapat menambahkan ID bila *context* ikut dikirim. Ubah setiap pemanggilan log di dalam penanganan *request* menjadi varian `...Context`:

```
// sebelum
s.log.Info("transfer completed", "transfer_id", t.ID, "amount", t.Amount)
// sesudah
s.log.InfoContext(ctx, "transfer completed", "transfer_id", t.ID, "amount", t.Amount)
```

Lokasi yang perlu diubah di `transfer`: semua log di `internal/transfer/service.go` (`Execute`, `run`, `compensate`, `finish`), `internalError` dan `readyz` di `internal/transfer/http.go` (dengan `r.Context()`), serta ketiga log `request rejected` di `internal/auth/http.go`. Log yang terjadi di luar *request*, misalnya `transfer service listening` atau perubahan status *circuit breaker*, tetap menggunakan `log.Info` atau `log.Warn`.

8.4.9 Meneruskan *Trace* melalui *Outbox* ke Kafka

Langkah ini menyambung *trace* dari `transfer` ke `notification`. Tambahkan dua kolom di akhir `transfer/internal/transfer/schema.sql`:

```
-- Observability (Lab 8): the correlation ID and the trace context
-- (traceparent, tracestate) of the request that wrote the event. The relay
-- runs later, outside that request; it reads both from here.
alter table outbox add column if not exists correlation_id varchar(100);
alter table outbox add column if not exists trace_context jsonb;
```

Di `transfer/internal/outbox/outbox.go`, `Message` mendapat dua *field* untuk kedua kolom tersebut, di akhir `struct` setelah `Payload`:

```
  // CorrelationID and TraceContext belong to the request that wrote the row
  // (Lab 8). Insert takes them from its ctx.
  CorrelationID string
  TraceContext  map[string]string
```

`Insert` menyalin *trace context* dari `ctx` ke `MapCarrier` lewat *propagator* (isinya `traceparent` dan, bila ada, `tracestate`), lalu menyimpannya bersama *correlation ID*. pgx menyimpan `map[string]string` ke kolom `jsonb` secara langsung.

```
func Insert(ctx context.Context, tx pgx.Tx, m Message) error {
  carrier := propagation.MapCarrier{}
  otel.GetTextMapPropagator().Inject(ctx, carrier)
  _, err := tx.Exec(ctx, `
    insert into outbox (id, aggregate_id, event_type, payload, correlation_id,
trace_context)
    values (@id, @aggregate_id, @event_type, @payload, nullif(@correlation_id, ''),
@trace_context)`,
    pgx.NamedArgs{"id": m.ID, "aggregate_id": m.AggregateID, "event_type": m.EventType,
      "payload": m.Payload, "correlation_id": telemetry.CorrelationID(ctx),
      "trace_context": map[string]string(carrier)})
  if err != nil {
    return fmt.Errorf("insert outbox message: %w", err)
  }
  return nil
}
```

`Relay` mendapat *field* `topic` dan `tracer`:

```
type Relay struct {
  pool   *pgxpool.Pool
```

```
  pub    Publisher
  topic  string
  tracer trace.Tracer
  log    *slog.Logger
}

// NewRelay takes topic only to name the spans; pub decides where messages go.
func NewRelay(pool *pgxpool.Pool, pub Publisher, topic string, log *slog.Logger) *Relay {
  return &Relay{pool: pool, pub: pub, topic: topic, tracer: otel.Tracer("transfer/outbox"),
log: log}
}
```

Query di `publishBatch` membaca kedua kolom baru. `coalesce` menangani baris lama dari Lab 5 hingga Lab 7 yang belum memiliki kolom tersebut:

```
  rows, err := tx.Query(ctx, `
    select id::text, aggregate_id::text, event_type, payload,
           coalesce(correlation_id, ''), coalesce(trace_context, '{}'::jsonb)
    from outbox
    ...
```

Baris hasil *query* diubah menjadi `Message` dengan `pgx.RowToStructByPos`, yang mencocokkan kolom dengan *field* berdasarkan urutan. Karena itu `CorrelationID` dan `TraceContext` harus berada di akhir `struct Message`, sesuai urutan kolom di `select`.

Fungsi baru `message` membuat *span* `publish transfer-events` dengan *parent* dari *trace context* yang tersimpan, lalu menulis *context* milik *span* tersebut ke *header* Kafka. Dengan begitu *span* di `notification` menjadi *child* dari *span* `publish`:

```
// message builds the Kafka message for m and starts its "publish" span as a
// child of the request that wrote m. The caller ends the span once Kafka has
// acknowledged the message.
func (r *Relay) message(ctx context.Context, m Message) (kafka.Message, trace.Span) {
  parent := otel.GetTextMapPropagator().Extract(ctx,
propagation.MapCarrier(m.TraceContext))
  spanCtx, span := r.tracer.Start(parent, "publish "+r.topic,
    trace.WithSpanKind(trace.SpanKindProducer),
    trace.WithAttributes(
      attribute.String("messaging.system", "kafka"),
      attribute.String("messaging.operation.type", "send"),
      attribute.String("messaging.destination.name", r.topic),
      attribute.String("messaging.message.id", m.ID),
      attribute.String("messaging.kafka.message.key", m.AggregateID),
      attribute.String("event.type", m.EventType),
    ))

  headers := []kafka.Header{
    {Key: "event_type", Value: []byte(m.EventType)},
    {Key: "event_id", Value: []byte(m.ID)},
  }
  if m.CorrelationID != "" {
    headers = append(headers, kafka.Header{Key: telemetry.CorrelationKafkaHeader, Value:
[]byte(m.CorrelationID)})
  }
  otel.GetTextMapPropagator().Inject(spanCtx, telemetry.KafkaHeaders{Headers: &headers})
  return kafka.Message{Key: []byte(m.AggregateID), Value: m.Payload, Headers: headers},
span
}
```

Di `publishBatch`, ganti pembuatan `kafka.Message` di dalam *loop* dengan `message`, lalu akhiri *span* setelah Kafka menerima *batch*. *Span* dari *batch* yang gagal sengaja tidak diakhiri, sehingga tidak pernah dikirim. Selama Kafka tidak berjalan, *batch* yang sama gagal setiap 500 ms. Bila *span* tersebut diakhiri, setiap *trace* transfer yang menunggu akan mendapat satu *span* gagal tambahan untuk setiap percobaan.

```
  spans := make([]trace.Span, len(pending))
  for i, m := range pending {
    msgs[i], spans[i] = r.message(ctx, m)
    ids[i] = m.ID
  }
  pubCtx, cancel := context.WithTimeout(ctx, 10*time.Second)
  defer cancel()
  if err := r.pub.WriteMessages(pubCtx, msgs...); err != nil {
    // The spans of a failed batch are not ended, so they are never exported.
    return 0, fmt.Errorf("publish %d events: %w", len(msgs), err)
  }
  for _, span := range spans {
    span.End()
  }
```

Bagian akhir `publishBatch`, setelah `update outbox set published_at`, kini memeriksa `Commit` lalu menulis satu log per *event* di dalam *trace event* tersebut:

```
  if err := tx.Commit(ctx); err != nil {
    return 0, err
  }
  // One line per event, in the event's own trace: Loki shows it under the
  // transfer's trace next to the log lines of the other services.
  for i, m := range pending {
    evCtx := telemetry.WithCorrelationID(trace.ContextWithSpan(ctx, spans[i]),
m.CorrelationID)
    r.log.InfoContext(evCtx, "event published", "event_id", m.ID,
      "event_type", m.EventType, "transfer_id", m.AggregateID)
  }
  return len(pending), nil
```

Terakhir, *gauge* `outbox_pending` mencatat jumlah baris yang belum terkirim. Dalam kondisi normal, nilainya 0 atau mendekati 0. Nilai yang terus naik berarti Kafka tidak dapat dihubungi atau *relay* macet, dan semua notifikasi tertunda. Tambahkan di atas `type Message`:

```
// pendingGauge is the backlog of the relay.
var pendingGauge = promauto.NewGauge(prometheus.GaugeOpts{
  Name: "outbox_pending",
  Help: "Outbox events not yet published to Kafka.",
})
```

Ubah *loop* di `Run` agar memperbarui *gauge* `outbox_pending` setelah setiap *batch*, termasuk setelah *batch* yang gagal:

```
    n, err := r.publishBatch(ctx)
    switch {
    case err != nil && ctx.Err() == nil:
      r.log.Warn("outbox publish failed, will retry", "error", err)
    case n > 0:
      r.log.Info("outbox events published", "count", n)
    }
    // After a failure too: a growing backlog while Kafka is down is exactly
    // what the gauge is for.
    r.updatePending(ctx)
  }
}
```

```
func (r *Relay) updatePending(ctx context.Context) {
  var n int64
  // Cheap: the partial index outbox_unpublished holds only unpublished rows.
  if err := r.pool.QueryRow(ctx, `select count(*) from outbox where published_at is
null`).Scan(&n); err != nil {
    if ctx.Err() == nil {
      r.log.Warn("counting pending outbox events failed", "error", err)
    }
    return
  }
  pendingGauge.Set(float64(n))
}
```

Import yang ditambahkan: `prometheus`, `promauto`, `go.opentelemetry.io/otel`, `.../otel/attribute`, `.../otel/propagation`, `.../otel/trace`, dan paket `telemetry`. Ambil *unit test* `transfer` dari *checkpoint*, lalu jalankan pengujian untuk memastikan semua bagian tersambung. `go mod tidy` dijalankan setelah *test* diambil agar *dependency* yang hanya digunakan oleh *test* ikut tercatat di `go.sum`:

```
git checkout lab-08 -- \
  transfer/config_test.go \
  transfer/internal/telemetry/telemetry_test.go \
  transfer/internal/outbox/outbox_test.go \
  transfer/internal/accountclient/client_test.go \
  transfer/internal/transfer/service_test.go
cd transfer && go mod tidy && make test && cd ..
```

8.4.10 Telemetri di `account`

Salin paket dari `transfer`, kecuali `kafka.go` karena `account` tidak menggunakan Kafka:

```
mkdir -p account/internal/telemetry
cp transfer/internal/telemetry/{telemetry.go,log.go,correlation.go,http.go} account/
internal/telemetry/
```

Di `account/internal/telemetry/correlation.go`, hapus konstanta `CorrelationKafkaHeader` dan ganti `UnaryClientCorrelation` dengan versi server. *Trace context* tidak membutuhkan kode di sini, karena *stats handler* `otelgrpc` sudah membacanya dan memulai *span* server sebelum *interceptor* berjalan.

```
// UnaryServerCorrelation is the gRPC counterpart of CorrelationMiddleware: the
// ID comes from the x-correlation-id metadata the transfer service sends. It
// runs first in the chain, so every later interceptor logs with the ID.
func UnaryServerCorrelation() grpc.UnaryServerInterceptor {
  return func(ctx context.Context, req any, _ *grpc.UnaryServerInfo, handler
grpc.UnaryHandler) (any, error) {
    var incoming string
    if md, ok := metadata.FromIncomingContext(ctx); ok {
      if v := md.Get(CorrelationMetadataKey); len(v) > 0 {
        incoming = v[0]
      }
    }
    id := AcceptCorrelationID(incoming)
    trace.SpanFromContext(ctx).SetAttributes(attribute.String("correlation.id", id))
    return handler(WithCorrelationID(ctx, id), req)
  }
}
```

Panggilan utama ke `account` berjalan lewat gRPC. Tanpa *metric* gRPC, `Debit` yang lambat tidak terlihat di satu pun *metric* latensi `account`. Buat `account/internal/interceptor/metrics.go`:

```
package interceptor

import (
  "context"
  "time"

  "github.com/prometheus/client_golang/prometheus"
  "github.com/prometheus/client_golang/prometheus/promauto"
  "google.golang.org/grpc"
  "google.golang.org/grpc/status"
)

// requestDuration is the gRPC counterpart of http_server_request_duration_seconds
// (Lab 8). method is the full method name: a fixed set, defined by the proto
// contract.
var requestDuration = promauto.NewHistogramVec(prometheus.HistogramOpts{
  Name:    "grpc_server_request_duration_seconds",
  Help:    "Duration of unary gRPC calls handled by this service.",
  Buckets: prometheus.DefBuckets, // 5 ms .. 10 s
}, []string{"method", "code"})

// Metrics observes every unary call with its status code. It runs before auth
// and the fault injector, so rejected calls and injected delays are measured
// like the caller experiences them.
func Metrics() grpc.UnaryServerInterceptor {
  return func(ctx context.Context, req any, info *grpc.UnaryServerInfo, handler
grpc.UnaryHandler) (any, error) {
    start := time.Now()
    resp, err := handler(ctx, req)
    requestDuration.WithLabelValues(info.FullMethod, status.Code(err).String()).
      Observe(time.Since(start).Seconds())
    return resp, err
  }
}
```

Fault injection di `account/internal/fault/fault.go` menambah *span event* agar gangguan simulasi dapat dibedakan dari gangguan sebenarnya di Tempo. Service yang benar-benar lambat muncul sebagai *span* panjang tanpa *event* tersebut. Ganti fungsi `inject`:

```
func (i *Injector) inject(ctx context.Context) bool {
  cfg := i.cfg.Load()
  // Lab 8: the span events tell a simulated fault from a real one in the trace.
  span := trace.SpanFromContext(ctx)
  if cfg.DelayMS > 0 {
    // Recorded before the wait: the caller usually gives up during it.
    span.AddEvent("fault injected: delay",
      trace.WithAttributes(attribute.Int("fault.delay_ms", cfg.DelayMS)))
    select {
    case <-time.After(time.Duration(cfg.DelayMS) * time.Millisecond):
    case <-ctx.Done():
      return false
    }
  }
  failed := cfg.ErrorRate > 0 && rand.Float64() < cfg.ErrorRate
  if failed {
    span.AddEvent("fault injected: error",
      trace.WithAttributes(attribute.Float64("fault.error_rate", cfg.ErrorRate)))
  }
  return failed
}
```

Di `account/main.go`, lakukan perubahan yang sama dengan `transfer`: *logger* JSON di `main`, `telemetry.Setup` beserta `defer Shutdown`, `GET /metrics` setelah `faults.Register(mux)`, dan `Handler: telemetry.HTTPHandler(mux)`. Server gRPC mendapat *stats handler* dan dua *interceptor* baru di awal *chain*:

```
grpcSrv := grpc.NewServer(
  // Lab 8: continues the caller's trace (traceparent in the metadata) with
  // one server span per call. A stats handler, not an interceptor: it sees
  // the call before the interceptors run, and the span covers them all,
  // fault delays included.
  grpc.StatsHandler(otelgrpc.NewServerHandler()),
  grpc.ChainUnaryInterceptor(
    telemetry.UnaryServerCorrelation(),
    interceptor.Metrics(),
    interceptor.Logging(log),
    auth.UnaryServerInterceptor(verifier, account.GRPCScopes, notAccountService, log),
    faults.UnaryServerInterceptor(notAccountService),
  ))
```

Urutan *interceptor* ini menentukan hasil pengukuran. `UnaryServerCorrelation` berada paling depan agar semua *interceptor* berikutnya menulis log dengan *correlation ID*. `Metrics` berada sebelum `auth` dan *fault injector*, sehingga durasi yang tercatat mencakup penolakan dan *delay* simulasi, sama seperti yang dialami pemanggil.

Ubah juga log di dalam penanganan *request* menjadi varian `...Context`: `mutate` dan `toStatus` di `internal/account/grpc.go`, termasuk log `caller gave up before the request finished` dari Lab 4. `toStatus` kini menerima `ctx` sebagai parameter pertama, sehingga semua pemanggilnya ikut diubah, termasuk pemanggilan saat `parseAmount` gagal di awal `mutate` (`s.toStatus(ctx, err)`). Lakukan hal yang sama pada ketiga log `grpc call rejected` dan log `no scope configured` di `internal/auth/grpc.go`, serta log di `internalError` dan `readyz` pada `internal/account/http.go`. `interceptor.Logging` sudah memanggil `log.Log(ctx, ...)`, sehingga tidak perlu diubah.

Ambil *unit test* `account` dari *checkpoint*, lalu jalankan pengujian:

```
git checkout lab-08 -- \
  account/config_test.go \
  account/internal/telemetry/correlation_test.go \
  account/internal/interceptor/metrics_test.go
cd account && go mod tidy && make test && cd ..
```

8.4.11 Telemetri di `notification`

`notification` tidak menggunakan gRPC. Salin paket dari `transfer`, termasuk `kafka.go`:

```
mkdir -p notification/internal/telemetry
cp transfer/internal/telemetry/{telemetry.go,log.go,correlation.go,http.go,kafka.go}
notification/internal/telemetry/
```

Di `notification/internal/telemetry/correlation.go`, hapus `CorrelationMetadataKey` dan `UnaryClientCorrelation` beserta *import* `grpc` dan `metadata`. Di `notification/main.go`, lakukan perubahan yang sama dengan `transfer`. Karena `Routes()` mengembalikan `*http.ServeMux`, `/metrics` dapat didaftarkan langsung:

```
mux := notification.NewHandler(store, log).Routes()
// Lab 8: Prometheus pulls the metrics from here. Open like the probes;
// production restricts it to the monitoring network.
mux.Handle("GET /metrics", promhttp.Handler())
srv := &http.Server{
  Addr:              cfg.HTTPAddr,
```

```
    Handler:           telemetry.HTTPHandler(mux),
    ReadHeaderTimeout: 5 * time.Second,
  }
```

Consumer membaca *trace context* dan *correlation ID* dari *header* Kafka, lalu memproses *event* di dalam *span* `process transfer-events`. Di `notification/internal/notification/consumer.go`, tambahkan *field* `tracer trace.Tracer` di `Consumer`, isi di `NewConsumer` dengan `otel.Tracer("notification/consumer")`, dan ganti `c.handle(ctx, m)` di `Run` menjadi `c.process(ctx, m)`. Fungsi `process`:

```
// process handles m inside a "process" span that continues the trace of the
// transfer: the outbox relay put the trace context (traceparent) and the
// correlation ID into the Kafka headers.
func (c *Consumer) process(ctx context.Context, m kafka.Message) error {
  headers := m.Headers
  ctx = otel.GetTextMapPropagator().Extract(ctx, telemetry.KafkaHeaders{Headers: &headers})
  ctx = telemetry.WithCorrelationID(ctx,
    telemetry.AcceptCorrelationID(header(m, telemetry.CorrelationKafkaHeader)))
  ctx, span := c.tracer.Start(ctx, "process "+m.Topic,
    trace.WithSpanKind(trace.SpanKindConsumer),
    trace.WithAttributes(
      attribute.String("messaging.system", "kafka"),
      attribute.String("messaging.operation.type", "process"),
      attribute.String("messaging.destination.name", m.Topic),
      attribute.Int("messaging.destination.partition.id", m.Partition),
      attribute.Int64("messaging.kafka.offset", m.Offset),
      attribute.String("messaging.message.id", header(m, "event_id")),
      attribute.String("correlation.id", telemetry.CorrelationID(ctx)),
    ))
  defer span.End()
  err := c.handle(ctx, m)
  if err != nil {
    span.SetStatus(codes.Error, "stopped before the event was handled")
  }
  return err
}
```

Paket `codes` di sini berasal dari `go.opentelemetry.io/otel/codes`. Hasil pemrosesan dihitung dengan `notifications_processed_total`. Tambahkan di atas `type Reader`:

```
// processed counts every event the consumer finished with (Lab 8):
//
//  sent       notification stored and sent
//  duplicate  event seen before, skipped (Lab 6); a burst after a replay or
//             a rebalance is expected
//  skipped    unreadable, invalid, rejected by the database, or without a
//             notification; should stay at 0
var processed = promauto.NewCounterVec(prometheus.CounterOpts{
  Name: "notifications_processed_total",
  Help: "Events handled by the notification consumer, by result.",
}, []string{"result"})

func init() {
  for _, r := range []string{"sent", "duplicate", "skipped"} {
    processed.WithLabelValues(r)
  }
}
```

Di `handle`, ambil *span* dengan `span := trace.SpanFromContext(ctx)` di awal fungsi, lalu di setiap cabang hasil naikkan *counter* yang sesuai dan ubah log menjadi varian `...Context`:

Cabang di `handle`	Tambahan
Payload tidak terbaca (`skipping unreadable event`)	`processed.WithLabelValues("skipped").Inc()` dan `span.SetStatus(codes.Error, "unreadable event")`
`event_id` atau `transfer_id` bukan UUID (`skipping event with invalid event_id or transfer_id`, Lab 5)	`processed.WithLabelValues("skipped").Inc()` dan `span.SetStatus(codes.Error, "invalid event_id or transfer_id")`
Setelah kedua ID lolos pemeriksaan	`span.SetAttributes(attribute.String("transfer.id", e.TransferID))`
Jenis *event* tanpa notifikasi (`skipping event type without notification`)	`processed.WithLabelValues("skipped").Inc()`; log menjadi `log.WarnContext(ctx, ...)`
Duplikat (`duplicate event skipped`)	`processed.WithLabelValues("duplicate").Inc()` dan `span.SetAttributes(attribute.Bool("event.duplicate", true))`
Berhasil (`notification sent`)	`processed.WithLabelValues("sent").Inc()`
Ditolak database, `permanentDBError` (`skipping event the database rejects`, Lab 5)	`processed.WithLabelValues("skipped").Inc()` dan `span.SetStatus(codes.Error, "event rejected by the database")`

Keempat cabang `skipped` mencatat *event* yang dilewati tanpa notifikasi. Nilai normalnya 0. Kenaikan nilai ini berarti ada *event* rusak atau berjenis tidak dikenal yang perlu diperiksa dari log `ERROR` atau `WARN` dengan pesan `skipping ...`. Di *production*, *event* seperti ini biasanya dikirim ke *dead-letter topic* untuk diperiksa petugas.

Log di dalam *loop* penyimpanan juga diubah: `skipping event the database rejects` menjadi `log.ErrorContext(ctx, ...)` dan `storing notification failed, retrying` menjadi `log.WarnContext(ctx, ...)`.

Ubah juga log `request failed` di `list` dan log di `readyz` pada `internal/notification/http.go` menjadi varian `...Context` (dengan `r.Context()`). Setelah itu ambil *unit test* `notification` dari *checkpoint* dan jalankan seluruh pengujian:

```
git checkout lab-08 -- \
  notification/config_test.go \
  notification/internal/notification/tracing_test.go
cd notification && go mod tidy && make test && cd ..
make test
```

8.4.12 Menelusuri Satu Transfer di Tempo

Jalankan ketiga service di tiga terminal, masing-masing dari folder `studi-kasus`:

```
make -C account run
make -C transfer run
make -C notification run
```

Kirim satu transfer dengan *correlation ID* buatan sendiri:

```
TOKEN=$(make -s token)
curl -s -i -X POST localhost:28082/transfers \
```

```
  -H "Authorization: Bearer $TOKEN" -H "X-Correlation-ID: lab8-1" \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"1000"}'
```

Respons membawa *correlation ID* yang sama. Go menulis nama *header* dalam *canonical form* (huruf besar di awal setiap kata), sehingga yang tampil adalah `X-Correlation-Id`:

```
HTTP/1.1 201 Created
Content-Type: application/json
X-Correlation-Id: lab8-1
...

{"id":"...","from_account":"1111111111","to_account":"2222222222",
 "amount":"1000","status":"COMPLETED",...}
```

Log ketiga service kini berbentuk JSON, dan baris yang ditulis dengan *context* milik *request* atau *event* membawa ID yang sama. Contoh berikut dipotong dan dipecah agar muat di halaman, sedangkan di terminal setiap *record* tertulis dalam satu baris:

```
account
{"level":"INFO","msg":"account mutated","number":"1111111111","type":"DEBIT",...,
 "correlation_id":"lab8-1","trace_id":"122d873a...","span_id":"c65886ff..."}
transfer
{"level":"INFO","msg":"transfer completed","transfer_id":"...","amount":"1000",
 "correlation_id":"lab8-1","trace_id":"122d873a...","span_id":"..."}
notification
{"level":"INFO","msg":"notification sent",...,
 "correlation_id":"lab8-1","trace_id":"122d873a...","span_id":"..."}
```

Di Grafana, buka *Explore*, pilih *data source* Tempo, lalu jalankan *query* TraceQL berikut:

```
{ span.correlation.id = "lab8-1" }
```

Trace yang muncul berisi *span* dari ketiga service, dengan susunan seperti diagram di bagian Gambaran. Pencarian Tempo dapat tertinggal beberapa detik dari *request*. Sebaliknya, *trace* yang dibuka langsung dengan `trace_id` dari baris log selalu tampil seketika.

8.4.13 Latihan: Menemukan Service yang Lambat

Latihan ini mensimulasikan insiden ketika `account` menjadi lambat. Peserta mencari penyebabnya hanya dari *dashboard*, *trace*, dan log. Buka *dashboard Studi Kasus: Transfer* (Grafana → *Dashboards* → *Studi Kasus*). *Dashboard* diperbarui setiap 5 detik dan menampilkan 15 menit terakhir.

Panel	***Query* Utama**
Transfer per status	`sum by (status) (rate(transfers_total[1m]))`
Latensi p95 REST per service	`histogram_quantile(0.95, ...)` atas `http_server_request_duration_seconds_bucket` per `job`
Circuit breaker transfer → account	`circuit_breaker_state{target="account"}`, ditampilkan sebagai `CLOSED`, `HALF-OPEN`, `OPEN`
Perubahan status circuit breaker	`increase(circuit_breaker_transitions_total[1m])` per `from` dan `to`
Outbox pending	`outbox_pending`
Notifikasi per hasil	`sum by (result) (rate(notifications_processed_total[1m]))`

Latensi p95 gRPC account per method	`histogram_quantile(0.95, ...)` atas `grpc_server_request_duration_seconds_bucket` per `method`
Latensi p95 REST per route	Seperti latensi REST, per `route`
Target scrape	`up`: `UP` bila Prometheus berhasil mengambil `/metrics` service

Di terminal keempat, jalankan beban transfer terus-menerus:

```
TOKEN=$(make -s token)
while true; do
  curl -s -o /dev/null -w "%{http_code}\n" -X POST localhost:28082/transfers \
    -H "Authorization: Bearer $TOKEN" \
    -d '{"from_account":"1111111111","to_account":"2222222222","amount":"1000"}'
  sleep 0.5
done
```

Setelah satu atau dua menit, ketika panel *Transfer per status* menunjukkan laju `COMPLETED` yang stabil, jalankan gangguan di terminal lain:

```
make fault-slow     # account menahan setiap panggilan 3 detik
make breaker        # setelah beberapa transfer: {"account":"open"}
make metrics
```

Amati hasilnya secara berurutan. *Query* untuk Tempo dan Loki ada di bawah tabel.

No.	Tempat	Yang Terlihat
1	Terminal beban	Respons berubah dari `201` menjadi `502` (transfer `FAILED` setelah semua percobaan habis), lalu `503` (ditolak *circuit breaker* yang berstatus `open`)
2	*Dashboard*	Latensi p95 gRPC `Debit` naik ke kisaran 2 detik, *circuit breaker* menjadi `OPEN`, dan laju `FAILED` naik
3	*Explore* → Tempo	*Trace* yang terjadi sebelum *circuit breaker* berstatus `open` berisi empat *span* `Debit` yang masing-masing sekitar 2 detik, yaitu satu percobaan awal ditambah tiga *retry*. *Span* server di `account` sama panjangnya dan membawa *event* `fault injected: delay`
4	*Explore* → Loki	Log `retrying account call` dan `transfer not completed` dari *trace* yang sama

Query TraceQL di Tempo untuk transfer yang gagal:

```
{ span.transfer.status = "FAILED" }
```

Query Loki untuk semua log satu *trace*, dengan `trace_id` disalin dari Tempo atau dari log:

```
{service_name=~"account|transfer|notification"} | trace_id="<trace_id>"
```

Beberapa hasil perlu dibaca dengan cermat:

- Latensi `Debit` tercatat sekitar 2 detik, padahal *delay* simulasi 3 detik. `transfer` memutus setiap percobaan setelah 2 detik (*timeout* per percobaan dari Lab 3). *Context* di `account` ikut dibatalkan,

sehingga *span* server juga berhenti di sekitar 2 detik. Nilai p95 di panel diperkirakan dari *histogram bucket* (batas 1 detik dan 2,5 detik), sehingga angkanya berada di antara kedua batas tersebut.
- Latensi p95 REST `transfer` dapat terlihat rendah ketika *circuit breaker* berstatus `open`. Transfer yang ditolak *circuit breaker* selesai dalam hitungan milidetik, karena panggilan ke `account` tidak pernah dikirim. Latensi yang rendah tidak berarti sistem sehat. Justru panel *Transfer per status* menunjukkan bahwa hampir semua transfer `FAILED`. Inilah alasan *metric* bisnis dibutuhkan di samping *metric* teknis.
- *Trace* transfer yang ditolak *circuit breaker* hanya berisi *span* `POST /transfers` tanpa *span* `Debit`.
- Selama gangguan masih aktif, panel *Perubahan status circuit breaker* menunjukkan siklus `open` → `half-open` → `open`. Setelah 10 detik, panggilan berikutnya menjadi percobaan *half-open*, gagal lagi, dan *circuit breaker* kembali ke `open`.

Akhiri gangguan:

```
make fault-clear
```

Panel *circuit breaker* kembali ke `CLOSED`, dan transfer kembali `COMPLETED`. Status `HALF-OPEN` hanya berlangsung sekitar satu detik, sehingga biasanya tidak tertangkap *scrape*. Meski begitu, perpindahan `open` → `half-open` → `closed` tetap terlihat di panel *Perubahan status circuit breaker*. Perpindahan dari `open` ke `half-open` hanya terjadi pada panggilan pertama setelah 10 detik. Tanpa beban transfer, *circuit breaker* dari latihan sebelumnya tetap berstatus `open` walaupun `account` sudah normal. Hentikan *loop* beban dengan Ctrl+C.

8.5 Verifikasi

Semua pengujian lulus:

```
make test
```

Pengujian baru yang diambil dari *checkpoint*:

Pengujian	Lokasi
`TestCheckEndpointURL`	`account`, `transfer`, `notification` (paket `main`)
`TestCorrelationMiddleware`, `TestLogCarriesCorrelationAndTraceIDs`, `TestMetricsUseRoutePattern`	`transfer/internal/telemetry`
`TestMessageContinuesTheRequestTrace`	`transfer/internal/outbox`
`TestSendsCorrelationID`, `TestBreakerMetrics`	`transfer/internal/accountclient`
`TestUnaryServerCorrelation`	`account/internal/telemetry`
`TestMetricsRecordsMethodAndCode`	`account/internal/interceptor`
`TestConsumerContinuesTraceAndCounts`	`notification/internal/notification`

Service berhenti saat *startup* bila konfigurasinya salah. Bila nilai `OTEL_EXPORTER_OTLP_ENDPOINT` di `transfer/Makefile` sementara diganti menjadi `localhost:14317`, `make -C transfer run` berhenti dengan:

```
{"time":"...","level":"ERROR","msg":"transfer service
stopped","error":"OTEL_EXPORTER_OTLP_ENDPOINT must be a URL like http://localhost:14317,
got \"localhost:14317\""}
```

Nilai `http://localhost`, tanpa port, juga ditolak:

```
{"time":"...","level":"ERROR","msg":"transfer service
stopped","error":"OTEL_EXPORTER_OTLP_ENDPOINT must name the port explicitly, like http://
localhost:14317, got \"http://localhost\""}
```

Metric domain dapat dibaca langsung dari `/metrics`, tanpa Grafana. Setelah beberapa transfer berhasil, `make metrics` menampilkan kira-kira:

```
# http://localhost:28081/metrics
# http://localhost:28082/metrics
circuit_breaker_state{target="account"} 0
outbox_pending 0
transfers_total{status="COMPENSATED"} 0
transfers_total{status="COMPENSATION_FAILED"} 0
transfers_total{status="COMPLETED"} 3
transfers_total{status="FAILED"} 0
# http://localhost:28083/metrics
notifications_processed_total{result="duplicate"} 0
notifications_processed_total{result="sent"} 3
notifications_processed_total{result="skipped"} 0
```

Bagian `28081` kosong karena `account` tidak memiliki *metric* domain. *Counter* perpindahan status *circuit breaker* baru muncul setelah perpindahan pertama.

Checklist akhir:

- Panel *Target scrape* menunjukkan `UP` untuk `account`, `transfer`, dan `notification`.
- Respons `POST /transfers` mengembalikan *header* `X-Correlation-Id`. Bila pemanggil tidak mengirim *header* tersebut, nilainya berupa UUID baru.
- Satu *trace* di Tempo berisi *span* dari `transfer`, `account`, dan `notification`.
- Log di Loki dapat difilter dengan `trace_id` dan menampilkan baris dari ketiga service.
- Latihan menunjukkan *circuit breaker* `OPEN` saat `make fault-slow` dan kembali `CLOSED` setelah `make fault-clear`.

8.6 Masalah Umum

Panel *Target scrape* menunjukkan `DOWN`. Prometheus di dalam *container* tidak dapat mengambil `/metrics`. Pastikan service berjalan dan `curl localhost:28082/metrics` berhasil dari laptop. Di Docker Engine Linux, pastikan `extra_hosts: host-gateway` ada di `compose.yaml`, dan *firewall* laptop mengizinkan koneksi dari jaringan Docker ke port 28081–28083. *Query* `up` di *Explore* menunjukkan status setiap target.

Log `telemetry export failed` di *stdout*. *Collector* belum siap atau `otel-lgtm` tidak berjalan. Periksa `docker compose ps otel-lgtm`. Service tetap melayani *request*, tetapi *span* dan log selama periode tersebut hilang. Selama kode lab ini dikembangkan, beberapa kali terjadi jeda 20–30 detik setelah *binary* hasil *build* baru pertama kali dijalankan. Selama jeda tersebut *export* OTLP gagal dengan `DeadlineExceeded`, dan pada satu kejadian panggilan gRPC serta pengambilan *token* dari `transfer` juga mengalami *timeout*. Gejala tersebut tidak muncul lagi dengan *binary* yang sama. Bila terjadi, tunggu hingga jeda tersebut selesai, lalu ulangi langkah yang sedang dikerjakan.

Baris log tidak memiliki `trace_id` atau `correlation_id`. Log ditulis tanpa *context* (`log.Info` alih-alih `log.InfoContext(ctx, ...)`), atau *logger* yang digunakan masih *logger* awal dari `main`, sebelum `log = tel.Logger`.

***Trace* berhenti di `transfer` dan tidak ada *span* `process transfer-events`.** Pastikan `Insert` mengisi `trace_context` (perintah di bawah), *relay* memanggil `span.End()` setelah `WriteMessages` berhasil, dan `notification` memanggil `Extract` sebelum `tracer.Start`. *Event* yang ditulis sebelum Lab 8 memang tidak memiliki *trace context*.

```
docker compose exec db-transfer psql -U transfer -d transfer -c \
  "select id, correlation_id, trace_context from outbox order by created_at desc limit
5"
```

Panel latensi per *route* menampilkan `unmatched` untuk semua *request*. Fungsi `route` tidak membungkus *mux* secara langsung, sehingga `r.Pattern` tidak terbaca. Urutan di `HTTPHandler` harus `otelhttp` → `CorrelationMiddleware` → `route` → *mux*.

Perubahan JSON *dashboard* tidak terlihat. Grafana membaca *file dashboard* saat *startup*. Jalankan `docker compose restart otel-lgtm`. Perubahan lewat UI Grafana tidak tersimpan ke *file* dan hilang ketika *container* dibuat ulang.

Port 13000 sudah digunakan. Ada program lain di laptop yang menggunakan port tersebut. Pada Linux dan macOS, `lsof -i :13000` menampilkan proses yang menggunakan port. Hentikan program tersebut sebelum `make infra-up`.

***Circuit breaker* tetap `open` padahal `account` sudah normal.** Perpindahan ke `half-open` hanya terjadi saat ada panggilan. Kirim beberapa transfer, lalu periksa lagi dengan `make breaker`.

Pencarian di Tempo tidak menemukan *trace* yang baru dibuat. Indeks pencarian Tempo tertinggal beberapa detik. Tunggu sebentar, atau buka *trace* langsung dengan `trace_id` dari log.

8.7 Eksplorasi Mandiri

1. Hentikan Kafka dengan `docker compose stop kafka`, kirim beberapa transfer, lalu amati panel *Outbox pending* dan log `outbox publish failed`. Jalankan kembali dengan `docker compose start kafka`. Periksa di Tempo apakah *trace* transfer yang tertunda tetap tersambung ke `notification`, dan berapa lama jarak antara *span* `POST /transfers` dan *span* `publish transfer-events`.
2. Jalankan `make fault-error RATE=0.5` alih-alih `make fault-slow`. Bandingkan *trace* yang dihasilkan: berapa percobaan `Debit` yang muncul, berapa lama setiap percobaan, dan *event* apa yang tercatat di *span* server `account`.
3. Tambahkan *counter* `account_mutations_total` dengan *label* `type` (`DEBIT`, `CREDIT`) dan `applied` (`true`, `false`) di `account`, lalu tambahkan panel baru di `observability/dashboards/studi-kasus.json`. Amati panel tersebut selama beban transfer dari latihan berjalan.
4. Ganti *sampler* di `telemetry.go` menjadi `sdktrace.ParentBased(sdktrace.TraceIDRatioBased(0.1))` di `transfer` saja. Amati berapa *trace* yang tersimpan dan apakah *span* `account` serta `notification` mengikuti keputusan *sampling* dari `transfer`.

8.8 Checkpoint

Kondisi kode di akhir lab ini tersimpan di tag `lab-08`. Untuk membandingkan hasil kerja dengan *checkpoint*:

```
git diff lab-08 -- .
```

Peserta yang ingin melanjutkan ke Lab 9 dari kondisi *checkpoint*:

```
git switch -c lab-09-kerja lab-08
```

9 Lab 9: *Container* dan *Deployment* ke k3d

9.1 Tujuan

Setelah menyelesaikan lab ini, peserta mampu:

- menulis *multi-stage Dockerfile* yang menghasilkan *binary* Go statis di atas *image* kosong (`FROM scratch`), berjalan sebagai *user* non-*root* dan tetap berfungsi dengan UID apa pun yang diberikan *platform*;
- menjalankan ketiga service sebagai *container* di Docker Compose dengan *root filesystem read-only*;

- membuat *cluster* Kubernetes lokal dengan k3d, memasukkan *image* ke *cluster* tanpa *registry*, lalu menerapkan *manifest* Kustomize yang sudah disediakan;
- membaca isi *manifest* (Deployment, Service, ConfigMap, Secret, Ingress, *probe*, *security context*) dan menjelaskan alasan setiap pengaturan;
- mengubah sistem yang sedang berjalan di *cluster*: *scale*, perubahan konfigurasi, *rolling update* dengan *tag image* baru, serta *rollback*.

9.2 Gambaran

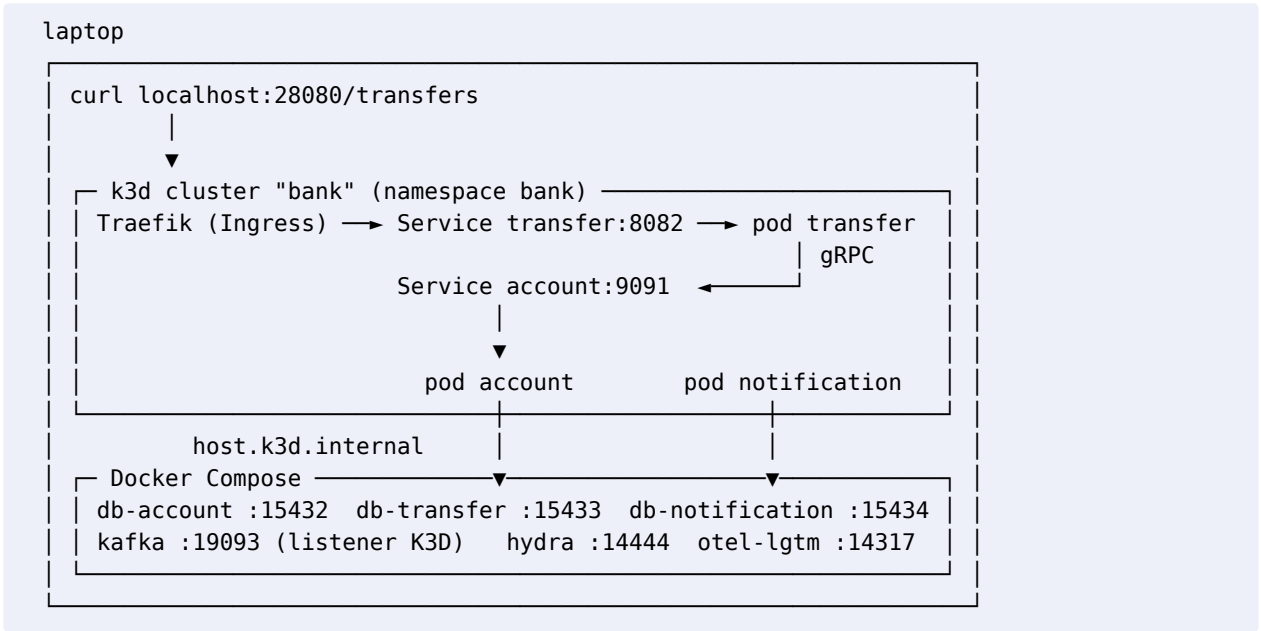
Hingga Lab 8, ketiga service dijalankan langsung di laptop dengan `make -C <service> run`, sedangkan infrastruktur (PostgreSQL, Kafka, Hydra, `otel-lgtm`) berjalan di Docker Compose. Cara tersebut cocok untuk pengembangan, tetapi tidak mewakili cara aplikasi dijalankan di *server*, tempat aplikasi dikemas sebagai *image* lalu dijalankan oleh *container platform* seperti Kubernetes atau OpenShift.

Lab ini memindahkan ketiga service ke dalam *container* dalam tiga tahap:

1. Setiap service mendapat `Dockerfile`. *Image* yang dihasilkan hanya berisi *binary* dan sertifikat CA, tanpa *shell* maupun *package manager*.
2. *Profile* `apps` di `compose.yaml` menjalankan ketiga *image* tersebut di Docker Compose. Di tahap ini, *image* diperiksa terlebih dahulu sebelum dijalankan di Kubernetes.
3. *Cluster* k3d menjalankan ketiga service sebagai Deployment. Infrastruktur tetap di Docker Compose, dan *pod* menjangkaunya melalui nama `host.k3d.internal`.

Manifest Kubernetes untuk tahap ketiga sudah disediakan di *checkpoint* `lab-09`. Peserta tidak perlu menulisnya dari awal, cukup menyalin *manifest* tersebut, membaca bagian pentingnya, menerapkannya ke *cluster*, lalu mengubahnya melalui beberapa latihan.

Susunan sistem di akhir lab:



Pembagian ini mengikuti praktik umum di *production*: aplikasi berjalan di *cluster*, sedangkan database, *message broker*, dan *authorization server* sering dikelola terpisah (*managed service* atau *cluster* khusus data).

9.2.1 Alasan Memilih Kubernetes dan k3d

Pertimbangan	Keterangan
Kebutuhan	Ketiga service harus dijalankan, diperbarui tanpa menghentikan layanan, dijalankan ulang otomatis saat macet, dan dapat menemukan alamat satu

Pertimbangan	Keterangan
	sama lain. Cara menjalankannya harus sama di laptop, di server *testing*, dan di *production*
Opsi yang dipertimbangkan	Docker Compose di beberapa VM, HashiCorp Nomad, atau Kubernetes (*vanilla*, OpenShift, atau layanan Kubernetes *managed* milik penyedia *cloud*). Untuk Kubernetes di laptop: k3d, minikube, kind, atau Kubernetes bawaan Docker Desktop
Pilihan di studi kasus	Setiap service dikemas sebagai *container image* dan dijalankan di Kubernetes. Untuk training, Kubernetes dijalankan dengan k3d
Yang diperoleh	*Image* yang sama digunakan dari laptop sampai *production*. Kubernetes bersifat deklaratif: kondisi yang diinginkan ditulis di *manifest*, lalu Kubernetes menyesuaikan isi *cluster* dengan *manifest* tersebut. Kubernetes juga menjalankan ulang *container* yang gagal (*self-healing*), menyediakan *rolling update*, nama DNS untuk setiap Service, dan *load balancing*. *Manifest* yang sama dapat digunakan di OpenShift dan di layanan Kubernetes *managed* milik penyedia *cloud*. k3d menjalankan k3s (distribusi Kubernetes yang ringan) di dalam *container* Docker: *cluster* dibuat dengan `make k3d-up` dan dihapus dengan `make k3d-down`, kebutuhan RAM-nya kecil, Traefik (*ingress controller*) sudah terpasang di k3s, *load balancer* milik k3d meneruskan *port* 28080 di laptop ke Traefik, dan `k3d image import` memasukkan *image* ke *cluster* tanpa *registry*
Yang dikorbankan	Kubernetes adalah platform tambahan yang harus dipelajari dan dioperasikan (*manifest*, RBAC, *upgrade cluster*). *Cluster* k3d hanya ditujukan untuk pengembangan dan pengujian di laptop. Komponen bawaan k3s seperti Traefik dan metrics-server belum tentu tersedia di *cluster* lain. k3s tidak sama dengan OpenShift: perbedaan seperti SCC dan Route dibahas di sesi OpenShift pada Hari 4

Dibandingkan k3d, minikube menyediakan banyak *add-on* dan dapat berjalan di dalam VM maupun *container*. kind juga menjalankan Kubernetes di dalam Docker, tetapi *Ingress* dan Service bertipe LoadBalancer membutuhkan komponen tambahan (cloud-provider-kind) yang dijalankan terpisah. Kubernetes bawaan Docker Desktop hanya tersedia bila menggunakan Docker Desktop.

9.3 Titik Awal

Lanjutkan dari kode hasil Lab 8. Peserta yang tertinggal mengambil *checkpoint* Lab 8:

```
git checkout lab-08
```

Lab ini membutuhkan Docker, k3d, dan kubectl yang sudah dipasang sesuai panduan persiapan. Periksa versinya:

```
docker version --format '{{.Server.Version}}'
k3d version
kubectl version --client
```

Semua perintah di lab ini dijalankan dari direktori `studi-kasus/`, kecuali jika disebutkan lain.

9.4 Langkah

9.4.1 Menghentikan Service yang Berjalan di Laptop

Hentikan ketiga service yang masih berjalan dari Lab 8 (tekan `Ctrl+C` di terminal tempat `make -C account run`, `make -C transfer run`, dan `make -C notification run` dijalankan). Ada dua alasan:

- port 28081–28083 di laptop akan digunakan oleh *container* di *profile* `apps`;
- di k3d, `notification` yang kedua akan bergabung ke *consumer group* yang sama dan berbagi *partition* dengan `notification` di laptop, sedangkan dua *outbox relay* `transfer` membaca tabel `outbox` yang sama. Hasilnya sulit dibaca karena sebagian *event* diproses oleh proses di laptop.

Infrastruktur di Docker Compose tetap berjalan.

9.4.2 *Dockerfile* untuk Service `account`

Image yang di-*build* di lab ini harus memenuhi tiga syarat:

- **Kecil dan minim isi.** *Image* akhir hanya berisi *binary* dan sertifikat CA. Tanpa *shell* dan *package manager*, tidak ada komponen lain yang perlu di-*patch* dan tidak ada *tools* yang dapat digunakan penyerang di dalam *container*.
- ***Binary* statis.** `CGO_ENABLED=0` menghasilkan *binary* yang tidak bergantung pada `libc`, sehingga dapat berjalan di *image* `scratch` yang kosong.
- **Non-*root* dengan UID numerik.** Kubernetes hanya dapat memeriksa `runAsNonRoot` terhadap angka, dan OpenShift dengan *restricted SCC* (*Security Context Constraints*) menjalankan *container* dengan UID acak dari rentang milik *namespace*. *Binary* yang tidak menulis ke *file* apa pun tetap berjalan dengan UID mana pun.

Multi-stage build memisahkan tahap kompilasi dari *image* akhir. Tahap pertama (`build`) menggunakan `golang:1.27.2-alpine` yang berisi *compiler*; tahap kedua (`FROM scratch`) hanya menyalin hasil kompilasi dari tahap pertama.

Build context adalah direktori `studi-kasus/`, satu tingkat di atas direktori service. Alasannya, `go.mod` service `account` dan `transfer` memiliki *replace directive* ke `../proto`, sehingga modul `proto` harus ikut masuk ke *build context*. `notification` tidak menggunakan `proto`, tetapi tetap di-*build* dengan *build context* yang sama agar ketiga *image* di-*build* dengan perintah yang sama.

Buat *file* `account/Dockerfile`:

```
# Multi-stage build of the account service. Build context: studi-kasus/ (not this
# directory), so the proto module that go.mod's replace directive points to
# (../proto) is part of the context:
#   docker build -f account/Dockerfile -t studi-kasus/account:1.0.0 .
FROM golang:1.27.2-alpine AS build

# GOWORK=off: build this module with its own go.mod/go.sum, not the workspace.
# The image then depends only on this service and proto; a change in another
# service's go.mod does not invalidate the cache here, and the CI pipeline of
# one service builds it alone (Lab 10).
# CGO_ENABLED=0: a static binary without libc, which runs in an empty image.
ENV GOWORK=off CGO_ENABLED=0

# Module files first: the download layer stays cached until go.mod/go.sum
# change, not on every source edit.
WORKDIR /src
COPY proto/go.mod proto/go.sum proto/
COPY account/go.mod account/go.sum account/
WORKDIR /src/account
RUN go mod download

COPY proto/ /src/proto/
```

```
COPY account/ /src/account/
# -trimpath drops the build machine's paths from the binary (reproducible,
# nothing about the builder leaks); -s -w drops the symbol table and DWARF
# debug data (smaller image; stack traces still have function names and lines).
RUN go build -trimpath -ldflags "-s -w" -o /out/account .

# Empty base image: no shell, no package manager, nothing to patch but the
# binary. The trade-off: no docker exec into a shell for debugging; use
# kubectl debug with an ephemeral container instead.
FROM scratch
# CA certificates for outgoing TLS (an https:// OAuth or OTLP endpoint in
# production). The lab endpoints are plain HTTP, but an image without them
# fails only later, on the first TLS connection.
COPY --from=build /etc/ssl/certs/ca-certificates.crt /etc/ssl/certs/
COPY --from=build /out/account /account
# Numeric UID:GID, not a name: scratch has no /etc/passwd, and Kubernetes can
# only check runAsNonRoot against a number. 65532 is the conventional
# "nonroot" ID. Nothing in the image needs a writable or owned path, so the
# binary also runs under the arbitrary UID that OpenShift's restricted SCC
# assigns, and with readOnlyRootFilesystem.
USER 65532:65532
# HTTP 8081 (REST, probes, /metrics) and gRPC 9091. Documentation only: the
# ports come from the environment (HTTP_ADDR, GRPC_ADDR).
EXPOSE 8081 9091
ENTRYPOINT ["/account"]
```

Beberapa hal yang perlu diperhatikan dari *file* tersebut:

- **Urutan `COPY`.** `go.mod` dan `go.sum` disalin dan `go mod download` dijalankan sebelum kode program disalin. Docker menyimpan setiap *layer* di *cache*. Selama `go.mod` tidak berubah, *layer* hasil `go mod download` digunakan ulang, sehingga *build* setelah perubahan kode hanya mengulang kompilasi.
- **`GOWORK=off`.** Di laptop, `go.work` menggabungkan ketiga modul. Di dalam *image*, setiap service dikompilasi hanya dengan `go.mod` miliknya, sehingga *image* satu service tidak terpengaruh perubahan dependensi service lain.
- **Tanpa *shell*.** `docker exec -it ... sh` tidak dapat digunakan pada *image* ini. Untuk *debugging* di Kubernetes, gunakan `kubectl debug` dengan *ephemeral container*, yaitu *container* sementara yang ditambahkan ke *pod* yang sedang berjalan (lihat Eksplorasi Mandiri).

9.4.3 *Dockerfile* untuk `transfer` dan `notification`

Kedua service lain menggunakan struktur yang sama dan hanya berbeda pada nama direktori, nama *binary*, dan port. Salin *file* milik `account`, lalu ganti nama service:

```
sed 's/account/transfer/g' account/Dockerfile > transfer/Dockerfile
sed 's/account/notification/g' account/Dockerfile > notification/Dockerfile
```

Perintah `sed` di atas tidak mengubah port. Di `transfer/Dockerfile`, ganti dua baris komentar `# HTTP 8081 ...` beserta baris `EXPOSE` di bawahnya (tepat di atas `ENTRYPOINT`) menjadi:

```
# HTTP 8082. Documentation only: the ports come from the environment
# (HTTP_ADDR).
EXPOSE 8082
```

Lakukan hal yang sama di `notification/Dockerfile`:

```
# HTTP 8083. Documentation only: the ports come from the environment
# (HTTP_ADDR).
EXPOSE 8083
```

`EXPOSE` hanya berfungsi sebagai dokumentasi. Port yang sebenarnya dibuka ditentukan oleh `HTTP_ADDR` dan `GRPC_ADDR` dari *environment variable*.

`notification` membutuhkan satu perubahan lagi. Service ini tidak mengimpor modul `proto`, karena *event* dikirim sebagai JSON di Kafka, sehingga `go.mod`-nya tidak memiliki *replace directive* ke `../proto`. Di `notification/Dockerfile`, hapus kedua baris yang menyalin `proto` (`COPY proto/go.mod proto/go.sum proto/` dan `COPY proto/ /src/proto/`), lalu sesuaikan komentarnya. Bagian atas *file* menjadi:

```
# Multi-stage build of the notification service. Build context: studi-kasus/ (not this
# directory), the same as for account and transfer, so make images builds all
# three the same way. Notification does not import the proto module (the
# events are JSON on Kafka), so only notification/ is copied:
#   docker build -f notification/Dockerfile -t studi-kasus/notification:1.0.0 .
FROM golang:1.27.2-alpine AS build

# GOWORK=off: build this module with its own go.mod/go.sum, not the workspace.
# The image then depends only on this service; a change in another
# service's go.mod does not invalidate the cache here, and the CI pipeline of
# one service builds it alone (Lab 10).
# CGO_ENABLED=0: a static binary without libc, which runs in an empty image.
ENV GOWORK=off CGO_ENABLED=0

# Module files first: the download layer stays cached until go.mod/go.sum
# change, not on every source edit.
WORKDIR /src
COPY notification/go.mod notification/go.sum notification/
WORKDIR /src/notification
RUN go mod download

COPY notification/ /src/notification/
```

Bagian sesudahnya (`RUN go build ...` hingga `ENTRYPOINT`) tetap sama dengan hasil `sed`.

9.4.4 *File* `.dockerignore`

Saat `docker build` dijalankan, seluruh isi *build context* dikirim ke *builder*. Direktori `k8s/`, `observability/`, dan dokumentasi tidak dibutuhkan untuk kompilasi; jika ikut terkirim, setiap perubahan pada *file* tersebut juga membuat *cache* tidak berlaku lagi. Buat `studi-kasus/.dockerignore`:

```
# Build context is studi-kasus/ (make images, compose profile apps). Only Go
# sources and module files are needed; everything else would only slow down
# the upload to the builder and invalidate its cache.
**/bin
k8s
observability
*.md
compose.yaml
Makefile
```

9.4.5 Target `make images`

Tambahkan bagian berikut di akhir `studi-kasus/Makefile`. `VERSION` sengaja tidak memiliki nilai *default*, sehingga *tag* setiap *image* selalu ditentukan secara eksplisit saat *build*.

```
# Lab 9 helpers.

# Images for k3d. VERSION is the tag, e.g. make images VERSION=1.0.0, and is
# required: the image name says which build it is. Nothing stops a rebuild under
# an existing tag, and it rolls nothing: the image reference is unchanged, so
# running pods keep the old image while pods created later (after another
# k3d-import) get the new one. A change = a new VERSION (1.0.1), make k3d-import
# with it, then
# kubectl set image or newTag in k8s/overlays/k3d/kustomization.yaml.
# Build context is this directory: account and transfer copy proto/ next to the
# service (go.mod replace ../proto); notification does not import it.
IMAGE_REPO := studi-kasus
CLUSTER := bank
NAMESPACE := bank
IMAGES = $(foreach s,$(SERVICES),$(IMAGE_REPO)/$(s):$(VERSION))

images:
  @test -n "$(VERSION)" || { echo "VERSION is required, e.g. make images VERSION=1.0.0"
>&2; exit 1; }
  @for s in $(SERVICES); do docker build -f $$s/Dockerfile -t $(IMAGE_REPO)/$$s:
$(VERSION) . || exit 1; done
```

Baris perintah di bawah target diawali karakter *tab*, bukan spasi. Tambahkan juga `images` ke daftar `.PHONY` di bagian atas `Makefile`. Variabel `SERVICES` sudah ada sejak lab sebelumnya (`account transfer notification`).

`make` tidak mencegah *build* ulang dengan *tag* yang sudah ada. Referensi *image* di Deployment tidak berubah, sehingga tidak terjadi *rollout*. *Pod* yang sedang berjalan tetap menggunakan *image* lama, sedangkan *pod* yang dibuat kemudian (setelah `make k3d-import` ulang) menggunakan *image* baru dengan *tag* yang sama. Akibatnya, dua *pod* dengan *tag* yang sama dapat menjalankan hasil *build* yang berbeda tanpa disadari. Karena itu setiap perubahan kode mendapat `VERSION` baru (`1.0.1`, `1.0.2`, dan seterusnya), seperti di latihan *rolling update*.

Build ketiga *image*:

```
make images VERSION=1.0.0
docker images 'studi-kasus/*'
```

Hasilnya tiga *image* dengan *tag* `1.0.0`. Isi setiap *image* hanya *binary* (sekitar 21–23 MB) dan sertifikat CA. Angka yang tampil di `docker images` bergantung pada *image store* Docker:

- *containerd image store* (Docker Engine 29, Docker Desktop versi baru) menampilkan dua kolom: `CONTENT SIZE` sekitar 8–9 MB (ukuran terkompresi) dan `DISK USAGE` sekitar 31–34 MB (ukuran terkompresi ditambah hasil ekstraknya);
- *image store* klasik menampilkan satu kolom `SIZE` sekitar 21–23 MB (ukuran tanpa kompresi).

Semua angka tersebut jauh lebih kecil dibanding *image* berbasis `golang:alpine`.

Pastikan *image* dapat berjalan dengan UID sembarang dan *root filesystem read-only*, seperti di OpenShift. Tanpa *environment variable* apa pun, service harus berhenti dengan pesan yang menyebut semua variabel yang belum diisi:

```
docker run --rm --user 1000690000:0 --read-only studi-kasus/account:1.0.0
```

Output yang diharapkan berupa satu baris log JSON:

```
{"time":"...","level":"ERROR","msg":"account service stopped","error":"missing required
environment variables: HTTP_ADDR, GRPC_ADDR, DATABASE_URL, OAUTH_ISSUER, OAUTH_JWKS_URL,
OTEL_EXPORTER_OTLP_ENDPOINT, OTEL_SERVICE_NAME"}
```

Pesan tersebut membuktikan bahwa *binary* dapat dieksekusi oleh UID selain 65532, dan bahwa service langsung berhenti jika ada variabel yang kosong, tanpa menebak nilainya.

9.4.6 *Listener* Kafka untuk *Pod* di k3d

Klien Kafka bekerja dalam dua tahap. Klien terhubung ke alamat *bootstrap*, lalu *broker* mengirim daftar alamat yang di-*advertise* (*advertised listener*), dan klien membuka koneksi baru ke alamat tersebut. *Listener* `HOST` meng-*advertise* `localhost:19092`. Di dalam *pod*, `localhost` berarti *pod* itu sendiri, sehingga koneksi kedua gagal walaupun koneksi *bootstrap* berhasil.

Karena itu Kafka membutuhkan *listener* ketiga yang meng-*advertise* `host.k3d.internal:19093`. Ubah bagian `kafka` di `compose.yaml`: komentar di atasnya, tiga variabel *listener*, dan daftar `ports`.

```
  # Single-node Kafka in KRaft mode: the same process is broker and controller,
  # no ZooKeeper. Three client listeners:
  #   HOST     localhost:19092 for the services running on the laptop
  #   INTERNAL kafka:29092 for containers on the compose network (kafka-init,
  #            profile apps)
  #   K3D      host.k3d.internal:19093 for pods in the k3d cluster (Lab 9);
  #            inside the cluster "localhost" is the pod itself
  # A client connects to the bootstrap address, then reconnects to the address
  # the broker advertises for its listener; that is why each audience needs its
  # own listener.
  kafka:
    image: apache/kafka:4.3.1
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller
      KAFKA_LISTENERS: HOST://:19092,INTERNAL://:29092,K3D://:19093,CONTROLLER://:9093
      KAFKA_ADVERTISED_LISTENERS: HOST://localhost:19092,INTERNAL://kafka:29092,K3D://
host.k3d.internal:19093
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP:
HOST:PLAINTEXT,INTERNAL:PLAINTEXT,K3D:PLAINTEXT,CONTROLLER:PLAINTEXT
      # ... variabel lain tidak berubah ...
    ports:
      - "19092:19092"
      - "19093:19093"
```

Terapkan perubahan tersebut. Docker Compose membuat ulang *container* `kafka` karena konfigurasinya berubah, sedangkan data *topic* tetap tersimpan di *volume*:

```
make infra-up
docker compose ps kafka
```

Kolom `PORTS` menampilkan `19092` dan `19093`.

9.4.7 Seluruh Sistem di Docker Compose (*Profile* `apps`)

Sebelum masuk ke Kubernetes, *image* diuji terlebih dahulu di Docker Compose. Tahap ini memisahkan dua jenis masalah: masalah pada *image* (*binary* gagal dijalankan, ada yang menulis ke *filesystem*) muncul di sini, sedangkan masalah jaringan k3d baru muncul di tahap berikutnya.

Alamat yang digunakan berubah. Di dalam *container*, `localhost` merujuk ke *container* itu sendiri, sehingga setiap alamat infrastruktur diganti dengan nama *compose service* (`db-account:5432`, `kafka:29092`, `hydra:4444`, `otel-lgtm:4317`). Pengecualiannya `OAUTH_ISSUER`: nilainya tetap `http://localhost:14444/`, karena Hydra selalu menulis *issuer* yang dikonfigurasi (`URLS_SELF_ISSUER`) ke *claim*

`iss`, apa pun alamat yang digunakan untuk meminta *token*. Jika `OAUTH_ISSUER` diubah menjadi `http://hydra:4444/`, setiap *token* ditolak karena nilai `iss` tidak cocok.

`read_only: true` menerapkan *root filesystem read-only* yang sama dengan `readOnlyRootFilesystem` di Kubernetes. Service yang mencoba menulis *file* akan gagal di sini lebih dahulu.

Tambahkan juga komentar berikut di akhir komentar pembuka `compose.yaml`, tepat sebelum baris `services:`:

```
#
# Lab 9: profile "apps" runs the three services as containers too (make
# apps-up); in k3d only the services run in the cluster and reach this
# infrastructure through host.k3d.internal and the published ports.
```

Tambahkan ketiga service berikut di `compose.yaml`, di bawah `hydra-init` dan sebelum bagian `volumes:`:

```
  # Lab 9: the three services as containers (make apps-up), built from the
  # Dockerfile of each service with studi-kasus/ as build context. Same
  # variables as in the service Makefiles, but the addresses are compose
  # service names: inside a container, localhost is the container itself.
  # Exception: OAUTH_ISSUER stays http://localhost:14444/, because that is the
  # "iss" Hydra writes into every token (URLS_SELF_ISSUER), whichever address
  # the token was requested from.
  # read_only: the same read-only root filesystem as in Kubernetes
  # (readOnlyRootFilesystem), so a service that tries to write a file fails
  # here first.
  # Stop the services started with make -C <service> run first: the published
  # ports are the same.
  account:
    profiles: ["apps"]
    build:
      context: .
      dockerfile: account/Dockerfile
    read_only: true
    environment:
      HTTP_ADDR: ":8081"
      GRPC_ADDR: ":9091"
      DATABASE_URL: postgres://account:account123@db-account:5432/account
      OAUTH_ISSUER: http://localhost:14444/
      OAUTH_JWKS_URL: http://hydra:4444/.well-known/jwks.json
      OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-lgtm:4317
      OTEL_SERVICE_NAME: account
    # REST only (28081 -> 8081): the gRPC API is internal, transfer reaches it as
account:9091.
    # Published, so make fault-* and the Prometheus scrape over
    # host.docker.internal keep working.
    ports:
      - "28081:8081"
    depends_on:
      db-account:
        condition: service_healthy
      hydra:
        condition: service_healthy

  transfer:
    profiles: ["apps"]
    build:
      context: .
```

```
      dockerfile: transfer/Dockerfile
    read_only: true
    environment:
      HTTP_ADDR: ":8082"
      DATABASE_URL: postgres://transfer:transfer123@db-transfer:5432/transfer
      ACCOUNT_GRPC_ADDR: account:9091
      KAFKA_BROKERS: kafka:29092
      KAFKA_TOPIC: transfer-events
      OAUTH_ISSUER: http://localhost:14444/
      OAUTH_JWKS_URL: http://hydra:4444/.well-known/jwks.json
      OAUTH_CLIENT_ID: transfer-service
      # Lab value, visible in the repository. Production: a secret store.
      OAUTH_CLIENT_SECRET: transfer-service-secret
      OAUTH_TOKEN_URL: http://hydra:4444/oauth2/token
      OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-lgtm:4317
      OTEL_SERVICE_NAME: transfer
    # Docker sends SIGKILL 10 s after SIGTERM by default; transfer waits up to
    # 60 s for sagas in flight (shutdownGrace in main.go) plus 5 s for the
    # telemetry flush.
    stop_grace_period: 70s
    ports:
      - "28082:8082"
    depends_on:
      db-transfer:
        condition: service_healthy
      kafka:
        condition: service_healthy
      hydra:
        condition: service_healthy
      account:
        condition: service_started

  notification:
    profiles: ["apps"]
    build:
      context: .
      dockerfile: notification/Dockerfile
    read_only: true
    environment:
      HTTP_ADDR: ":8083"
      DATABASE_URL: postgres://notification:notification123@db-notification:5432/
notification
      KAFKA_BROKERS: kafka:29092
      KAFKA_TOPIC: transfer-events
      KAFKA_GROUP_ID: notification
      OTEL_EXPORTER_OTLP_ENDPOINT: http://otel-lgtm:4317
      OTEL_SERVICE_NAME: notification
    ports:
      - "28083:8083"
    depends_on:
      db-notification:
        condition: service_healthy
      kafka:
        condition: service_healthy
```

`stop_grace_period: 70s` pada `transfer` mengikuti *timeout graceful shutdown* `transfer`. Setelah menerima `SIGTERM`, `transfer` menunggu transfer yang sedang berjalan hingga 60 detik agar tidak ada *saga* yang terpotong di antara *debit* dan *credit*. Batas ini ditetapkan oleh konstanta `shutdownGrace` di

`transfer/main.go` pada *checkpoint* `lab-08`. Nilainya mencakup durasi terburuk satu *saga*: *debit* dan *credit* masing-masing 11,75 detik (4 percobaan × 2 detik + 3 jeda *backoff* × 1,25 detik), *reversal* 32 detik, ditambah beberapa detik untuk penulisan ke database. Jika `transfer/main.go` hasil kerja peserta masih menggunakan `10*time.Second` sebagai *timeout* `srv.Shutdown`, tambahkan konstanta berikut di atas `func main()`:

```
// shutdownGrace is how long the HTTP server waits on SIGTERM for transfers in
// flight. A saga cut mid-way leaves a PENDING or DEBITED row that nothing
// resumes (see README), so the wait covers the worst case of one saga under
// accountclient.ProductionSettings:
//
//  debit    4 attempts x 2 s + 3 backoffs x 1.25 s  = 11.75 s
//  credit   same                                    = 11.75 s
//  reversal 30 s retry window + one last 2 s attempt = 32 s
//                                                      55.5 s
//
// plus a few seconds for the database writes in between. The platform must
// give the process at least this long before SIGKILL (Kubernetes:
// terminationGracePeriodSeconds).
const shutdownGrace = 60 * time.Second
```

lalu ganti `10*time.Second` pada `context.WithTimeout` sebelum `srv.Shutdown` dengan `shutdownGrace`.

Docker secara *default* mengirim `SIGKILL` 10 detik setelah `SIGTERM`, sehingga tanpa pengaturan ini *saga* yang sedang berjalan dapat terhenti di tengah.

Karena berada di *profile* `apps`, ketiga service tersebut tidak ikut berjalan saat `make infra-up` dan baru berjalan jika *profile* tersebut diaktifkan. Sisipkan kedua target berikut tepat di bawah komentar `# Lab 9 helpers.` di `Makefile`, sebelum `IMAGE_REPO :=`, dan tambahkan keduanya ke daftar `.PHONY`:

```
# The three services as containers next to the infrastructure (compose profile
# apps), rebuilt from the Dockerfiles. Run make infra-up first, and stop the
# services started with make -C <service> run: the ports are the same.
apps-up:
  docker compose --profile apps up -d --build account transfer notification

apps-down:
  docker compose --profile apps rm -sf account transfer notification
```

Ubah juga `infra-down` dan `infra-reset`. Perintah `docker compose down` melewati service dari *profile* yang tidak aktif, dan *container* service yang tertinggal mencegah *network* dihapus:

```
# --profile init/apps: down skips services of inactive profiles, and a leftover
# kafka-init or service container would keep the network from being removed.
infra-down:
  docker compose --profile init --profile apps down

# Removes the database and Kafka volumes too: all lab data is lost.
infra-reset:
  docker compose --profile init --profile apps down -v
```

Jalankan dan coba satu transfer. Port yang digunakan sama dengan Lab 8, sehingga perintah dari lab sebelumnya tetap berlaku:

```
make apps-up
docker compose ps account transfer notification
TOKEN=$(make -s token)
```

```
curl -i -X POST localhost:28082/transfers -H "Authorization: Bearer $TOKEN" \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
make apps-down
```

Transfer yang berhasil menghasilkan `HTTP/1.1 201 Created` dengan `"status":"COMPLETED"`. Jika rekening `1111111111` atau `2222222222` belum ada (misalnya setelah `make infra-reset`), buat terlebih dahulu dengan `POST /accounts` seperti di Lab 1, misalnya `curl -X POST localhost:28081/accounts -d '{"number":"1111111111","owner_name":"Budi","initial_balance":"1000000"}'`.

Jalankan `make apps-down` sebelum melanjutkan. Jika *container* `notification` dari *profile* `apps` tetap berjalan, *container* tersebut berbagi *consumer group* dengan *pod* `notification` di k3d.

9.4.8 *Manifest* Kubernetes

Manifest untuk lab ini sudah disediakan. Ambil direktori `k8s/` dan `README.md` dari *checkpoint* `lab-09` tanpa mengubah *file* lain. `README.md` berisi bagian Lab 9 sebagai bahan rujukan:

```
git checkout lab-09 -- k8s README.md
```

Strukturnya mengikuti pola Kustomize *base* dan *overlay*:

```
k8s/
  base/                 sama di semua environment
    kustomization.yaml
    account/            configmap, secret (kosong), deployment, service
    transfer/           configmap, secret (kosong), deployment, service, ingress
    notification/       configmap, secret (kosong), deployment, service
  overlays/k3d/         namespace bank, alamat infrastruktur, credential lab, tag image
    kustomization.yaml
    namespace.yaml
    config.yaml
    secrets.yaml
```

Base berisi konfigurasi yang berlaku sama di semua *environment*: k3d di laptop, *server* latihan bersama di Lab 10, dan *production*. *Base* tidak dapat di-*deploy* sendiri, karena alamat infrastruktur, *credential*, *tag image*, dan *namespace* diisi di *overlay*. Jika ada *key* yang tidak diisi di *overlay*, service berhenti saat *startup* dengan pesan `missing required environment variables: ...`, dan *pod* masuk status `CrashLoopBackOff`. Perilaku ini lebih mudah didiagnosis dibandingkan service yang berjalan dengan nilai tebakan.

Hasil gabungan *base* dan *overlay* dapat dilihat tanpa menerapkannya ke *cluster*:

```
kubectl kustomize k8s/overlays/k3d | less
```

Bagian berikut membahas pengaturan penting di *manifest* tersebut. Baca *file*-nya sambil mengikuti penjelasan.

9.4.8.1 Deployment: *Probe* dan *Rolling Update*

Kutipan dari `k8s/base/account/deployment.yaml`:

```
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
  ...
          readinessProbe:
```

```
            httpGet:
              path: /readyz
              port: http
            periodSeconds: 5
            timeoutSeconds: 3
            failureThreshold: 2
          livenessProbe:
            httpGet:
              path: /healthz
              port: http
            periodSeconds: 10
            failureThreshold: 3
```

Konsep *liveness* dan *readiness*, termasuk tabel kondisinya, sudah dibahas di Lab 1 (bagian 1.4.7). *Manifest* di atas menghubungkan kedua *endpoint* tersebut dengan tindakan Kubernetes:

Probe	Pemeriksaan	Tindakan Kubernetes saat Gagal
`readinessProbe`	`GET /readyz` setiap 5 detik dengan batas waktu 3 detik. Dianggap gagal setelah 2 kegagalan berturut-turut	*Pod* dikeluarkan dari daftar *endpoint* Service sehingga tidak menerima *request* baru, tetapi *container* tidak di-*restart*. Kolom `READY` di `kubectl get pods` menjadi `0/1`. *Pod* kembali menerima *request* setelah `/readyz` kembali berhasil
`livenessProbe`	`GET /healthz` setiap 10 detik. Dianggap gagal setelah 3 kegagalan berturut-turut	kubelet menghentikan lalu menjalankan ulang *container*. Kolom `RESTARTS` bertambah

`/readyz` hanya memeriksa database milik service itu sendiri. `transfer` tidak memeriksa `account` maupun Kafka di *readiness probe*, karena gangguan pada keduanya sudah ditangani *circuit breaker* dan *outbox*. `/healthz` tidak memeriksa dependensi apa pun, sehingga database yang mati tidak memicu *restart* massal. *Readiness probe* diberi `timeoutSeconds: 3`, lebih panjang dari batas 2 detik untuk *ping* database di `/readyz`. Dengan begitu, hasil *ping* yang menentukan *pod* siap atau tidak. *Liveness probe* tetap menggunakan nilai bawaan Kubernetes (`timeoutSeconds: 1`), karena `/healthz` tidak menunggu apa pun.

Manifest tidak memuat `startupProbe`, karena *service* Go di studi kasus selesai *startup* dalam waktu kurang dari satu detik. *Service* dengan *startup* lambat menambahkan `startupProbe`. Selama *probe* ini belum berhasil, kubelet menunda *liveness* dan *readiness probe*, sehingga *container* tidak di-*restart* sebelum selesai *startup*.

- **`maxSurge: 1`, `maxUnavailable: 0`.** Saat *rolling update*, Kubernetes membuat satu *pod* baru terlebih dahulu dan baru menghentikan *pod* lama setelah *pod* baru lolos *readiness probe*. Jika *image* baru gagal berjalan, *rollout* terhenti dan *pod* lama tetap melayani *request*.

9.4.8.2 Deployment: Menghentikan *Pod* dengan Aman

Kutipan lain dari `k8s/base/account/deployment.yaml`:

```
      terminationGracePeriodSeconds: 30
      containers:
        - name: account
          ...
          lifecycle:
            preStop:
```

```
            sleep:
              seconds: 5
```

Saat *pod* dihentikan (*rolling update*, *scale down*), Kubernetes mengeluarkan *pod* dari daftar *endpoint* Service dan dari Traefik bersamaan dengan saat *pod* mulai dihentikan. Akibatnya, selama beberapa detik *request* baru masih dikirim ke *pod* tersebut. Tanpa jeda, server di dalam *pod* sudah mulai ditutup dan menolak *request* tersebut.

Hook `preStop` dijalankan kubelet sebelum `SIGTERM`. Aksi `sleep` menunda `SIGTERM` selama 5 detik untuk memberi waktu hingga *pod* tidak lagi menerima *request* baru. Setelah itu, *graceful shutdown* di `main.go` menyelesaikan *request* yang sedang diproses. Aksi `sleep` bawaan Kubernetes (tersedia sejak versi 1.30) tidak membutuhkan *shell*. Bentuk `exec` dengan perintah `sleep 5` tidak dapat digunakan, karena *image* `scratch` tidak memiliki *shell* maupun *binary* `sleep`.

`terminationGracePeriodSeconds` dihitung sejak `preStop` dimulai. Setelah batas tersebut lewat, kubelet mengirim `SIGKILL`. Nilainya harus mencakup jeda `preStop`, *timeout shutdown* service, dan 5 detik untuk mengirim sisa telemetri:

Service	`terminationGracePeriodSeconds`	Rincian
`account`	30	5 detik `preStop` + 10 detik untuk panggilan HTTP dan gRPC yang sedang berjalan + 5 detik telemetri, ditambah cadangan
`notification`	30	5 detik `preStop` + 10 detik untuk *request* HTTP yang sedang diproses (setelah *consumer* Kafka dihentikan) + 5 detik telemetri, ditambah cadangan
`transfer`	75	5 detik `preStop` + 60 detik `shutdownGrace` agar *saga* tidak terpotong di antara *debit* dan *credit* + 5 detik telemetri, ditambah cadangan

Nilai *default* Kubernetes (30 detik) tidak cukup untuk `transfer`. Jika `shutdownGrace` di `transfer/main.go` diperbesar, `terminationGracePeriodSeconds` harus ikut diperbesar.

9.4.8.3 *Security Context* tanpa `runAsUser`

```
      securityContext:
        runAsNonRoot: true
        seccompProfile:
          type: RuntimeDefault
      ...
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
```

Kombinasi ini sesuai dengan *Pod Security Standard* `restricted` di Kubernetes dan dengan *restricted SCC* di OpenShift. `runAsUser` sengaja tidak diisi. *Image* sudah berjalan sebagai UID 65532 karena instruksi `USER` di `Dockerfile`, sedangkan OpenShift menetapkan UID sendiri dari rentang milik *namespace* dan menolak *pod* yang meminta UID tetap di luar rentang tersebut. `runAsNonRoot: true` membuat kubelet menolak menjalankan *image* yang *user*-nya `root`. Pemeriksaan ini hanya dapat dilakukan karena `USER` di `Dockerfile` berupa angka.

9.4.8.4 Service dan Koneksi gRPC

Service `account` (`k8s/base/account/service.yaml`) membuka dua port: 8081 untuk REST dan 9091 untuk gRPC. ConfigMap `transfer` mengisi `ACCOUNT_GRPC_ADDR` dengan `account:9091`, dan DNS *cluster* me-*resolve* nama pendek `account` ke Service di *namespace* yang sama.

Service bertipe ClusterIP melakukan *load balancing* per koneksi TCP. gRPC menggunakan satu koneksi HTTP/2 yang terus terbuka per klien, sehingga semua panggilan dari satu *pod* `transfer` tetap menuju *pod* `account` yang sama, walaupun `account` di-*scale* menjadi beberapa *replica*. *Load balancing* per panggilan membutuhkan *headless Service* dengan *load balancing* `round_robin` di sisi klien, atau *service mesh*.

9.4.8.5 Ingress

`k8s/base/transfer/ingress.yaml` meneruskan *path* `/transfers` ke Service `transfer`. k3s menyertakan Traefik sebagai *ingress controller*, dan `make k3d-up` memetakan port 28080 di laptop ke port 80 Traefik. Hanya `/transfers` yang diteruskan, sedangkan *probe*, `/metrics`, dan `/debug/circuit-breaker` tetap berada di dalam *cluster* dan dijangkau dengan `kubectl port-forward`. OpenShift menggunakan objek Route untuk keperluan yang sama, tetapi juga menerima Ingress dan membuat Route dari objek tersebut.

9.4.8.6 *Overlay* k3d: Alamat Infrastruktur

`k8s/overlays/k3d/config.yaml` menambahkan *key* khusus k3d ke ConfigMap dari *base*:

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: transfer
data:
  KAFKA_BROKERS: host.k3d.internal:19093
  KAFKA_TOPIC: transfer-events
  OAUTH_ISSUER: http://localhost:14444/
  OAUTH_JWKS_URL: http://host.k3d.internal:14444/.well-known/jwks.json
  OAUTH_CLIENT_ID: transfer-service
  OAUTH_TOKEN_URL: http://host.k3d.internal:14444/oauth2/token
  OTEL_EXPORTER_OTLP_ENDPOINT: http://host.k3d.internal:14317
```

`host.k3d.internal` adalah nama yang ditambahkan k3d ke DNS *cluster* dan mengarah ke laptop. Melalui nama tersebut, *pod* menjangkau port yang dibuka Docker Compose di laptop:

Infrastruktur	Alamat dari *pod*
PostgreSQL	`host.k3d.internal:15432`, `15433`, `15434` (di Secret, sebagai bagian dari `DATABASE_URL`)
Kafka	`host.k3d.internal:19093`, *listener* `K3D` dari langkah sebelumnya
Hydra	JWKS dan *token endpoint* melalui `http://host.k3d.internal:14444`; `OAUTH_ISSUER` tetap `http://localhost:14444/`
OTLP	`http://host.k3d.internal:14317`
gRPC `account`	`account:9091`, Service di *namespace* yang sama

Alasan `OAUTH_ISSUER` tetap `http://localhost:14444/` sama dengan di *profile* `apps`: nilai tersebut dibandingkan dengan *claim* `iss`, dan Hydra menulis *issuer* yang dikonfigurasi, apa pun alamat yang digunakan untuk meminta *token*. Yang berubah hanya alamat untuk mengambil *key* (JWKS) dan *token*.

Trace dan log tetap masuk ke `otel-lgtm` melalui OTLP. *Metric* dari *pod* tidak terambil, karena Prometheus di `otel-lgtm` mengambil data dari port 28081–28083 di laptop, sedangkan *pod* tidak membuka port di laptop.

9.4.8.7 *Overlay* k3d: Secret dan *Tag Image*

`k8s/overlays/k3d/secrets.yaml` mengisi `DATABASE_URL` setiap service dan `OAUTH_CLIENT_SECRET` milik `transfer` melalui `stringData`. *Credential* lab ini boleh ikut di-*commit* karena hanya berlaku untuk infrastruktur di laptop peserta. Di *environment* sungguhan, Secret tidak pernah disimpan di *repository*: Secret diambil dari *vault* atau *secret operator* (External Secrets, Sealed Secrets), atau dibuat oleh *pipeline* dari *credential store* miliknya.

Secret di Kubernetes hanya di-*encode* base64 di etcd, tidak dienkripsi, kecuali *cluster* mengaktifkan *encryption at rest*. Hak akses untuk membaca Secret diatur dengan RBAC.

Bagian `images:` di `k8s/overlays/k3d/kustomization.yaml` menetapkan *tag* ketiga *image*:

```
images:
  - name: studi-kasus/account
    newTag: "1.0.0"
  - name: studi-kasus/transfer
    newTag: "1.0.0"
  - name: studi-kasus/notification
    newTag: "1.0.0"
```

Deployment di *base* hanya menyebut nama *image* (`studi-kasus/account`) dengan `imagePullPolicy: IfNotPresent`. *Image* dimasukkan langsung ke *cluster* dengan `k3d image import`, sehingga kubelet menggunakan *image* yang sudah ada dan tidak mencoba melakukan *pull* dari *registry*.

9.4.9 Target `make` untuk k3d

Tambahkan target berikut ke bagian Lab 9 di `Makefile` (dan ke daftar `.PHONY`):

```
# One server node; port 28080 on the laptop goes to the cluster's ingress
# controller (Traefik, port 80). The k3s image is pinned; k3d itself pulls its
# helper images (k3d-proxy, k3d-tools) in the version of the k3d binary.
# k3d writes host.k3d.internal into the CoreDNS ConfigMap, but CoreDNS answers
# for it only after the ConfigMap reaches its pod, up to a minute later; pods
# started before that crash with "lookup host.k3d.internal ... no such host".
# So k3d-up waits until the cluster DNS (k3s default 10.43.0.10) resolves the
# name, asked from the server node container (its image has nslookup).
k3d-up:
  k3d cluster create $(CLUSTER) --image rancher/k3s:v1.36.5-k3s1 -p
28080:80@loadbalancer
  @echo "waiting until the cluster DNS resolves host.k3d.internal"
  @for i in $$(seq 90); do \
    docker exec k3d-$(CLUSTER)-server-0 nslookup host.k3d.internal 10.43.0.10 >/dev/null
2>&1 && exit 0; \
    sleep 2; \
  done; \
  echo "host.k3d.internal does not resolve in the cluster DNS after 180 s" >&2; exit 1

# Copies the images from the local Docker into the cluster's containerd, so no
# registry is needed (Lab 10 pushes to one).
k3d-import:
  @test -n "$(VERSION)" || { echo "VERSION is required, e.g. make k3d-import
VERSION=1.0.0" >&2; exit 1; }
  $(call k3d-load,$(IMAGES))

# $(call k3d-load,IMAGE ...) imports images into the cluster and checks that
```

```
# the node's containerd really has each one (crictl resolves the name the way
# the kubelet does). k3d image import can report success while an image never
# arrives; the pod then tries to pull it from the network (ImagePullBackOff).
# A missing image is imported once more, directly into the node.
K3D_NODE = k3d-$(CLUSTER)-server-0
k3d-load = k3d image import -c $(CLUSTER) $(1) && \
  for img in $(1); do \
    docker exec $(K3D_NODE) crictl inspecti -q $$img >/dev/null 2>&1 && continue; \
    echo "$$img not found in the cluster after import, importing again" >&2; \
    k3d image import --mode direct -c $(CLUSTER) $$img && \
    docker exec $(K3D_NODE) crictl inspecti -q $$img >/dev/null 2>&1 || \
    { echo "$$img is not in cluster $(CLUSTER) after two imports" >&2; exit 1; }; \
  done

# Applies the k3d overlay, then waits until every Deployment is rolled out
# (readiness probes pass) or reports which one is not.
k8s-apply:
  kubectl apply -k k8s/overlays/k3d
  @for s in $(SERVICES); do kubectl -n $(NAMESPACE) rollout status deployment/$$s --
timeout=120s || exit 1; done

k3d-down:
  k3d cluster delete $(CLUSTER)
```

Image k3s di-*pin* ke `rancher/k3s:v1.36.5-k3s1`. k3d me-*pull* sendiri *image* pendukungnya (`k3d-proxy` untuk *load balancer* dan `k3d-tools` untuk `image import`) dengan versi yang sama dengan *binary* k3d.

Bagian kedua `k3d-up` menunggu DNS *cluster*. k3d menulis nama `host.k3d.internal` ke ConfigMap CoreDNS, tetapi CoreDNS baru menjawab nama tersebut setelah perubahan ConfigMap dimuat oleh *pod* CoreDNS, hingga sekitar satu menit kemudian. *Pod* yang dijalankan sebelum itu berhenti dengan pesan `lookup host.k3d.internal ... no such host` dan di-*restart* berulang kali. Perulangan di atas menanyakan nama tersebut ke DNS *cluster* (alamat *default* k3s `10.43.0.10`) dari *container node* k3d setiap 2 detik, dan berhenti dengan pesan *error* jika nama tersebut belum dapat di-*resolve* setelah 180 detik.

`k3d-import` tidak berhenti pada pesan sukses `k3d image import`. Fungsi `k3d-load` memeriksa setiap *image* di *containerd* milik *node* k3d dengan `crictl inspecti`, yaitu pencarian nama yang sama dengan yang dilakukan kubelet. Pemeriksaan ini diperlukan karena `k3d image import` pernah melaporkan sukses, padahal *image* tidak sampai ke *node*. Akibatnya *pod* mencoba mengambil *image* dari jaringan dan berakhir di `ImagePullBackOff`. *Image* yang tidak ditemukan di-*import* ulang satu kali langsung ke *node* (`--mode direct`). Bila *image* tetap tidak ada, *target* berhenti dengan pesan yang menyebut nama *image* tersebut.

9.4.10 Menjalankan Sistem di k3d

Pastikan infrastruktur berjalan dan *image* sudah di-*build*, lalu buat *cluster*, masukkan *image*, dan terapkan *manifest*:

```
make infra-up
make images VERSION=1.0.0
make k3d-up
make k3d-import VERSION=1.0.0
make k8s-apply
```

`make k3d-up` membuat *cluster* bernama `bank` dan menjadikan *context* `k3d-bank` sebagai *context* aktif kubectl. Setelah baris `waiting until the cluster DNS resolves host.k3d.internal`, perintah ini

biasanya menunggu sekitar 40–60 detik sebelum selesai. `make k8s-apply` selesai jika ketiga Deployment sudah *rolled out*:

```
namespace/bank created
configmap/account created
...
ingress.networking.k8s.io/transfer created
deployment "account" successfully rolled out
deployment "transfer" successfully rolled out
deployment "notification" successfully rolled out
```

Periksa objek yang dibuat:

```
kubectl -n bank get pods,svc,ingress
```

Ketiga *pod* berstatus `Running` dengan kolom `READY` bernilai `1/1`, dan kolom `RESTARTS` bernilai `0`.

9.4.11 Transfer melalui Ingress

`make fault-*` dan pembuatan rekening menggunakan REST `account` di `localhost:28081`. Di k3d, port tersebut tidak terbuka di laptop. Buka terminal lain dan teruskan port Service `account` ke laptop; biarkan perintah ini berjalan:

```
kubectl -n bank port-forward svc/account 28081:8081
```

Rekening dari lab sebelumnya tetap ada, karena `account` di k3d menggunakan database `db-account` yang sama. Periksa saldonya, lalu kirim transfer melalui Ingress di port 28080:

```
curl -s localhost:28081/accounts/1111111111
TOKEN=$(make -s token)
curl -i -X POST localhost:28080/transfers -H "Authorization: Bearer $TOKEN" \
  -d '{"from_account":"1111111111","to_account":"2222222222","amount":"100000"}'
```

`make -s token` meminta *token* ke Hydra di `localhost:14444` dari laptop, sehingga *claim* `iss` bernilai `http://localhost:14444/`, sama dengan nilai `OAUTH_ISSUER` di *pod*. Respons yang diharapkan:

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /transfers/3f0c...

{"id":"3f0c...","from_account":"1111111111","to_account":"2222222222","amount":"100000","status":"COMPL
```

Log *pod* `transfer` memuat baris `"msg":"transfer completed"`, dan log *pod* `notification` memuat `"msg":"notification sent"` dengan `transfer_id` yang sama:

```
kubectl -n bank logs deploy/transfer --tail=20
kubectl -n bank logs deploy/notification --tail=20
```

9.4.12 Latihan Mengubah Sistem di *Cluster*

Latihan berikut mengubah sistem yang sedang berjalan. Selama latihan, buka terminal lain dan amati perubahan *pod*:

```
kubectl -n bank get pods -w
```

9.4.12.1 *Scale* `notification`

Topic `transfer-events` memiliki tiga *partition* dan semua *pod* `notification` bergabung ke *consumer group* `notification`. Kafka membagi *partition* di antara anggota *group*, sehingga hingga tiga *pod* dapat berbagi pekerjaan:

```
kubectl -n bank scale deployment/notification --replicas=3
docker compose exec kafka /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server localhost:19092 --describe --group notification
```

Setelah *rebalance* selesai, setiap *partition* (0, 1, 2) memiliki `CONSUMER-ID` yang berbeda. Ulangi dengan `--replicas=4`: satu *pod* tidak mendapat *partition* dan tidak memproses *event* apa pun, walaupun statusnya `Running` dan *ready*. Paralelisme *consumer group* dibatasi oleh jumlah *partition*.

Kembalikan ke satu *replica* setelah selesai:

```
kubectl -n bank scale deployment/notification --replicas=1
```

Jeda *rebalance*. *Pod* yang dihentikan dengan `SIGTERM` menutup *reader* Kafka dan keluar dari *consumer group* secara teratur, sehingga *partition* miliknya segera dibagi ulang. Jika sebuah *container* `notification` dihentikan paksa (`SIGKILL`, misalnya `kubectl delete pod --force --grace-period=0` atau *OOM kill*), Kafka baru membagi ulang *partition* milik *container* tersebut setelah *session timeout*, sekitar 30 detik. Selama jeda tersebut, *event* di *partition* yang bersangkutan belum diproses. Kondisi ini normal; tunggu sekitar 30 detik sebelum menyimpulkan bahwa *event* hilang.

9.4.12.2 Mengubah Konfigurasi

Environment variable hanya dibaca saat proses dimulai. Perubahan ConfigMap tidak terlihat oleh *pod* yang sedang berjalan, sehingga *pod* baru harus dibuat dengan `kubectl rollout restart`.

Latihan ini sengaja membuat konfigurasi salah untuk mengamati perlindungan `maxUnavailable: 0`. Hapus baris `KAFKA_GROUP_ID: notification` dari ConfigMap `notification` di `k8s/overlays/k3d/config.yaml`, lalu terapkan dan *restart*:

```
kubectl apply -k k8s/overlays/k3d
kubectl -n bank rollout restart deployment/notification
kubectl -n bank rollout status deployment/notification --timeout=60s
```

`rollout status` berhenti dengan pesan *timeout*. *Pod* baru masuk status `CrashLoopBackOff`, sedangkan *pod* lama tetap `Running` dan terus memproses *event*, karena Kubernetes tidak menghentikan *pod* lama sebelum *pod* pengganti *ready*. Log *pod* baru menyebut variabel yang hilang:

```
kubectl -n bank get pods -l app.kubernetes.io/name=notification
kubectl -n bank logs <nama-pod-baru>
```

```
{"time":"...","level":"ERROR","msg":"notification service stopped","error":"missing
required environment variables: KAFKA_GROUP_ID"}
```

Kembalikan baris `KAFKA_GROUP_ID: notification`, terapkan lagi, lalu *restart*:

```
kubectl apply -k k8s/overlays/k3d
kubectl -n bank rollout restart deployment/notification
kubectl -n bank rollout status deployment/notification
```

`rollout restart` tetap diperlukan setelah nilai dikembalikan. *Pod* yang berstatus `CrashLoopBackOff` baru mencoba berjalan lagi setelah jeda *backoff*, yang dapat mencapai 5 menit, sedangkan `rollout restart` langsung membuat *pod* baru.

Setiap `kubectl apply -k k8s/overlays/k3d` melaporkan Secret sebagai `configured`, walaupun isinya tidak berubah. Penyebabnya bagian `stringData` di *overlay*: API *server* memindahkan isinya ke `data`, sehingga `kubectl apply` selalu melihat perbedaan. Laporan tersebut tidak menandakan masalah.

> **Mengapa latihan ini mengubah ConfigMap, bukan Secret.** `kubectl apply` menghapus *key* yang dihilangkan dari bagian `data` ConfigMap. Untuk Secret yang ditulis dengan `stringData`, hal tersebut tidak terjadi: API *server* menyimpan isinya di bagian `data` dalam bentuk base64, sehingga *key* yang dihapus dari `stringData` tetap ada di objek Secret yang tersimpan di *cluster*. Untuk menghapus *key* dari Secret, hapus objek Secret-nya (`kubectl -n bank delete secret notification`), lalu *apply* lagi.

9.4.12.3 *Rolling Update* dengan *Tag* Baru

Setiap versi aplikasi mendapat *tag image* sendiri. *Build image* dengan *tag* `1.0.1`, masukkan ke *cluster*, lalu ganti *image* Deployment `transfer`:

```
make images VERSION=1.0.1
make k3d-import VERSION=1.0.1
kubectl -n bank set image deployment/transfer transfer=studi-kasus/transfer:1.0.1
kubectl -n bank rollout status deployment/transfer
```

Kode tidak perlu diubah. Perubahan *tag* di *pod template* sudah cukup untuk memicu *rollout*. Sebaliknya, *build* ulang dengan *tag* `1.0.0` yang sama tidak memicu *rollout* apa pun, karena referensi *image* di Deployment tidak berubah. Di terminal `get pods -w`, *pod* baru dibuat dan menjadi *ready* (`1/1`) terlebih dahulu, baru kemudian *pod* lama berstatus `Terminating`. Riwayat *rollout* dapat dilihat dengan:

```
kubectl -n bank rollout history deployment/transfer
kubectl -n bank get deployment transfer -o
jsonpath='{.spec.template.spec.containers[0].image}'
```

Cara lain yang lebih sesuai dengan prinsip *manifest* sebagai *source of truth* (acuan tunggal kondisi *cluster*): ubah `newTag` milik `studi-kasus/transfer` di `k8s/overlays/k3d/kustomization.yaml` menjadi `"1.0.1"`, lalu jalankan `kubectl apply -k k8s/overlays/k3d`. `kubectl set image` mengubah objek di *cluster* tanpa mengubah *file*, sehingga `make k8s-apply` berikutnya mengembalikan *tag* ke nilai di `kustomization.yaml`.

9.4.12.4 *Rollback*

```
kubectl -n bank rollout undo deployment/transfer
kubectl -n bank rollout status deployment/transfer
kubectl -n bank get deployment transfer -o
jsonpath='{.spec.template.spec.containers[0].image}'
```

Image kembali ke `studi-kasus/transfer:1.0.0`, dengan pola penggantian *pod* yang sama seperti *rolling update*. `rollout undo` dapat menampilkan peringatan tentang anotasi `last-applied-configuration`. Peringatan tersebut tidak berpengaruh pada *rollback* dan dapat diabaikan.

9.4.13 Menghapus *Cluster*

Setelah selesai, *cluster* dapat dihapus. Infrastruktur di Docker Compose dan isi database tidak terpengaruh:

```
make k3d-down
```

Untuk menjalankan sistem di k3d lagi, buat ulang *cluster* dengan `make k3d-up`, `make k3d-import VERSION=1.0.0`, dan `make k8s-apply`. *Image* perlu di-*import* ulang karena *image* tersimpan di dalam *cluster* yang sudah dihapus.

9.5 Verifikasi

Jalankan pemeriksaan berikut dari direktori `studi-kasus/` dengan *cluster* `bank` dan infrastruktur yang berjalan.

1. Ketiga *image* ada, masing-masing dengan `CONTENT SIZE` sekitar 8–9 MB (*containerd image store*) atau `SIZE` sekitar 21–23 MB (*image store* klasik):

   ```
   docker images 'studi-kasus/*'
   ```

2. *Image* berjalan dengan UID sembarang dan *root filesystem read-only*, lalu berhenti dengan pesan `missing required environment variables: ...` karena tidak ada *environment variable*:

   ```
   docker run --rm --user 1000690000:0 --read-only studi-kasus/transfer:1.0.0
   ```

3. *Manifest* dapat di-*render* tanpa *error*, dan Deployment hasil *render* tidak memuat `runAsUser`. Perintah kedua tidak menghasilkan *output* apa pun:

   ```
   kubectl kustomize k8s/overlays/k3d > /dev/null && echo OK
   kubectl kustomize k8s/overlays/k3d | grep runAsUser
   ```

4. Ketiga *pod* `Running` dan `1/1`, Ingress `transfer` terdaftar dengan *class* `traefik`:

   ```
   kubectl -n bank get pods,ingress
   ```

5. Transfer melalui Ingress menghasilkan `201 Created` dan `"status":"COMPLETED"`, sedangkan *request* tanpa *header* `Authorization` mendapat respons `401 Unauthorized`:

   ```
   TOKEN=$(make -s token)
   curl -s -X POST localhost:28080/transfers -H "Authorization: Bearer $TOKEN" \
     -d '{"from_account":"1111111111","to_account":"2222222222","amount":"1000"}'
   curl -i -X POST localhost:28080/transfers \
     -d '{"from_account":"1111111111","to_account":"2222222222","amount":"1000"}'
   ```

6. Notifikasi tercatat untuk transfer tersebut. Teruskan port `notification` di terminal lain, lalu ganti `<id>` dengan nilai `id` dari respons transfer:

   ```
   kubectl -n bank port-forward svc/notification 28083:8083
   curl -s 'localhost:28083/notifications?transfer_id=<id>'
   ```

   Hasilnya *array* JSON berisi satu notifikasi dengan `"event_type":"TransferCompleted"`.

7. *Trace* transfer dari *pod* tetap masuk ke Grafana (`http://localhost:13000`, Tempo), dengan *span* dari `transfer`, `account`, dan `notification` dalam satu *trace*.

9.6 Masalah Umum

***Pod* berstatus `ErrImagePull` atau `ImagePullBackOff`.** *Image* dengan *tag* yang diminta tidak ada di *cluster*, sehingga kubelet mencoba melakukan *pull* dari Docker Hub. Penyebabnya `make k3d-import` belum dijalankan, dijalankan dengan `VERSION` yang berbeda, atau *cluster* dibuat ulang setelah *import*. Jalankan `make k3d-import VERSION=<tag>` dengan *tag* yang sama dengan `newTag` di `kustomization.yaml`, lalu hapus *pod* yang gagal agar dibuat ulang.

***Pod* berstatus `CrashLoopBackOff` setelah Docker atau laptop di-*restart*.** *Container* k3d berjalan lagi tanpa nama `host.k3d.internal` di DNS *cluster*, dan log *pod* memuat `lookup host.k3d.internal ... no such host`. Jalankan `k3d cluster stop bank`, lalu `k3d cluster start bank`. k3d menulis nama tersebut lagi, dan *pod* berjalan normal dalam satu hingga dua menit tanpa perintah tambahan.

***Pod* berstatus `CrashLoopBackOff`.** Baca lognya dengan `kubectl -n bank logs <pod> --previous`. Pesan `missing required environment variables` berarti ada *key* yang belum diisi di ConfigMap atau Secret *overlay*. Pesan koneksi database atau Kafka yang gagal berarti port di laptop tidak dapat dijangkau melalui `host.k3d.internal`; pastikan `make infra-up` sudah berjalan dan `docker compose ps` menampilkan port yang dibutuhkan.

***Pod* `Running` tetapi `READY` bernilai `0/1`.** *Readiness probe* gagal karena database milik service tidak dapat dijangkau. Jalankan `kubectl -n bank describe pod <pod>` dan baca bagian `Events`.

`notification` tidak menerima *event*, log menyebut `localhost:19092`. Kafka masih menggunakan konfigurasi lama tanpa *listener* `K3D`. Periksa `KAFKA_ADVERTISED_LISTENERS` di `compose.yaml`, lalu jalankan `make infra-up` agar *container* `kafka` dibuat ulang.

`curl localhost:28080/transfers` menghasilkan `404 page not found`. Traefik membutuhkan beberapa puluh detik setelah `make k3d-up` hingga siap, dan Ingress baru dikenali setelah *manifest* diterapkan. Periksa `kubectl -n kube-system get pods` dan `kubectl -n bank get ingress`. *Path* selain `/transfers` memang tidak diteruskan oleh Ingress.

`curl: (52) Empty reply from server` tepat setelah `make k8s-apply`. `rollout status` sudah melaporkan *pod* siap, tetapi Traefik baru menerima daftar *endpoint* yang baru satu atau dua detik kemudian. Tunggu sebentar, lalu ulangi *request*. Jawaban `401` untuk *request* tanpa *token* menandakan bahwa *request* sudah sampai ke `transfer`.

Transfer ditolak `401` dengan *token* yang baru dibuat. `OAUTH_ISSUER` di ConfigMap tidak sama dengan nilai `iss` di *token*. Nilainya harus tepat `http://localhost:14444/`, termasuk garis miring di akhir. Jika Hydra di-*restart*, `make infra-up` membuat ulang *client* OAuth2, dan *token* lama tidak berlaku lagi.

Transfer `FAILED` atau `503` dengan alasan `account` tidak tersedia. *Pod* `account` belum *ready* atau DNS *cluster* belum menjawab `account`. Periksa `kubectl -n bank get endpoints account` dan log `transfer`.

`make k3d-up` gagal karena port 28080 sudah digunakan. Hentikan proses lain yang membuka port 28080, lalu jalankan `make k3d-down` dan `make k3d-up` lagi.

Notifikasi ganda atau *event* diproses oleh proses di laptop. Masih ada `notification` atau `transfer` di luar *cluster* (`make -C ... run` atau *profile* `apps`). Hentikan service di laptop dan jalankan `make apps-down`.

`kubectl port-forward` berhenti dengan sendirinya. `port-forward` terhubung ke satu *pod*. Jika *pod* tersebut diganti (misalnya saat *rolling update*), jalankan ulang perintahnya.

9.7 Eksplorasi Mandiri

1. ***Scale* `account` dan koneksi gRPC.** Jalankan `kubectl -n bank scale deployment/account --replicas=3`, kirim sepuluh transfer, lalu bandingkan log ketiga *pod* `account` (`kubectl -n bank logs <pod>`). Semua panggilan gRPC dari satu *pod* `transfer` tetap menuju satu *pod* `account`. Jelaskan penyebabnya dan cari tahu cara mengubahnya dengan *headless Service* serta konfigurasi `round_robin` di klien gRPC. Perhatikan juga bahwa `make fault-slow` melalui `port-forward` hanya mengenai satu *pod* `account`, karena *fault injection* disimpan di memori setiap *pod*.
2. ***Debugging* tanpa *shell*.** *Image* `scratch` tidak memiliki *shell*. Masukkan *image* `golang:1.27.2-alpine` (yang sudah di-*pull* untuk *build* dan memiliki *shell*) ke *cluster* dengan `k3d image import -c bank golang:1.27.2-alpine`, lalu jalankan perintah berikut untuk menambahkan

ephemeral container ke *pod* tersebut: `echo '{"securityContext":{"runAsUser":65532}}' > /tmp/debug.json`, kemudian `kubectl -n bank debug -it <pod-transfer> --image=golang:1.27.2-alpine --target=transfer --custom=/tmp/debug.json -- sh`. `runAsNonRoot: true` di *security context pod* juga berlaku untuk *ephemeral container*, sedangkan `golang:1.27.2-alpine` tidak memiliki instruksi `USER` sehingga berjalan sebagai `root`; tanpa *profile* `--custom` tersebut, kubelet menolak menjalankan *container* dengan status `CreateContainerConfigError`. Dari dalamnya, coba `wget -qO- localhost:8082/readyz`. *Ephemeral container* berbagi *network namespace* dengan *pod* target, sehingga `localhost` di dalamnya adalah *pod* `transfer`.

3. **Simulasi UID OpenShift.** Tambahkan *patch* di *overlay* k3d yang mengisi `runAsUser: 1000690000` pada `securityContext` *pod* `transfer`, terapkan, dan pastikan *pod* tetap *ready*. Setelah itu, coba *image* dengan `USER root` (atau tanpa `USER`) dan amati pesan kubelet ketika `runAsNonRoot: true` menolak *image* tersebut (`kubectl describe pod`).
4. **Dua *replica* `transfer`.** *Scale* `transfer` menjadi 2, kirim beberapa transfer, lalu pastikan dengan `make outbox` bahwa setiap baris *outbox* hanya dikirim sekali ke Kafka, karena *relay* mengunci baris dengan `for update skip locked`. Aktifkan `make fault-error RATE=1` dan amati `/debug/circuit-breaker` di setiap *pod* `transfer` (dengan `kubectl port-forward pod/<nama> 28082:8082`): setiap *replica* memiliki status *circuit breaker* sendiri.

9.8 Checkpoint

Kondisi kode di akhir lab ini tersedia di *tag* `lab-09`. Bandingkan hasil kerja dengan *checkpoint* tersebut:

```
git diff lab-09 -- .
```

Peserta yang ingin langsung melanjutkan ke Lab 10 dari kondisi akhir Lab 9:

```
git checkout lab-09
```

10 Lab 10: *Pipeline* CI/CD per Service

10.1 Tujuan

Setelah lab ini, peserta mampu:

- menyusun *target* `make` yang sama untuk laptop, runner CI, dan Jenkins: `image`, `push`, `apply`, `deploy`;
- menulis *workflow* per service yang hanya berisi *step* `run:` dan dipicu oleh perubahan di direktori service tersebut;
- memetakan setiap *step workflow* ke *stage declarative pipeline* Jenkins;
- menjalankan tahapan *pipeline* satu service di laptop hingga *deploy* ke k3d, lalu menelusuri versi yang berjalan lewat `kubectl rollout history`;
- menjelaskan *drift* (selisih antara isi repository dan isi *cluster*) yang menjadi alasan GitOps.

10.2 Gambaran

Hingga Lab 9, *image* di-*build* dan di-*deploy* ke k3d dengan perintah manual: `make images`, `make k3d-import`, lalu `kubectl set image`. Lab 10 mengubah urutan perintah itu menjadi *pipeline*: setiap *commit* ke `main` yang mengubah satu service menjalankan *test*, mem-*build image* dengan *tag* berupa SHA *commit*, mengirim *image* ke *container registry*, lalu menjalankan *rolling update* Deployment service tersebut.

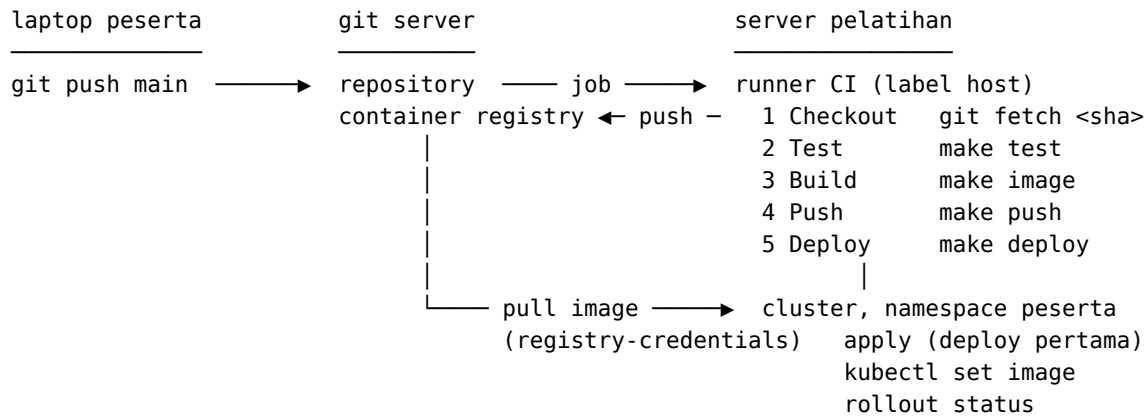
Ada tiga keputusan desain dalam lab ini.

Satu *pipeline* per service. *Commit* yang hanya mengubah `account` tidak perlu menguji ulang dan men-*deploy* ulang `notification`. Setiap service punya *workflow* dan `Jenkinsfile` sendiri dengan filter *path*. Perubahan di direktori bersama (`proto/`, `go.work`, `pipeline.mk`) memicu *pipeline* semua service yang bergantung padanya.

***Step* hanya memanggil `make`.** *Workflow* tidak menggunakan *action* siap pakai (`uses:`). Setiap *step* berisi perintah `make` yang juga bisa dijalankan di laptop, sehingga setiap *step* setara dengan satu *stage* Jenkins dan perpindahan dari satu CI ke CI lain cukup mengganti *file* definisi tanpa mengubah logikanya.

Tidak ada nilai *default*. `REGISTRY`, `IMAGE_OWNER`, `VERSION`, dan `NAMESPACE` wajib diisi oleh pemanggil. Lebih aman bila *pipeline* berhenti dengan pesan error dibandingkan mengirim *image* ke registry yang salah atau men-*deploy* ke *namespace* orang lain.

Alur *pipeline* di server pelatihan:



Alur yang sama di laptop, tanpa git server dan registry:

```
git commit → make pipeline-k3d SERVICE=transfer
                make test → make image → k3d image import → make deploy → k3d
(namespace bank)
```

> **Status server pelatihan.** *Workflow* dan *overlay* `k8s/overlays/server` sudah dijalankan dari awal hingga akhir di server pelatihan (Langkah 10): *deploy* pertama ke *namespace* kosong, *rolling update*, dan dua peserta yang melakukan *push* bersamaan. Ketiga `Jenkinsfile` sudah dijalankan di Jenkins LTS 2.580.1 dengan *deploy* ke k3d (Langkah 6). Latihan yang bisa dijalankan sepenuhnya di laptop adalah `make pipeline-k3d` pada Langkah 8.

10.3 Titik Awal

Lab ini melanjutkan kondisi akhir Lab 9: *image* ketiga service bisa di-*build*, k3d bisa dibuat, dan *manifest* `k8s/overlays/k3d` bisa di-*apply*. Peserta yang belum menyelesaikan Lab 9 memulai dari *tag* `lab-09`:

```
cd training-microservices-2026-01
git checkout lab-09
git switch -c lab-10-saya
```

Perintah `git switch -c` membuat *branch* kerja. Langkah 8 mengharuskan peserta membuat *commit*, padahal *commit* yang dibuat langsung di atas *tag* berada dalam kondisi *detached HEAD* dan mudah hilang setelah `git checkout` berikutnya.

Pastikan prasyarat Lab 9 masih berjalan:

```
cd studi-kasus
make infra-up
k3d version
kubectl version --client
```

10.4 Langkah

10.4.1 Langkah 1: *Target pipeline* bersama di `pipeline.mk`

Ketiga service membutuhkan *target* yang sama: mem-*build image*, mengirimnya ke registry, membuat objek service di *cluster* pada *deploy* pertama, dan mengganti *image* pada *deploy* berikutnya. Agar tidak ditulis tiga kali, *target* tersebut diletakkan di satu *file* `studi-kasus/pipeline.mk` yang di-*include* oleh `Makefile` setiap service. `Makefile` service cukup mengisi variabel `SERVICE`; nilai lainnya diisi oleh pemanggil.

Beberapa hal yang perlu diperhatikan sebelum menulis *file*-nya:

- Fungsi `require` memeriksa variabel yang kosong dan menghentikan `make` dengan daftar nama variabelnya, sama seperti `LoadConfig` di service.
- `image` menggunakan *build context* `..` (direktori `studi-kasus/`), sama seperti Lab 9, karena `Dockerfile` menyalin `proto/`. Label `org.opencontainers.image.revision` menghubungkan *image* dengan *commit* asalnya.
- `push` membuat `DOCKER_CONFIG` sementara untuk setiap pemanggilan dan menghapusnya ketika *recipe* (baris perintah di bawah *target*) selesai, sehingga *token* registry tidak tertinggal di `~/.docker/config.json` milik pengguna runner setelah *job* selesai. Mekanisme ini tidak mengisolasi peserta satu sama lain: runner digunakan bersama, dan siapa pun yang bisa mengubah *workflow* bisa menjalankan perintah apa saja di *host* runner. `REGISTRY_USER` dan `REGISTRY_PASSWORD` hanya dibaca dari *environment variable*, karena argumen baris perintah terlihat di daftar proses. Setelah *push* berhasil, *image* lokal dihapus dengan `docker image rm`. *Image* itu sudah tersimpan di registry. Bila tidak dihapus, *image* dari setiap *commit* setiap peserta akan memenuhi disk runner. *Build cache* tetap ada, sehingga *build* berikutnya menggunakan ulang *layer* yang tidak berubah.
- `apply` memasang bagian *overlay* `k8s/overlays/server` milik satu service: ConfigMap, Service, dan Deployment, ditambah Ingress khusus untuk `transfer`. *Overlay* itu tidak menyebut *namespace* dan *tag image*. `apply` menulis *kustomization* sementara di atas *overlay* yang mengisi keduanya dari `NAMESPACE` dan `IMAGE`, lalu `kubectl apply -l app.kubernetes.io/name=<service>` hanya memasang objek yang berlabel service tersebut (label dipasang di Langkah 7). Tanpa filter label, Deployment service lain ikut terpasang dengan nama *image* yang belum diganti. Direktori sementara `.apply.XXXXXX` dibuat di samping `Makefile` service, bukan di `/tmp`, karena *kustomize* menolak *path* absolut di `resources`. `apply` juga berhenti dengan pesan error selama *overlay* masih berisi *placeholder* `replace-me` (Langkah 7).
- `deploy` memeriksa terlebih dahulu apakah Deployment service sudah ada. `kubectl get --ignore-not-found` tidak mencetak apa pun hanya bila Deployment belum ada. Dalam kondisi itu `deploy` memanggil `apply`, sehingga *pipeline* pertama setiap service membuat objeknya sendiri di *namespace* kosong. Error lain, misalnya akses ditolak atau *cluster* tidak bisa dihubungi, menghentikan `deploy` di baris itu. Setelah itu `deploy` menjalankan `kubectl set image`, lalu menulis anotasi `kubernetes.io/change-cause`. Anotasi disalin *controller* ke ReplicaSet yang sedang aktif, sehingga `kubectl rollout history` menampilkan versi setiap revisi. Urutannya penting: anotasi yang ditulis sebelum `set image` akan menimpa catatan versi revisi sebelumnya. Bila *rollout* gagal, `kubectl rollout status` ikut gagal sehingga *pipeline* berhenti, sementara *pod* lama tetap melayani karena Deployment menggunakan `maxUnavailable: 0`.

Buat *file* `studi-kasus/pipeline.mk`:

```
# Pipeline targets shared by the three service Makefiles (Lab 10). The
# workflows in .forgejo/workflows and the Jenkinsfile of each service call
# these targets and nothing else, so a laptop, the git server's runner and a
# Jenkins agent run the same commands.
#
# The including Makefile sets SERVICE. Everything else comes from the caller
# (command line or environment) and has no default: an image pushed to the
```

```
# wrong registry or a deploy into the wrong namespace is worse than a stopped
# pipeline.
#   REGISTRY       registry host, e.g. registry.example.com
#   IMAGE_OWNER    user or organization that owns the image in the registry
#   VERSION        image tag; the pipeline passes the commit SHA, so every
#                  commit has its own image and a tag is never overwritten
#   NAMESPACE      Kubernetes namespace of the deploy target
#   REGISTRY_USER, REGISTRY_PASSWORD   push credentials, environment only
#                  (a command-line argument shows up in the process list)

IMAGE = $(REGISTRY)/$(IMAGE_OWNER)/$(SERVICE):$(VERSION)

# $(call require,VAR ...) stops with the names of the empty variables, the
# same message style as the services' LoadConfig.
missing = $(strip $(foreach v,$(1),$(if $(strip $($(v))),,$(v))))
require = $(if $(call missing,$(1)),$(error missing required variables: $(call missing,
$(1))))

.PHONY: image push apply deploy

# Build context is studi-kasus/ (..), as in Lab 9: the Dockerfile copies proto/
# next to the service. The revision label ties a running image back to its
# commit (docker inspect, or the registry UI).
image:
  $(call require,REGISTRY IMAGE_OWNER VERSION)
  docker build -f Dockerfile --label org.opencontainers.image.revision=$(VERSION) -t
$(IMAGE) ..

# A throwaway DOCKER_CONFIG per call, removed when the recipe ends: the
# registry token is not left in the runner user's ~/.docker/config.json after
# the job. That is all it does. It is no isolation between participants on a
# shared runner: anyone who can edit a workflow runs commands on the runner
# host as the runner user, and can read whatever another job has on disk or in
# its environment while it runs.
# The local image is removed after a successful push: the registry has it,
# and on a shared runner one image per commit per participant fills the disk.
# The build cache stays, so the next build reuses unchanged layers.
push:
  $(call require,REGISTRY IMAGE_OWNER VERSION REGISTRY_USER REGISTRY_PASSWORD)
  @cfg=$$(mktemp -d) && trap 'rm -rf "$$cfg"' EXIT && \
  printf '%s' "$$REGISTRY_PASSWORD" | DOCKER_CONFIG="$$cfg" docker login $(REGISTRY) -u
"$$REGISTRY_USER" --password-stdin && \
  DOCKER_CONFIG="$$cfg" docker push $(IMAGE)
  docker image rm $(IMAGE)

# Applies this service's part of OVERLAY into NAMESPACE with the image
# IMAGE: the ConfigMap, Service, Deployment (and Ingress of transfer). The
# overlay has no namespace and no image tag; a generated kustomization on top
# of it sets both, and kubectl apply -l keeps only the objects labelled with
# this service (the other services' Deployments would get an unresolved
# image). Refuses while the overlay still has a replace-me placeholder.
# make deploy calls it when the Deployment does not exist yet; make
# k8s-apply-server (top-level Makefile) calls it to bring a config change to
# the cluster.
# The generated kustomization lives in a temporary directory next to this
# Makefile, removed when the recipe ends: kustomize refuses an absolute path
# in resources, so the overlay is referenced relative to it.
OVERLAY := ../k8s/overlays/server
```

```
SELECTOR := app.kubernetes.io/name=$(SERVICE)

apply:
  $(call require,REGISTRY IMAGE_OWNER VERSION NAMESPACE)
  @if kubectl kustomize $(OVERLAY) | grep -n replace-me; then \
    echo "$(OVERLAY) still has replace-me placeholders (lines above)" >&2; exit 1; fi
  @k=$$(mktemp -d .apply.XXXXXX) && trap 'rm -rf "$$k"' EXIT && \
  printf '%s\n' \
    'apiVersion: kustomize.config.k8s.io/v1beta1' \
    'kind: Kustomization' \
    'namespace: $(NAMESPACE)' \
    'resources:' \
    '  - ../$(OVERLAY)' \
    'images:' \
    '  - name: studi-kasus/$(SERVICE)' \
    '    newName: $(REGISTRY)/$(IMAGE_OWNER)/$(SERVICE)' \
    '    newTag: "$(VERSION)"' > "$$k/kustomization.yaml" && \
  echo "apply $(OVERLAY) to namespace $(NAMESPACE), objects $(SELECTOR), image $(IMAGE)"
&& \
  kubectl -n $(NAMESPACE) apply -k "$$k" -l $(SELECTOR)

# Rolling update to the new image: the same "kubectl set image" + "rollout
# status" as the Lab 9 exercise. The first run of a service, in an empty
# namespace, creates the Deployment with make apply instead; set image then
# changes nothing and rollout status waits for the first pods.
# "get --ignore-not-found" prints nothing only when the Deployment does not
# exist; any other error (no access, cluster unreachable) stops here.
# The change-cause annotation is copied to the new ReplicaSet, so "kubectl
# rollout history" lists the version per revision. It comes after set image:
# the controller copies it to the ReplicaSet that is current at that moment,
# and annotating first would relabel the previous revision with the new
# version.
# A failed rollout fails the pipeline and leaves the old pods serving
# (maxUnavailable: 0); rolling back stays a human decision
# (kubectl rollout undo).
# kubectl uses KUBECONFIG; the pipeline points it at a kubeconfig that is
# limited to NAMESPACE.
deploy:
  $(call require,REGISTRY IMAGE_OWNER VERSION NAMESPACE)
  @d=$$(kubectl -n $(NAMESPACE) get deployment/$(SERVICE) --ignore-not-found -o name) ||
exit 1; \
  if [ -z "$$d" ]; then \
    echo "deployment/$(SERVICE) not in namespace $(NAMESPACE): first deploy, applying
the overlay"; \
    $(MAKE) --no-print-directory apply || exit 1; \
  fi
  kubectl -n $(NAMESPACE) set image deployment/$(SERVICE) $(SERVICE)=$(IMAGE)
  kubectl -n $(NAMESPACE) annotate deployment/$(SERVICE) kubernetes.io/change-
cause="$(SERVICE) $(VERSION)" --overwrite
  kubectl -n $(NAMESPACE) rollout status deployment/$(SERVICE) --timeout=180s
```

Baris *recipe* di `Makefile` harus diawali karakter *Tab*, bukan spasi. Saat kode disalin dari PDF, *Tab* biasanya berubah menjadi spasi, dan `make` lalu berhenti dengan pesan `missing separator`.

10.4.2 Langkah 2: *Include* di `Makefile` setiap service

Tambahkan dua baris berikut di akhir `account/Makefile`, `transfer/Makefile`, dan `notification/Makefile`. Ketiga *file* itu sudah mengisi `SERVICE := <nama service>` di baris awal sejak service tersebut dibuat (Lab 1, 2, dan 5), dan `pipeline.mk` membaca variabel tersebut.

```
# image, push, deploy (Lab 10).
include ../pipeline.mk
```

Periksa bahwa *target* baru berhenti dengan pesan error bila variabelnya kosong:

```
make -C transfer image
make -C transfer deploy REGISTRY=r IMAGE_OWNER=o VERSION=v
```

Output yang diharapkan:

```
../pipeline.mk:31: *** missing required variables: REGISTRY IMAGE_OWNER VERSION.  Stop.
../pipeline.mk:100: *** missing required variables: NAMESPACE.  Stop.
```

Angka setelah `pipeline.mk:` adalah nomor baris pemanggilan `require`, sehingga bisa berbeda bila komentar ditulis dengan kata-kata sendiri. GNU make 4.x (umumnya di Linux) juga mencetak baris `make: Entering directory '.../studi-kasus/transfer'` dan `make: Leaving directory ...` di sekitar *output* setiap perintah `make -C`, termasuk perintah `make -n -C` di bawah. GNU make 3.81 bawaan macOS tidak mencetak baris tersebut.

Opsi `-n` menampilkan perintah tanpa menjalankannya. Dengan opsi ini, isi *target* `image` bisa diperiksa tanpa Docker:

```
make -n -C transfer image REGISTRY=k3d.local IMAGE_OWNER=studi-kasus VERSION=abc
```

```
docker build -f Dockerfile --label org.opencontainers.image.revision=abc -t k3d.local/
studi-kasus/transfer:abc ..
```

Opsi `-n` tidak cocok untuk `deploy`. Baris *recipe* yang berisi `$(MAKE)` tetap dijalankan walaupun ada `-n`, dan baris pertama `deploy` berisi `$(MAKE) apply`. Akibatnya `kubectl get deployment` benar-benar dijalankan, dan tanpa *cluster* perintah itu berhenti dengan error koneksi. Urutan perintah `deploy` cukup dibaca dari `pipeline.mk`: `kubectl get` (dan `apply` bila Deployment belum ada), `kubectl set image`, `kubectl annotate`, lalu `kubectl rollout status`.

Nama *container* di `kubectl set image` (`transfer=...`) mengacu pada nama *container* di `k8s/base/transfer/deployment.yaml`, yaitu nama service itu sendiri.

10.4.3 Langkah 3: `.dockerignore` dan `.gitignore`

`Jenkinsfile` dan `pipeline.mk` tidak dibutuhkan untuk mem-*build* *binary*. `Jenkinsfile` berada di dalam direktori service yang disalin oleh `COPY transfer/ /src/transfer/`, sehingga setiap perubahan `Jenkinsfile` membuat *cache layer* tersebut tidak terpakai lagi dan membuat *build* berikutnya lebih lama. `pipeline.mk` tidak disalin ke *image*, tetapi tetap ikut terkirim ke *builder* sebagai bagian dari *build context*. Tambahkan keduanya ke `.dockerignore`, sama seperti `Makefile` dan `compose.yaml`, dengan dua baris di akhir `studi-kasus/.dockerignore`:

```
**/Jenkinsfile
pipeline.mk
```

Direktori sementara `.apply.XXXXXX` dari *target* `apply` dihapus ketika *recipe* selesai. Agar direktori yang tertinggal karena proses terhenti di tengah jalan tidak ikut ter-*commit*, tambahkan satu baris di akhir `.gitignore` di direktori *root* repository:

```
.apply.*
```

10.4.4 Langkah 4: *Helper* Lab 10 di `Makefile` utama

`Makefile` di `studi-kasus/` mendapat dua *target* baru:

- `pipeline-k3d` menjalankan tahapan *pipeline* satu service di laptop hingga *deploy* ke k3d. Laptop tidak punya registry, sehingga `push` diganti `k3d image import`. *Tag image* diambil dari 12 karakter pertama SHA *commit* saat ini. Karena itu *target* ini berhenti dengan pesan error selama ada perubahan yang belum di-*commit* di `studi-kasus/`: tanpa pemeriksaan tersebut, *image* bisa berisi kode yang tidak ada di *commit* yang namanya tertulis di *tag*.
- `k8s-apply-server` memanggil *target* `apply` milik satu service. *Pipeline* pertama sudah membuat objek service lewat `deploy`, sehingga *target* ini hanya diperlukan untuk membawa perubahan `config.yaml` atau `ingress.yaml` di *overlay* server ke *cluster* (Langkah 10). Variabelnya sama dengan `apply`: `SERVICE`, `VERSION`, `REGISTRY`, `IMAGE_OWNER`, `NAMESPACE`.

Tambahkan kedua nama *target* ke baris `.PHONY` yang sudah ada:

```
.PHONY: infra-up infra-down infra-reset test proto fault-slow fault-error fault-clear
fault-status breaker \
  events outbox replay duplicates token metrics apps-up apps-down images k3d-up k3d-
import k8s-apply k3d-down \
  pipeline-k3d k8s-apply-server
```

Variabel `NAMESPACE := bank` dari Lab 9 perlu diganti nama menjadi `K3D_NAMESPACE`. `pipeline.mk` mengambil `NAMESPACE` dari pemanggil. Bila `NAMESPACE` diekspor di *environment*, nilai `bank` di `Makefile` utama akan menimpanya, lalu ikut diteruskan ke `make` service yang dipanggil `k8s-apply-server`. Ganti definisinya:

```
# K3D_NAMESPACE, not NAMESPACE: pipeline.mk takes NAMESPACE from the caller,
# and a top-level value would be passed on to it.
K3D_NAMESPACE := bank
```

Ganti juga penggunaannya di *target* `k8s-apply`:

```
  @for s in $(SERVICES); do kubectl -n $(K3D_NAMESPACE) rollout status deployment/$$s --
timeout=120s || exit 1; done
```

Lalu tambahkan blok berikut di akhir `studi-kasus/Makefile`, setelah *target* `k3d-down`. Variabel `CLUSTER` dan `IMAGE_REPO` sudah didefinisikan di Lab 9.

```
# Lab 10 helpers. The pipeline itself is in .forgejo/workflows (git server's
# runner) and <service>/Jenkinsfile; its steps are the targets image, push and
# deploy in pipeline.mk.

# The pipeline of one service on the laptop, against k3d: test, image, then
# k3d image import in place of push (no registry on the laptop), then deploy.
# Usage: make pipeline-k3d SERVICE=transfer (after make k8s-apply of Lab 9).
# The tag is the current commit, as in the pipeline; uncommitted changes would
# end up in an image whose tag names a commit without them, so commit first.
# A later make k8s-apply sets the images back to the tag in overlays/k3d.
PIPELINE_REGISTRY := k3d.local
pipeline-k3d:
  @test -n "$(SERVICE)" || { echo "SERVICE is required, e.g. make pipeline-k3d
SERVICE=transfer" >&2; exit 1; }
  @test -z "$$(git status --porcelain -- .)" || { echo "uncommitted changes in studi-
kasus/: commit first, the image tag is the commit" >&2; exit 1; }
  @v=$$(git rev-parse --short=12 HEAD) && \
  $(MAKE) -C $(SERVICE) test && \
```

```
  $(MAKE) -C $(SERVICE) image REGISTRY=$(PIPELINE_REGISTRY) IMAGE_OWNER=$(IMAGE_REPO)
VERSION=$$v && \
  $(call k3d-load,$(PIPELINE_REGISTRY)/$(IMAGE_REPO)/$(SERVICE):$$v) && \
  $(MAKE) -C $(SERVICE) deploy REGISTRY=$(PIPELINE_REGISTRY) IMAGE_OWNER=$(IMAGE_REPO)
VERSION=$$v NAMESPACE=$(K3D_NAMESPACE)

# The server overlay for one service, the same apply that the pipeline's
# first deploy runs (make apply in pipeline.mk): to bring a change in
# k8s/overlays/server (config.yaml, ingress.yaml) to the cluster. VERSION is
# the image to keep or to roll out, usually the commit already deployed
# (kubectl rollout history). Pods read ConfigMaps only at start: kubectl
# rollout restart deployment/<service> after a config change.
# Refuses while a placeholder is left: the instructor hands out the training
# server's addresses (~/lab10.txt on the server).
# Usage: make k8s-apply-server SERVICE=transfer VERSION=<sha> REGISTRY=...
#        IMAGE_OWNER=... NAMESPACE=...  (KUBECONFIG of the training server)
k8s-apply-server:
  @test -n "$(SERVICE)" || { echo "SERVICE is required, e.g. make k8s-apply-server
SERVICE=transfer VERSION=<sha>" >&2; exit 1; }
  $(MAKE) -C $(SERVICE) apply
```

10.4.5 Langkah 5: *Workflow* per service

Runner CI membaca *workflow* dari direktori `.forgejo/workflows/` di *root* repository, bukan di `studi-kasus/`. Format *file*-nya sama dengan format *workflow* CI berbasis YAML yang umum: `on` untuk *trigger*, `jobs` untuk *job*, `steps` untuk urutan perintah.

Bagian-bagian *workflow* `transfer`:

- `on.push.paths` membatasi *trigger* pada perubahan di `studi-kasus/transfer/`, `studi-kasus/proto/` (kontrak gRPC yang di-*import* `transfer`), `go.work`, `pipeline.mk`, dan *file workflow* itu sendiri.
- `concurrency` dengan `cancel-in-progress: false` membuat *workflow* satu service berjalan satu per satu. *Push* berikutnya menunggu *pipeline* yang sedang berjalan selesai, bukan membatalkannya. Dua *deploy* satu service yang berjalan bersamaan akan saling berebut Deployment yang sama. Padanannya di Jenkins adalah `disableConcurrentBuilds()`.
- `runs-on: host` menjalankan *job* langsung di server tempat runner berjalan, tanpa *container job*. Server tersebut sudah menyediakan `git`, `make`, `go`, `docker`, dan `kubectl`. Runner digunakan bersama oleh semua peserta tanpa pemisahan hak akses: *job* berjalan sebagai pengguna runner, sehingga siapa pun yang bisa mengubah *workflow* bisa menjalankan perintah apa saja di *host* tersebut.
- `env` di tingkat *job* mengisi variabel untuk `pipeline.mk`. `REGISTRY` dan `NAMESPACE` diambil dari *variable* repository, `IMAGE_OWNER` diisi nama pemilik repository, dan `VERSION` diisi SHA *commit* yang memicu *pipeline*. Satu *commit* menghasilkan satu *image*, sehingga *tag* tidak pernah tertimpa.
- *Step* Checkout menggunakan `git` biasa dengan *token* yang diterbitkan runner untuk *job* ini (akses baca ke repository, berlaku selama *job* berjalan). Karena isi *workspace* runner *host* bisa tersisa dari *job* sebelumnya, `fetch`, `checkout -f`, dan `clean -ffdx` menyamakan isinya tepat dengan *commit* ini.
- *Secret* `REGISTRY_USER` dan `REGISTRY_PASSWORD` hanya tersedia di *step* Push.
- *Step* Deploy menulis *secret* `KUBE_CONFIG` ke *file* sementara yang dihapus ketika *step* selesai (`trap ... EXIT`). Dengan `KUBECONFIG` yang menunjuk ke *file* itu, `kubectl` tidak pernah menggunakan *kubeconfig* milik pengguna runner. RBAC *kubeconfig* tersebut harus mengizinkan pembuatan ConfigMap, Service, Deployment, dan Ingress di *namespace* peserta, karena *deploy* pertama membuat objek-objek itu.

Bila *variable* atau *secret* belum diisi, *step* menerimanya sebagai *string* kosong. `pipeline.mk` menghentikan *step* tersebut dengan pesan `missing required variables: ...`, dan *step* Deploy memeriksa `KUBE_CONFIG` sendiri.

Buat *file* `.forgejo/workflows/transfer.yaml` di direktori *root* repository:

```yaml
# Pipeline of the transfer service (Lab 10): test, build the image, push it to the
# git server's container registry, roll it out on the training server.
# One workflow per service, so a change to transfer builds and deploys transfer only.
#
# Only run: steps that call make targets (no marketplace actions): every step
# maps 1:1 to a stage of studi-kasus/transfer/Jenkinsfile, and the same commands
# run on a laptop.
#
# Verified end to end on the training server (git server 16.0.6, runner
# 13.2.0, k3s v1.36.5+k3s1): first deploy into an empty namespace, rolling
# update, two participants pushing at the same time.
#
# The runner is shared by all participants and trusts them all: a job runs on
# the runner host as the runner user, so anyone who can push a workflow change
# can run any command there.
#
# What the runner and the repository must provide:
#   runner label "host"   jobs run directly on the server (no job container),
#                         where git, make, go, docker and kubectl are installed
#   variable  REGISTRY    registry host (the git server's host name)
#   variable  NAMESPACE   the participant's namespace on the training server
#   secret    REGISTRY_USER, REGISTRY_PASSWORD   an account and token allowed
#                         to push packages
#   secret    KUBE_CONFIG kubeconfig limited to NAMESPACE (RBAC), allowed to
#                         create ConfigMaps, Services, Deployments and
#                         Ingresses there (the first deploy creates them)
# A missing variable or secret arrives as an empty string; the make targets
# stop on it with "missing required variables: ...".
name: transfer

on:
  push:
    branches: [main]
    # proto/, go.work and pipeline.mk are shared: a gRPC contract change
    # rebuilds both account and transfer.
    paths:
      - studi-kasus/transfer/**
      - studi-kasus/proto/**
      - studi-kasus/go.work
      - studi-kasus/pipeline.mk
      - .forgejo/workflows/transfer.yaml

# One run of this workflow at a time: a newer push waits until the running one
# has finished instead of cancelling it (cancel-in-progress is true by default
# for push). Two deploys of one service at once would race on the same
# Deployment. The Jenkinsfile does the same with disableConcurrentBuilds().
concurrency:
  group: pipeline-transfer
  cancel-in-progress: false

jobs:
  pipeline:
    runs-on: host
    env:
      REGISTRY: ${{ vars.REGISTRY }}
      # Packages in the registry belong to the repository owner.
      IMAGE_OWNER: ${{ github.repository_owner }}
      NAMESPACE: ${{ vars.NAMESPACE }}
```

```
      # One image per commit: the tag says exactly which code runs.
      VERSION: ${{ github.sha }}
    steps:
      # Checkout with plain git and the token the runner issues for this job
      # (read access to this repository, valid while the job runs). The
      # workspace of a host runner can survive between jobs: fetch + checkout
      # -f + clean make it exactly this commit either way.
      - name: Checkout
        env:
          SERVER_URL: ${{ github.server_url }}
          REPOSITORY: ${{ github.repository }}
          TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          set -eu
          auth=$(printf 'x-access-token:%s' "$TOKEN" | base64 | tr -d '\n')
          git init -q .
          git -c http.extraHeader="Authorization: basic $auth" fetch -q --depth 1
"$SERVER_URL/$REPOSITORY.git" "$VERSION"
          git checkout -q -f FETCH_HEAD
          git clean -q -ffdx

      - name: Test
        run: make -C studi-kasus/transfer test

      - name: Build image
        run: make -C studi-kasus/transfer image

      - name: Push
        env:
          REGISTRY_USER: ${{ secrets.REGISTRY_USER }}
          REGISTRY_PASSWORD: ${{ secrets.REGISTRY_PASSWORD }}
        run: make -C studi-kasus/transfer push

      # The kubeconfig secret goes to a file that exists only during this
      # step; kubectl never falls back to the runner user's own kubeconfig.
      # The first run in an empty namespace creates the service's objects from
      # k8s/overlays/server with this commit's image; later runs only change
      # the image (pipeline.mk, target deploy).
      - name: Deploy
        env:
          KUBE_CONFIG: ${{ secrets.KUBE_CONFIG }}
        run: |
          set -eu
          test -n "$KUBE_CONFIG" || { echo "missing required secret: KUBE_CONFIG" >&2;
exit 1; }
          KUBECONFIG=$(mktemp)
          trap 'rm -f "$KUBECONFIG"' EXIT
          printf '%s\n' "$KUBE_CONFIG" > "$KUBECONFIG"
          export KUBECONFIG
          make -C studi-kasus/transfer deploy
```

Salin *file* tersebut menjadi `account.yaml` dan `notification.yaml`, lalu ganti setiap `transfer` dengan nama service. Ada satu perbedaan: `notification` tidak meng-*import* kontrak gRPC, sehingga baris `studi-kasus/proto/**` dihapus dari `paths` di `notification.yaml`.

```
cd ..   # akar repository
sed 's/transfer/account/g' .forgejo/workflows/transfer.yaml > .forgejo/workflows/
```

```
account.yaml
sed -e 's/transfer/notification/g' -e '/studi-kasus\/proto/d' \
  .forgejo/workflows/transfer.yaml > .forgejo/workflows/notification.yaml
```

Periksa hasilnya dengan `git diff --no-index`. Perintah `sed` juga mengganti kata `transfer` di dalam komentar, sehingga komentar tentang `proto/` di kedua *file* baru perlu diperbaiki. Di `account.yaml`, baris kedua komentar menjadi:

```
    # rebuilds both account and transfer.
```

Di `notification.yaml`, ganti kedua baris komentar di atas `paths:` dengan:

```
    # No proto/: notification does not import the gRPC contract, it only reads
    # the events on Kafka. go.work and pipeline.mk are shared by all services.
```

10.4.6 Langkah 6: `Jenkinsfile` untuk pemetaan ke Jenkins

Banyak organisasi menjalankan CI *on-premise* dengan Bitbucket dan Jenkins. `Jenkinsfile` *declarative* berikut menjalankan *target* `make` yang sama dengan *workflow* pada Langkah 5. Setiap service mendapat satu *job* Jenkins bertipe *Multibranch Pipeline* dengan *Script Path* `studi-kasus/<service>/Jenkinsfile`. Tipe *job* lain tidak bisa digunakan. Kondisi `when { branch 'main' }` membaca `BRANCH_NAME`, dan variabel ini hanya diisi oleh *job Multibranch*; di *job Pipeline* biasa semua *stage* akan dilewati.

Stage `Test` diawali blok `script` yang menghentikan *build* bila `GIT_COMMIT` kosong. `GIT_COMMIT` diisi oleh *checkout* SCM bawaan *declarative pipeline*. Bila *checkout* ini dimatikan, misalnya dengan `options { skipDefaultCheckout() }`, `VERSION = "${env.GIT_COMMIT}"` berisi *string* `null`, dan pemeriksaan variabel kosong di `pipeline.mk` akan meloloskan *image* dengan *tag* `null`. Pemeriksaan ini diletakkan di dalam *stage* `Test`, bukan di *stage* tersendiri, agar daftar *stage* tetap setara satu per satu dengan *step workflow*.

Pemetaan antara kedua format:

Workflow	**Jenkins**	**Keterangan**
Step Checkout	*Checkout* SCM bawaan	Jenkins melakukan *checkout* sebelum *stage* pertama
Step Test	*Stage* `Test`	`make -C studi-kasus/<service> test`
Step Build image	*Stage* `Build Image`	`make ... image`
Step Push	*Stage* `Push`	`make ... push`
Step Deploy	*Stage* `Deploy`	`make ... deploy`
`github.sha`	`env.GIT_COMMIT`	Nilai `VERSION`
Variable `REGISTRY`, `NAMESPACE`	*Environment variable* folder atau global	`IMAGE_OWNER` juga diisi di sini
Secret `REGISTRY_USER`, `REGISTRY_PASSWORD`	*Credential* `registry` jenis *username with password*	Dibuka dengan `withCredentials` hanya di *stage* `Push`
Secret `KUBE_CONFIG`	*Credential* `kubeconfig` jenis *secret file*	`withCredentials` mengisi `KUBECONFIG` dengan *path file* sementara
`concurrency`	`disableConcurrentBuilds()`	Satu *build* per service dalam satu waktu

`on.push.branches` dan `paths`	`when { branch ...; changeset ... }`	Lihat catatan di bawah
Re-run sebuah *run*	*Build Now*	`triggeredBy 'UserIdCause'`: semua *stage* berjalan untuk *commit* terakhir tanpa memeriksa *changeset*

***Changeset* pada *build* pertama dan setelah *build* gagal.** Kondisi `changeset` membandingkan *build* saat ini dengan *build* sebelumnya pada *branch* yang sama, apa pun hasil *build* sebelumnya. *Build* pertama sebuah *branch* tidak punya pembanding, sehingga *changeset*-nya kosong dan semua *stage* dilewati. Hal yang sama terjadi pada *build* ulang setelah *build* yang gagal: *commit* yang sudah tercatat di *build* gagal tidak masuk lagi ke *changeset* berikutnya. Karena itu kondisi `when` juga menerima `triggeredBy 'UserIdCause'`. *Build* yang dijalankan manual dengan *Build Now* selalu menjalankan semua *stage* untuk *commit* terakhir, sama seperti *re-run* di runner lab. *Build* yang dipicu *webhook* atau *branch indexing* tetap mengikuti *changeset*.

Buat *file* `studi-kasus/transfer/Jenkinsfile`:

```
// Declarative pipeline of the transfer service for Jenkins, the mapping of
// .forgejo/workflows/transfer.yaml: the same stages calling the same make
// targets. Job setup: a Multibranch Pipeline on this repository with Script
// Path studi-kasus/transfer/Jenkinsfile, one job per service. Multibranch
// only: "when { branch 'main' }" reads BRANCH_NAME, which only a
// Multibranch job sets; in a plain Pipeline job every stage would be skipped.
//
// Verified on Jenkins LTS 2.580.1 (Multibranch Pipeline, plain Git branch
// source, built-in node with git, make, go, docker and kubectl) deploying to
// k3d: path filter per service, first deploy into an empty namespace, rolling
// update, disableConcurrentBuilds(), a missing variable stopping make.
//
// What Jenkins must provide (no defaults here; make stops with "missing
// required variables: ..." when one is absent):
//   REGISTRY, IMAGE_OWNER, NAMESPACE   environment variables of the folder or
//                                      global properties, per environment
//   credentials "registry"             Username with password, push access
//   credentials "kubeconfig"           Secret file, kubeconfig limited to
//                                      NAMESPACE; the first deploy creates
//                                      the service's objects there
//   agent                              git, make, go, docker and kubectl
pipeline {
    agent any

    options {
        // Two builds of one service deploying at once would race on the
        // same Deployment.
        disableConcurrentBuilds()
        timeout(time: 20, unit: 'MINUTES')
    }

    environment {
        // One image per commit, as in the workflow (github.sha). GIT_COMMIT
        // is set by the SCM checkout; without it this becomes the string
        // "null", which the Test stage rejects.
        VERSION = "${env.GIT_COMMIT}"
    }
```

```
    stages {
        // The workflow's "on: push: branches/paths" filter. changeset
        // compares with the previous build of the branch, whatever its
        // result: the first build of a branch has no changeset, and a build
        // after a failed one sees only the commits pushed since. Both would
        // skip the stages; "Build Now" (UserIdCause) runs them for the
        // current commit, the counterpart of re-running a workflow run.
        stage('transfer') {
            when {
                allOf {
                    branch 'main'
                    anyOf {
                        triggeredBy 'UserIdCause'
                        changeset 'studi-kasus/transfer/**'
                        changeset 'studi-kasus/proto/**'
                        changeset 'studi-kasus/go.work'
                        changeset 'studi-kasus/pipeline.mk'
                    }
                }
            }
            stages {
                stage('Test') {
                    steps {
                        // Fail before anything is built: make's check for
                        // empty variables would let an image tagged "null"
                        // through.
                        script {
                            if (!env.GIT_COMMIT || env.GIT_COMMIT == 'null') {
                                error('GIT_COMMIT is empty: the job must check out this
repository from SCM (Multibranch Pipeline)')
                            }
                        }
                        sh 'make -C studi-kasus/transfer test'
                    }
                }
                stage('Build Image') {
                    steps {
                        sh 'make -C studi-kasus/transfer image'
                    }
                }
                stage('Push') {
                    steps {
                        withCredentials([usernamePassword(credentialsId: 'registry',
                                usernameVariable: 'REGISTRY_USER', passwordVariable:
'REGISTRY_PASSWORD')]) {
                            sh 'make -C studi-kasus/transfer push'
                        }
                    }
                }
                stage('Deploy') {
                    steps {
                        // The secret file exists only inside this block;
                        // KUBECONFIG points kubectl at it.
                        withCredentials([file(credentialsId: 'kubeconfig', variable:
'KUBECONFIG')]) {
                            sh 'make -C studi-kasus/transfer deploy'
                        }
                    }
```

```
                }
            }
        }
    }
}
```

Buat `account/Jenkinsfile` dan `notification/Jenkinsfile` dengan cara yang sama seperti Langkah 5: ganti `transfer` dengan nama service, dan hapus baris `changeset 'studi-kasus/proto/**'` untuk `notification`.

```
sed 's/transfer/account/g' studi-kasus/transfer/Jenkinsfile > studi-kasus/account/
Jenkinsfile
sed -e 's/transfer/notification/g' -e '/studi-kasus\/proto/d' \
  studi-kasus/transfer/Jenkinsfile > studi-kasus/notification/Jenkinsfile
```

Ketiga `Jenkinsfile` ini sudah dijalankan di Jenkins LTS 2.580.1: *job Multibranch Pipeline* dengan sumber Git, *environment variable* global `REGISTRY`, `IMAGE_OWNER`, dan `NAMESPACE`, *credential* `registry` dan `kubeconfig` (*ServiceAccount* dengan RBAC satu *namespace*), lalu *deploy* ke k3d. Hasil yang teramati:

- *commit* yang mengubah `studi-kasus/transfer/` hanya menjalankan *stage* di *job* `transfer`; *job* `account` dan `notification` ikut berjalan, tetapi semua *stage*-nya dilewati;
- *commit* yang mengubah `studi-kasus/proto/` menjalankan `account` dan `transfer`, tetapi tidak menjalankan `notification`;
- *commit* yang hanya mengubah dokumentasi melewati semua *stage* di ketiga *job*;
- *deploy* pertama membuat ConfigMap, Service, Deployment, dan Ingress dari *namespace* kosong, dan *deploy* berikutnya menambah satu revisi di `kubectl rollout history` dengan anotasi `<service> <SHA>`;
- *build* kedua untuk service yang sama menunggu di antrean dengan pesan `Build #N is already in progress`;
- tanpa `REGISTRY`, *stage* `Build Image` berhenti dengan `missing required variables: REGISTRY`.

Di kelas, *file* ini digunakan sebagai bahan perbandingan; peserta tidak menjalankan Jenkins.

10.4.6.1 *Pipeline* yang Sama di GitHub Actions

Repository materi ini juga menjalankan *pipeline* yang sama di GitHub Actions, dan hasil setiap *run* dapat dibuka publik di `github.com/artivisi/training-microservices-2026-01/actions`. *Workflow* `.github/workflows/<service>.yaml` menjalankan *target* `make` yang sama dengan runner git server dan Jenkins, lalu mengirim *image* ke `ghcr.io`. *Runner* GitHub tidak memiliki *cluster*, sehingga *step* Deploy diganti *workflow* `e2e-k3d.yaml`. *Workflow* ini membuat k3d di *runner*, menjalankan Lab 9, menguji satu transfer lewat Ingress, lalu menjalankan `make pipeline-k3d SERVICE=transfer` sambil mengirim transfer terus-menerus. *Run* dinyatakan gagal bila ada satu saja respons yang bukan 2xx selama *rolling update*.

Tahap	Runner git server	Jenkins	GitHub Actions
Checkout	`git fetch` dengan *token job*	*Checkout* SCM bawaan	`git fetch` dengan `GITHUB_TOKEN`
Tools	Terpasang di host runner	Terpasang di *agent*	*Step* `Install tools: .github/install-tools.sh`, versi dan SHA-256 tetap
Test	`make ... test`	`stage('Test')`	`make ... test`

Build image	`make ... image`	`stage('Build Image')`	`make ... image`
Push	`make ... push` ke registry git server	`stage('Push')`	`make ... push` ke `ghcr.io`
Credential registry	*Secret* `REGISTRY_USER`, `REGISTRY_PASSWORD`	*Credential* `registry`	`github.actor` dan `GITHUB_TOKEN` dengan `packages: write`
Deploy	`make ... deploy`	`stage('Deploy')`	*Workflow* `e2e-k3d.yaml`: `make pipeline-k3d` ke k3d di *runner*
Satu *run* per service	`concurrency`	`disableConcurrentBuilds()`	`concurrency`

GitHub Actions hanya menambah satu *step*, yaitu `Install tools`. *Runner* GitHub dimulai dari *image* generik, sedangkan runner git server dan *agent* Jenkins sudah dipasangi Go, Docker, dan kubectl.

10.4.7 Langkah 7: *Overlay* server pelatihan

Sama seperti *manifest* Lab 9, *overlay* server sudah disediakan sehingga peserta tidak perlu menulisnya dari awal. Ambil dari *tag* `lab-10`:

```
git checkout lab-10 -- studi-kasus/k8s/overlays/server studi-kasus/README.md
ls studi-kasus/k8s/overlays/server
```

```
config.yaml  ingress.yaml  kustomization.yaml  secrets.yaml
```

Perintah yang sama mengambil `studi-kasus/README.md` versi Lab 10, yang menambahkan bagian ringkasan Lab 10.

Overlay `server` sengaja tidak berisi `namespace:` dan blok `images:`. *Namespace* dan *tag image* baru diketahui saat *pipeline* berjalan (`NAMESPACE` dan `VERSION`), dan keduanya diisi oleh *target* `apply` pada Langkah 1. Dengan demikian peserta tidak perlu memasang *overlay* secara manual sebelum *pipeline* pertama, dan tidak ada *tag* fiktif di repository.

Perbedaan *overlay* `server` dengan *overlay* `k3d`:

Hal	*Overlay* `k3d`	*Overlay* `server`
Namespace	`bank`, dibuat oleh *overlay* (`namespace.yaml`)	Tidak ditulis di *overlay*; diisi `apply` dari `NAMESPACE`. *Namespace* peserta dibuat instruktur bersama *kubeconfig* dengan RBAC yang hanya berlaku di *namespace* itu. Tidak ada objek Namespace karena peserta tidak boleh membuat *namespace*
Image	Di-*import* ke k3d, tanpa registry. *Tag* ditulis di `newTag`	Tidak ditulis di *overlay*; diisi `apply` menjadi `<REGISTRY>/<IMAGE_OWNER>/<service>:<VERSION>`. Di-*pull* dari *container registry* git server dengan Secret `registry-credentials`

		(`imagePullSecrets`, ditambahkan ke setiap Deployment lewat *patch*)
Secret	Berisi nilai lokal untuk Docker Compose	Dihapus dari hasil *render* (`$patch: delete`). Instruktur membuat Secret `account`, `transfer`, `notification` berisi `DATABASE_URL` (dan `OAUTH_CLIENT_SECRET` untuk `transfer`), sehingga *credential* tidak pernah masuk ke repository
Kafka	`host.k3d.internal:19093`, *topic* `transfer-events`	Kafka bersama, *topic* dan *consumer group* diberi awalan nama peserta
OAuth2, OTLP	Hydra dan otel-lgtm di Docker Compose laptop	Server OAuth2 dan *collector* bersama
Ingress	Semua *host* di `localhost:28080`	*Host* per peserta, karena *ingress controller* digunakan bersama

Setiap nilai yang ditulis `replace-me` di `config.yaml` dan `ingress.yaml` adalah *placeholder* untuk alamat atau nama yang dibagikan instruktur ketika server siap. Domain `.invalid` adalah *top-level domain* yang dicadangkan. Bila ada *placeholder* yang terlewat, nama tersebut tidak bisa di-*resolve*, sehingga tidak ada koneksi yang sampai ke *host* sungguhan.

Target `apply` memilih objek dengan `kubectl apply -l app.kubernetes.io/name=<service>`. Deployment, Service, dan Ingress dari Lab 9 sudah membawa label itu, tetapi ConfigMap dan Secret belum. Tambahkan blok `labels` ke `k8s/base/<service>/kustomization.yaml` ketiga service, sehingga *kustomize* memasang label tersebut di setiap objek service. Untuk `transfer`:

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
# Every object of this service carries its name, also the ConfigMap and the
# Secret: the pipeline's first deploy (make deploy in Lab 10) applies only the
# objects of one service with kubectl apply -l app.kubernetes.io/name=transfer.
# Metadata labels only: selectors and pod labels are already in the manifests.
labels:
  - pairs:
      app.kubernetes.io/name: transfer
resources:
  - configmap.yaml
  - secret.yaml
  - deployment.yaml
  - service.yaml
  - ingress.yaml
```

Untuk `account` dan `notification`, nilai labelnya adalah nama service masing-masing. Daftar `resources` kedua *file* itu tidak berubah. Blok `labels` tanpa `includeSelectors` hanya mengubah `metadata.labels`; *selector* Deployment dan Service tetap seperti Lab 9. Hasil *render* `k8s/overlays/k3d` hanya bertambah label pada ConfigMap dan Secret, sehingga `make k8s-apply` tidak memicu *rollout*.

Selama masih ada *placeholder*, `make k8s-apply-server` berhenti tanpa memasang apa pun, bahkan sebelum menghubungi *cluster*:

```
cd studi-kasus
make k8s-apply-server SERVICE=transfer VERSION=abc REGISTRY=r IMAGE_OWNER=o NAMESPACE=n
```

Output-nya menampilkan baris-baris hasil *render* yang masih berisi `replace-me`, diikuti:

```
../k8s/overlays/server still has replace-me placeholders (lines above)
```

Path diawali `../` karena pemeriksaan ini dijalankan oleh *target* `apply` dari direktori service. Tanpa `SERVICE`, perintah berhenti dengan `SERVICE is required, e.g. make k8s-apply-server SERVICE=transfer VERSION=<sha>`; tanpa variabel lain, berhenti dengan `missing required variables: ....`

Hasil *render overlay* bisa diperiksa tanpa *cluster* dengan `kubectl kustomize k8s/overlays/server`.

Nilai pengganti setiap `replace-me` dibagikan instruktur di server pelatihan (`~/lab10.txt`, Langkah 10). *Namespace* dan registry tidak termasuk di dalamnya: keduanya sudah diisi sebagai *variable* repository.

10.4.8 Langkah 8: Menjalankan *pipeline* di laptop hingga *deploy* ke k3d

Langkah ini menjalankan tahapan *pipeline* yang sama dengan runner CI, tetapi di laptop: *test*, *image* dengan *tag* berupa SHA *commit*, *import* ke k3d sebagai pengganti `push`, lalu *deploy* dengan `kubectl set image` dan `rollout status`.

Siapkan infrastruktur, *cluster*, dan *manifest* dari Lab 9. Perintah kedua menggunakan *cluster* `bank` yang sudah ada, dan baru membuatnya bila belum ada:

```
make infra-up
k3d cluster get bank || make k3d-up
make images VERSION=1.0.0
make k3d-import VERSION=1.0.0
make k8s-apply
```

Cluster yang dihentikan dengan `k3d cluster stop bank` dijalankan lagi dengan `k3d cluster start bank`. `make k8s-apply` di sini juga memasang label baru dari Langkah 7 ke ConfigMap dan Secret.

Commit hasil Langkah 1 hingga 7 terlebih dahulu, karena `pipeline-k3d` berhenti dengan pesan error bila `studi-kasus/` masih berisi perubahan yang belum di-*commit*:

```
cd ..
git add .forgejo .gitignore studi-kasus
git commit -m "Lab 10: pipeline per service"
cd studi-kasus
```

Buat perubahan kecil yang terlihat di log `transfer`. Di `transfer/main.go`, ganti pesan log saat server HTTP mulai berjalan:

```
    log.Info("transfer service started", "addr", cfg.HTTPAddr, "account_grpc",
cfg.AccountGRPCAddr,
```

Baris aslinya berbunyi `log.Info("transfer service listening", ...` di dalam *goroutine* yang memanggil `srv.ListenAndServe()`. *Commit* perubahan itu, lalu jalankan *pipeline*:

```
git commit -am "transfer: change startup log message"
make pipeline-k3d SERVICE=transfer
```

Urutan yang terlihat di *output*: `go vet` dan `go test` untuk `transfer`, `docker build` dengan *tag* `k3d.local/studi-kasus/transfer:<12 karakter SHA>`, `k3d image import`, lalu perintah `kubectl` dari *target* `deploy`.

Deployment `transfer` sudah ada dari `make k8s-apply`, sehingga `deploy` tidak memanggil `apply` dan langsung menjalankan `kubectl set image`, `kubectl annotate`, dan `kubectl rollout status`. Baris terakhirnya:

```
deployment "transfer" successfully rolled out
```

`k3d.local` hanya nama registry fiktif untuk *tag* lokal. *Image* tidak pernah dikirim ke registry; `k3d image import` menyalinnya langsung ke *containerd* di dalam *cluster*. Karena Deployment menggunakan `imagePullPolicy: IfNotPresent`, *kubelet* menjalankan salinan itu tanpa mem-*pull image* dari registry.

10.4.9 Langkah 9: Menelusuri versi yang berjalan dan mengamati *drift*

Lihat riwayat *rollout*:

```
kubectl -n bank rollout history deployment/transfer
```

```
deployment.apps/transfer
REVISION  CHANGE-CAUSE
1         <none>
2         transfer 3c9d1e0a7b42
```

SHA di atas hanya contoh; setiap laptop menghasilkan SHA yang berbeda. Nomor revisi bisa lebih banyak bila latihan *rolling update* Lab 9 sudah dijalankan. Revisi terakhir mencatat nama service dan SHA *commit* hasil `pipeline-k3d`. Bandingkan dengan `git log`:

```
git rev-parse --short=12 HEAD
kubectl -n bank get deploy transfer -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

```
3c9d1e0a7b42
k3d.local/studi-kasus/transfer:3c9d1e0a7b42
```

Sekarang jalankan ulang `make k8s-apply`, perintah Lab 9 yang memasang *manifest* dari repository:

```
make k8s-apply
kubectl -n bank get deploy transfer -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
```

```
studi-kasus/transfer:1.0.0
```

Image kembali ke *tag* `1.0.0` yang tertulis di `k8s/overlays/k3d/kustomization.yaml`, dan hasil *pipeline* hilang. Penyebabnya: *pipeline* mengubah *cluster* langsung dengan `kubectl set image`, sedangkan repository tetap menyebut `1.0.0`. Isi *cluster* dan isi repository berbeda (*drift*), dan siapa pun yang meng-*apply* ulang repository membatalkan *deploy* terakhir tanpa sadar.

GitOps menghilangkan selisih tersebut dengan menjadikan repository satu-satunya jalan untuk mengubah *cluster*: *pipeline* tidak memanggil `kubectl set image`, tetapi meng-*commit* *tag* baru ke *manifest*, lalu sebuah *agent* di *cluster* menyamakan isi *cluster* dengan isi repository. Lab ini tidak memasang *agent* GitOps. Cukup dipahami bahwa *drift* yang baru saja terlihat itulah masalah yang diselesaikan GitOps.

> Setelah `make k8s-apply`, kolom `CHANGE-CAUSE` revisi terbaru bisa tetap menampilkan `transfer <sha>` walaupun *image*-nya sudah `1.0.0`. Anotasi `change-cause` ditulis oleh `make deploy`, bukan

> oleh *manifest*, sehingga `kubectl apply` tidak menghapusnya. Untuk mengetahui *image* yang benar-benar berjalan, baca `.spec.template.spec.containers[0].image`, bukan anotasi.

Untuk mengulang latihan, ubah kode lagi, *commit*, lalu jalankan `make pipeline-k3d SERVICE=transfer` sekali lagi.

10.4.10 Langkah 10: *Pipeline* di server pelatihan

Instruktur sudah menyiapkan hal berikut untuk setiap peserta:

- repository `<peserta>/training-microservices-2026-01` di git server, berisi semua *branch* dan *tag* repository publik, dengan *variable* `REGISTRY` dan `NAMESPACE` serta *secret* `REGISTRY_USER`, `REGISTRY_PASSWORD`, dan `KUBE_CONFIG`. *Kubeconfig* ini hanya berlaku di *namespace* peserta, dan RBAC-nya mengizinkan pembuatan ConfigMap, Service, Deployment, dan Ingress di sana;
- akun Linux di server pelatihan dengan nama yang sama dengan akun git server, `kubectl` yang sudah terhubung ke *namespace* peserta, dan *file* `~/lab10.txt` berisi semua nilai untuk langkah ini;
- *namespace* peserta berisi Secret `registry-credentials` dan Secret `account`, `transfer`, `notification`;
- *topic* Kafka `<peserta>.transfer-events` dan dua *client* OAuth2: `<peserta>-transfer-service` (digunakan `transfer` untuk memanggil `account`) dan `<peserta>-channel-app` (digunakan peserta untuk memanggil REST `transfer`).

Langkah persiapan di git server:

1. Masuk ke git server lewat *browser* dengan *password* awal dari instruktur, lalu ganti *password*. Selama *password* awal belum diganti, git server menolak `git clone` dan `git push` lewat HTTPS dengan pesan `remote: Update your password`.
2. Daftarkan *public key* SSH di menu *Settings* → *SSH / GPG Keys*. Kunci yang sama digunakan untuk git lewat SSH dan untuk masuk ke server pelatihan.
3. Masuk ke server pelatihan dan baca nilai milik sendiri:

```
ssh <peserta>@app.<domain>
cat ~/lab10.txt
kubectl get pods
```

`kubectl get pods` menjawab `No resources found in <peserta> namespace.` sebelum *pipeline* pertama. *Namespace* peserta lain tidak bisa dibaca: `kubectl -n <peserta-lain> get pods` berhenti dengan `Forbidden`.

Alur peserta:

1. *Clone* repository dari git server, di laptop atau di server pelatihan. Repository itu sudah berisi *overlay* `k8s/overlays/server`:

   ```
   git clone git@<git server>:<peserta>/training-microservices-2026-01.git
   cd training-microservices-2026-01
   ```

2. Ganti setiap `replace-me` di `studi-kasus/k8s/overlays/server/config.yaml` dan `ingress.yaml` dengan nilai dari `~/lab10.txt`. *File* tersebut juga memuat satu perintah `sed` (GNU sed, Linux) yang mengganti semuanya sekaligus. Periksa hasilnya dengan `kubectl kustomize studi-kasus/k8s/overlays/server | grep replace-me`, yang tidak boleh mencetak apa pun.
3. Tambahkan satu baris komentar di akhir `studi-kasus/pipeline.mk`, lalu *commit* dan *push* `main`. Perubahan yang hanya menyentuh `k8s/overlays/server` tidak memicu *workflow* mana pun, karena *path* itu tidak tercantum di `on.push.paths`. Perubahan `pipeline.mk` memicu ketiga *workflow*, sehingga *pipeline* pertama membuat ketiga service. `transfer` membutuhkan `account` untuk setiap transfer.

```
echo '# first deploy to the training server' >> studi-kasus/pipeline.mk
git commit -am "Fill server overlay"
git push origin main
```

4. Buka halaman *Actions* repository di git server (`https://<git server>/<peserta>/training-microservices-2026-01/actions`). Ketiga *workflow* berjalan bersamaan; log setiap *step* bisa dibuka dari halaman tersebut. Pada *pipeline* pertama, Deployment belum ada, sehingga log *step* Deploy menampilkan `deployment/transfer not in namespace <peserta>: first deploy, applying the overlay`, lalu `apply` membuat ConfigMap, Service, Deployment, dan Ingress dengan *image commit* tersebut. *Image* ketiga service muncul di menu *Packages* akun peserta.
5. Di server pelatihan, periksa hasilnya:

```
kubectl get deploy,pods,ingress
kubectl rollout history deployment/transfer
```

6. Buat dua rekening lewat `kubectl port-forward`, ambil *token* `channel-app`, lalu kirim transfer lewat Ingress. Port lokal untuk `port-forward` dan *client secret* ada di `~/lab10.txt`. Server pelatihan digunakan bersama, sehingga setiap peserta mendapat nomor port sendiri.
7. Ubah pesan log di `studi-kasus/transfer/main.go` seperti Langkah 8, *commit*, lalu *push*. Hanya *workflow* `transfer` yang berjalan. `kubectl rollout history deployment/transfer` kini menampilkan dua revisi, masing-masing dengan SHA *commit*-nya.

Pada uji coba server pelatihan, *pipeline* pertama (ketiga service sekaligus, *cache* Go dan Docker masih kosong) selesai dalam sekitar 3,5 menit. *Pipeline* berikutnya untuk satu service selesai dalam sekitar 1 menit, termasuk *rolling update*. Selama *rolling update* tersebut, 81 transfer yang dikirim terus-menerus lewat Ingress semuanya dijawab `201`. Runner menjalankan paling banyak empat *job* sekaligus; *push* berikutnya menunggu di antrean.

Hasil kerja sendiri dari laptop (*branch* `lab-10-saya`) juga bisa dikirim ke repository ini. *Branch* tersebut tidak berasal dari `main` repository git server, sehingga *push* biasa ditolak sebagai *non-fast-forward*. Repository ini milik peserta sendiri, jadi `main` boleh ditimpa:

```
git remote add git-server git@<git server>:<peserta>/training-microservices-2026-01.git
git push --force git-server lab-10-saya:main
```

Setelah Deployment ada, *pipeline* tidak lagi menyentuh ConfigMap dan Ingress. Perubahan `config.yaml` atau `ingress.yaml` dikirim ke *cluster* dengan `make k8s-apply-server`, satu service per perintah, dengan `VERSION` berisi *commit* yang sedang berjalan agar *image* tidak berubah:

```
kubectl -n <namespace> rollout history deployment/transfer
make k8s-apply-server SERVICE=transfer VERSION=<sha> REGISTRY=<registry>
IMAGE_OWNER=<pemilik> NAMESPACE=<namespace>
kubectl -n <namespace> rollout restart deployment/transfer
```

Pod membaca ConfigMap hanya saat *startup*, sehingga `rollout restart` diperlukan agar nilai baru terbaca. Selisih antara repository dan *cluster* ini juga termasuk *drift* yang dihilangkan GitOps.

Di server pelatihan, sama seperti di k3d, Prometheus tidak mengambil *metric* dari *pod*. Yang sampai ke *collector* hanya *trace* dan log.

10.5 Verifikasi

Jalankan ulang `make pipeline-k3d SERVICE=transfer` setelah Langkah 9, karena `make k8s-apply` di Langkah 9 mengembalikan *image* ke `1.0.0`.

Perintah	Hasil yang Diharapkan

`make -C transfer image`	Berhenti dengan `missing required variables: REGISTRY IMAGE_OWNER VERSION`
`make -C transfer push REGISTRY=r IMAGE_OWNER=o VERSION=v`	Berhenti dengan `missing required variables: REGISTRY_USER REGISTRY_PASSWORD`
`make pipeline-k3d`	Berhenti dengan `SERVICE is required, e.g. make pipeline-k3d SERVICE=transfer`
`make pipeline-k3d SERVICE=transfer` dengan perubahan yang belum di-*commit*	Berhenti dengan `uncommitted changes in studi-kasus/: commit first, the image tag is the commit`
`make pipeline-k3d SERVICE=transfer` setelah *commit*	Berakhir dengan `deployment "transfer" successfully rolled out`
`kubectl -n bank rollout history deployment/transfer`	Revisi terakhir berisi `transfer <12 karakter SHA>`
`kubectl -n bank logs deployment/transfer`	Ada baris JSON dengan `"msg":"transfer service started"`
`make -C transfer deploy REGISTRY=r IMAGE_OWNER=o VERSION=v`	Berhenti dengan `missing required variables: NAMESPACE`
`make k8s-apply-server SERVICE=transfer VERSION=abc REGISTRY=r IMAGE_OWNER=o NAMESPACE=n`	Menampilkan baris `replace-me`, lalu berhenti dengan `../k8s/overlays/server still has replace-me placeholders (lines above)`
`kubectl -n bank get configmap,secret -l app.kubernetes.io/name=transfer -o name`	`configmap/transfer` dan `secret/transfer`: label dari Langkah 7 sudah terpasang

Periksa juga bahwa ketiga *workflow* dan ketiga `Jenkinsfile` hanya berbeda pada nama service dan baris `proto`:

```
cd ..
git diff --no-index .forgejo/workflows/transfer.yaml .forgejo/workflows/
notification.yaml
git diff --no-index studi-kasus/transfer/Jenkinsfile studi-kasus/account/Jenkinsfile
```

10.6 Masalah Umum

`missing separator. Stop.` Baris *recipe* di `pipeline.mk` atau `Makefile` diawali spasi. Ganti spasi di awal baris *recipe* dengan satu karakter *Tab*.

`uncommitted changes in studi-kasus/.` `pipeline-k3d` hanya berjalan bila `studi-kasus/` tidak berisi perubahan yang belum di-*commit*. Jalankan `git status`, lalu *commit* atau `git stash` perubahan yang tersisa. Direktori `bin/` hasil *build* lokal sudah tercantum di `.gitignore`.

`deployment/transfer not in namespace bank: first deploy, applying the overlay`, lalu `../k8s/overlays/server still has replace-me placeholders`, saat `pipeline-k3d`. Deployment `transfer` belum ada di k3d, sehingga `deploy` mencoba memasang *overlay* server, yang ditolak karena masih berisi *placeholder*. Di laptop, Deployment dibuat oleh `make k8s-apply` dari *overlay* `k3d`. Jalankan `make k8s-apply`, lalu ulangi `make pipeline-k3d`.

`The connection to the server ... was refused` saat `make -n -C <service> deploy`. Opsi `-n` tetap menjalankan baris *recipe* yang berisi `$(MAKE)`, termasuk `kubectl get deployment` di awal `deploy` (Langkah 2). Baca urutan perintahnya langsung di `pipeline.mk`.

`rollout status` berhenti setelah 180 detik. *Pod* baru tidak lolos *readiness probe*. Periksa `kubectl -n bank get pods` dan `kubectl -n bank logs <pod baru>`. Penyebab yang sering muncul: infrastruktur Docker Compose belum berjalan (`make infra-up`), atau kode baru gagal saat *startup*. *Pod* lama tetap melayani; kembalikan dengan `kubectl -n bank rollout undo deployment/transfer` bila perlu. `rollout undo` bisa mencetak peringatan tentang anotasi `last-applied-configuration`; peringatan ini tidak berbahaya dan bukan error.

`ErrImageNeverPull` atau `ErrImagePull` untuk *tag* `k3d.local/...`. *Image* tidak ada di *node* k3d, sehingga kubelet mencoba mengambilnya dari jaringan. `k3d-load` sudah memeriksa setiap *image* setelah *import* dan mengulang *import* satu kali, sehingga kondisi ini biasanya berarti *import* diarahkan ke *cluster* lain. Pastikan *cluster* bernama `bank` (`k3d cluster list`), lalu jalankan ulang `make pipeline-k3d`.

Hasil *pipeline* hilang. `make k8s-apply` dijalankan setelah `pipeline-k3d` dan mengembalikan *image* ke `1.0.0`. Ini perilaku yang diharapkan (Langkah 9).

***Build* Jenkins pertama melewati semua *stage*.** *Changeset* kosong pada *build* pertama sebuah *branch* (lihat catatan di Langkah 6). Jalankan *Build Now* pada *branch* `main`. Bila semua *build* melewati semua *stage*, termasuk *Build Now*, periksa tipe *job*: hanya *Multibranch Pipeline* yang mengisi `BRANCH_NAME` untuk kondisi `branch 'main'`.

***Build* Jenkins setelah *build* yang gagal melewati semua *stage*.** *Changeset* dibandingkan dengan *build* sebelumnya, termasuk *build* yang gagal, sehingga *build* berikutnya tanpa *commit* baru di direktori service tidak menjalankan apa pun. Perbaiki penyebabnya, lalu jalankan *Build Now* atau *push commit* perbaikan.

***Build* Jenkins berhenti dengan `GIT_COMMIT is empty`.** *Checkout* SCM bawaan *declarative pipeline* dimatikan, misalnya dengan `options { skipDefaultCheckout() }`, sehingga `VERSION` tidak berisi SHA *commit*. Hapus opsi tersebut.

10.7 Eksplorasi Mandiri

1. Ubah *file* di `studi-kasus/proto/`, lalu tentukan *workflow* mana yang akan terpicu dengan membaca `on.push.paths` ketiga *file*. Jalankan `make pipeline-k3d` untuk setiap service yang terpicu.
2. Buat *commit* yang membuat `transfer` gagal saat *startup* (misalnya mengganti nama *environment variable* yang wajib di `config.go`), lalu jalankan `make pipeline-k3d SERVICE=transfer`. Amati bahwa `rollout status` gagal, *pod* lama tetap melayani transfer di `http://localhost:28080/transfers`, dan `kubectl rollout undo` mengembalikan revisi sebelumnya.
3. Coba alur GitOps secara manual: ubah `newTag transfer` di `k8s/overlays/k3d/kustomization.yaml` ke *tag* hasil `pipeline-k3d`, *commit*, lalu `make k8s-apply`. Bandingkan hasilnya dengan Langkah 9: kali ini `make k8s-apply` tidak membatalkan *deploy*.
4. Tambahkan *stage* `Approve` berisi *step* `input` sebelum *stage* `Deploy` di `Jenkinsfile`, sehingga *deploy* menunggu persetujuan manual. Rancang padanannya di *workflow*, misalnya dengan memindahkan *step* Deploy ke *workflow* terpisah yang dipicu manual.

10.8 Checkpoint

Kondisi akhir lab ini tersedia di *tag* `lab-10`. Bandingkan hasil kerja dengan *tag* tersebut, atau lanjutkan dari sana:

```
git diff lab-10 -- studi-kasus .forgejo .gitignore
git checkout lab-10
```

Hasil kerja wajar berbeda dari *tag* `lab-10` pada perubahan pesan log di `transfer/main.go` dari Langkah 8, karena perubahan itu tidak termasuk di *tag*. Komentar di `account.yaml` dan `notification.yaml` yang ditulis ulang dengan kata-kata sendiri juga wajar berbeda.